

The CoCoT_EX Framework

A modular package suite for
automatic, flexible typesetting

Version v0.6.2
(2025/07/14)

Lupino
lupino@le-tex.de

Table of contents

Introduction	vii
1 Basic concepts	vii
1.1 Types, Inheritance and Abstract Containers	vii
1.2 Complex Components	vii
1.3 Relation to L ^A T _E X Templates	viii
2 How to Read This Documentation	viii
2.1 Keyword Markings	viii
2.2 Data Types of Properties	ix
2.3 Types of Components	ix
 Module 1 cocotex.dtx	 3
1 Hard-coded requirements	3
2 Class Options	3
2.1 Options passed down to mandatory standard L ^A T _E X packages	3
2.2 The Publication Type	4
2.3 User-Level Macro Names and Debugging Options	4
2.4 Accessibility Features	5
2.5 Options for Other CoCoT _E X Modules	6
3 PDF-Version and Standard Conformity	7
4 Class Hook	8
5 Internal Requirement	9
6 Loading and Adjusting Underlying DocumentClass	9
6.1 General Typography	9
7 Loading other CoCoT _E X Modules	10
7.1 coco-accessibility	10
7.2 coco-script	10
7.3 coco-headings	11
7.4 coco-floats	11
7.5 coco-title	11
7.6 coco-notes	11
8 Further Hard Dependencies	11
8.1 Index	11
8.2 Hyperref	11
9 End of Document Class Hook	12
 Module 2 coco-kernel.dtx	 15
1 Preamble	15
1.1 Hard dependencies	15
1.2 Package Options	15
2 Exception handlers	16
3 Global Switches	18
4 Containers	18
5 Components	21
5.1 Simple Components	21
5.2 Counted Components	25
6 Properties	31
6.1 Setting Properties	31

6.2	Using Properties	32
6.3	Processing Instructions	34
6.4	Property Conditionals.....	34
7	Helper macros	35
7.1	Handling of Optional Arguments	35
7.2	Iterators	35
7.3	Attributes	35
7.4	Style Classes	39
7.5	The CoCoTeX Logo	40
Module 3 coco-common.dtx		41
1	Package options	41
1.1	Accessibility Features.....	41
1.2	Output Options	42
2	Commonly Used Low-Level Macros and Registers	42
2.1	Hard Dependencies	43
2.2	Common Variables.....	43
2.3	Helper macros.....	44
2.4	Masks	44
2.5	Arithmetics	45
2.6	Determine actual page number	46
3	Re-Thinking L ^A T _E X Core Functions	47
3.1	Keeping .aux-Files Up-to-Date	47
3.2	Content lists	47
3.3	Indentation and Left Margins of Potentially Numbered Items	50
3.4	Labelling and Cross referencing	54
3.5	Linguistic Name generation and selection	55
3.6	Link Generation	55
Module 4 coco-accessibility.dtx		59
1	LaTeX code	59
1.1	General Processing.....	59
1.2	Activating and Deactivating Accessibility Features	60
1.3	Accessibility-specific additions	61
1.4	Generic Macro to Declare Accessibility Features	62
1.5	Language Tagging	66
1.6	Lua injection	67
1.7	Alternative Text Elements	67
1.8	Hyperlink handling	67
1.9	Tagging Page Styles as Artifacts	69
1.10	generic artifacts	70
1.11	Tagging for Floats.....	70
1.12	Transformation of Typographic Unicode characters	72
1.13	Automatic PDF Tagging	72
1.14	Math Tagging	73
1.15	Default Role Mapping	73
2	Lua code	74
2.1	Local Variables and Tables	74
2.2	Meta Data Extraction	74
2.3	Public Methods.....	75
Module 5 coco-meta.dtx		77
1	Counted Container Handlers.....	77
1.1	Generic Blocks	77
1.2	Contributor Roles	78
1.3	Declaring Contributor Role Blocks	78

2	Labeled Components	82
3	Meta Data Rolemaps for Tagged PDFs	83
4	Common Meta Data	83
4.1	Affiliations	85
5	Meta Data Properties	86
5.1	Initials	86
5.2	Member Role Composition Properties	86
5.3	Overrides Within a Role Counted Component	86
5.4	Format of Single Role Collection Component Items	87
5.5	Collection Component Properties Specific to Author Role	88
5.6	Format of Affiliation Lists	89
5.7	Properties for Labeled Componetns	90

Module 6 coco-headings.dtx 93

1	Facility for declaring heading levels and their layouts	93
1.1	Initializers for New Heading Levels	99
1.2	Initializers for Instances of Heading Levels	100
2	Externalisation of Heading Compoents	100
2.1	Common Stuff	100
2.2	Table of Contents Entry	101
2.3	Facility to create the running title macros	102
2.4	Facility to create PDF bookmarks	103
3	Rendering the Headings	103
3.1	Inline Headings	104
3.2	Block Headings	104
4	The Heading environment	104
4.1	Environment Macros	104
4.2	Content Handlers	105
5	Defaults	107
6	Accessibility Features	114
7	Miscellaneous	117
7.1	Alternative paragraph separation	117

Module 7 coco-notes.dtx 119

1	Internal Switches and Package Options	119
1.1	Package Switches	119
1.2	Package Options	119
1.3	Hard Requirements	120
2	Endnote Handling	120
3	Processing Package Options	122
3.1	Endnotes With Chapters	122
3.2	Chapter-wise Resetting	123
3.3	Back-Referencing Endnotes	123
3.4	Allow Non-Numerical Endnote Counters	123
4	Adjusting Regular Footnotes	124
4.1	Allowing Multiple Paragraphs in Footnotes	124
4.2	Allowing Non-Numeric Footnote Counters	124
5	Tagging Footnotes	125

Module 8 coco-script.dtx 127

1	Fallback fonts	128
1.1	Default Fallback Font: Noto	128
1.2	Emojis	129
1.3	Support for medieval scripts and special characters	129
1.4	International Phonetic Alphabet	130
2	Generic Fonts Declaration Mechanism	130

3	Predefined script systems	131
3.1	Support for Armenian script.....	131
3.2	Support for Chinese script.....	131
3.3	Support for Japanese script	132
3.4	Support for Korean script.....	132
3.5	Support for Hebrew script	132
3.6	Support for Arabic script	133
3.7	Support for Greek script	133
3.8	Support for Ethiopian/Amharic script	133
3.9	Support for Georgian script	134
3.10	Support for Syrian script	134

Module 9 coco-title.dtx 137

1	Top-Level Interface.....	137
2	Processing of PDF Meta Data	139
2.1	Processing of the Document's Main Language.....	141
2.2	Processing of the Document's Title	141
2.3	Processing of the Document's Author	141
2.4	Processing of the PDF's Creator, Producer, and Keywords Meta Data.....	142
2.5	Including the XMP Meta Data	142
3	Intermediate Level Interfaces	143
3.1	Funds, Grants, and Supporters	144
3.2	Simple Component Declarations	145
4	Default Settings	153
5	Accessibility Features	159
5.1	Encoding of the PDF-A Conformance	159
5.2	Titlepage Specific Role Maps	160

Module 10 coco-floats.dtx 161

1	Package Setup	161
1.1	Hard requirements	161
1.2	Adjustments at the Beginning of the Document	162
1.3	Document Class-Option Overrides	162
1.4	Internal Registers	163
2	Internal macros	164
2.1	Generic resetter	164
2.2	Wrapper for L ^A T _E X's Native float Environments	165
3	The Generic float Container	166
3.1	Common Float Components.....	166
3.2	Common Float Properties	167
3.3	The Generic float Environment	173
3.4	The SubFloat Environment	175
3.5	Attribute Handlers	176
3.6	Handling of List-of Entries	177
3.7	Label and Referencing mechanisms.....	179
3.8	Processing the Float.....	181
3.9	Caption mechanism	184
4	Generic User-Level Float Containers.....	187
5	Image Containers	188
5.1	Abstract Graphics Container	188
5.2	Floating Figure Container	188
5.3	Measuring Single Fig Components for Variable Width Captions	189
5.4	Figure Output Routines	190
5.5	Inline Figures.....	191
6	Table Containers.....	192
6.1	The Abstract Tabular Container	192

6.2	The User-Level Table Container	193
6.3	The Table Output Handler	194
7	Other Float-Related Macros	195
Module 11 coco-frame.dtx		197
1	Top-Level Interface.....	197
2	Cropmark printer	198
Module 12 coco-lists.dtx		203
1	Preamble	203
1.1	Package Options	203
2	The List Container	204
2.1	List Properties.....	204
2.2	List Components	208
3	Declaring List Types	208
4	Declare Lists.....	209
4.1	The List Environment.....	211
4.2	Unpacking the List Type-Specific Handlers	213
5	Default List Types	214
5.1	Unnumbered Lists.....	214
5.2	Numbered Lists	215
5.3	Description Lists	217
5.4	Replacing L ^A T _E X's Default Lists	218
Index		219
Macro and Environment Index.....		219
Container Index		224
Component Index.....		225
Property Index		227
Hook Index		230
Tag Index.....		231
Attribute Index.....		232

Introduction

1 Basic concepts

The core concept of the CoCoTeX Framework to view typographical objects, such as *floats*, *headings*, *title pages*, etc., as closed units that contain a fixed set of elements that determine the exact nature of each occurrence. For a *heading*, such elements may be the heading's *title*, an optional *subtitle*, a *counter* or a list of *authors* responsible for the part of a publication introduced by the *heading*.

In CoCoTeX those typographical units are referred to as *Containers*. The occurrence of a Container in a specific TeX document is an *Instance* of that Container. The elements inside each Container instance are called *Components*.

The final realization of a *Container* in the rendered output is done in local style files with so-called *Properties*; short snippets of L^AT_EX code, which tell the L^AT_EX interpreter how the Components in the Instances of Containers are to be read, processed and eventually rendered.

Typically, Containers are L^AT_EX environments that contain the Components in the form of L^AT_EX macros or other, embedded, environments. In the simpler cases, Component macros take the value for the Component in that specific *Instance* of the Container as their mandatory argument. Most Containers follow an *read first – process later* approach, i.e., the L^AT_EX interpreter reads the whole content of the Container and the processing is done at the `\end` macro of the corresponding environment.

1.1 Types, Inheritance and Abstract Containers

Components and Properties are both seen as (*Data*-)*Types* specific to each Container. A Container can be abstract, meaning that the Container is by itself not directly used in an end-user's tex file, but serves as “blueprint” for other, more “user-level”, Containers. As such, Containers can *inherit* the Types of another Container. Containers that inherit Types from other Containers are called *Sub Containers* or *Child Containers*, while the inherited Container is called a *Parent Container*.

Containers are therefore somewhat comparable to *classes* in object-oriented programming languages, an Instance of a Container can be seen as an *object* (i.e., an *instance of a class*). Components are *object variables*, while Properties take the place of *class variables* and/or *methods*, depending on how exactly a certain Property is implemented. Sometimes, a Property holds only a simple value (which makes it a *class variable*), while another Property may contain a complex set of instructions and calls to other Properties and Component values (which would make it a *method*).

1.2 Complex Components

Components can also be more complex than simple data storage devices. Usually, a Component occurs only once in a Container, for instance, there can be only one (main) “Title” in each “Heading”.

Other Components may occur more than once in the same Container Instance, for example, a “chapter” (which itself may be a Sub Container of a more abstract Parent Container “Heading”) may have more than one “Author”. Such Components are called *Group Components*.

They are usually realized as L^AT_EX environments within a Container's environment and contain themselves other Components. Those “second-level” Components are called *Counted Components*, as they are “enumerated” across all Group Component instances within the same Container Instance. For each Group Component, there is a *Collection Component*, in which all instances of a Group Component are collected during processing. How this collection is put together is controlled by a special *Collection Property*.

1.3 Relation to L^AT_EX Templates

Newer version of L^AT_EX adopt a quite similar design principle with the introduction of *Templates* into the L^AT_EX Kernel in mid 2024¹

The template system in L^AT_EX provides three levels of abstraction: *Object Types*, *Templates* and *Template Instances*. An *Object Type* represents the general idea of a typographical element, like “heading”, “float”, or “list”. The *Object Type* also determines the exact number of “Arguments” each *Template Instance* must or may have. *Templates* define how the instances may be manipulated by the end-user by adding a list of pre-defined key-value pairs. Finally, *Template Instances* are what the end-user is supposed to be using in their documents, often masked behind more user-friendly interface macros.

As an example, an *Object Type* may be “sectioning” that provides the Arguments title, short title and number. A *Template* heading derived from that *Object Type* may introduce the Interface key-value pairs pre-skip, font-size and after-skip. A *Template Instance* then might be *subsection*, which defines the pre-skip to `2\baselineskip`, the after-skip to `1\baselineskip`

and the font-size to `{10}{12}`. The *Template Instance* is then being called in the definition of a user level macro like `\subsection[<label>][<shorttitle>]{<title>}`.

In CoCoT_EX terms, an *Object Type* would be an abstract Container that defines the exact number and nature of Components each Instance of that Container could have, but does not declare any Properties. L^AT_EX *Templates* are equivalent to CoCoT_EX’s second-level Child Containers whose parent is the abstract *Object-Type* Container, but it only declares the Properties that can be used to manipulate the output of each Container Instance. *Template Instances* are equivalent to third-level Sub Containers of the *Template* Child Container, that define fixed values for some or all of the *Properties*.

In summary, CoCoT_EX has no formal distinction between *Object Types*, *Templates* and *Template Instances*. However, those “layers of abstraction” can be realized by the *Inheritance* mechanism, but there is no hard restrictions about *if* and *when* *Properties* and/or *Components* are introduced.

2 How to Read This Documentation

The documented source code is printed in red code boxes with line numbers referring to lines in the corresponding unpacked .sty files:

29 This is the documented source code

Code and usage examples are printed in blue boxes without line numbering:

This is a {L^AT_EX} example.

2.1 Keyword Markings

Certain Parts of this documentation are icon- and color-coded:

-  **Containers** are orange and marked with a box symbol ,
-  **Hooks** are green and marked with an insertion icon ,
-  **Components** are blue and marked with an arrow to box symbol ,
-  **Properties** are purple and marked with a gear symbol ,
-  **PDF-Tags** are cyan and marked with a tag symbol ,
-  **Attributes** are dark green and marked with a chain symbol , and
- L^AT_EX-Macros** are red and have no symbol.

¹It is however noteworthy that the principal functionality has been available for much longer in form of the `xtemplate` package.

2.2 Data Types of Properties

Whenever a Property is declared, the documentation contains a list of expected values for that property. The following list gives an overview over the various expected data types:

<code><dimen></code>	means that the Property is expected to return a dimensional value (or “length”) or a dimension register.
<code><skip></code>	means that the Property is expected to return a skip, i. e., a $\text{\LaTeX}$ dimension with or without glue, or a skip register.
<code><num></code>	means that the Property is expected to return a number or counter register.
<code><CS token></code>	means that one previously defined control sequence token (i. e., a $\text{\LaTeX}$ macro) is expected.
<code>[word1 word2]</code>	indicates that either exact word1 or word2 is expected. This notation may also contain other fixed data types, and more than one option could be given.
<code><name></code>	means that the name of a specific Component, Property or Container is expected. Details are usually in the description.
<code><any></code>	means that the Property can take any value.

2.3 Types of Components

LC	means that the Component is a Labeled Component
CC	means that the Component is a Counted Component
GC	means that the Component is a Group Component
CL	means that the Component is a Collection Component
OR	means that the Component is an Override

One more driver function

22 `<*driver>`

If we want to run the splitted development dtx locally, this macro prevents undefined control sequence errors and actually includes the dtx chunks.

23 `\def\includeDTX#1{\input src/#1.dtx}`

End driver function

24 `</driver>`

Module 1

cocotex.dtx

```
<*class>
```

This is the main class file for the CoCoT_EX Framework.

File Preamble

```
23 %%
24 %% Common document class for \textit{xerif} projects.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesClass{cocotex}
30 [2025/07/14 v0.6.2 cocotex]
```

1 Hard-coded requirements

First, we set the default hook label for all CoCoT_EX modules. Since some modules are “stand-alone”, we do this in all kernel files, i.e., here, in `coco-kernel` and in `coco-common`.

```
31 \SetDefaultHookLabel{cc}
```

2 Class Options

2.1 Options passed down to mandatory standard L^AT_EX packages

The `main` option controls the document’s main language passed down to the `babel` package.

```
32 \ExplSyntaxOn
33 \keys_define:nn { cocotex/cls }
34 {
35   main .code:n = { \PassOptionsToPackage{main=#1}{babel} },
```

The next two options are used for Spanish documents:

```
36   es-noindentfirst .code:n = { \PassOptionsToPackage{es-noindentfirst}{babel} },
37   es-noshorthands .code:n = { \PassOptionsToPackage{es-noshorthands}{babel} }
```

By default, we disable `babel` shorthands as its character encoding may interfere with some CoCoT_EX functionality.

```
38 }
39 \PassOptionsToPackage{shorthands=off}{babel}
```

The `no-hyperindex` switch prevents `hyperref` from auto-linking index terms.

```
40 \keys_define:nn { cocotex/cls }
41 {
42   no-hyperindex .code:n = {\global\let\cc@no@hyperindex\relax}
43 }
```

2.2 The Publication Type

The option `pubtype` (short for “publication type”) has four possible values: `mono`, `collection`, `journal`, and `article`. `mono` (also the default when no `pubtype` is given) and `collection` are used to switch between single and multiple contributor documents; `collection` and `journal` to switch between one-time text collections and periodicals, respectively. All three types implicitly load the L^AT_EX standard class `book`.

`collection` is used when the document’s components (i. e., chapters) are contributed by different authors like collections or proceedings. `journal` is used for collections where each contribution is accompanied by a myriad of meta data. `mono` stands for monographs, i.e., whole books that are written by the same author(s).

The publication type `article` is intended for single articles of a journal. It loads the L^AT_EX standard class `article`.

```
44 \newif\ifcollection
45 \newif\ifarticle
46 \newif\ifmonograph
47 \newif\ifjournal
48 \keys_define:nn { cocotex/cls }
49 {
50   pubtype .choices:nn = { collection, article, journal, mono }
51   {
52     \global\collectionfalse
53     \global\articlefalse
54     \global\monographfalse
55     \global\journalfalse
56     \str_case:Vn \l_keys_choice_tl
57     {
58       { collection } { \global\collectiontrue }
59       { article } { \global\articletrue }
60       { journal } { \global\journaltrue }
61       { mono } { \global\monographtrue }
62     }
63   },
64   pubtype .initial:n = mono,
65 }
```

2.3 User-Level Macro Names and Debugging Options

Next, we capture all options that needs passing down to the various CoCoT_EX modules.

The prefix option is used to define prefix for user level macros. If CoCoT_EX is used in conjunction with *xerif*, this will be `tp`.

```
66 \keys_define:nn { cocotex/cls }
67 {
68   prefix .code:n = { \PassOptionsToPackage{prefix=#1}{coco-kernel} },
```

The debugging options: trigger `debug` will toggle on debug mode, valued `debug-domain` will determine the kinds of messages printed to the console.

```

69 debug .code:n = { \PassOptionsToPackage{debug}{coco-kernel} },
70 debug-domain .code:n = { \PassOptionsToPackage{debug-domain=#1}{coco-kernel} },

```

The `silent` options attempts to suppress most unnecessary messages sent to the shell.

```

71 silent .code:n = {\PassOptionsToPackage{silent}{coco-common}},

```

The `nofigs` options disables the `\includegraphics` command and prints a placeholder instead. This is intended to ensure successful LaTeX runs even though image files may be missing.

```

72 nofigs .code:n = {\PassOptionsToPackage{nofigs}{coco-floats}},
73 }

```

2.4 Accessibility Features

The next two options enable accessibility features and control the PDF standard used: `a11y` generates PDF version 1.7, `a11y20` generates PDF version 2.0.

`\ifcc@pdf@two` is a switch that indicates PDF version 2.0 is used (true) instead of 1.x (false).

```

74 \newif\ifcc@pdf@two \cc@pdf@twofalse

```

```

75 \str_new:N \g_cc_pdf_standard_str

```

```

76 \keys_define:nn { cocotex/cls }
77 {

```

Options for the aimed PDF standard. The value should match the following regex pattern:

`(UA|A|X)(-[0-9])(+[A-Z](:[0-9]+)?)?`

The choices are case-insensitive and the selected standard also determines the PDF version later.

```

78 pdf-standard .code:n =
79 {
80   \exp_args:Nnx
81   \keys_set:nn {cocotex/cls} {_pdfstandard=\str_uppercase:n{#1}}
82 },
83 _pdfstandard .str_set:N = \g_cc_pdf_standard_str,
84 _pdfstandard .initial:n = A-2A,

```

WARNING!
The following section is deprecated and will be changed or deleted in future releases.

The following two options are there for backward compatibility and are considered legacy code. Both cause PDF/UA, `a11y` UA-1 and PDF v1.7, `a11y20` UA-2 with PDF v2.0.

```

85 a11y .code:n = { \keys_set:nn { cocotex/cls } { pdf-standard = UA-1 } },
86 a11y20 .code:n = { \keys_set:nn { cocotex/cls } { pdf-standard = UA-2 } },

```

WARNING!
The following section is deprecated and will be changed or deleted in future releases.

`lang-id` is the ISO-639/2 identifier of the document's main language. This is necessary for the PDF meta data and *different* from the main language name given via the `main` key.

```
87 lang-id .code:n = {},
```

If set, `nodetree` triggers extensive debugging output from the `ltpdfa` package.

```
88 nodetree .code:n = {\PassOptionsToPackage{nodetree}{coco-accessibility}},
```

`showspaces` enables whitespaces processed by `ltpdfa` to be visible in the PDF.

```
89 show-spaces .code:n = {\PassOptionsToPackage{show-spaces}{coco-accessibility}},
```

`no-spaces` disables whitespace processing by `ltpdfa`.

```
90 no-spaces .code:n = {\PassOptionsToPackage{no-spaces}{coco-accessibility}},
```

`no-para` disables paragraph tagging via `ltpdfa`.

```
91 no-para .code:n = {\PassOptionsToPackage{no-para}{coco-accessibility}},
```

`no-compress` disables PDF compression; useful for debugging the PDF source code.

```
92 no-compress .code:n =
93 {
94   \sys_ensure_backend:
95   \ifx\pdfobjcompresslevel\@undefined
96     \edef\pdfobjcompresslevel{\pdfvariable objcompresslevel}%
97   \fi
98   %\pdfcompresslevel=0
99   \pdfobjcompresslevel=0
100   \pdf_uncompress:
101 },
```

`color-enc` serves two purposes: First, it controls the colour space for colours invoked via the `xcolor` package. Second, it controls which default ICC colour profile is embedded into the PDF file when no explicit ICC profile is provided by the user.

```
102 color-enc .code:n = {\PassOptionsToPackage{color-enc=#1}{coco-common}},
```

2.5 Options for Other CoCoT_EX Modules

Options for the Script Module

The option `usescript` takes a comma-separated list of language names. Languages that use non-latin scripts may require fallback fonts when the script's glyphs are not included in the main font. This option tells the `coco-scripts` module which fonts to pre-load. The values in is also passed down to `\babelprovide`.

```
103 usescript .code:n = {\PassOptionsToPackage{usescript={#1}}{coco-script}},
```

Options for the Notes Module

The switch `endnotes` triggers all footnotes to be collected and printed in a specific endnote section at the end of the document.

```
104 endnotes .code:n = {\PassOptionsToPackage{endnotes}{coco-notes}},
```


The switch `ennotoc` triggers headings in the Endnotes area to *not* appear in the table of contents (read: *end-notes-no-toc*).

```
105 ennotoc .code:n = {\PassOptionsToPackage{ennotoc}{coco-notes}},
```

The switch `endnoteswithchapters` triggers chapter headings to be repeated as subsections within the endnote section.

```
106 endnoteswithchapters .code:n = {\PassOptionsToPackage{endnoteswithchapters}{coco-notes}},
```

The switch `resetnotesperchapter` causes foot- and endnote counters to be reset at the beginning of each new chapter.

```
107 resetnotesperchapter .code:n = {\PassOptionsToPackage{resetnotesperchapter}{coco-notes}},
```

Remaining Options

All other unprocessed options are passed down to the base document class:

```
108 }
109 \DeclareOption*{\PassOptionsToClass{\CurrentOption}{article}}
110 \DeclareOption*{\PassOptionsToClass{\CurrentOption}{book}}
```

Process the options.

```
111 \ProcessOptions
112 \ProcessKeyOptions[cocotex/cls]
113 \cs_if_exist:NT \ccClassOptions{\exp_args:Nx\keys_set:nn {cocotex/cls}{\ccClassOptions}}
```

3 PDF-Version and Standard Conformity

After processing the options, we determine which PDF standard and PDF version the document is supposed to conform to. If no standard is selected, we default to PDF version 1.7.

```
114 \sys_ensure_backend:
```

`\g_cc_pdfstd_seq` is a sequence that contains the PDF conformity from the value of the `pdf-standard` class option. It has four elements: `[standard, part, level, issue]`.

```
115 \seq_new:N \g_cc_pdfstd_seq
```

`\g_cc_pdfstd_macros_seq` is a list of CS names that the values from `\g_cc_pdfstd_seq` are mapped onto. If the `pdf-standard` Option lacks a field, the corresponding macro is un-defined.

```
116 \seq_const_from_clist:Nn \g_cc_pdfstd_macros_seq {cc@pdf@std, cc@pdf@part, cc@pdf@level,
cc@pdf@issue }
```

First, we extract the data fields from the option value:

```
117 \regex_extract_all:nVN { ^([UAX]+)\-([0-9]+)([A-Z]*)?:?(.*)? } \g_cc_pdf_standard_str \g_cc_
pdfstd_seq
```

The first element contains the entire string, so we pop it from the stack and throw it away.

```
118 \seq_pop_left:NN \g_cc_pdfstd_seq \l_tmpa_tl
```

`\cc_map_pdfstd:nn` is a helper function that takes two arguments:

`{#1}` is the value extracted from `\g_cc_pdf_standard_str`

`{#2}` is the CS name of the macro that the value should be mapped to if it is not empty

```

119 \cs_new:Npn \cc_map_pdfstd:nn #1#2 {%
120   \str_if_empty:nTF { #1 }
121     { \cs_gset:cpn{#2}{} }
122     { \cs_gset:cpe{#2}{ \str_if_eq:nnTF{#1}{UA}{U}{#1} } }
123 }
```

Now we can apply the macro to the list of extracted values:

```

124 \seq_map_pairwise_function:NNN \g_cc_pdfstd_seq \g_cc_pdfstd_macros_seq \cc_map_pdfstd:nn
```

Finally, we parse the original `\g_cc_pdf_standard_str` value a second time to determine the PDF version. We default to the highest version possible to each given conformity level. If one of the PDF/UA-Standards is used, we also initialize the coco-accessibility module.

```

125 \prg_generate_conditional_variant:Nnn \regex_match_case:nn {nV} {T,F,TF}
126 \regex_match_case:nVF
127 {
128   { ^UA\ -1 } { \pdf_version_gset:n { 1.7 } \PassOptionsToPackage{init}{coco-accessibility} }
129   { ^UA\ -2 } { \pdf_version_gset:n { 2.0 } \PassOptionsToPackage{init}{coco-accessibility} \
130     global\@cc@pdf@twotrue }
131   { ^A\ -1A } { \pdf_version_gset:n { 1.4 } \PassOptionsToPackage{init}{coco-accessibility} }
132   { ^A\ -1B } { \pdf_version_gset:n { 1.4 } }
133   { ^A\ -[23][AU] } { \pdf_version_gset:n { 1.7 } \PassOptionsToPackage{init}{coco-accessibility} }
134   { ^A\ -[23]B } { \pdf_version_gset:n { 1.7 } }
135   { ^A\ -4A } { \pdf_version_gset:n { 2.0 } \PassOptionsToPackage{init}{coco-accessibility} \
136     global\@cc@pdf@twotrue }
137   { ^A\ -4B } { \pdf_version_gset:n { 2.0 } \global\@cc@pdf@twotrue }
138   { ^X\ -[123] } { \pdf_version_gset:n { 1.4 } }
139   { ^X\ -[45] } { \pdf_version_gset:n { 1.6 } }
140   { ^X\ -6 } { \pdf_version_gset:n { 2.0 } \global\@cc@pdf@twotrue }
141 } \g_cc_pdf_standard_str
142 { \pdf_version_gset:n { 1.7 } }
143 \ExplSyntaxOff
```

4 Class Hook

`\ccAfterClassHook` Almost all user level macros have been renamed when CoCoTeX became independent from `xerif`. In order to ensure backwards-compatibility, we define a hook that holds aliases from the old names to the new ones. Those are defined in the `coco-xerif` module (which is *not* part of CoCoTeX itself, but included in `xerif`'s common files¹). The hook is expanded at the very end of the `cocotex.cls` file. The `coco-xerif` module itself is loaded early in `coco-common.sty`.

```

142 \def\ccAfterClassHook{}
```

`\ccToggleCountedConditionalsHook` is a hook to ensure backwards-compatibility within the processing of Counted Components

```

143 \def\ccToggleCountedConditionalsHook{}
```

¹see <https://github.com/transpect/xerif-latex>

5 Internal Requirement

```
144 \RequirePackage{coco-common}
```

6 Loading and Adjusting Underlying DocumentClass

All publication types supported by CoCoT_EX are based on one of L^AT_EX's default classes `article` (when `pubtype=article`) or `book` (all other pubtypes):

```
145 \ifarticle
146   \LoadClass[10pt,a4paper]{article}
147 \else
148   \LoadClass[10pt,a4paper]{book}
149 \fi
```

6.1 General Typography

Offsets are the removed to make all values relative to the upper left corner of the page to ease maintainance.

```
150 \voffset-1in\relax
151 \hoffset-1in\relax
```

Automatted typesetting needs some room to play

```
152 \emergencystretch=2em
```

and strong restrictions:

```
153 \frenchspacing
154 \clubpenalty10000
155 \widowpenalty10000
```

Empty Pagestyle

Page style without any headers or footers

```
156 \def\ps@empty{%
157   \let\@oddhead\@empty
158   \let\@evenhead\@empty
159   \let\@oddfoot\@empty
160   \let\@evenfoot\@empty
161 }
```

Vacancy Pages

Vacancy pages in general need to have page style `empty`:

```
162 \def\cleardoublepage{\clearpage\if@twoside \ifodd\c@page\else
163   \hbox{}\thispagestyle{empty}\newpage\if@twocolumn\hbox{}\newpage\fi\fi\fi}
```

Book Parts

The macros `\frontmatter`, `\mainmatter` and `\backmatter` are re-defined to make front- and backmatter components in book derivatives distinguishable.

Note that we need to (re-)define the conditionals outside, because `\if@mainmatter` is undefined for articles and the conditional inside the definition of `\mainmatter` disturbs the outer `\ifarticle`.

```

164 \newif\if@frontmatter \@frontmatterfalse
165 \newif\if@mainmatter \@mainmatterfalse
166 \newif\if@backmatter \@backmatterfalse
167 \ifarticle\else
168   \renewcommand\frontmatter{%
169     \cchResetNesting
170     \cleardoublepage
171     \global\@mainmatterfalse
172     \global\@backmatterfalse
173     \global\@frontmattertrue
174     \ccaVstructStart{Frontmatter}%
175     \pagenumbering{arabic}}
176   \renewcommand\mainmatter{%
177     \cchResetNesting
178     \if@frontmatter
179       \ccaVstructEnd{Frontmatter}%
180     \fi
181     \cleardoublepage
182     \global\@frontmatterfalse
183     \global\@backmatterfalse
184     \global\@mainmattertrue
185     \ccaVstructStart{Mainmatter}%
186   }
187   \renewcommand\backmatter{%
188     \cchResetNesting
189     \if@mainmatter
190       \ccaVstructEnd{Mainmatter}
191     \fi%
192     \cleardoublepage
193     \global\@mainmatterfalse
194     \global\@frontmatterfalse
195     \global\@backmattertrue
196     \ccaVstructStart{Backmatter}%
197   }
198 \fi% \ifarticle

```

7 Loading other CoCoT_EX Modules

7.1 coco-accessibility

We load the accessibility module always, even if we don't end up actually using it.

```

199 \RequirePackage{coco-accessibility}

```

7.2 coco-script

Inclusion of the script module which also loads the babel package

```

200 \ifLuaTeX
201 \RequirePackage{coco-script}
202 \else
203 \RequirePackage{babel}
204 \fi

```

7.3 coco-headings

```

205 \RequirePackage{coco-headings}

```

7.4 coco-floats

Inclusion of the float module

```

206 \RequirePackage{coco-floats}

```

7.5 coco-title

Inclusion of the title page module

```

207 \RequirePackage{coco-title}

```

7.6 coco-notes

Inclusion of the end-/footnotes module

```

208 \RequirePackage{coco-notes}

```

Fallback, in case, `coco-headings.sty` is not loaded for some reason.

8 Further Hard Dependencies

8.1 Index

Some more hard dependencies:

```

209 \RequirePackage{index}
210 \makeindex

```

8.2 Hyperref

The hyperref package allows some limited interactivens of PDF files insofar as that internal and external references become click-able.

For PDF/A and PDF/UA, we enable/allow linking by invoking the normal hyperref package...

```
211 \RequirePackage{hyperref}
```

...with some preset options:

```
212 \hypersetup{%
```

first, we want links to be breakable

```
213 breaklinks%
```

and the table of contents not to be automatically linked, as this causes problems with the `ltpdfa` package and we add the links via the `coco-common` module, anyways.

```
214 ,linktoc=none%
```

pdf borders are controlled via the `coco-frame` module, if desired

```
215 ,pdfborder={0 0 0}%
```

The next option causes `hyperref` to calculate the encoding of `DocumentInfo` and other direct-to-PDF data (bookmarks, etc.) automatically

```
216 ,pdfencoding=unicode
217 ,unicode=true
```

Bookmarks are numbered and open by default.

```
218 ,bookmarksnumbered=true%
219 ,bookmarksopen=false%
```

index is linked by default unless the `no-hyperindex` class option is set.

```
220 ,hyperindex=\ifx\cc@no@hyperindex\relax false\else true\fi
221 }
222 \if\cc@pdf@std X
```

PDF/X standards, however, must not be interactive, so we disable all `hyperref` markup by setting `hyperrefs draft` option, which disables all `hyperref` markup without rendering the respective user-level macros undefined. This way, we can generate linked and unlinked PDFs from the same sources without the need to remove macros from the tex files.

```
223 \hypersetup{draft}
```

We also check if the `frame` module is already loaded. If not, we load it with the `pdfboxes` option to draw the pdf borders, which are required in the PDF/X standard.

```
224 \AddToHook{begindocument}{%
225 \ifpackageloaded{coco-frame.sty}{}%
226 \RequirePackage[pdfboxes]{coco-frame}%
227 }%
228 }%
229 \fi
```

9 End of Document Class Hook

Expanding backwards-compatibility aliases from the `coco-xerif` module:

230 \ccAfterClassHook

</class>

Module 2

coco-kernel.dtx

```
<*kernel>
```

This file provides the object-oriented interfaces for all other CoCoTeX modules.

1 Preamble

```
23 %%
24 %% CoCoTeX Kernel
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-kernel}
30 [2025/07/14 v0.6.2 cocotex kernel]
```

Before we do anything, we check if the user uses a (more or less) current L^AT_EX kernel version. If not, we issue a hard error message. The pivot that separates “new” from “old” is June 1, 2020.

```
31 \ifx\IfFormatAtLeastTF\@undefined
32 \providecommand\IfFormatAtLeastTF{\@ifl@t@r\fmtversion}%
33 \fi
34 \IfFormatAtLeastTF{2020/06/01}{}%
35 {\PackageError{CoCoTeX Kernel}
36 {LaTeX kernel too old!}
37 {CoCoTeX v0.5.0 and newer needs at least LaTeX kernel version 2023/11/01!}}
```

1.1 Hard dependencies

```
38 \RequirePackage{etoolbox}
```

Default hook label for CoCoTeX modules:

```
39 \SetDefaultHookLabel{cc}
```

1.2 Package Options

The `debug` option triggers the output of additional information messages to the shell.

```
40 \newif\if@cc@debug \@cc@debugfalse
41 \ExplSyntaxOn
42 \keys_define:nn { cocotex/kernel }
```

```

43 {
44   debug .code:n = {\global\@cc@debugtrue}
45 }

```

The `debug-domain` option serves as a filter for log messages. It takes a comma separated list of log message categories as argument. Only messages whose domain match any item of that list are printed. If the option is omitted or its argument empty, all messages are printed.

Implies `debug`.

```

46 \global\let\debug@domain@list\relax
47 \keys_define:nn { cocotex/kernel }
48 {
49   debug-domain .code:n = {%
50     \global\@cc@debugtrue
51     \if!#1!\else
52       \def\do##1{\listadd\debug@domain@list{##1}}%
53       \docsvlist{#1}%
54     \fi
55   }%
56 }

```

The `prefix` option will be explained below in Sect. 3.

```

57 \let\cc@prefix\@empty
58 \keys_define:nn { cocotex/kernel }
59 {
60   prefix .code:n = {\gdef\cc@prefix{#1}},%
61   prefix .initial:n = {}
62 }
63 \ProcessKeyOptions[cocotex/kernel]
64 \ExplSyntaxOff

```

2 Exception handlers

The CoCoTeX kernel provides some macros to unify exception handling. There are four levels of output: `error`, `warning`, `info`, and `debug`.

`\ccPackageError` creates an error message specific to the Framework.

`{#1}` is the module
`{#2}` is the type of error
`{#3}` is the immediate error message
`{#4}` is the help string

```

65 \def\ccPackageError#1#2#3#4{%
66   \GenericError{%
67     (#1)\@spaces\@spaces\@spaces\@spaces
68   }{%
69     [CoCoTeX #1 #2 Error] #3%
70   }-#{#4}%
71 }

```

`\cc@patch@error` issues an error when a CoCoTeX module `{#1}` could not patch macro `{#2}`.

```

72 \def\cc@patch@error#1#2{%

```

```

73 \ccPackageError{#1}{compatibility}
74 {Could not patch \noexpand#2}
75 {You probably use a LaTeX package that re-defines the \noexpand#2 control sequence. It is
    apparently not compatbile with CoCoTeX. Sorry}}

```

`\ccPackageWarning` is a macro to create warnings specific to the Framework.

`{#1}` is the module
`{#2}` is the type of error
`{#3}` is the immediate warning message

```

76 \def\ccPackageWarning#1#2#3{%
77   \GenericWarning{%
78     (#1)\@spaces\@spaces\@spaces\@spaces
79   }{%
80     [CoCoTeX #1 \if!#2!\else#2 \fi Warning] #3%
81   }%
82 }

```

`\ccPackageInfo` is a macro to create shell output specific to the Framework.

`{#1}` is the module
`{#2}` is the type of message
`{#3}` is the immediate info string

```

83 \def\ccPackageInfo#1#2#3{%
84   \GenericInfo{%
85     (#1)\@spaces\@spaces\@spaces\@spaces
86   }{%
87     [CoCoTeX #1\if!#2!\else\space#2\fi] #3%
88   }%
89 }

```

While the macros defined above are meant to be used in all CoCoTeX modules, the following is only for the Kernel.

`\cc@debug@message` is a generic debug message that is displayed whenever the debug option is set. Otherwise, it gobbles its argument.

```

90 \ifcc@debug
91   \def\do#1{\csgdef{cc@debug@#1@message}##1{\typeout{[CoCo #1 Debug]\space##1}}}%
92   \dolistloop{\debug@domain@list}%
93   \def\cc@debug@message#1{\typeout{[CoCo Debug]\space#1}}
94 \else
95   \let\cc@debug@message\@gobble
96 \fi

```

`\ccDebugMsg` prints a domain specific debug message.

`[#1]` is the debug domain for filtering
`{#2}` is the message

The whole mechanism works by *dynamicly* defining the underlying, domain specific, debug message macros, which all follow the scheme `cc@debug@<domain>@message`. If the debug domain is requested via the debug-domains class option, the macros whose domain is listed in this key are defined on runtime. Otherways, the mandatory argument gets gobbled.

The following debug domain filters are currently used:

a11y Messages related to PDF tagging and accessibility features
 eval final values of Type expansions

inheritance Inheritance mechanism of Types

```

97 \def\ccDebugMsg{\cc@opt@empty\cc@debug@msg}
98 \def\cc@debug@msg[#1]#2{%
99   \expandafter\ifx\csname cc@debug@#1@message\endcsname\relax
100     \gobble{#2}%
101   \else
102     \csname cc@debug@#1@message\endcsname{#2}%
103   \fi
104 }
```

3 Global Switches

`\ccPrefix` is the prefix that is added to Component macros and (some) Container environments.

This has mostly historic reasons: back when CoCoTeX was specific to the *xerif* typesetting automaton, all macros produced by the xml converter had a `tp` prefix (from *transp*ect, the XML conversion tool in the backend of *xerif*). After CoCoTeX became stand-alone, the `tp` prefix became obsolete, but the converters running at the time needed to be backward-compatible. Therefore, all *xerif*-bound CoCoTeX instances still set this macro to ensure user-level macros bear the `tp`-prefix.

```

105 \ifx\ccPrefix\@undefined\edef\ccPrefix{\cc@prefix}\fi
106 \ccPackageInfo{Kernel}{Info}{The macro prefix is now '\ccPrefix'.}
```

`\ifcc@is@final` is a boolean switch that indicates whether or not a process is final. This is mainly used in the accessibility module where it matters if a macro is actually used to print stuff, or if it is just processed.

```

107 \newif\ifcc@is@final \cc@is@finalfalse
108 \AtBeginDocument{\cc@is@finaltrue}
```

`\ccWhenAlly` is a stub that eats its argument. It only does stuff when the `coco-accessibility` package is loaded, which we cannot know, yet.

```

109 \let\ccWhenAlly\@gobble
```

`\ccUnlessAlly` is a stub that does nothing.

```

110 \let\ccUnlessAlly\@iden
```

`\ccIfAlly` is the same as `\ccWhenAlly`, but it takes two arguments, one for the *true* case and a second for the *false* case. We default to the *else* case, so we always gobble the first argument. This will be altered if the `coco-accessibility` package is loaded later.

```

111 \let\ccIfAlly\@gobble
```

4 Containers

Containers are the package's core data structure. They are basically sets of properties that are processed in the same way.

`\ccSetContainer` is used to change the currently active (Sub-)Container.

`{#1}` is the name of the new active Container

```
130 \def\ccSetContainer#1{\def\cc@cur@cont{#1}\ignorespaces}
```

`\ccAddToType` add additional content (i.e., the next token) to a Type `{#1}` of a previously declared Container `{#2}` globally.

```
131 \def\ccAddToType#1#2{\csgappto{cc@type@#1@#2}}
```

`\ccAddToTypeLocal` add additional content (i.e., the next token) to a Type `{#1}` of a previously declared Container `{#2}` locally.

```
132 \def\ccAddToTypeLocal#1#2{\csappto{cc@type@#1@#2}}
```

`\ccEvalType` calls the declaration list for Data Type `{#2}`. With optional `[#1]`, the Type's Container name can be overridden locally.

```
133 \def\ccEvalType{\cc@opt@curcont\cc@eval@type}
134 \def\cc@eval@type[#1]#2{%
135   \expandafter\ifx\csname cc@type@#2@#1\endcsname\relax
136   \ccPackageError{Kernel}{Class}
137     {Data Type #2 in Container #1 undefined!}
138     {You try to evaluate a data type `#2' from container `#1', but that data type has not been
      declared.}%
139   \else
140     \ccDebugMsg[eval]{Evaluating cc@type@#2@#1:^^J \csmeaning{cc@type@#2@#1}}%
141     \csname cc@type@#2@#1\endcsname
142   \fi\ignorespaces}
```

`\ccCheckParent` checks if a Container `{#1}` is declared so that another Container `{#2}` can inherit.

```
143 \def\ccCheckParent#1#2{%
144   \expandafter\ifx\csname cc@container@#1\endcsname\relax
145   \ccPackageError{Kernel}{Class}
146     {Parent Container `#1' undeclared}
147     {You tried to make a Container named `#2' inherit from a Container named `#1', but a
      Container with that name does not exist.\MessageBreak
148     Please make sure that parent Containers are declared before their descendents.}%
149   \else
150     \csgdef{cc@parent@#2}{#1}%
151   \fi
152   \ignorespaces}
```

`\cc@inherit` is the low-level inherit function.

`{#1}` is a comma-separated list of things to be inherited

`{#2}` is the Container-list that should be inherited from

`{#3}` is the name of the inheriting Container

```
153 \def\cc@inherit#1#2#3{\cc@parse@inherit #1,,\@nil #2,,\@nil #3\@@nil}
```

`\cc@parse@inherit` is a low-level function to recursively parse the parameters of the `\cc@inherit` macro, above.

```
154 \def\cc@parse@inherit #1,#2,\@nil #3,#4,\@nil #5\@@nil{%
155   \let\next\relax
156   \if!#1!\else
```

```

157 \if!#3!\else
158 \cc@do@inherit{#1}{#3}{#5}%
159 \def\@argii{#2}\def\@argiv{#4}%
160 \ifx\@argii\@empty
161 \ifx\@argiv\@empty\else
162 \def\next{\cc@parse@inherit #1,,\@nil #4,\@nil #5\@nil}%
163 \fi
164 \else
165 \ifx\@argiv\@empty
166 \def\next{\cc@parse@inherit #2,\@nil #3,,\@nil #5\@nil}%
167 \else
168 \def\next{%
169 \cc@parse@inherit #1,,\@nil #4,\@nil #5\@nil
170 \cc@parse@inherit #2,\@nil #3,#4,\@nil #5\@nil
171 }%
172 \fi\fi\fi\fi
173 \next}

```

`\cc@do@inherit` is the macro that causes the parent's (unexpanded) Type list to be appended to the child's type list.

`{#1}` is the name of a Type
`{#2}` is the name of the Container that Type `{#1}` is *inherited* from
`{#3}` is the name of the Container that *inherits* Type `{#1}`

```

174 \def\cc@do@inherit#1#2#3{%
175 \ccDebugMsg[inheritance]{#3 inherits #1 from #2.}%
176 \ccCheckParent{#2}{#3}%
177 \expandafter\ifx\csname cc@type@#1@#2\endcsname\relax
178 \ccPackageError{Kernel}{Type}{Type `#1' was not declared}{Type `#1' was not declared for
179 Container `#2'.}%
180 \else
181 \edef\x{\noexpand\csgappto{cc@type@#1@#3}}%
182 \expandafter\x\expandafter{\csname cc@type@#1@#2\endcsname}%
183 \ccDebugMsg[inheritance]{value cc@type@#1@#3:^^J \csmeaning{cc@type@#1@#3}}%
184 \fi
185 \ignorespaces}

```

5 Components

5.1 Simple Components

“Simple Components” are basically data storages. They are used within Containers to obtain data and store them for further processing at the end of the Container, or even beyond.

`\ccDeclareComponent` defines a simple Component macro. The internal macro that is used to store the Component's value is `\csname cc@<current Container name>@<#1>\endcsname`.

`{#1}` is the Component's identifier. If omitted, `{#1}` is the same as `{#2}`.
`{#2}` is the Component's name
`{#3}` is code that is executed *before* assignment of the user's value
`{#4}` is code that is executed *after* assignment of the user's value

```

185 \def\ccDeclareComponent{\cc@opt@second\cc@declare@comp}
186 \def\cc@declare@comp[#1]#2#3#4{%
187 \ltx@LocalExpandAfter\global\expandafter\let\csname cc@\cc@cur@cont @#1\endcsname\relax

```

```

188 \expandafter\long\expandafter\def\csname \ccPrefix#2\endcsname##1{%
189   #3\expandafter\long\expandafter\def\csname cc@cc@cur@cont @#1\endcsname{##1}#4\ignorespaces
190   }%
191 \ignorespaces}

```

`\ccDeclareGlobalComponent` is a shortcut to declare simple, globally available Components with the name `{#2}` and an optional initial value `[#1]`. They are usually empty.

```

191 \def\ccDeclareGlobalComponent{\cc@opt@empty\cc@declare@global@comp}%
192 \def\cc@declare@global@comp[#1]#2{%
193   \ccDeclareComponent{#2}{\expandafter\global}{}}%
194   \if!#1!\else\csname \ccPrefix #2\endcsname{#1}\fi%
195   \ignorespaces}

```

Once declared, a component can be set in two ways: The first way is to use `\ccPrefix<name>` with one argument for its value. The second, preferred, way is to use the `\ccComponent` macro.

`\ccCompWarning` issues a warning if a value is assigned to a non-existing Component. `{#1}` is the name of the unknown Component.

```

196 \def\ccCompWarning#1{\ccPackageWarning{Kernel}{}{Assigning value to previously undeclared~^J%
197   \cc@warning@spaces Component `#1'. Declaring now.}}

```

`\ccComponent` is the preferred way to fill a Component with content.

`{#1}` is the Component's name

`{#2}` is the Instance value.

```

198 \long\protected\def\ccComponent#1#2{%
199   \ifx\cc@is@counted\relax
200     \ifcsdef{cc@\cc@cur@cont @#1}{%
201       {\ccCompWarning{#1}\cc@def@counted@comp{\cc@counted@comp@scheme{#1}}{#1}{}}}%
202     \csgdef{cc@\cc@cur@cont @\cc@counted@comp@scheme{#1}}{#2}%
203   \else
204     \ifcsdef{cc@\cc@cur@cont @#1}{%
205       {\ccCompWarning{#1}\ccDeclareComponent{#1}{}}}%
206     \csdef{cc@\cc@cur@cont @#1}{#2}%
207   \fi\ignorespaces}

```

`\ccGlobalComponent` is a global variant of `\ccComponent`.

`{#1}` is the Component's name

`{#2}` is the Instance value.

```

208 \long\protected\global\def\ccGlobalComponent#1#2{%
209   \ifx\cc@is@counted\relax
210     \ifcsdef{cc@\cc@cur@cont @#1}{%
211       {\ccCompWarning{#1}\cc@def@counted@comp{\cc@counted@comp@scheme{#1}}{#1}{}}}%
212     \csgdef{cc@\cc@cur@cont @\cc@counted@comp@scheme{#1}}{#2}%
213   \else
214     \ifcsdef{cc@\cc@cur@cont @#1}{%
215       {\ccCompWarning{#1}\ccDeclareGlobalComponent{#1}{}}}%
216     \csgdef{cc@\cc@cur@cont @#1}{#2}%
217   \fi\ignorespaces}

```

`\ccComponentEA` is a variant of `\ccComponent` but it expands the Content in `{#2}` once before it is assigned to the Component `{#1}`.


```

218 \long\protected\def\ccComponentEA#1#2{%
219   \def\x{\ccComponent{#1}}\expandafter\x\expandafter{#2}%
220   \ignorespaces}

```

`\ccUseComp` is a high level command to return (or print) the material stored as a Component with the name `{#1}`.

```

221 \def\ccUseComp#1{\csname cc@cc@cur@cont @#1\endcsname}

```

`\ccdefFromComp` is a user-level command to store the value of a Component `{#2}` into a CS token `{#1}`.

```

222 \def\ccdefFromComp#1#2{\cc@store@comp{e}#1{#2}}

```

`\ccgdefFromComp` is the global variant of `\ccdefFromComp`.

```

223 \def\ccgdefFromComp#1#2{\cc@store@comp{x}#1{#2}}

```

`\ccpgdefFromComp` is a global variant of `\ccdefFromComp` that takes a CS name as `{#1}` and prepends the `\ccPrefix` before assigning the Contents of Component `{#2}` to the resulting CS token.

```

224 \def\ccpgdefFromComp#1#2{\def\x{\cc@store@comp{x}}\expandafter\x\csname\ccPrefix #1\endcsname
    {#2}}

```

`\ccdefTlFromComp` defines control sequence `{#1}` such that it expands to the exact token list stored in Component `{#2}`.

```

225 \def\ccdefTlFromComp#1#2{\edef#1{\expandafter\unexpanded\expandafter\expandafter\expandafter{
    \csname cc@cc@cur@cont @#2\endcsname}}}

```

`\ccgdefTlFromComp` global variant of `\ccdefTlFromComp`.

```

226 \def\ccgdefTlFromComp#1#2{\xdef#1{\expandafter\unexpanded\expandafter\expandafter\expandafter{
    \csname cc@cc@cur@cont @#2\endcsname}}}

```

`\strip@longprefix` is a helper macro to strip the prefix from the `\meaning` of a `\long` macro.

```

227 \def\strip@longprefix#1\long macro:->#2{#2}

```

`\cc@store@comp` is a generalized macro to store a component's unexpanded internal definition in a TeX macro.

`{#1}` is a scope quantifier (either 'e' or 'x')

`{#2}` is a CS token

`{#3}` is the name of a component

```

228 \long\def\cc@store@comp#1#2#3{%
229   \edef\@tempa{\expandonce{\csname protected@#1def\endcsname}\noexpand#2}%
230   \protected@edef\@tempb{\csname cc@cc@cur@cont @#3\endcsname}%
231   \ifx\@tempb\relax
232     \let#2\relax
233   \else
234     \expandafter\@tempa\expandafter{\@tempb}%
235   \fi
236   \ignorespaces}

```

`\ccUseComponentFrom` is a high level command to return (or print) the material stored as a global Component from the Container `{#1}` with the name `{#2}`.

```
237 \def\ccUseComponentFrom#1#2{\csname cc@#1@#2\endcsname}
```

`\ccGetComp` is a user-level command to return the contents stored in a Component of name `{#1}` as a paragraph iff the Component is neither empty nor `\relax`. If Accessibility features are activated, the returned content of the Component is automatically tagged with a `<P/>` tag.

```
238 \def\ccGetComp{\@ifstar\cc@sget@comp\cc@get@comp}
239 \def\cc@get@comp#1{%
240   \ccWhenComp{#1}
241   {\ccWhenAlly{\ccaStructStart{P}}}%
242   \ccUseComp{#1}%
243   \ccWhenAlly{\ccaStructEnd{P}}}%
244   \par}}
```

`\ccGetComp*` The starred version of `\ccGetComp` suppresses automated tagging for that Component (only if accessibility features are active).

```
245 \def\cc@sget@comp#1{\ccWhenComp{#1}{\ccUseComp{#1}\par}}
```

`\ccIfComp` is a high level macro that executes `{#2}` if the Component macro `{#1}` is used in a Container (empty or non-empty), and `{#3}` if not.

```
246 \long\def\ccIfComp#1#2#3{\expandafter\ifx\csname cc@cc@cur@cont @#1\endcsname\relax#3\else#2\fi}
```

`\ccWhenComp` is a high level variant of `\ccIfComp` that omits the `else`-branch.

`{#1}` is the name of the Component

`{#2}` is code that is expanded when the Component `{#1}` is used in a container (empty or non-empty)

```
247 \long\def\ccWhenComp#1#2{\expandafter\ifx\csname cc@cc@cur@cont @#1\endcsname\relax\else#2\fi}
```

`\ccUnlessComp` is a high level variant of `\ccIfComp` that omits the `then`-branch.

`{#1}` is the name of the Component

`{#2}` is the code that is expanded when a Container `{#1}` is *not* used in a Container (neither empty nor non-empty)

```
248 \long\def\ccUnlessComp#1#2{\expandafter\ifx\csname cc@cc@cur@cont @#1\endcsname\relax#2\fi}
```

`\ccIfCompFrom` is the global variant of `\ccIfComp`.

`{#1}` is the name of the Container

`{#2}` is the name of the Component

`{#3}` is the then branch

`{#4}` is the else branch

```
249 \long\def\ccIfCompFrom#1#2#3#4{\expandafter\ifx\csname cc@#1@#2\endcsname\relax#4\else#3\fi}
```

`\ccIfCompFromVal` is a conditional to check if a Global Component holds a specific value.

`{#1}` is the name of the Container

`{#2}` is the name of the Component

`{#3}` is the comparison value

`{#4}` is the then branch

`{#5}` is the else branch

```
250 \long\def\ccIfCompFromVal#1#2#3#4#5{\protected@edef\@argiii{#3}\expandafter\ifx\csname cc@#1@#2\endcsname\@argiii#4\else#5\fi}
```

`\cc@long@empty` is a helper macro used as comparator when checking whether a `\long` macro is empty or not.

```
251 \long\def\cc@long@empty{}
```

`\ccIfCompEmpty` is a high level macro that executes `{#2}` if the Component macro `{#1}` is empty (or `{}`) within its Container, and `{#3}` if it is either not existant or non-empty.

```
252 \long\def\ccIfCompEmpty#1#2#3{\expandafter\ifx\csname cc@cc@cur@cont @#1\endcsname\cc@long@empty#2\else\expandafter\ifx\csname cc@cc@cur@cont @#1\endcsname\@empty#2\else#3\fi\fi}
```

`\ccIfCompFromEmpty` is a global variant of `\ccIfCompEmpty`.

`{#1}` is the name of the Container
`{#2}` is the name of the Component
`{#3}` is the then-branch
`{#4}` is the else-branch

```
253 \long\def\ccIfCompFromEmpty#1#2#3#4{\expandafter\ifx\csname cc@#1@#2\endcsname\cc@long@empty#3\else\expandafter\ifx\csname cc@#1@#2\endcsname\@empty#3\else#4\fi\fi}
```

`\cc@check@empty` handles the distinction between empty and un-used components: First, check if `#4#3` is set (i. e., anything but `\relax`). If it is set, check if it is empty. If empty, set `#4#3` to `\relax`, meaning further occurrences of `\ccIfComp{#4#3}` will execute the `else` branch. If `#4#3` is non-empty, do nothing.

If `#4#3` is already `\relax`, check if the fallback `#1#3` is set. If so, make `#4#3` an alias of `#1#3`. If not, do nothing.

`[#1]` is the prefix of the fallback component
`{#2}` is the Container name
`{#3}` is the name of the Component
`{#4}` is the Override's prefix

```
254 \def\cc@check@empty{\cc@opt@empty\@cc@check@empty}%
255 \def\@cc@check@empty[#1]#2#3#4{%
256   \ccIfComp{#4#3}
257   {\ccIfCompEmpty{#4#3}
258     {\expandafter\global\expandafter\let\csname cc@#2@#4#3\endcsname\relax}
259     {}}
260   {\ccIfComp{#1#3}
261     {\expandafter\expandafter\expandafter\let\expandafter\csname cc@#2@#4#3\expandafter\endcsname\csname cc@#2@#1#3\endcsname}
262     {}}}
```

5.2 Counted Components

Counted Components are Components that may occur in the same parent Container multiple times. They may be multiple instances of single-macro Components, or recurring collections of multiple Components, called **Component Groups**.

Component Groups

`\ccDeclareComponentGroup` is a user-level macro to declare a new Component Group with the name `{#2}` and the body `{#3}`. Optional `[#1]` holds Attribute handlers specific to that Component Group's Instances.

```

263 \def\ccDeclareComponentGroup{\cc@opt@empty\cc@declare@component@group}%
264 \def\cc@declare@component@group[#1]#2#3{%
265   \csnumgdef{cc#2Cnt}{\z@}%
266   \def\@argi{#1}\ifx\@argi@empty
267     \csgdef{cc@type@Attributes@\cc@cur@cont @cc#2}{}%
268   \else
269     \csgappto{cc@type@Attributes@\cc@cur@cont @cc#2}{#1}%
270   \fi
271   \long\csdef{\ccPrefix#2}{\cc@opt@empty{\csname cc@group@#2\endcsname}}%
272   \long\csdef{cc@group@#2}[##1]{%
273     \def\cc@cnt@grp{cc#2}%
274     \csxdef{cc#2Cnt}{\expandafter\the\expandafter\numexpr\csname cc#2Cnt\endcsname+\@ne\relax}%
275     \if##1!\else
276       \ccEvalAttributes[\cc@cur@cont @cc#2]{##1}%
277       \csgdef{cc@\cc@cur@cont @#2-\csname cc#2Cnt\endcsname @attrs}{##1}%
278     \fi
279     #3%
280     \csname cc@group@#2@hook\endcsname
281     \ignorespaces
282   }%
283   \csdef{end\ccPrefix#2}{\ccToggleCountedConditionals\cuse{cc@compose@group@#2}}\aftergroup\
    ignorespaces}%
284 }

```

`\ccAddToComponentGroup` can be used to locally add code `{#2}` to a previously defined Component Group `{#1}`.

```

285 \def\ccAddToComponentGroup#1#2{\csappto{cc@group@#1@hook}{#2}}

```

`\ccDeclareGroupHandler` is used to declare a new group handler. A Group Handler is a hook for code `{#2}` that is expanded at the end of a Component Group `{#1}`'s environment. It is mostly used to process Components within a Group instance and store the result in their own components. For instance, a Group Handler can be used to combine a First Name and a Surname to a combined Component "FullName".

```

286 \def\ccDeclareGroupHandler#1#2{%
287   \ifcsdef{cc@group@#1}
288     {\ifcsdef{cc@compose@group@#1}
289       {\csgappto{cc@compose@group@#1}{#2}}
290       {\csgdef{cc@compose@group@#1}{#2}}}
291     {\ccPackageError{Kernel}{Type}{Component Group `#1' unknown!}{You tried to declare a Group
      Handler for a Component Group that has not been declared, yet! Use \string\
      ccDeclareComponentGroup{#1}{ } to declare the Component Group first.}}%
292 \ignorespaces}

```

`\cc@cnt@grp` is a designated group name. Counted Components of the same group use the same counter.

```

293 \let\cc@cnt@grp@empty

```

`\ccUseCompByIndex` picks a Component with name `{#3}` and index `{#2}` from a group `{#1}`.

```

294 \def\ccUseCompByIndex#1#2#3{\csname cc@\cc@cur@cont @#1-#3-#2\endcsname}

```

`\ccUsePropFrom` picks a Counted Component with the index `{#2}` from a Group `{#1}` and renders it using Property `{#3}`.

```

295 \def\ccUsePropFrom#1#2#3{%
296   \beginngroup
297     \@tempcnta\numexpr#2\relax

```

```

298 \letcs\ccTotalCount{cc#1Cnt}%
299 \def\cc@cnt@grp{cc#1}%
300 \ccToggleCountedConditionals
301 \csnumdef{cc#1Cnt}{\the\@tempcnta}%
302 \ccCurCount=\the\@tempcnta\relax%
303 \csname cc@\cc@cur@cont @#3\endcsname%
304 \endgroup}

```

Iterating over Component Groups

The following two macros iterate over all instances of a Component Group `{#1}` in the current Container and applies for each instance the Property `{#2}`. The result is appended to the the Collector Component `{#3}`, if and only if that Component is not yet set for the current Container at the time of the first iteration.

While the first macro only writes the Property *definition* into the Collector Component, the second fully expands the macros inside the Property and stores the result in Component `{#3}`.

Use the former to print and the latter to further process the respective results.

`\ccCurCount` stores the number of the current instance of a Counted Component. Use this in the declarations of Properties that are expanded within the Component Group.

```

305 \newcount\ccCurCount

```

`\cc@assign@res` assigns the result of the Component collection to a control sequence with the name `{#1}` and resets the temporary storage.

```

306 \def\cc@assign@res#1{%
307   \ifx\cc@iterate@res\undefined
308     \cslet{#1}\relax
309   \else
310     \ifx\cc@iterate@res\relax
311       \cslet{#1}\relax
312     \else
313       \expandafter\csname #1\expandafter\endcsname\expandafter{\cc@iterate@res}%
314     \fi
315   \fi
316   \global\let\cc@iterate@res\relax
317 }

```

`\ccIfComponentOverride` is a switch to apply either `{#2}`, if the Collection Component `{#1}` has been set manually within a container; or `{#3}`, if it has been generated from Counted Components.

```

318 \def\ccIfComponentOverride#1#2#3{\expandafter\ifx\csname cc@used@#1@override\endcsname\@empty#2\
    else#3\fi}

```

`\ccComposeCollection` is used to create an unexpanded Collection Component `{#3}` from all instances of Component Group `{#1}` using the instructions given by property `{#2}`.

```

319 \def\ccComposeCollection#1#2#3{%
320   \ccIfComp{#3}
321   {\cslet{cc@used@#3@override}\@empty}
322   {\cc@compose@collection{#1}{#2}%
323     \cc@assign@res{\ccPrefix#3}}

```

`\ccApplyCollection` is an alternative version of `\ccComposeCollection` and fully expands the result of the application of Property `{#2}` before it is stored inside the Component `{#3}`.

```

324 \def\ccApplyCollection#1#2#3{%
325   \ccIfComp{#3}
326   {\cslet{cc@used@#3@override}\empty}
327   {\cc@apply@collection{#1}{#2}%
328     \cc@assign@res{\ccPrefix#3}}

```

`\cc@compose@collection` is a low-level macro used to compose a resource object from the unexpanded values of a Component Group {#1} using Property {#2}. The result is stored in `\cc@iterate@res` and can be retrieved with `\cc@assign@res`.

```

329 \def\cc@compose@collection#1#2{%
330   \ccaProtect
331   \expandafter\ifnum\csname cc#1Cnt\endcsname > \z@\relax
332     \edef\cc@iterate@res{%
333       \noexpand\bgroup
334       \noexpand\def\noexpand\ccTotalCount{\csname cc#1Cnt\endcsname}%
335       \noexpand\ccToggleCountedConditionals
336       \noexpand\def\noexpand\cc@cur@cont{\cc@cur@cont}%
337       \noexpand\def\noexpand\cc@cnt@grp{cc#1}%
338       \expandafter\@tempcntb=\csname cc#1Cnt\endcsname\relax
339       \cc@iterate{\@tempcnta}{\@one}{\@tempcntb}{%
340         \edef\@tempb{%
341           \noexpand\begingroup
342           %% top-level counter for user interaction
343           \noexpand\ccCurCount=\the\@tempcnta\relax
344           %% evaluating the current group's Attributes
345           \ifcsdef{cc@cc@cur@cont @#1-\the\@tempcnta @attrs}
346             {\noexpand\ccEvalAttributes[\cc@cur@cont @cc#1]
347               {\csname cc@cc@cur@cont @#1-\the\@tempcnta @attrs\endcsname}}{}}%
348           %% internal counter for macro grabbing
349           \noexpand\csnumdef{cc#1Cnt}{\ccCurCount}%
350           \noexpand\ccUseProperty{#2}%
351           \noexpand\endgroup
352         }%
353         \expandafter\expandafter\expandafter\def
354         \expandafter\expandafter\expandafter\cc@iterate@res
355         \expandafter\expandafter\expandafter{\expandafter\cc@iterate@res\@tempb}%
356       }%
357       \expandafter\def\expandafter\cc@iterate@res\expandafter{\cc@iterate@res\egroup}%
358     \fi
359     \ccaEnable}

```

`\cc@apply@collection` is the low-level macro used to fully expand a Component Group {#1} using Property {#2}. The result is stored in `\cc@iterate@res` and can be retrieved with `\cc@assign@res`.

```

360 \def\cc@apply@collection#1#2{%
361   \ccaProtect
362   \begingroup
363     \global\let\cc@iterate@res\relax
364     \letcs\ccTotalCount{cc#1Cnt}%
365     \cc@iterate{\@tempcnta}{\@one}{\ccTotalCount}{%
366       \bgroup
367       \ccToggleCountedConditionals
368       \def\cc@cnt@grp{cc#1}%
369       \csnumdef{cc#1Cnt}{\the\@tempcnta}%
370       \ifcsdef{cc@cc@cur@cont @#1-\the\@tempcnta @attrs}
371         {\ccEvalAttributes[\cc@cur@cont @cc#1]
372           {\csname cc@cc@cur@cont @#1-\the\@tempcnta @attrs\endcsname}}{}}%
373       \ccCurCount=\the\@tempcnta

```

```

374 \protected@xdef\@tempb{\csname cc@\cc@cur@cont @#2\endcsname}%
375 \@temptokena \expandafter{\@tempb}%
376 \def\@tempc{\csgappto{cc@iterate@res}}%
377 \expandafter\@tempc\expandafter{\@tempb}%
378 \egroup
379 }%
380 \endgroup
381 \ccaEnable
382 }

```

`\cc@comp@edef` is used to pass a Counted Component into a TeX macro.

`{#1}` is a prefix to the def command, e.g., `\global` or `\protected`

`{#2}` is a CS token

`{#3}` is the Name of the Counted Component

`{#4}` is the Property that should be applied to all Members of the Counted Component

```

383 \def\cc@comp@edef{\cc@opt@empty\cc@comp@edef}
384 \def\@cc@comp@edef[#1]#2#3#4{%
385 \cc@apply@collection{#3}{#4}%
386 \ifx\cc@iterate@res\relax
387 #1\let#2\relax%
388 \else
389 \def\@tempa{#1\def#2}%
390 \cc@assign@res{@tempa}%
391 \fi
392 }

```

```

393 \def\cc@comp@def{\cc@opt@empty\cc@comp@def}
394 \def\@cc@comp@def[#1]#2#3#4{%
395 \cc@compose@collection{#3}{#4}%
396 \ifx\cc@iterate@res\relax
397 #1\let#2\relax%
398 \else
399 \def\@tempa{#1\def#2}%
400 \cc@assign@res{@tempa}%
401 \fi
402 }

```

```

403 \def\ccdefFromCountedComp{\cc@comp@def}
404 \def\ccgdefFromCountedComp{\cc@comp@def[\global]}

```

`\ccedefFromCountedComp` is the user-level command for *local* `\cc@comp@edef`.

```

405 \def\ccedefFromCountedComp{\cc@comp@edef}

```

`\ccxdefFromCountedComp` is the user-level command for *global* `\cc@comp@edef`.

```

406 \def\ccxdefFromCountedComp{\cc@comp@edef[\global]}

```

`\ccpxdefFromCountedComp` is the user-level command for *global* `\cc@comp@edef`. In contrast to `\ccxdefFromCountedComp`, it takes a CS name as first argument and prepends the `\ccPrefix` to the CS token to be defined.

```

407 \def\ccpxdefFromCountedComp#1{\expandafter\ccxdefFromCountedComp\csname \ccPrefix #1\endcsname}

```

Declaring Counted Component

`\cc@counted@comp@scheme` gives the scheme how counted components are defined internally.

`{#1}` the name of the Counted Component.

```
408 \def\cc@counted@comp@scheme#1{\cc@cnt@grp-#1-\csname \cc@cnt@grp Cnt\endcsname}
```

`\ccDeclareCountedComponent` is a user-level macro to create a new Counted Component.

`{#1}` is the user-level name of the Component

```
409 \def\ccDeclareCountedComponent#1{%
410   \cc@def@counted@comp
411   {\cc@counted@comp@scheme{#1}}
412   {#1}
413   {}
414   {\expandafter\global}%
415   \ignorespaces}
```

`\cc@def@counted@comp` is used to declare Counted Components.

`{#1}` is the internal name of the Component which is composed out of the group name, the value of the group counter and the user-level macro name `{#2}`

`{#2}` is the name of the Counted Component

`{#3}` is some custom code passed to the second argument of `\ccDeclareComponent`

`{#4}` is a modifier to the internal macro definition.

```
416 \def\cc@def@counted@comp#1#2#3#4{%
417   \ccDeclareComponent[#1]{#2}
418   {\bgroup#3\expandafter\global}
419   {\def\@tempa{\@cc@reset@components@\cc@cur@cont}}%
420   \edef\@tempb{\noexpand\csgundef{\cc@noexpand\cc@cur@cont @#1}}%
421   \expandafter\expandafter\expandafter\csgappto\expandafter\@tempa\expandafter{\@tempb}%
422   \egroup}%
423   \ignorespaces
424   #4\expandafter\long\expandafter\def\csname cc@\cc@cur@cont @#2\endcsname{\csname cc@
425     cc@cur@cont @#1\endcsname}%
426   \ignorespaces}
```

Resetting Counted Component

`\cc@reset@components` is used to reset Counted Components to prevent later Containers of a given type to feed the components from the previous Container of the same type. Usually, this is prevented by keeping Component definitions strictly local.

In some cases, however, Components may be declared globally, i.e., they may be re-used after the Container is ended. In this so-called Asynchronous Processing of Components, the reset should be done at the very beginning of the next instance of the container type to prevent bleeding of one container's components into the next one, specifically if a container occurs more than once in the same document.

`{#1}` is the name of the Component Group

```
426 \def\cc@reset@components#1{%
427   \csname @cc@reset@components@#1\endcsname
428   \global\cslet{\cc@reset@components@#1}\relax%
429 }
```


Toggling Conditionals for Counted Components

`\ccToggleCountedConditionals` In order to process Counted Components, we need to re-define the Conditionals in a way such that the Component is expanded twice before the comparison takes place to correctly resolve the Component counter.

Warning! Use this macro only within local groups!

```
430 \long\def\ccToggleCountedConditionals{%
431   \let\cc@is@counted\relax
```

This re-definitions of `\ccIfComp` cannot use `etoolbox`'s `\cs...` macros since the conditional can be embedded inside itself. If an inner csname is undefined, the condition for the outer one would be reset before it can be expanded by `\ifx`.

```
432 \long\def\ccIfComp##1{%
433   \expandafter\expandafter\expandafter\expandafter\expandafter\expandafter\expandafter\ifx\
      csname cc@\cc@cur@cont @##1\endcsname\relax\expandafter\@secondoftwo\else\expandafter\
      @firstoftwo\fi%
434 }%
435 \long\def\ccWhenComp##1{%
436   \expandafter\expandafter\expandafter\expandafter\expandafter\expandafter\expandafter\ifx\
      csname cc@\cc@cur@cont @##1\endcsname\relax\expandafter\@gobble\else\expandafter\
      @firstofone\fi%
437 }%
438 \long\def\ccUnlessComp##1{%
439   \expandafter\expandafter\expandafter\expandafter\expandafter\expandafter\expandafter\ifx\
      csname cc@\cc@cur@cont @##1\endcsname\relax\expandafter\@firstofone\else\expandafter\
      @gobble\fi%
440 }%
441 \long\def\ccIfCompEmpty##1{%
442   \expandafter\expandafter\expandafter\ifx\csname cc@\cc@cur@cont @##1\endcsname\cc@long@empty
      \expandafter\@firstoftwo\else\expandafter\@secondoftwo\fi}%
443 \ccToggleCountedConditionalsHook% legacy
444 }
```

`\ccUntoggleCountedConditionals` is used to locally reset the conditional definitions in the case that a Container instance occurs in the value of a Counted Component.

```
445 \let\cc@reserved@if@comp\ccIfComp
446 \let\cc@reserved@when@comp\ccWhenComp
447 \let\cc@reserved@unless@comp\ccUnlessComp
448 \let\cc@reserved@if@comp@empty\ccIfCompEmpty
449 \def\ccUntoggleCountedConditionals{%
450   \let\cc@is@counted\@undefined
451   \let\ccIfComp\cc@reserved@if@comp
452   \let\ccWhenComp\cc@reserved@when@comp
453   \let\ccUnlessComp\cc@reserved@unless@comp
454   \let\ccIfCompEmpty\cc@reserved@if@comp@empty
455 }
```

6 Properties

6.1 Setting Properties

`\ccSetProperty` is a user-level macro that provides the Property–Value interface for Containers.

`{#1}` is the name of the Property
`{#2}` is the Value assigned to that Property.

```
456 \long\def\ccSetProperty#1#2{\long\csdef{cc@\cc@cur@cont @#1}{#2}\ignorespaces}
```

`\ccAppToProp` can be used add material to the *end* of an existing Property value.

`{#1}` is the name of the Property
`{#2}` is the material to be added to previous value of that Property

```
457 \def\ccAppToProp#1#2{%
458   \long\csappto{cc@\cc@cur@cont @#1}{#2}%
459   \ignorespaces}
```

`\ccPreToProp` can be used add material to the *beginning* of an existing Property.

`{#1}` is the name of the Property
`{#2}` is the material to be inserted before the previous value of that Property

```
460 \def\ccPreToProp#1#2{%
461   \long\cspreto{cc@\cc@cur@cont @#1}{#2}%
462   \ignorespaces}
```

`\ccPropertyLet` can be used to create an alias Property `{#1}` of a given Property `{#2}`. Is is equivalent to `\ccSetProperty{#1}{\ccUseProperty{#2}}`.

```
463 \long\def\ccPropertyLet#1#2{\long\csedef{cc@\cc@cur@cont @#1}{\expandonce{\csname cc@\cc@cur@cont @#2\endcsname}}\ignorespaces}
```

`\ccPropertyLetX` creates a Property `{#1}` with the fully expanded value of another Property `{#2}`. Is is equivalent to `\ccSetPropertyX{#1}{\ccUseProperty{#2}}`.

```
464 \long\def\ccPropertyLetX#1#2{\long\csedef{cc@\cc@cur@cont @#1}{\csname cc@\cc@cur@cont @#2\endcsname}\ignorespaces}
```

`\ccSetPropertyVal` is a variant of `\ccSetProperty` that expands the value `{#2}` *once* before assigning it to the Property macro with the name `{#1}`. This can be used to assign the current value of a variable macro, dimension, counter or length to a Property.

```
465 \long\def\ccSetPropertyVal#1#2{\def\@tempa{\ccSetProperty{#1}}\expandafter\@tempa\expandafter{#2}\ignorespaces}
```

`\ccSetPropertyX` is another variant of `\ccSetProperty`, but it *fully expands* the value (using `\edef`) defined in `{#2}` before the Property is stored in the Property macro named `{#1}`. Use this if you need to use conditionals to determine the actual values of Properties that otherwise expect fixed named or dimensional values.

```
466 \long\def\ccSetPropertyX#1#2{\long\csedef{cc@\cc@cur@cont @#1}{#2}\ignorespaces}
```

`\ccAddToProperties` adds the material in `{#2}` to a Container of name `{#1}`'s *Properties* List.

```
467 \long\def\ccAddToProperties#1#2{\ccAddToType{Properties}{#1}{#2}\ignorespaces}
```

6.2 Using Properties

`\ccUseProperty` is a user-level command to directly access a previously set Property with the name `{#1}`.

```
468 \def\ccUseProperty#1{\csuse{cc@\cc@cur@cont @#1}}
```

`\cc@store@prop` stores the result of the application of property `{#3}` in the control sequence `{#2}`. The optional `[#1]` can hold a definition modifier like `\global` or `\long`.

```
469 \def\cc@store@prop{\cc@opt@empty\cc@store@prop}%
470 \long\def\cc@store@prop[#1]#2#3{%
471   \protected@edef\@tempa{\ccUseProperty{#3}}%
472   #1\expandafter\def\expandafter#2\expandafter{\@tempa}%
473   \ignorespaces}
```

`\ccdefFromProperty` expands an (implicit) Property `{#2}` and stores the result in (implicit) control sequence `{#1}`.

```
474 \def\ccdefFromProperty{\cc@store@prop}
```

`\cggdefFromProperty` is the `\global` variant of `\ccdefFromProperty`.

```
475 \def\cggdefFromProperty{\cc@store@prop[\global]}
```

`\ccpgdefFromProperty` is a `\global` variant of `\ccdefFromProperty` that takes a CS name as `{#1}` and prepends the `\ccPrefix` to it before assigning the Property result to the macro.

```
476 \def\ccpgdefFromProperty#1{\expandafter\cggdefFromProperty\csname \ccPrefix #1\endcsname}
```

`\ccUsePropertyEnv` is a user-level command to access a previously set Property and make it an environment accessible to Property specific processing instructions (see below).

```
477 \def\ccUsePropertyEnv#1{\cslet{cc@#1@active}{\relax}\csuse{cc@\cc@cur@cont @#1}\csundef{cc@#1
  @active}}
```

`\ccIfStrEqual` is a variant of etoolbox's `\ifstrequal` that first fully expands both arguments `{#1}` and `{#2}` (using `\edef`) before comparing them.

```
478 \def\ccIfStrEqual#1#2{%
479   \edef\@argi{#1}\edef\@argii{#2}%
480   \expandafter\expandafter\expandafter\ifstrequal
481     \expandafter\expandafter\expandafter{\expandafter\@argi\expandafter}%
482     \expandafter{\@argii}}
```

Local Property Overrides

`\cc@set@property@local` is a low-level macro to locally manipulate Properties.

`{#1}` is the CS token representing a method to alter the property (`\ccSetProperty`, `\ccAppToProp`, or `\ccPreToProp`)

`{#2}` is the name of the Property to be altered

`{#3}` is the new (or added) Value

```
483 \def\cc@set@property@locally#1#2#3{%
484   \let\cc@cur@cont\cc@cur@cont
485   \ifdefstring\cc@cur@cont{Heading}{\let\cc@cur@cont\ccCurSecName}{}%
486   \csappto{cc@type@Properties@\cc@cur@cont}{#1{#2}{#3}}%
487 }
```

The User level macros are Prefix sensitive. They exist in three flavours depending on whether the global Value of a Property should be kept or be replaced.

They all take two arguments:

`{#1}` is the name of the Property

`{#2}` is the value to be set, appended, or prepended to that Property, respectively.

`\ccSetPropLocal` sets a Property `{#1}` to a new value `{#2}`.

```
488 \def\ccSetPropLocal{\cc@set@property@locally\ccSetProperty}
489 \cslet{\ccPrefix SetPropLocal}\ccSetPropLocal%
```

`\ccAppPropLocal` appends the value `{#2}` to the *end* of an existing Property `{#1}`.

```
490 \def\ccAppPropLocal{\cc@set@property@locally\ccAppToProp}
491 \cslet{\ccPrefix AppPropLocal}\ccAppPropLocal%
```

`\ccPrePropLocal` appends the value `{#2}` to the *beginning* of an existing Property `{#1}`.

```
492 \def\ccPrePropLocal{\cc@set@property@locally\ccPreToProp}
493 \cslet{\ccPrefix PrePropLocal}\ccPrePropLocal%
```

6.3 Processing Instructions

In general, processing instructions are commands that are only visible to a specific process and ignored by others. In CoCoT_EX, Processing Instructions (PIs) are commands placed inside a Component that should only take effect when that Component is processed through a specific Property.

`\ccPI` is a Processing Instruction that executes `{#2}` when a Property with the name `{#1}` is currently processed with the `\ccUsePropertyEnv` macro.

```
494 \DeclareRobustCommand\ccPI[2]{\ifcsdef{cc@#1@active}{#2}{}}
```

6.4 Property Conditionals

`\ccIfProp` checks if a Property with the name `{#1}` is defined and non-empty. If so, do `{#2}`, otherwise do `{#3}`.

```
495 \long\def\ccIfProp#1#2#3{%
496   \expandafter\ifx\csname cc@\cc@cur@cont @#1\endcsname\relax#3\else
497     \expandafter\ifx\csname cc@\cc@cur@cont @#1\endcsname\cc@long@empty#3\else#2\fi
498   \fi
499   \ignorespaces}
```

`\ccWhenProp` is the same as `\ccIfProp` but omits the `else`-branch.

```
500 \long\def\ccWhenProp#1#2{%
501   \expandafter\ifx\csname cc@\cc@cur@cont @#1\endcsname\relax\else
502     \expandafter\ifx\csname cc@\cc@cur@cont @#1\endcsname\cc@long@empty\else#2\fi
503   \fi
504   \ignorespaces}
```

`\ccIfPropVal` checks if a Property `{#1}` expands to `{#2}`. If so, do `{#3}`, otherwise do `{#4}`.

Warning: Do not use this conditional in Properties that are used in `\ccApplyCollection`!

```

505 \long\def\ccIfPropVal#1#2#3#4{\long\def\@tempa{#2}%
506 \expandafter\ifx\csname cc@\cc@cur@cont @#1\endcsname\@tempa\relax#3\else#4\fi\ignorespaces}

```

7 Helper macros

7.1 Handling of Optional Arguments

Two simple internal macros to ease up the handling of optional arguments.

`\cc@opt@curcont` overrides stores the currently active Container name as future `#1`, unless the control sequence `{#1}` is called with an optional argument. In this case, the future `#1` is the value of that optional argument.

```

507 \long\def\cc@opt@curcont#1{\@ifnextchar [{#1}{#1[\cc@cur@cont]}}%

```

`\cc@opt@empty` passes an empty string as future `#1` if the optional argument is missing.

```

508 \long\def\cc@opt@empty#1{\@ifnextchar [{#1}{#1[]}}%

```

`\cc@opt@second` passes the first *mandatory* argument as value to the *optional* argument if the latter is missing.

```

509 \let\cc@opt@second\@dblarg

```

7.2 Iterators

`\cc@iterate` traverses in `[#1]`-th steps (defaults to +1) through counter `{#2}`, starting at number `{#3}` until and including number `{#4}` and does `{#5}` at every iteration (from `forloop.sty`, **Be aware** that incrementation of counter `{#2}` takes place after `{#5}` is called!):

```

510 \long\def\cc@iterate{\@ifnextchar [{\cc@iterate}{\@cc@iterate[\@ne]}}%
511 \long\def\@cc@iterate[#1]#2#3#4#5{%
512   #2=#3\relax%
513   \expandafter\ifnum#2>#4\relax%
514   \else
515     #5%
516     \advance#2 by #1\relax
517     \cc@iterate[#1]{#2}{\the#2}{#4}{#5}%
518   \fi}%

```

7.3 Attributes

Many macros and environments deal with optional arguments that are used to alter the behaviour of that macro or environment. The combination of a parameter and its set of possible values are called **Attributes**. In this section, we define the parsers for those parameters.

In order to catch the `babel` package's messing with the quote symbol, we make sure it has the correct cat-code.

```

519 \begingroup
520 \catcode`"=12

```

`\ccParseAttributes` High level wrapper for the attribute parser.

`{#1}` is the domain of the attribute

`{#2}` is the raw attribute list

```

521 \gdef\ccParseAttributes#1#2{%
522   \if!#1!\else
523     \if!#2!\else
524       \def\cc@cur@domain{#1}%
525       \cc@parse@attributes #2,,\@nil
526     \fi\fi}

```

The actual, recursively applying, parser comes in two parts:

`\cc@parse@attributes` parses the single attributes in an optional argument,

```

527 \gdef\cc@parse@attributes #1,#2,\@nil{%
528   \if!#1!\else
529     \cc@parse@kv#1==\@nil
530     \if!#2!\else
531       \cc@parse@attributes#2,\@nil
532     \fi\fi}

```

and

`\cc@parse@kv` distinguishes between the attribute name and its value(s).

```

533 \gdef\cc@parse@kv#1=#2=#3\@nil{%
534   \edef\@argii{#2}%
535   \ifx\@argii\@empty
536     \expandafter\let\csname cc@\cc@cur@domain @attr@#1\endcsname\@empty%
537   \else
538     \ifx #2 =\else
539       \expandafter\def\csname cc@\cc@cur@domain @attr@#1\endcsname{#2}%
540     \fi
541   \fi}

```

`\cc@parse@csv` takes a fallback macro `{#1}` and feeds it as argument to each item of the comma-separated list in the control sequence `{#2}`. The macro `{#1}` is stored internally as `\cc@parser@callback`.

```

542 \gdef\cc@parse@csv#1#2{%
543   \if!#1!\else
544     \let\cc@parser@callback#1%
545     \edef\cc@tempa{\csname #2\endcsname}%
546     \ifx\cc@tempa\@empty\else
547       \expandafter\cc@@parse@csv\cc@tempa,,\@nil
548     \fi
549   \fi}

```

`\cc@@parse@csv` applies `\cc@parser@callback` to the first item of a comma-separated pair and feeds the second item to itself.

```

550 \gdef\cc@@parse@csv #1,#2,\@nil{%
551   \if!#1!\else
552     \cc@parser@callback{#1}%
553   \fi
554   \if!#2!\else
555     \cc@@parse@csv#2,\@nil
556   \fi
557   \ignorespaces}

```

```
558 \endgroup
```

`\ccEvalAttributes` is a special Type Evaluator for Containers that define their Instance's attributes as Data Type. The Type then contains a list of `\ccDeclareAttributeHandler` statements for each of the allowed attributes.

`{#1}` is the Attribute Domain (defaults to the current Container name)

`{#2}` is the Container Instance's raw Attribute list.

```
559 \def\ccEvalAttributes{\cc@opt@curcont\cc@eval@attributes}%
560 \def\cc@eval@attributes[#1]#2{%
```

First we check if the Container Instance has a dedicated Attribute Type defined

```
561 \expandafter\ifx\csname cc@type@#1@Attributes\endcsname\relax
```

If so, we parse the Attribute list.

```
562 \ccParseAttributes{#1}{#2}%
```

After reading the Attribute list, we prepare unpacking the Attribute Data Type. Usually, the Type contains of a list of `\ccDeclareAttributeHandler` statements, but it can also handle the Attributes directly. The Attribute handler macro is defined locally:

`\ccDeclareAttributeHandler*` declares an Attribute handler. The *starred* version is for Attributes that are not expected to hold a value (i. e., switches), while the *non-starred* version is for Attributes that hold a value (key-value pairs). The value(s) for each matching Attribute is stored in `\ccAttrVal`. You may want to copy that value into another macro inside the third argument of the Handler macro for later evaluation, as it will be redefined by an Attribute Handler that is further down the Handler list.

`{#1}` is the name of the attribute (i. e., the part before the '=')

`{#2}` is code that is called when the Attribute *does not* occur in the Attribute list `{#1}`

`{#3}` is code that is called when the Attribute *does* occur in the Attribute list `{#1}`.

```
563 \def\ccDeclareAttributeHandler{%
564 \let\cc@is@starred\@undefined
565 \@ifstar
566 {\let\cc@is@starred\relax\cc@declare@attribute@handler}
567 {\cc@declare@attribute@handler}}%
568 \def\cc@declare@attribute@handler##1{\cc@opt@empty{\@cc@declare@attribute@handler{##1}}}%
569 \def\cc@declare@attribute@handler##1[##2]##3{%
570 \let\ccAttrVal\relax
571 \ifx\cc@is@starred\relax
572 \ccIfAttrIsSet{#1}{##1}{##3}{##2}%
573 \else
574 \ccIfAttr{#1}{##1}
575 {\letcs\ccAttrVal{cc@#1@attr@##1}##3}
576 {##2}%
577 \fi\ignorespaces
578 }%
```

With the Handler macro in place, we evaluate the Attributes data Type, thus parsing the Attributes.

```
579 \ccEvalType[#1]{Attributes}%
580 \else
```

If the Container has no Attributes type defined, we check if the Container instance has, in fact, Attributes

```
581 \if!#2!\else
```

If so, we issue a warning since we cannot know how to deal with the Attributes.

```

582 \ccPackageWarning{Kernel}{Attribute}
583 {Container instance on line \inputlineno\space has Attributes,^^Jbut Container `#1'
    provides no Attribute handlers!}
584 \fi
585 \fi
586 \ignorespaces}

```

`\ccGetAttribute` returns the value of an attribute.

`{#1}` is the attribute domain

`{#2}` is the attribute name

```

587 \def\ccGetAttribute#1#2{\csuse{cc@#1@attr@#2}}

```

`\ccIfAttr` can be used to call macros depending on whether an attribute is set, or not.

`{#1}` is the attribute domain

`{#2}` is the attribute name

`{#3}` is the then case

`{#4}` is the else case

```

588 \def\ccIfAttr#1#2#3#4{\ifcsdef{cc@#1@attr@#2}{#3}{#4}}

```

`\ccWhenAttr` is a variant of `\ccIfAttr` that omits the *else* branch.

`{#1}` is the attribute domain

`{#2}` is the attribute name

`{#3}` is the then case

```

589 \def\ccWhenAttr#1#2#3{\ifcsdef{cc@#1@attr@#2}{#3}{}}

```

`\ccUnlessAttr` is a variant of `\ccIfAttr` that omits the *then* branch.

`{#1}` is the attribute domain

`{#2}` is the attribute name

`{#3}` is the else case

```

590 \def\ccUnlessAttr#1#2#3{\ifcsdef{cc@#1@attr@#2}{#3}}

```

`\ccIfAttrIsStr` can be used to call macros depending if an attribute is set to the current (sub)container or group and what value it has.

`{#1}` is the attribute domain

`{#2}` is the attribute name

`{#3}` is the comparing value

`{#4}` is the then case

`{#5}` is the else case

```

591 \def\ccIfAttrIsStr#1#2#3#4#5{\ccIfAttr{#1}{#2}{\ifcsstring{cc@#1@attr@#2}{#3}{#4}{#5}}{#5}}

```

`\ccIfAttrIsSet` can be used to check if a value-less attribute has been set (i.e., it expands to `\@empty`).

`{#1}` is the attribute domain

`{#2}` is the attribute name

`{#3}` is the then case

`{#4}` is the else case


```

592 \def\ccIfAttrIsSet#1#2#3#4{\ccIfAttr{#1}{#2}{\expandafter\ifx\csname cc@#1@attr@#2\endcsname\
    @empty#3\else#4\fi}{#4}}

```

7.4 Style Classes

Style Classes are locally usable sub-Containers.

TODO
 Style Classes should be
 container-dependent.
 Better, yet, incorporate
 style classes into the
 new Attribute Type!

`\ccDeclareClass` The top-level macro `\ccDeclareClass[#1]{#2}[#3]{#4}` has four arguments, two of which are optional. `{#2}` is the name of the class. If this argument is empty, the special class name `default` is used. `{#4}` is the declaration block of the class. This argument usually contains a set of Property assignments using the `\ccSetProperty{<prop>}{<val>}` macro, see Sect. 6. The first optional argument `[#1]` is the Style Class' parent Container. Using parent Containers, you can have Style Classes of the same name for different (sub-)Containers, e.g., a `default` class for each float and heading Container. The second optional argument `[#3]` is the parent Style Class. Properties from that Style Class are loaded automatically prior to the loading of the current Style Class's Properties. This applies recursively allowing for a cascading of property values, as in CSS.

```

593 \long\def\ccDeclareClass{\@ifnextchar [{\@cc@set@class}{\@cc@set@class[default]}}%
594 \long\def\@cc@set@class[#1]#2{\cc@opt@empty{\cc@set@class[#1]{#2}}}%
595 \long\gdef\cc@default@class@default{}
596 \long\def\cc@set@class[#1]#2[#3]#4{%
597   \def\@argii{#2}\ifx\@argii\@empty\let\@argii\cc@str@default\fi%
598   \if!#3!\else
599     \expandafter\long\expandafter\def\csname cc@#1@class@\@argii @parent\endcsname{#3}%
600   \fi
601   \expandafter\long\expandafter\def\csname cc@#1@class@\@argii\endcsname{#4}%
602   \ignorespaces}

```

`\ccUseStyleClass` is a user-level macro to expand and “activate” a Style Class' Properties, those of its recursive ancestor Style Classes, and the default Style Class respecting the current Container.

`{#1}` is the Style Class name

`{#2}` is the Container name

```

603 \def\ccUseStyleClass#1#2{%
604   \expandafter\ifx\csname cc@#2@class@#1\endcsname\relax
605     \expandafter\ifx\csname cc@default@class@#1\endcsname\relax
606       \PackageError{cocotex.cls}{Class `#1' with scope `#2' not defined!}{Please declare the
        class `#1'!}%
607     \else
608       \PackageInfo{cocotex.cls}{Class `#1' with scope `#2' not defined; using unscoped class.}%
609     \fi
610   \fi
611   \csname cc@default@class@#1\endcsname%
612   \expandafter\ifx\csname cc@#2@class@#1@parent\endcsname\relax\else
613     \expandafter\ccUseStyleClass\expandafter{\csname cc@#2@class@#1@parent\endcsname}{#2}%
614   \fi
615   \csname cc@#2@class@#1\endcsname\ignorespaces}

```

7.5 The CoCoT_EX Logo

`\CoCoTeX` the CoCoT_EX Logo.

```
616 \DeclareRobustCommand\CoCoTeX{\texorpdfstring{{C\kern-.1em o\kern-.033emC\kern-.1em o}\kern-.133  
em\TeX}}{\CoCoTeX}}
```

```
</kernel>
```

Module 3

coco-common.dtx

```
<*common>
```

This file provides some macros that are used in more than one CoCoT_EX module.

```
23 %%
24 %% module for CoCoTeX that provides some commonly used base macros.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-common}
30 [2025/07/14 v0.6.2 CoCoTeX common module]
```

Default hook label for CoCoT_EX modules:

```
31 \SetDefaultHookLabel{cc}
```

1 Package options

1.1 Accessibility Features

Default color encoding passed as option to the `xcolor` package.

```
32 \def\cc@str@cmyk{cmyk}
33 \def\cc@str@rgb{rgb}
34 \def\cc@str@gray{gray}
35 \def\cc@str@natural{natural}
36 \let\cc@color@enc\cc@str@cmyk
37 \ExplSyntaxOn
38 \keys_define:nn { cocotex/common }
39 {
40   color-enc .choice:,
41   color-enc / srgb .code:n = { \global\let\cc@color@enc\cc@str@rgb },
42   color-enc / rgb .code:n = { \global\let\cc@color@enc\cc@str@rgb },
43   color-enc / gray .code:n = { \global\let\cc@color@enc\cc@str@gray },
44   color-enc / grey .code:n = { \global\let\cc@color@enc\cc@str@gray },
45   color-enc / cmy .code:n = { \global\let\cc@color@enc\cc@str@cmyk },
46   color-enc / cmyk .code:n = { \global\let\cc@color@enc\cc@str@cmyk },
47   color-enc / natural .code:n = { \global\let\cc@color@enc\cc@str@natural },
48   color-enc / none .code:n = { \global\let\cc@color@enc\relax },
49   color-enc .initial:n = cmyk
50 }
```

1.2 Output Options

The `silent` options attempts to suppress most unnecessary messages sent to the shell.

```
51 \newif\ifcc@silent \@cc@silentfalse
52 \keys_define:nn { cocotex/common }
53 {
54   silent .code:n = { \global\@cc@silenttrue }%
55 }
```

Process the options

```
56 \ProcessKeyOptions[cocotex/common]
```

First, we set up the requested color model with both the newer LaTeX Kernel and the xcolor package.

```
57 \ifx\cc@color@enc\relax\else
58   \tl_gset:Nn \l_color_fixed_model_tl {\cc@color@enc}%
59   \PassOptionsToPackage{\cc@color@enc}{xcolor}%
```

We also set the built-in defaults in case they are not overridden via the corresponding Components inside the  Meta instance.

```
60 \ifx\cc@color@enc\cc@str@cmyk
61   \def\cct@default@icc@comp{4}%
62   \def\cct@default@icc@iden{Coated FOGRA39}%
63 \else\ifx\cc@color@enc\cc@str@rgb
64   \def\cct@default@icc@comp{3}%
65   \def\cct@default@icc@iden{IEC 61966-2.1 Default RGB colour space - sRGB}%
66 \else
67   \def\cct@default@icc@comp{1}%
68   \def\cct@default@icc@iden{ISO Coated v2 - GREY 1c - (basIColor)}%
69 \fi
70 \fi
71 \fi%
72 \ExplSyntaxOff
```

`\ccIfPreamble` is true as long as there has not been a `\begin{document}`.

```
73 \def\cc@if@preamble{\ifx\nodocument\relax\expandafter\@secondoftwo\else\expandafter\@firstoftwo
74   \fi}
75 \let\ccIfPreamble\cc@if@preamble
```

2 Commonly Used Low-Level Macros and Registers

If CoCoTeX is used in conjunction with `xerif`¹, we include the `coco-xerif` module, which, albeit not an official part of the CoCoTeX framework, is essential for the Framework to work with `xerif` generated `.tex` files.

```
75 \IfFileExists{coco-xerif.sty}{\RequirePackage{coco-xerif}}{}
```

The `coco-kernel` module contains the core functions of the CoCoTeX framework.

```
76 \RequirePackage{coco-kernel}
```

¹See <https://github.com/transpect/xerif/>

2.1 Hard Dependencies

Hard requirements for all CoCoTeX modules:

```
77 \RequirePackage{xcolor}
```

Including the `graphicx` package and catching case-insensitive graphics file's endings from Word:

```
78 \RequirePackage{graphicx}
79 \ifcc@silent\let\Gin@log\@gobble\fi
80 \DeclareGraphicsRule{.EPS}{eps}{.EPS}{}
```

2.2 Common Variables

String Variables for Value Comparisons

`\cc@str@default` is a CS token that holds the string “default” for comparisons.

```
81 \def\cc@str@default{default}
```

`\cc@str@table` is a CS token that holds the string “table” for comparisons.

```
82 \def\cc@str@table{table}
```

`\cc@str@figure` is a CS token that holds the string “figure” for comparisons.

```
83 \def\cc@str@figure{figure}
```

`\cc@str@top` is a CS token that holds the string “top” for comparisons.

```
84 \def\cc@str@top{top}
```

`\cc@str@bottom` is a CS token that holds the string “bottom” for comparisons.

```
85 \def\cc@str@bottom{bottom}
```

Box Registers

Some temporary boxes that won't interfere with LaTeX's temporary boxes.

`\cc@tempboxa` is a temporary box register used throughout CoCoTeX.

```
86 \newbox\cc@tempboxa
```

`\cc@tempboxb` is another temporary box register used throughout CoCoTeX.

```
87 \newbox\cc@tempboxb
```

Temporary Length and Skip Registers

`\cc@tempskipa` is a temporary skip register used throughout CoCoTeX.

```
88 \newskip\cc@tempskipa
```

2.3 Helper macros

`\cc@topstrut` is a `\strut` that has the height of `\topskip` and the depth of the difference between the `\baselineskip` and `\topskip`.

```
89 \def\cc@topstrut{\vrule\@width\z@\@height\topskip\@depth\dimexpr\baselineskip-\topskip\relax}
```

`\cc@afterbox` prevents indentation and additional spacing after environments. Intended to be used in combination with `\aftergroup`.

```
90 \def\cc@afterbox{%
91   \everypar{%
92     \if@nobreak
93       \@nobreakfalse
94       \clubpenalty \@M
95     \if@afterindent \else
96       {\setbox\z@\lastbox}%
97       \everypar{}%
98     \fi
99   \else
100     \clubpenalty \@clubpenalty
101     {\setbox\z@\lastbox}%
102     \everypar{}%
103   \fi}}
```

`\ccSanitizeStr` is used to clean macro content from any non-expandable markup. It is intended to sanitize strings before they are sent to the PDF writer, like DocumentInfo strings or alternative texts.

`{#1}` is the macro that is defined to hold the sanitized text

`{#2}` is the content to be sanitized

```
104 \ExplSyntaxOn
105 \cs_new:Npn\ccSanitizeStr #1 #2
106 {
107   \edef #1 { \text_purify:n { #2 } }
108 }
109 \ExplSyntaxOff
```

2.4 Masks

These macros are intended to mask non-content markup, like page- or line breaking commands in order to find and remove or alter them easier.

`\hack` intended to mask line breaking macros.

```
110 \let\hack\@firstofone
```

`\hackfor` intended to hide line breaking macros.

```
111 \let\hackfor\gobble
```

`\Hack` intended to mask page breaking macros.

```
112 \let\Hack\@firstofone
```

`\Hackfor` intended to hide page breaking macros.

```
113 \let\Hackfor\@gobble
```

`\@gobbleopt` intended to nullify a macro's argument with a possible optional argument interfering.

Use it like this: `\let\yourMacroWithOptArg\@gobbleopt`

```
114 \long\def\@gobbleopt{\@ifnextchar[\@gobbleopt{\@gobbleopt[]}}%
115 \long\def\@gobbleopt[#1]#2{%
```

`\ccGobble` is used to de-activate certain macros to prevent them from being called multiple times while processing contents. An example is a footnote inside a caption while calculating the height of the caption. In this case, we need the space the footnote symbol requires without the actual footnote being written into the footnote insert, since that should happen when we actually print the caption.

```
116 \def\ccGobble{%
117   \renewcommand\footnote[2][\the\c@footnote]{\def\@thefnmark{##1}\@makefnmark}%
118   \renewcommand\index[2][]{}%
119   \renewcommand\marginpar[2][]{}%
120   \renewcommand\glossary[2][]{}%
121   \let\hypertarget\@gobbletwo
122   \let\label\@gobble
123   \@cc@is@finalfalse
124 }%
```

2.5 Arithmetics

`\CalcRatio` is used to calculate the ratio between two rational numbers `{#1}` and `{#2}`.

```
125 \def\CalcRatio#1#2{\strip@pt\dimexpr\number\numexpr\number\dimexpr#1\relax*65536/\number\dimexpr
    #2\relax\relax sp}
```

`\CalcModulo` is used to calculate the remainder of integer division of `{#1}` by `{#2}`. This needs a different approach than the common modulo definition, which would return negative results in some cases, as TeX rounds up the quotient of `{#1}` and `{#2}` if the first decimal place is equal to or greater 5.

```
126 \def\CalcModulo#1#2{\number\numexpr#1+#2-((#1+#2/2)/#2)*#2\relax}
```

`\minusvspace` Counterpart to L^AT_EX's `\addvspace`: if the value of `\minusvspace` is larger than `\lastskip`, `\lastskip` is used. Otherwise, the value of `\minusvspace` is used.

```
127 \def\@xminusvskip{%
128   \ifdim\lastskip<\@tempskipb
129   \else
130     \ifdim\lastskip<\z@
131     \else
132       \ifdim\@tempskipb<\z@
133         \advance\@tempskipb\lastskip
134       \fi
135       \vskip-\lastskip
136       \vskip \@tempskipb
137     \fi
138   \fi}
139 \def\minusvspace#1{%
140   \ifvmode
141     \if@minipage\else
```

```
142 \ifdim \lastskip =\z@
```

Compatibility to texlive pre 2020:

```
143 \ifx\@vspace@calcify\@undefined
144 \vskip #1\relax
145 \else
146 \@vspace@calcify{#1}%
147 \fi
148 \else
149 \setlength\@tempskipb{#1}%
150 \@xminusvskip
151 \fi
152 \fi
153 \else
154 \@noitemerr
155 \fi}
```

2.6 Determine actual page number

We need to determine the real page a floating object is printed. This mechanism is largely an adaption of the mechanism used in the `marginnote` package.

Counting absolute page numbers, however, may be misleading when the `coco-title` module is loaded and the cover page is not followed by an empty page. Therefore, we save the default page counter from $\text{\LaTeX}$ to evaluate it independently from the actual manner of counting.

`\the@cc@thispage` temporarily stores the current page number.

```
156 \def\the@cc@thispage{}
```

`\cc@abspage` is a counter for the absolute page number.

```
157 \newcount\cc@abspage \cc@abspage\z@
```

`\thecc@abspage` is the output formatter for the `\cc@abspage` counter.

```
158 \def\thecc@abspage{\the\cc@abspage}
```

`\if@cc@odd` is a conditional that is set to true if the current absolute page number is not divisible by 2.

```
159 \newif\if@cc@odd \cc@oddtrue
```

The absolute page counter is injected directly into $\text{\LaTeX}$'s output routine:

```
160 \AtBeginDocument{%
161 \global\cc@abspage=\c@page\relax%
162 \g@addto@macro\@outputpage{\global\cc@abspage\c@page}%
163 }
```

We split the testing mechanism into two parts.

`\ccTestPage` is run before the floating object is placed. It will store the page according to the placement in the tex source code.

```
164 \def\ccTestPage{}
```



```

165 \expandafter\ifx\csname the@cc@thispage\endcsname\@empty
166   \gdef\the@cc@atthispage{1}%
167 \else
168   \expandafter\ifnum \the@cc@thispage=\cc@abspage%
169     \begingroup
170     \@tempcnta\the@cc@atthispage\relax
171     \advance\@tempcnta\@ne\relax
172     \xdef\the@cc@atthispage{\the\@tempcnta}%
173   \endgroup
174 \else
175   \gdef\the@cc@atthispage{1}%
176 \fi
177 \fi
178 \xdef\the@cc@thispage{\the\cc@abspage}%
179 \let\cc@currpage\relax
180 \expandafter\ifx\csname \cc@cur@cont-\the@cc@thispage-\the@cc@atthispage\endcsname\relax
181   \ifodd\cc@abspage\relax\cc@oddtrue\else\cc@oddfalse\fi
182 \else
183   \edef\cc@currpage{\expandafter\expandafter\expandafter\@firstofone\csname \cc@cur@cont-\
184     the@cc@thispage-\the@cc@atthispage\endcsname}%
185   \ifodd\cc@currpage\relax\cc@oddtrue\else\cc@oddfalse\fi
186 \fi
187 }

```

`\ccSavePage` is the second macro, which writes the actual page number into the aux files.

```

187 \def\ccSavePage{%
188   \protected@write\@auxout{\def\the@cc@cur@cont{\cc@cur@cont}\let\thecc@abspage\relax}{%
189     \string\expandafter\string\gdef\string\csname\space \cc@cur@cont-\the@cc@thispage-\
190       the@cc@atthispage\string\endcsname{\thecc@abspage}}%
191 }

```

3 Re-Thinking L^AT_EX Core Functions

3.1 Keeping .aux-Files Up-to-Date

`\ccBreak` is a general line break macro intended to be re-defined if necessary without touching LaTeX’s kernel page and line breaking macros.

```

191 \DeclareRobustCommand*\ccBreak{\hfill\break}
192 \ifx\ccPrefix\@empty\else\cslet{\ccPrefix break}\ccBreak\fi

```

3.2 Content lists

Default L^AT_EX Content Lists

This part contains macros to “simplify” the generation of content lists like the Table of Contents or List of Figures/Tables, etc.

Entries in the list-files (e.g., `\jobname.toc`, `\jobname.lof`, etc.) usually contain `\contentsline` macros that expand to `l@<level>`. Whenever a level of Components that are to be written into content lists is declared, the package automatically generates a `\cc@l@<level>` macro for this level of entries. The content-baring argument of `\ccContentsline` (or `\cc@l@<level>`, resp.) contains Components.

Once a list file is read, those `\cc@l@<level>` macros are expanded in two steps. Each entry constitutes a Container in its own right. It therefore can have multiple Components. The first step is the extraction phase, where the entry's Container is dynamically declared, the corresponding properties are initialised, and its Components are extracted

`\cc@init@l@` is a low-level macro used to dynamically define `\cc@l@<level>` macros.

- `[#1]` is an override for counters that have to be restored
- `{#2}` is the list file ending (raw entries being stored in a file `\jobname.\#2`)
- `{#3}` is a number that indicated the nesting depth
- `{#4}` is the nested level's unique name.

```

193 \def\cc@init@l@{\cc@opt@empty\@cc@init@l@}%
194 \def\cc@init@l@[#1]#2#3#4{%
195   \expandafter\ifx\csname c@#2depth\endcsname\relax
196     \expandafter\global\expandafter\newcount\csname c@#2depth\endcsname
197     \expandafter\global\csname c@#2depth\endcsname=0\relax
198   \fi
199   \expandafter\ifx\csname cc@#2@extract@data\endcsname\relax
200     \expandafter\let\csname cc@#2@extract@data\endcsname\cc@extract@generic
201   \fi
202   \expandafter\ifx\csname cc@#2@print@entry\endcsname\relax
203     \expandafter\let\csname cc@#2@print@entry\endcsname\cc@print@generic
204   \fi
205   \expandafter\long\expandafter\gdef\csname cc@l@#4\endcsname##1##2{%
206     \ifLuaTeX\suppresslongerror=1\fi
207     \expandafter\ifnum \csname c@#2depth\endcsname<#3\relax
208       \else
209         \bgroup

```

`\ccTocLink` is used to link list entries to their destination.

```

210   \long\def\ccTocLink####1{\hyper@linkstart{link}{\@contentsline@destination}{####1}\
    hyper@linkend}%

```

```

211   \csname cc@#2@extract@data\endcsname{#3}{#4}{##1}{##2}%
212   \csname cc@#2@print@entry\endcsname{#4}%
213   \egroup
214   \fi
215   \ifLuaTeX\suppresslongerror=0\fi
216 }}

```

`\ccContentsline` is our version of L^AT_EX's `\contentsline`.

- `{#1}` is the name of the list counter
- `{#2}` is the name of the list entry
- `{#3}` is the page number
- `{#4}` is the hyperref destination

```

217 \long\def\ccContentsline#1#2#3#4{\gdef\@contentsline@destination{#4}%
218   \csname cc@l@#1\endcsname{#2}{#3}}

```

`\cc@extract@generic` is a fallback extractor for a list entry. It is used when the list handler does not provide a case-specific extractor for the entries.

- `{#1}` is the name of the list counter
- `{#2}` is the name of the list entry
- `{#3}` is the page number
- `{#4}` is the hyperref destination

```
219 \def\cc@extract@generic#1#2#3#4{}
```

`\cc@print@generic` is the fallback output generator for the composed list entry `{#1}`.

```
220 \def\cc@print@generic#1{}
```

`\cc@expand@l@contents` expands the content of the `cc@l@<level>` macro and contains some code to catch and handle standard L^AT_EX headings.

`{#1}` is the content of the `cc@l@`-Macro

`{#2}` is the name of the handling Container

`{#3}` is the Component prefix

`{#4}` is the name of the Content component

```
221 \def\cc@expand@l@contents#1#2#3#4{%
222   \global\let\cc@tempa\relax
223   \sbox\z@{\def\numberline##1{\xdef\cc@tempa{\noexpand\csdef{cc@#2@#3Number}{##1}}}{#1}%
224   \ifdim\wd\z@>\z@
225     \let\numberline\@gobble%
226     \protected@csdef{cc@#2@#3#4}{#1}%
227     \cc@tempa
228   \else
229     #1%
230   \fi
231   \global\let\cc@tempa\relax
232 }
```

Custom Content Lists

`\ccDeclareContentList` provides an interface for additional content lists.

`{#1}` is the name of the custom content

`{#2}` is a comma separated list of container names the instances of which should be listed in the custom contents list

```
233 \def\ccDeclareContentList#1#2{%
234   \def\cc@add@extra@cl##1{%
235     \expandafter\ifx\csname cc@##1@extra@cl\endcsname\relax
236       \csgdef{cc@##1@extra@cl}{#1}%
237     \else
238       \csgappto{cc@##1@extra@cl}{, #1}%
239     \fi}%
240   \edef\@argii{#2}%
241   \cc@parse@cvs\cc@add@extra@cl{\@argii}%
242   \expandafter\newwrite\csname cc@cl@#1\endcsname\relax
243 }
```

`\ccCreateContentListEntries` creates entries for Custom Content Lists. It is called during the processing of a container's instance.

`{#1}` is the name of the calling Container

`{#2}` is the name of the file stream

`{#3}` is the level of the entry

`{#4}` is the current page counter

`{#5}` is the current hyperref label

```
244 \def\ccCreateContentListEntries#1#2#3#4#5{%
245   \def\cc@add@extra@cl##1{%
```

```

246 \expandafter\protected@write\csname cc@cl@##1\endcsname
247 {\ccGobble\ccaProtect}%
248 {\protect\ccContentsline{#2}{#3}{#4}{#5}\protected@file@percent}\relax
249 \expandafter\protected@write\csname cc@cl@##1\endcsname
250 {\ccaEnable}{}%
251 }%
252 \ifcsdef{cc@#1@extra@cl}{%
253 \cc@parse@csv\cc@add@extra@cl{cc@#1@extra@cl}}}%
254 }

```

3.3 Indentation and Left Margins of Potentially Numbered Items

The **left margin** means the space between the left border of the page area and the imaginary line that multi-line text aligns to. The **indent** is the offset of the very first line of that block of text relative to that value.

If the **indent** is a negative value you'll get a hanging indent; if it is positive, you get a paragraph style indent, and if it is set to **Opt**, you get a clean alignment of the whole item.

CoCoTeX provides a feature that allows the indentation of counted elements to be just as wide as the widest Number of the same level (if **indent** is set to **auto**), as well as a feature that allows the indent to be as wide as all Numbers of the same container type (if **indent** is set to **auto-global**).

The approach to set the **indent**, **margin-left** and the position of the Number Component in numbered items such as Headings, entries in ToC and listof-X, captions, etc. is to store the maximum width for each level and the maximum width across all Numbers of a Container Type in the .aux file at the very end of the compilation after it has been constantly updated during the entire L^AT_EX runtime. That way, for the next L^AT_EX run, the maximum values are available immediately and can be used to fortify those parameters.

`\cc@store@latest` is a low-level macro that stores the maximum value of a dimension Property `{#1}`. An internal Property `\#1-local` is constantly updated whenever the macro is called and the previously stored value is lower than the one given in `{#2}`.

The first call of the macro for a given Property triggers an addendum to the `\@enddocumenthook` which causes the last value for that dimension to be stored in the .aux file. If the Property hasn't been set from a previous L^AT_EX run or a previous call to the `\cc@store@latest` macro for the same Property and the same level, it is set to `{#2}`.

`{#1}` is the internal name of the property

`{#2}` is the check value.

```

255 \def\cc@store@latest#1#2{%
256 \expandafter\ifx\csname cc-\cc@cur@cont-#1\endcsname\relax
257 \csxdef{cc-\cc@cur@cont-#1}{#2}%
258 \else
259 \expandafter\ifdim\csname cc-\cc@cur@cont-#1\endcsname<#2\relax
260 \csxdef{cc-\cc@cur@cont-#1}{#2}%
261 \fi
262 \fi
263 \expandafter\ifx\csname cc-\cc@cur@cont-#1-local\endcsname\relax
264 \csxdef{cc-\cc@cur@cont-#1-local}{#2}%
265 \else
266 \expandafter\ifdim\csname cc-\cc@cur@cont-#1-local\endcsname<#2\relax
267 \csxdef{cc-\cc@cur@cont-#1-local}{#2}%
268 \fi
269 \fi

```

The second step is to store the highest values in the .aux file for later LaTeX runs. A `\write\@auxout` command for the storage macro is therefore added to the `\@enddocumenthook` and a flag is set that indicates that the write command has already been added to the hook, since that needs to be done only once for each to-be-stored dimension.

Note that the value that is eventually stored, is the updated *local* maximum, not the value that is retrieved at the beginning of the run. This allows the values to be down-graded if the LaTeX source changed during two consecutive runs. However, if values change, you still need to do at least two more L^AT_EX runs before the values stabilize.

```

270 \ifcsdef{cc-\cc@cur@cont-#1-stored-trigger}{%
271   {\edef\@tempa{%
272     \noexpand\immediate\noexpand\write\noexpand\@auxout{%
273       \noexpand\string\noexpand\csgdef{cc-\cc@cur@cont-#1}{%
274         \noexpand\csname cc-\cc@cur@cont-#1-local\noexpand\endcsname}}}%
275     \expandafter\AtEndDocument\expandafter{\@tempa}%
276     \csgdef{cc-\cc@cur@cont-#1-stored-trigger}{\@empty}}}
```

`\cc@format@number` calculates number widths and prepares macros to be used by the user.

`{#1}` is the internal Property prefix
`{#2}` is the user-level Component prefix
`{#3}` is the numerical list level.

```

277 \def\cc@format@number#1#2#3{%
278   \ccSetPropertyVal{#1curr-number-level}{#3}%
```

First step: measuring the natural width of the Number if it exists for the current item.

```

279 \ccIfComp{#2Number}
280   {\sbox\z@{\ccUseProperty{#1number-format}}}%
281   {\sbox\z@{}}%
```

Second step: we store the width of `\box0` if it is wider than the previously stored width for that level. The end value will be written into the .aux file during expansion of the `\@enddocumenthook`. We do the same for the maximum across *all* levels of the same Container Type.

```

282 \cc@store@latest{#1number-#3-maxwd}{\the\wd\z@}%
283 \cc@store@latest{#1number-maxwd}{\the\wd\z@}%
```

We provide the maximum level as a user-level Property `#1number-width-level-max`, the global maximum across all levels as `#1number-width-max`, and the width of the current number as `#1number-width`.

```

284 \ccSetPropertyVal{#1number-width-level-max}{\csname cc-\cc@cur@cont-#1number-#3-maxwd\
    endcsname}%
285 \ccSetPropertyVal{#1number-width-max}{\csname cc-\cc@cur@cont-#1number-maxwd\endcsname}%
286 \ccSetPropertyVal{#1number-width}{\the\wd\z@}%
```

Third step: we calculate and fortify the actual `#1margin-left` (i.e., the overall left indent of the whole item) and `#1indent` (offset of the first line) of the entry.

```

287 \cc@get@indent{#1}{#3}%
288 \cc@set@hang{#1}%
289 }
```

`\cc@set@hang` determines and sets the hanging indent of a counter.

`{#1}` is the internal Property prefix

```

290 \def\cc@set@hang#1{%
```

First, we set the `#1hang-number` to be an alias of `#1number-format` as fallback.

```

291 \ccPropertyLet{#1hang-number}{#1number-format}%
```

Then, we check for `#1indent`.

```

292 \ccIfProp{#1indent}
293   {\ifdim\ccUseProperty{#1indent}<\z@

```

If it is set and negative, we alter the `#1hang-number` Property in such a way that it is shifted to the left by `#1indent` amount and put into a hbox of `-#1indent` width (remember that the value is negative).

```

294   \ccSetProperty{#1hang-number}{%
295     \hskip\ccUseProperty{#1indent}%
296     \hbox to -\ccUseProperty{#1indent}{%
297       \ccIfPropVal{#1number-align}{left}{\hss}%
298       \ccUseProperty{#1number-format}%
299       \ccIfPropVal{#1number-align}{right}{\hss}}}%
300   \fi}{}}

```

In all other cases, we stick to the default (`#1number-format`) we set in the first step.

`\cc@calc@margin@left` determines the left margin of the current level by subtracting the current level's indent from the left margin of the next-higher level. “Next-higher” meaning “hierarchically”, i.e., the level counter is *lower*. Remember that for hang indent, the indent is negative, so `margin-left` grows larger.

`{#1}` is the Property prefix

`{#2}` is the current numerical list level.

```

301 \def\cc@calc@margin@left#1#2{%
302   \@tempcnta\numexpr#2-\@ne\relax
303   \expandafter\ifx\csname cc-\cc@cur@cont-#1\the\@tempcnta-margin-left\endcsname\relax
304     \@tempdima=-\ccUseProperty{#1indent}\relax%
305   \else
306     \@tempdima=\dimexpr\csname cc-\cc@cur@cont-#1\the\@tempcnta-margin-left\endcsname-
307       \ccUseProperty{#1indent}\relax
308   \fi
309   \cc@store@latest{#1#2-margin-left}{\the\@tempdima}%
310   \ccSetProperty{#1margin-left}{\the\@tempdima}}

```

`\cc@get@indent` Eventually, write the actually used values for `margin-left` and `indent` into the current container's Property list.

`{#1}` is the CS token of a method that is called to calculate the actual left margin of the list item. It defaults to above's `\cc@calc@margin@left` and is fed the two mandatory arguments of the `\cc@get@indent` macro, namely

`{#2}` for the internal property prefix, and

`{#3}` for the numerical list level.

The callback method should set and store the `#2margin-left` Property.

```

310 \def\cc@get@indent{\@ifnextchar[{\@cc@get@indent}{\@cc@get@indent[\cc@calc@margin@left]}}
311 \def\@cc@get@indent[#1]#2#3{%

```

First, we need to store the initial values for both `#2margin-left` and `#2indent` since, first their values might be non-dimensional, and second, they will be altered during macro expansion to ultimately being passed to `\hskip`.

```

312   \ccPropertyLetX{int-#2margin-left}{#2margin-left}%
313   \ccPropertyLetX{int-#2indent}{#2indent}%
314   \ccIfPropVal{#2indent}{auto-global}

```

If `#2indent` is set to `auto-global`, the item gets an `indent` that is set to the negative value of the maximum width of all numbers across all Levels of the same Container Type. The same maximum is added to the user-set value of `margin-left`.

```

315   {\ccSetPropertyX{#2indent}{-\ccUseProperty{#2number-width-max}}}%

```

If the user has set `#2margin-left` to `auto`, we reset it to empty.

```
316 \ccIfPropVal{#2margin-left}{auto}{\ccSetProperty{#2margin-left}{}}{}}%
```

If the user has not set `margin-left`, we set it to `\z@`.

```
317 \ccIfPropVal{#2margin-left}{  
318   {\ccSetProperty{int-#2margin-left}{\z@}}  
319   {\ccPropertyLetX{int-#2margin-left}{#2margin-left}}}%  
320 \ccSetPropertyX{#2margin-left}{\dimexpr\ccUseProperty{#2number-width-max}+\ccUseProperty{int  
  -#2margin-left}\relax}}
```

Next, we check if `#2margin-left` is set to `auto`.

```
321 {\ccIfPropVal{int-#2margin-left}{auto}}
```

If `#2margin-left` is set to `auto`, all items of the same level get the same left margin that is determined by the sums of the indents of all higher levels.

```
322 {\ccIfPropVal{int-#2indent}{auto}}
```

if `#2indent` is also set to `auto`, the `indent` of the current item is set to the widest Number of the same level.

```
323 {\ccSetPropertyX{#2indent}{-\ccUseProperty{#2number-width-level-max}}}
```

otherwise it is set to the value of `indent`, or `Opt` if it was not set at all.

```
324 {\ccIfProp{int-#2indent}  
325   {\ccSetPropertyX{#2indent}{\ccUseProperty{int-#2indent}}}  
326   {\ccSetProperty{#2indent}{\z@}}}%
```

the final value for `margin-left` is calculated. If no optional argument is given, the method called is the `\cc@calc@margin@left` macro, above.

```
327 #1{#2}{#3}}
```

This branch is reached when the left margin is not set to `auto`.

```
328 {\ccIfProp{int-#2margin-left}  
329   {\ccIfPropVal{int-#2indent}{auto}}
```

If `margin-left` is set to a specific value and `indent` is set to `auto`, set the actual indent to the width of the level's widest Number.

```
330 {\ccSetPropertyX{#2indent}{-\ccUseProperty{#2number-width-level-max}}}  
331 {\ccIfProp{int-#2indent}}
```

Otherwise, if `indent` is set to a specific width, apply that value, or else set the inden to `Opt`.

```
332 {\ccSetPropertyX{#2indent}{\ccUseProperty{int-#2indent}}}  
333 {\ccSetProperty{#2indent}{\z@}}}
```

If `margin-left` is not set,

```
334 {\ccIfPropVal{int-#2indent}{auto}}
```

and `indent` is set to `auto`, set `margin-left` to the width of the level's widest Number and the actual `indent` to the negative of that.

```

335     {\ccPropertyLetX{#2margin-left}{#2number-width-level-max}%
336      \ccSetPropertyX{#2indent}{-\ccUseProperty{#2number-width-level-max}}%
337      {\ccIfProp{int-#2indent}

```

If `margin-left` is not set, and `indent` is set to a specific value, apply that value for `indent` and set `margin-left` to `Opt`. In this branch, `indent` should have a positive value, otherwise the content would probably lap over the left edge of the type area.

```

338     {\ccSetPropertyX{#2indent}{\ccUseProperty{int-#2indent}}%
339     \ccSetProperty{#2margin-left}{\z@}}

```

otherwise set both `indent` nad `margin-left` to `Opt`.

```

340     {\ccSetProperty{#2indent}{\z@}%
341     \ccSetProperty{#2margin-left}{\z@}}}}}}

```

3.4 Labelling and Cross referencing

CoCoT_EX provides two ways to put labels on Container instances: one via the label attribute at the begin of a (Sub-)Containers corresponding environment, or via the `RefLabel` Component inside the (Sub-)Container.

```

342 \AtBeginDocument{%

```

Storing the final definitions of `\label`

`\cc@ltx@label` stores the definition of LaTeX's `\label` macro at the beginning of the document.

```

343 \global\let\cc@ltx@label\label
344 }

```

`\ccCreateLabel` is a high level macro to generate hyperref anchors and/or ref targets.

`{#1}` is the type of anchor

This macro looks for both the label attribute in the begin of a Container's environment, as well as for a `RefLabel` Components inside the environment. If both exist, both apply. If none exists, we adopt the generic anchor point generated by the `hyperref` package.

TODO
Check if the hyperref macros need to be configured in any way for various reference types!

```

345 \def\ccCreateLabel#1{%
346   \ifx\Hy@MakeCurrentHrefAuto\undefined\else
347     \Hy@MakeCurrentHrefAuto{cc:#1}%
348     \Hy@raisedlink{\hyper@anchorstart{\@currentHref}\hyper@anchorend}%
349   \fi
350   \let\cc@ref@label\relax
351   \ccWhenComp{RefLabel}
352     {\ccgdefFromComp\cc@ref@label{RefLabel}%
353     \expandafter\cc@create@label\expandafter{\cc@ref@label}}%
354   \ccIfAttr{\cc@cur@cont}{label}
355     {\cc@parse@csv\cc@create@label{\cc@cc@cur@cont @attr@label}}%
356     {\ifx\cc@ref@label\relax\cc@create@label{\@currentHref}\fi}}

```


`\cc@create@label` generates the actual anchor for document-internal cross-references (i.e., a L^AT_EX `\label`).

`{#1}` is the label ID

```

357 \def\cc@create@label#1{%
358   \ccIfComp{Number}
359   {\ifx\cc@labelname@comp\@undefined
360     \def\cc@labelname@comp{Title}%
361     \fi
362     \begingroup
363       \ccGobble\ccaProtect
364       \ccgdefFromComp{@currentlabel}{Number}%
365       \ccgdefFromComp{@currentlabelname}{\cc@labelname@comp}%
366     \endgroup}%
367   {\cc@fallback@anchor}%
368   %% leaving this will generate lots of "duplicate destination"
369   %% messages from pdfbackend
370   %\expandafter\hypertarget\expandafter{#1}{}%
371   \@tempswafalse\ifx\cc@saved@ccaStructStart\ccaStructStart\@tempswatrue\fi
372   \ccaProtect
373   \expandafter\label\expandafter{#1}%
374   \if@tempswa\ccaEnable\else\ccaDisable\fi
375 }
376 \def\cc@fallback@anchor{\phantomsection}%

```

3.5 Linguistic Name generation and selection

`\ccSetBabelLabel` defined a language-dependent string macro for German and English varieties.

`{#1}` is the language

`{#2}` is the internal reference name

`{#3}` is the language specific label

```

377 \def\ccSetBabelLabel#1#2#3{%
378   \def\ccc@lang{#1}%
379   \expandafter\def\expandafter\ccc@tempa\expandafter{\expandafter\def\csname #2name\endcsname
380     {#3}}%
381   \ifdefstring\ccc@lang{german}{%
382     \expandafter\addto\expandafter\captionsgerman\expandafter{\ccc@tempa}%
383     \expandafter\addto\expandafter\captionsgerman\expandafter{\ccc@tempa}%
384   }\relax%
385   \ifdefstring\ccc@lang{english}{%
386     \expandafter\addto\expandafter\captionsbritish\expandafter{\ccc@tempa}%
387     \expandafter\addto\expandafter\captionbUKenglish\expandafter{\ccc@tempa}%
388     \expandafter\addto\expandafter\captionsgenglish\expandafter{\ccc@tempa}%
389     \expandafter\addto\expandafter\captionssamerican\expandafter{\ccc@tempa}%
390     \expandafter\addto\expandafter\captionssUSenglish\expandafter{\ccc@tempa}%
391   }\relax%
392 }

```

3.6 Link Generation

Links to Components

`\ccCompLink` creates a hyperlink with the target taken from a Component with the name `{#1}` and the label `{#2}`.

```

392 \def\ccCompLink#1#2{%
393   \protected@edef\@argi{\expandonce{\ccUseComp{#1}}}%

```

```

394 \expandafter\href\expandafter{\@argi}{#2}%
395 }

```

`\ccPageLabel` enables referencing pages via `\pageref` by using `\phantomsection` to create a hyperref anchor for label `{#1}`.

```

396 \def\ccPageLabel#1{\phantomsection\label{#1}}

```

Breakable URLs

The following macros define a command to markup plain URLs that are allowed to breaks either after each character or after a special character. This character can be given individually for each URL or set globally by re-defining `\ccBurlTokenSep`. By default, no hyphenchar is set to avoid ambiguities, and there is minimal glue after each breaking point to keep lines nicely justified. Alternatively, one could insert a discretionary by using the `\ccUrlSep` macro.

`\ccBurlTokenSep` is the default break point character. The global default is `\@empty`, which means that plain URLs may break after each character.

```

397 \let\ccBurlTokenSep\@empty

```

`\ccBurl` is the high-level user macro for breakable URLs. It takes two arguments, `[#1]` is an optional break point marker and `{#2}` the URL. Special characters like `#` or `%` can (but don't need to) be escaped, since the value is sanitized through hyperref's `\hyper@normalise` macro.

The break point marker in the optional argument is a single character that may or may not occur in the URL to designate where the URL may break across lines. The character is neither printed nor does it occur in the hyperlink destination.

```

398 \ExplSyntaxOn
399 \DeclareRobustCommand*\ccBurl[#1][\ccBurlTokenSep]{%
400   \beginingroup
401     \tl_if_blank:eF { #1 }
402     {
403       \token_if_active:NT #1 { \edef#1{ \token_to_str:N #1 } }
404       \edef\ccBurlTokenSep{#1}
405     }
406     \hyper@normalise\cc@url}
407 \ExplSyntaxOff%
408 \cslet{\ccPrefix Burl}\ccBurl

```

`\cc@url` is where the break point character is processed. It is handled differently for the actual URI, where it is simply removed, and the text that is printed on a page, where it is replaced by the value of `\ccUrlSep`. If no break point character is given, the URL may break after any character.

```

409 \def\cc@url#1{%
410   \toks@{#1}\edef\Url@String{\the\toks@}%
411   \edef\Url@String{\expandafter\strip@prefix\meaning\Url@String}%
412   \edef\x{%
413     \noexpand\def\noexpand\cc@break@link\unexpanded{##1}\ccBurlTokenSep\unexpanded{##2}\
414       ccBurlTokenSep\noexpand\@nil{%
415       \noexpand\ifx\noexpand\cc@url@sep\noexpand\@empty\noexpand\else
416       \noexpand\cc@url@sep\noexpand\hbox\noexpand\bgroup
417       \noexpand\fi
418       \unexpanded{##1}%
419       \noexpand\ifx\noexpand\cc@url@sep\noexpand\@empty\noexpand\else
420       \noexpand\egroup%
421       \noexpand\fi
422       \noexpand\if\noexpand\space\unexpanded{##2}\noexpand\space\noexpand\else

```

```

422 \ifx\ccBurlTokenSep\@empty
423 \noexpand\cc@break@link\unexpanded{##2}\noexpand\@nil
424 \else
425 \noexpand\if\ccBurlTokenSep\unexpanded{##1}\noexpand\else
426 \noexpand\if\ccBurlTokenSep\unexpanded{##2}\noexpand\else
427 \noexpand\cc@break@link\unexpanded{##2}\ccBurlTokenSep\noexpand\@nil
428 \noexpand\fi
429 \noexpand\fi
430 \fi
431 \noexpand\fi
432 }}\x
433 \begingroup
434 \let\cc@url@sep\@empty
435 \edef\x{\noexpand\cc@break@link\unexpanded{#1}\ccBurlTokenSep\ccBurlTokenSep\noexpand\@nil}%
436 \xdef\Url@String{\x}%
437 \endgroup
438 \expandafter\def\expandafter\x\expandafter{\expandafter\href\expandafter{\Url@String}}%
439 \expandafter\x\expandafter{\@cc@url{#1}}

```

`\@cc@url` correspond's to the `url` package's `\Url` macro, just without the “movable” check.

```

440 \def\@cc@url{%
441 \let\do\@makeother \dospecials % verbatim catcodes
442 \catcode`\@one \catcode`\tw@ % with exceptions
443 \catcode`\ =10 % allow "\url {x}"
444 \catcode`\# =6 % always # -> ##, so I can later convert ## -> #
445 \obeyspaces\obeylines\cc@urlz}

```

`\cc@urlz` is a fork of `url` package's `\@urlz`. Note that `{#1}` is the *printed* URL, not the URI written into the PDF as hyperlink destination.

```

446 \def\cc@urlz#1{\toks@{#1}\edef\Url@String{\the\toks@}%
447 \edef\Url@String{\expandafter\strip@prefix\meaning\Url@String}%
448 \Url@ObeySp %
449 \Url@acthash
450 \Url@actpercent
451 \Url@HyperHook
452 \edef\z{\noexpand\expandafter\noexpand\cc@break@link\noexpand\Url@String\ccBurlTokenSep\
453 \ccBurlTokenSep\noexpand\@nil}\z
454 \endgroup}

```

`\ccUrlSep` is used to set the code inserted in between each token of a link without (`{#1}`) and with (`{#2}`) break characters. by default, this is a 0sp `\hskip` without any glue for break chars and with 1sp glue for links without break char. But it can also be a `\discretionary`.

```

454 \def\ccUrlSep#1#2{\def\cc@url@sep{\ifx\ccBurlTokenSep\@empty#1\else#2\fi}}
455 \ccUrlSep{\hskip\z@\@plus1sp}{\hskip\z@}

```

</common>

Module 4

coco-accessibility.dtx

This file provides code for the interaction between the CoCoTeX framework and the `ltpdfa` package.

Please consider this module as highly experimental!

There are two files created from this dtx: one `coco-accessibility.sty` and one `coco-accessibility.lua`.

1 LaTeX code

```
<*a11y-sty>
```

1.1 General Processing

The `coco-accessibility.sty` starts with some general package information like name, current version and date of last changes.

```

23 %%
24 %% Accessibility features for \textit{xerif} projects.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 %% requires LuaLaTeX and ltpdfa!
29 %%
30 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
31 \ProvidesPackage{coco-accessibility}
32   [2025/07/14 v0.6.2 CoCoTeX accessibility module]
33 \ifpackageloaded{etoolbox} {}{\RequirePackage{etoolbox}}
34 \ifpackageloaded{graphicx} {}{\RequirePackage{graphicx}}
35 \ifpackageloaded{iftex} {}{\RequirePackage{iftex}}
36 \ifpackageloaded{coco-kernel}{}{\newif\ifcc@is@final\cc@is@finaltrue}

```

The `ltpdfa` package re-defines too many standard LaTeX macros, so we only use its lua code and define the interface ourself. For that, we use `etoolbox`'s patch commands to inject our tagging code into the standard macros rather than to create hard copies. This should increase compatibility with other packages and make all our lives easier.

`\ifcc@do@ally` is a simple switch that indicates active (`\relax`) or inactive (everything else) accessibility features.

```

37 \newif\ifcc@do@ally \cc@do@allyfalse
38 \ExplSyntaxOn
39 \keys_define:nn { cocotex/a11y }
40 {
41   init .code:n = {\global\cc@do@allytrue},

```

`\cca@do@nodetree` if `\relax`, show the node tree in the log and in the shell output.

```
42 \nodetree .code:n = {\let\cca@do@nodetree\relax},
```

`\cca@do@showspaces` if `\relax`, show spaces in the pdf.

```
43 \show-spaces .code:n = {\let\cca@do@showspaces\relax},
```

`\cca@do@dospaces` if `\relax`, add ASCII space characters to the PDF. $\text{\LaTeX}$ doesn't write physical spaces into the output document but moves letters via skips, which allows variable word spacing beyond a font's space width definition, but it is a hard barrier for screen readers which rely on real space characters. This options causes the `ltpdfa` package to insert real space characters that are immediately followed by a negative skip by the font-dependent width of that space to keep $\text{\LaTeX}$'s typeface intact. This is activated by default.

```
44 }
45 \let\cca@do@dospaces\relax
46 \keys_define:nn { cocotex/a11y }
47 {
48   no-spaces .code:n = {\let\cca@do@dospaces\@undefined}
49 }
```

`\cca@debug` if `\relax`, print exhaustive debugging information.

```
50 \keys_define:nn { cocotex/a11y }
51 {
52   debug .code:n = {\let\cca@debug\relax}
53 }
```

`\cca@do@doparas` if `\relax`, add paragraph tagging.

```
54 \let\cca@do@doparas\relax
55 \keys_define:nn { cocotex/a11y }
56 {
57   no-paras .code:n = {\let\cca@do@doparas\@undefined}
58 }
```

Processing the options.

```
59 \ProcessKeyOptions[cocotex/a11y]
60 \ExplSyntaxOff
```

`\cca@patch@error` is a generic error message that is thrown whenever a $\text{\LaTeX}$ kernel macro could not be patched. This is usually the case when the macro definition does not match coco-accessibility's expectation, e.g., when another package messes with the macro's original definition. #1 is the CS token of the un-patchable macro.

```
61 \ifdefined\cc@patch@error
62   \def\cca@patch@error{\cc@patch@error{a11y}}
63 \else
64   \def\cca@patch@error#1{\PackageError{coco-accessibility}
65     {Could not patch \noexpand#1}
66     {You probably use a LaTeX package that re-defines the \noexpand#1 control sequence. It is
        apparently not compatible with CoCoTeX. Sorry}}
67 \fi
```

1.2 Activating and Deactivating Accessibility Features

`\ccIfAll` is a switch to distinct between compilation with (implicit #1) or without (implicit #2) activated accessibility features.

```

68 \def\cc@if@ally{\ifcc@do@ally\expandafter\@firstoftwo\else\expandafter\@secondoftwo\fi}
69 \let\ccIfAlly\cc@if@ally

```

`\ccWhenAlly` is a variant of `\ccIfAlly` that omits the else branch.

```

70 \def\ccWhenAlly{\ifcc@do@ally\expandafter\@firstofone\else\expandafter\@gobble\fi}

```

`\ccUnlessAlly` is a variant of `\ccIfAlly` that omits the then branch.

```

71 \def\ccUnlessAlly{\ifcc@do@ally\expandafter\@gobble\else\expandafter\@firstofone\fi}

```

1.3 Accessibility-specific additions

Loading Further Dependencies

Activated coco-accessibility requires two packages: `luatexbase-attr` (possibly deprecated?) provides an interface to add attributes to lua code; `atveryend` provides a hook to inject code to the final stages of PDF rendering.

```

72 \ccWhenAlly{%
73   \ifluatex\else
74     \ifdefined\ccPackageError
75       \def\@tempa{\PackageError{coco-accessibility}}
76     \else
77       \def\@tempa{\ccPackageError{a11y}{engine}}
78     \fi
79     \@tempa{accessibility features require luatex!}
80     {You tried to use the accessibility features of CoCoTeX with an other TeX engine than
      luatex. This will not work; luatex is a hard requirement. Sorry.}
81   \fi
82   \RequirePackage{luatexbase-attr}
83   \RequirePackage{atveryend}

```

Additional Hyperref Setup

Additional hyperref setup to be executed at the very end of the preamble.

```

84 \AddToHook{env/document/before}{\ifpackageloaded{hyperref}{}{\RequirePackage{hyperref}}}%
85 \AtBeginDocument{%
86   \hypersetup{%
87     % pdfa=true% already set elsewhere
88     ,unicode=true%
89     ,pdfinfo={}%
90     % ,pdfpagelabels=true% already set elsewhere
91     ,pageanchor=true%
92   }%
93   \Hy@pdfatrue
94 }

```

Loading and Configuring ltpdfa's Lua Modules

Now, we set the configuration of the `ltpdfa` lua facility by passing some of the `coco-accessibility` package options:

```

95 \directlua{ltpdfa = require('ltpdfa')}
96 \directlua{ltpdfa.config.final = true}
97 \ifdefined\debug@domain@list

```

```

98 \ifinlist{ltpdfa}\debug@domain@list
99   {\directlua{ltpdfa.config.debug = true}}
100   {\directlua{ltpdfa.config.debug = false}}
101 \else
102   \directlua{ltpdfa.config.debug = \ifx\cca@debug\relax true\else false\fi}%
103 \fi
104 \directlua{ltpdfa.config.nodetree = \ifx\cca@do@nodetree\relax true\else false\fi}
105 \directlua{ltpdfa.config.showspaces = \ifx\cca@do@showspaces\relax true\else false\fi}
106 \directlua{ltpdfa.config.dospaces = \ifx\cca@do@dospaces\relax true\else false\fi}
107 \directlua{ltpdfa.config.doparas = \ifx\cca@do@doparas\relax true\else false\fi}

```

We implement our own auto-close mechanism to end document partitions, so we disable ltpdfa's native one:

```

108 \directlua{ltpdfa.tagger.doautoclose = false}

```

CoCoTeX with accessibility support is `\luaTeX` only, so we hard-code `pdfTeX` as render engine:

```

109 \directlua{ltpdfa.config.driver = "\luaescapestring{pdfTeX}"}

```

`\cca@lang@id` is the IETF language tag for the document's main language.

```

110 \AtBeginDocument{%
111   \@ifpackageloaded{babel}{\edef\cca@lang@id{\localeinfo*{tag.bcp47}}}% tag.bcp47
112   \directlua{ltpdfa.config.lang = '\luaescapestring{\cca@lang@id}'}%
113   }\directlua{ltpdfa.config.lang = '\luaescapestring{en}'}%
114   \directlua{ltpdfa.init()}%
115 }%

```

Initial setup of ltpdfa

```

116 \edef\@ltpdfa@pattr{\directlua{ltpdfa.getAttribute('\luaescapestring{parentattr}')}}
117 \edef\@ltpdfa@tattr{\directlua{ltpdfa.getAttribute('\luaescapestring{typeattr}')}}
118 \attributedef\@ltpdfa@typeattr=\@ltpdfa@tattr
119 \attributedef\@ltpdfa@parentattr=\@ltpdfa@pattr
120 \def\ltpdfa@last@page{\ifx\r@LTLastPage\undefined\@empty\else\expandafter\@secondoftwo\r@LTLastPage\fi}%

```

We need the absolute last page of the document

```

121 \AfterLastShipout{\immediate\write\@mainaux{\string\newlabel{LTLastPage}{\LTLastPage}{\directlua{ltpdfa.getPageNum()}}}}%
122 }%/ccWhenAlly

```

1.4 Generic Macro to Declare Accessibility Features

In order to selectively enable and (temporarily) disable accessibility macros during runtime, we need each tagging markup macro to exist in three states: one where they trigger tagging into the pdf, one where they do nothing, and one where they can be rescued for later expansion.

The enabled, disabled and protected versions of each macro are stored inside three separate lists:

`\cca@relaxed@defs` is the list that stores the *disabled* ltpdfa interface command variants,

```

123 \def\cca@relaxed@defs{}

```

`\cca@saved@defs` is a list that stores the *enabled* ltpdfa interface command variants.

```

124 \def\cca@saved@defs{}

```


`\cca@protected@defs` stores the protected macros where they do nothing when called, but also are not removed from the token list.

```
125 \def\cca@protected@defs{}
```

The next three macros are used to disable, enable, and protect accessibility markup locally, iff accessibility features are globally activated:

`\ccaDisable` disables all ltpdfa commands

```
126 \def\ccaDisable{\ccWhenAlly{\cca@relaxed@defs}}
```

and

`\ccaEnable` enables all ltpdfa commands.

```
127 \def\ccaEnable{\ccWhenAlly{\cca@saved@defs}}
```

`\ccaProtect` protects accessibility commands such that they expand to itself (useful when material containing tagging macros are stored in a macro using `\protected@edef`.)

```
128 \def\ccaProtect{\ccWhenAlly{\cca@protected@defs}}
```

`\CsToStr` is a helper macro which returns the name of a control sequence #1.

```
129 \ExplSyntaxOn
130 \newcommand{\CsToStr}[1]{\cs_to_str:N #1}
131 \ExplSyntaxOff
```

`\DeclareAccessibilityCommand` is the wrapper for our interface macros. It has the same argument signature as L^AT_EX's `\newcommand*`, albeit without the whole checking for already defined control sequences.

```
132 \def\DeclareAccessibilityCommand#1{\@ifnextchar[{\cca@declare@cmd@firstopt#1}{\cca@declare@cmd
#1}}%
```

First, we need to take care of the optional arguments:

`\cca@temp@signature` is the temporary storage for the argument signature.

```
133 \let\cca@temp@signature\@empty
```

`\cca@declare@cmd@firstopt` is the handler for the first optional argument, which holds the overall number of the arguments of our interface macro:

```
134 \def\cca@declare@cmd@firstopt#1[#2]{\edef\cca@temp@signature{[\unexpanded{#2}]}%
135 \@ifnextchar[{\cca@declare@cmd@secopt#1}{\cca@declare@cmd#1}}%
```

`\cca@declare@cmd@secopt` is the handler for the second optional argument, which indicates that the first of the first-level arguments is optional and which itself holds the default value for that optional argument. Its unexpanded value is added to the argument signature.

```
136 \def\cca@declare@cmd@secopt#1[#2]{\eappto\cca@temp@signature{[\unexpanded{#2}]} \cca@declare@cmd
#1}
```

`\cca@declare@cmd` , eventually, is the actual wrapper for the newcommand calls.

```
137 \def\cca@declare@cmd#1#2{%
```

First, we create a string `\savedDef` that includes the *active* definition of our interface macro and store it in an internal macro named `\cc@saved@#1`. This macro is immediately called.

```
138 \edef\savedDef{\noexpand\newcommand*\expandafter\noexpand\csname cc@saved@\CsToStr{#1}\endcsname\expandonce{\cca@temp@signature}{\unexpanded{#2}}}\savedDef%
```

Then, we create a `\let` sequence that maps the plain CS name `#1` onto that newly created internal macro. The String containing the let-sequence is then stored in the `\cca@saved@defs` list, so whenever this list is expanded, the desired CS-token “`#1`” is defined to the active definition.

```
139 \edef\x{\noexpand\let\noexpand#1\expandafter\noexpand\csname cc@saved@\CsToStr{#1}\endcsname}%
140 \global\expandafter\appto\expandafter\cca@saved@defs\expandafter{x}%
```

Then, we repeat the same procedure, but this time, we define the whole internal CS token with the same argument structure to expand to `\relax`.

```
141 \edef\relaxDef{\noexpand\newcommand*\expandafter\noexpand\csname cc@no@\CsToStr{#1}\endcsname\expandonce{\cca@temp@signature}{\relax}}\relaxDef%
```

The whole `\let` sequence for the `\relax` version of our internal macro is then stored in the `\cca@relaxed@defs` list.

```
142 \edef\y{\noexpand\let\noexpand#1\expandafter\noexpand\csname cc@no@\CsToStr{#1}\endcsname}%
143 \global\expandafter\appto\expandafter\cca@relaxed@defs\expandafter{y}%
```

Now, we can decide which of the two `\let`-sequences should be the used to define the initial value of the `#1` CS token, depending on the value of the `\ccIfAlly` conditional:

```
144 \ccIfAlly{x}{y}%
```

Eventually, we define the protected version of the CS token `{#1}` which expands to itself, and thus is protected from expansion inside `\edef`, et al.

```
145 \edef\z{\noexpand\def\noexpand#1{\noexpand\protect\noexpand#1}}%
146 \global\expandafter\appto\expandafter\cca@protected@defs\expandafter{z}%
```

Finally, we reset the macro that contained the argument signature.

```
147 \let\cca@temp@signature\@empty
148 }
```

Some macros from `ltpdfa.sty`:

```
149 \DeclareAccessibilityCommand{\ccaAddToConfig}[2]{\directlua{ltpdfa.addToConfig('\luaescapestring{#1}', '\luaescapestring{#2}')}}
150 \DeclareAccessibilityCommand{\ccaAddKeyword}[1]{\directlua{ltpdfa.tagger.addToKeywords('\luaescapestring{#1}')}}
151 \DeclareAccessibilityCommand{\ccaAddAuthor}[1]{\directlua{ltpdfa.tagger.addToAuthors('\luaescapestring{#1}')}}
152 \@onlypreamble\ccaAddToConfig
```

`\ccaStructStart` inserts a structural tag with the name `#2`. Optional `#1` is the name of a forced parent.

This tagging macro inserts `\bgroup` at the start of the tagged area.

```
153 \DeclareAccessibilityCommand{\ccaStructStart}[2][\ifcc@is@final\directlua{ltpdfa.tagger.structStart('\luaescapestring{#2}', '\luaescapestring{#1}'))}\fi}
```

`\ccaStructEnd` inserts the an `\egroup` and an end tag with the name #1.

```
154 \DeclareAccessibilityCommand{\ccaStructEnd}[1]{\ifcc@is@final\directlua{ltpdfa.tagger.structEnd
    ('\luaescapestring{#1}')}\fi}
```

`\ccaVstructStart` is the same as `\ccaStructStart`, but without inserting a group at the beginning of the tagging area.

```
155 \DeclareAccessibilityCommand{\ccaVstructStart}[2][]{\ifcc@is@final\directlua{ltpdfa.tagger.
    vstructStart('\luaescapestring{#2}','\luaescapestring{#1}')}\fi}
```

`\ccaVstructEnd` ends an ungrouped tagging area. #1 is the name of the tag.

```
156 \DeclareAccessibilityCommand{\ccaVstructEnd}[1]{\ifcc@is@final\directlua{ltpdfa.tagger.
    vstructEnd('\luaescapestring{#1}')}\fi}
```

`\ccaPstructStart` is the same as `\ccaStructStart` but no grouping and no setting of any attributes applies. Implies that the element has no content children, at all.

```
157 \DeclareAccessibilityCommand{\ccaPstructStart}[2][]{\directlua{ltpdfa.tagger.pstructStart('\
    luaescapestring{#2}','\luaescapestring{#1}')}}
```

`\ccaPstructEnd` ends an unattributed tagging area.

```
158 \DeclareAccessibilityCommand{\ccaPstructEnd}[1]{\directlua{ltpdfa.tagger.pstructEnd('\
    luaescapestring{#1}')}}
```

`\ccaGetCurStruct` returns the internal ID of the currently open structural element. #1 is table attribute that should be returned. The following code gives an example on how to use the macro:

```
\ccaStructStart{Leela}
\edef\LeelaID{\ccaGetCurStruct{idx}}%
\ccaStructEnd{Leela}
```

This stores the internal node index of the `Leela` tag node in the `\LeelaID` macro so it can be referenced by other lua interface macros like `\ccaAddToStruct` or `\ccaMoveStruct`, as shown below.

```
159 \DeclareAccessibilityCommand{\ccaGetCurStruct}[1]{\directlua{ltpdfa.tagger.getCurrentStruct('\
    luaescapestring{#1}')}}
160 \DeclareAccessibilityCommand{\ccaSaveCurStruct}[1]{\protected@csxdef{#1}{\ccaGetCurStruct{idx}}}
```

`\ccaAddToStruct` adds the current structural element to the structural element #1 previously retrieved using `\ccaGetCurStruct`, e.g.,

```
% \ccaStructStart{Fry}
% \edef\FryID{\ccaGetCurStruct{idx}}%
% \ccaStructEnd{Fry}
% \ccaStructStart{Hubert}
% \ccaAddToStruct{\FryID}%
% \ccaStructEnd{Hubert}
```

makes `Hubert` into a child node of `Fry` and detaches it from its current parent node (which, in this case, is also the current parent of `Foo`). Note that the parent has to be tagged *before* the child node.

```
161 \DeclareAccessibilityCommand{\ccaAddToStruct}[1]{\ifdef{#1}{\directlua{ltpdfa.tagger.addToStruct
    ('\luaescapestring{#1}')}}{}}
```

`\ccaMoveStruct` removes the Node with the ID #1 from its current parent and attaches it as child to the current node. `\ccaMoveStruct` is the logical counter-part of above's `\ccaAddToStruct`. The child's node ID can be retrieved with the `\ccaGetCurrentStruct` command, for example:

```
\ccaStructStart{Hubert}
  \xdef\HubertID{\ccaGetCurStruct{idx}}
\ccaStructEnd{Hubert}
\structStart{Fry}
  \ccaMoveStruct{\HubertID}
\structEnd{Fry}
```

This will make **Hubert** a child of **Fry**. In contrast to `\ccaAddToStruct`, this allows to attach a previously tagged child node to a later tagged parent node.

```
162 \DeclareAccessibilityCommand{\ccaMoveStruct}[1]{\relax\directlua{ltpdfa.tagger.moveStruct('\luaescapestring{#1}')}}}
```

`\ccaReplaceStruct` takes a previously added tag node with the index #1 and *replaces* it with the current tag node.

```
163 \DeclareAccessibilityCommand{\ccaReplaceStruct}[1]{\relax\directlua{ltpdfa.tagger.replaceStruct
  ('\luaescapestring{#1}')}}}
164 \DeclareAccessibilityCommand{\ccaMoveChildren}[1]{\relax\directlua{ltpdfa.tagger.moveChilds('\luaescapestring{#1}')}}}
```

`\ccaAddID` renames the index attribute of the current tag node to #1. If #1 is “auto”, the index is calculated by `ltpdfa`.

```
165 \DeclareAccessibilityCommand{\ccaAddID}[1]{\directlua{ltpdfa.tagger.addID('\luaescapestring
  {#1}')}}}
```

`\cca@set@docinfo` sets the PDF docinfo. #2 is a key, #3 is the value, optional #1 is an encoding.

```
166 \DeclareAccessibilityCommand{\ccaSetDocinfo}[3][\directlua{ltpdfa.setDocInfo('\luaescapestring
  {#2}', '\luaescapestring{#3}', '\luaescapestring{#1}')}}}
```

`\ccaAddRolemap` is used to map a custom LaTeX tag to a well-defined PDF tag. #1 is the name of the LaTeX Tag, #2 is the name of the PDF role.

```
167 \DeclareAccessibilityCommand{\ccaAddRolemap}[2]{\directlua{ltpdfa.tagger.addRolemap('\luaescapestring{#1}', '\luaescapestring{#2}')}}}
```

`\ccaAddPlacement` tells the tagger if a floating object is placed as a “Block” or “Inline”.

```
168 \DeclareAccessibilityCommand{\ccaAddPlacement}[1]{\directlua{ltpdfa.tagger.addPlacement('\luaescapestring{#1}')}}}
```

`\ccaAddNumbering` ???

```
169 \DeclareAccessibilityCommand{\ccaAddNumbering}[1]{\directlua{ltpdfa.tagger.addNumbering('\luaescapestring{#1}')}}}
```

1.5 Language Tagging

`\ccaAddLang` adds a `/Lang(uage)` attribute to the current node with the value `{#1}`.

```
170 \DeclareAccessibilityCommand{\ccaAddLang}[1]{\directlua{ltpdfa.tagger.setLang('\luaescapestring
    {#1}')}}}
```

The following code patches the unstarred `\foreignlanguage` command in such a way that it automatically tags its content with a `</Span/>` and adds a `/Lang` attribute to the Tag that is collected from the language definition file of the currently activated language.

```
171 \ccWhenAlly{%
172   \AddToHook{babel/*/foreign}{%
173     \pretocmd\BabelText
174       {\ccaVstructStart{Span}\ccaAddLang{\localeinfo{tag.bcp47}}}%
175     }{\cca@patch@error\BabelText}%
176   \apptocmd\BabelText
177     {\ccaVstructEnd{Span}}
178     {\cca@patch@error\BabelText}%
179   }%
180   \AtBeginDocument{\ActivateGenericHook{babel/*/foreign}}%
181 }% /ccWhenAlly
```

1.6 Lua injection

Some features are realized by Lua code, so we tell LuaLaTeX to include the code that is generated from material later in this source file. However, the lua code is currently only needed in conjunction with the CoCoTeX's `title` module.

```
182 \ccWhenAlly{\IfFileExists{coco-accessibility.lua}{\directlua{ally = require('coco-accessibility
    ')}}{}}
```

1.7 Alternative Text Elements

Some structure elements in the PDF/UA standards require alternative texts.

`\ccaAddAltText` is used to add an Alternative Text node, given in `{#1}`, to the PDF structTree.

It is assumed that the input is always UTF-8, so we convert the value in `{#1}` to PDF encoding using the `utf8toPDFenc` method from `ltpdfa`. If the entire string contains only the 256 ASCII characters, the string is written as-is into the PDF. However, if it also contains characters further down the Unicode range, the entire string is converted to hexadecimal UTF-16.

```
183 \DeclareAccessibilityCommand{\ccaAddAltText}[1]{\ifcc@is@final\directlua{ltpdfa.tagger.
    addAltText(ltpdfa.metadata.utf8toPDFenc('\luaescapestring{#1}'))}\fi}
```

`\cca@Gin@alt` is the captured value of the `alt` key from the optional argument of the `\includegraphics` command. This can be used to pass its value to the `\ccaAddAltText` macro defined above.

```
184 \define@key{Gin}{alt}{\gdef\cca@Gin@alt{#1}}
```

1.8 Hyperlink handling

`\ccaAddLastLink` adds the last Link node to the PDF structTree.

```
185 \DeclareAccessibilityCommand{\ccaAddLastLink}{\ifcc@is@final\directlua{ltpdfa.tagger.
    addLastLink()}\fi}
```

`\ccaGetStructParent` returns the current parent structure. This is needed in case a link breaks across columns (or pages).

```
186 \DeclareAccessibilityCommand{\ccaGetStructParent}{\directlua{lua.tagger.getStructParent()}}
```

We prepare the link interface macros to be patched into `hyperref` at the begin document hook if accessibility features are activated.

First we add the start tag for a Link node.

```
187 \begingroup
188 \makeother\#
189 \ccWhenAlly{%
190 \AtBeginDocument{%
191   \patchcmd\Hy@StartlinkName
192     {\pdfstartlink}
193     {\Hy@pstringdef\@argii{#2}\ccaStructStart{Link}\ccaAddAltText{\@argii}\edef\@ltpdfmy@parent
194       {\ccaGetStructParent}%
195     {\pdfstartlink}
196     {}{\cca@patch@error\Hy@StartlinkName}}
```

and the parent node inside the link attribute:

```
196   \patchcmd\Hy@StartlinkName
197     {#1}
198     {#1
199       \ifx\ccaGetStructParent\cc@no\ccaGetStructParent\else
200         /StructParent \@ltpdfmy@parent
201         /Contents(\@argii)
202       \fi
203     }
204     {}{\cca@patch@error\Hy@StartlinkName}}
```

then we patch `hyperref`'s general link macro, twice. Once for the Link's start tag

```
205   \patchcmd\hyper@linkurl
206     {\pdfstartlink}
207     {\Hy@pstringdef\@argii{#2}\ccaStructStart{Link}\ccaAddAltText{\@argii}\edef\@ltpdfmy@parent
208       {\ccaGetStructParent}%
209     {\pdfstartlink}
210     {}{\cca@patch@error\hyper@linkurl}}
```

and secondly for the Parent:

```
210   \patchcmd\hyper@linkurl
211     {/C[\@urlbordercolor]%
212     \fi
213   }
214     {/C[\@urlbordercolor]%
215     \fi
216     \ifx\ccaGetStructParent\cc@no\ccaGetStructParent\else
217       /StructParent \@ltpdfmy@parent%
218       /Contents(\@argii)
219     \fi
220     }{\cca@patch@error\hyper@linkurl}}
```

finally, we patch the end tag for the link node into the `\close@pdflink` macro:

```
221   \patchcmd\close@pdflink
222     {\pdfendlink}
223     {\pdfendlink}
```

```

224 \ccaAddLastLink\ccaStructEnd{Link}}
225 {}{\cca@patch@error\close@pdf@link}

```

For internal references, we patch the tagging into the `\@setref` macro. Unfortunately, `hyperref` redefines this macro and links to both the original version (when `\ref*` is used), and its own re-definition (else), so we need to patch both versions. We start by resetting `\@setref` to its vanilla state and inject our tagging, once for the start tag and a second time for the end tag:

```

226 \let\cca@hy@setref\@setref
227 \let\@setref\real@setref
228 \patchcmd\@setref
229   {\else}
230   {\else\ccaStructStart{Reference}}
231   {}{\cca@patch@error\orig@setref@new}%
232 \patchcmd\@setref
233   {\fi}
234   {\ccaStructEnd{Reference}\fi}
235   {}{\cca@patch@error\orig@setref@new}%

```

Now, we restore `hyperref`'s version and inject the tagging there as well:

```

236 \let\real@setref\@setref
237 \let\@setref\cca@hy@setref
238 \patchcmd\@setref
239   {\expandafter\Hy@setref@link}
240   {\ccaStructStart{Reference}\expandafter\Hy@setref@link}
241   {}{\cca@patch@error\@setref}
242 \patchcmd\@setref
243   {{#2}}
244   {{#2}\ccaStructEnd{Reference}}
245   {}{\cca@patch@error\@setref}
246 }% /AtBeginDocument
247 }% /ccWhenAlly
248 \endgroup

```

1.9 Tagging Page Styles as Artifacts

Page styles, i.e., headers and footers, need to be tagged as artifacts unless they contain semantic information. To avoid inserting the tagging by hand into each publisher's page style definitions, we inject the tagging automatically by using `etoolbox`'s patch commands to insert the start and end tags inside the internal header and footer macros, respectively.

`\ccaPagestyleArtifacts` contains the code to patch the `\@oddhead`, `\@evenhead`, `\@oddfoot` and `\@evenfoot` macros.

```

249 \DeclareAccessibilityCommand{\ccaPagestyleArtifacts}{%
250   \ifx\@oddhead\@empty\else
251     \pretocmd\@oddhead{\ccaStructStart[Document]{header}\ccaDisable}{-}{-}%
252     \apptocmd\@oddhead{\ccaEnable\ccaStructEnd{header}}{-}{-}%
253   \fi
254   \ifx\@evenhead\@empty\else
255     \pretocmd\@evenhead{\ccaStructStart[Document]{header}\ccaDisable}{-}{-}%
256     \apptocmd\@evenhead{\ccaEnable\ccaStructEnd{header}}{-}{-}%
257   \fi
258   \ifx\@oddfoot\@empty\else
259     \pretocmd\@oddfoot{\ccaStructStart[Document]{footer}\ccaDisable}{-}{-}%
260     \apptocmd\@oddfoot{\ccaEnable\ccaStructEnd{footer}}{-}{-}%
261   \fi

```

```

262 \ifx\@evenfoot\@empty\else
263   \pretocmd\@evenfoot{\ccaStructStart [Document]{footer}\ccaDisable}{-}{-}%
264   \apptocmd\@evenfoot{\ccaEnable\ccaStructEnd{footer}}{-}{-}%
265 \fi}

```

The standard pagestyles from the L^AT_EX kernel are patched by the module.

```

266 \AtBeginDocument{%
267   \apptocmd\ps@empty{\ccaPagestyleArtifacts}{-}{-}%
268   \apptocmd\ps@plain{\ccaPagestyleArtifacts}{-}{-}%
269   \apptocmd\ps@headings{\ccaPagestyleArtifacts}{-}{-}%
270   \apptocmd\ps@myheadings{\ccaPagestyleArtifacts}{-}{-}%
271 }

```

Finally, we register the `footer` and `header` PDF tags as `artifacts` with `ltpdfa`:

```

272 \ccWhenAlly{%
273   \ccaAddToConfig{artifact}{header={Type:Pagination}{Subtype:Header}}
274   \ccaAddToConfig{artifact}{footer={Type:Pagination}{Subtype:Footer}}

```

1.10 generic artifacts

```

275 \ccaAddToConfig{artifact}{leaders={Type:Layout}}
276 \ccaAddToConfig{artifact}{footnoterule={Type:Layout}}
277 \ccaAddToConfig{artifact}{Rule={Type:Layout}}
278 \ccaAddToConfig{artifact}{Artifact={Type:Layout}}
279 \ccaAddToConfig{artifact}{Background={Type:Background}}
280 }

```

`\ccaArtifact` starts an Artifact environment within which all Tagging is disabled.

```

281 \def\ccaArtifact{\ccaStructStart [Document]{Artifact}\ccaDisable}

```

`\endccaArtifact` ends an Artifact environment.

```

282 \def\endccaArtifact{\ccaEnable\ccaStructEnd{Artifact}}

```

1.11 Tagging for Floats

Tagging for Figures

`\ccaAddFigure` #1, #2, #3, and #4 are the x and y coordinates of the image, first x and y of the lower left corner, then x and y of the upper right corner; #5 and #6 are the x and y scales, respectively; and #7 is “true” or “false” depending on whether or not the clipping option is active.

```

283 \DeclareAccessibilityCommand{\ccaAddFigure}[7]{\directlua{ltpdfa.tagger.addFigure(
284   '\luaescapestring{#1}',
285   '\luaescapestring{#2}',
286   '\luaescapestring{#3}',
287   '\luaescapestring{#4}',
288   '\luaescapestring{#5}',
289   '\luaescapestring{#6}',
290   '\luaescapestring{#7}')}}}

```


`\ccaFigureStart` injects the starting tag for images to the pdf

```
291 \DeclareAccessibilityCommand{\ccaFigureStart}[1]{\directlua{ltpdfa.tagger.figureStart('\luaescapestring{#1}')}}}
```

`\ccaFigureEnd` injects the ending tag for images

```
292 \DeclareAccessibilityCommand{\ccaFigureEnd}[1]{\directlua{ltpdfa.tagger.figureEnd('\luaescapestring{#1}')}}}
```

which we add to the beginning and the end of `graphics` package's `\Gininclude@graphics` macro, respectively:

```
293 \AtBeginDocument{%
294   \let\ltx@Gininclude@graphics\Gininclude@graphics
295   \def\Gininclude@graphics#1{\ifcc@is@final\ccaFigureStart{}\fi\ltx@Gininclude@graphics{#1}\ifcc@is@final\ccaFigureEnd{}\fi}%
296 }
```

```
297 \apptocmd\Gininclude@pdfTeX{\ifcc@is@final%
298   \def\@tempa{!}%
299   \ccaAddFigure{\Gin@llx}{\Gin@lly}{\Gin@urx}{\Gin@ury}
300   {\ifx\Gin@scalex\@tempa\else \Gin@scalex\fi}
301   {\ifx\Gin@scaley\@tempa\else \Gin@scaley\fi}
302   {\ifGin@clip true\else false\fi}\fi}%rwi/rhi
303   {}{}
304 \AtBeginDocument{%
305   \ifpackageloaded{htmltabs}{%
306     \let\ltx@ht@valign@box\ht@valign@box
307     \def\ht@valign@box{\ifht@final@render\cc@is@finaltrue\fi\ltx@ht@valign@box}
308     \let\ltx@ht@RenderCell\ht@RenderCell
309     \def\ltx@ht@RenderCell{\@cc@is@finalfalse\ltx@ht@RenderCell}}{}}
```

Tagging for Tables

`\ccaAddScope` is used to indicate the scope of in table's head cells. The value should be either `Column` or `Row`.

```
310 \DeclareAccessibilityCommand{\ccaAddScope}[1]{\relax\directlua{ltpdfa.tagger.addScope('\luaescapestring{#1}')}}}
```

`\ccaAddColSpan` is used to mark a cell to span horizontally over #1 columns (including it's own).

```
311 \DeclareAccessibilityCommand{\ccaAddColSpan}[1]{\relax\directlua{ltpdfa.tagger.addColSpan('\luaescapestring{#1}')}}}
```

`\ccaAddRowSpan` is used to mark a cell to span vertically over #1 rows (including it's own).

```
312 \DeclareAccessibilityCommand{\ccaAddRowSpan}[1]{\relax\directlua{ltpdfa.tagger.addRowSpan('\luaescapestring{#1}')}}}
```

`\ccaAddKeep` is inserted into empty Tags to tell the ltpdfa-tagger to not remove the Tag even if it may be empty.

```
313 \DeclareAccessibilityCommand{\ccaAddKeep}{\relax\directlua{ltpdfa.tagger.addKeep()}}
```

1.12 Transformation of Typographic Unicode characters

In order for screen readers to work correctly, some unicode characters that mask purely typographic glyphs (e.g., ligatures) need to be mapped to their underlaying orthographic characters. This is done via pdfTeX's `glyphtounicode` tables:

```
314 \ifx\pdfextension\@undefined\else
315 \protected\def\pdfglyphtounicode{\pdfextension glyphtounicode}
316 \input glyphtounicode
317 \edef\pdfgentounicode{\pdfvariable gentounicode}
318 \pdfgentounicode = 1
319 \fi
```

`\ccaConvertPdfString` takes a `utf-16` string as `{#1}` and converts it to `utf-8`. This is intended to resolve octal tokens in the output of `hyperref`'s `\pdfstringdef`, which `ltpdfa`'s `ltpdfa.setDocInfo()` method does not seem to handle well.

```
320 \DeclareAccessibilityCommand{\ccaConvertPdfString}[1]{\directlua{tex.print(ltpdfa.metadata.utf16
    ToUtf8('\luaescapestring{#1}'))}}
```

1.13 Automatic PDF Tagging

Document Root Node

The following code causes the `ltpdfa` package to tag the `document` environment as the structural representation's root node:

```
321 \ccWhenAlly{%
```

`cca/document/begin` is a hook to add accessibility specific declarations right before begin document.

```
322 \NewHook{cca/document/begin}%
323 \AtBeginDocument{%
324   \directlua{ltpdfa.beginDocument('\luaescapestring{ltpdfa@last@page})}
325   \UseHook{cca/document/begin}%
326   \ccaVstructStart{Document}%
```

`\cca@id@document` is the internal ID of the document root node.

```
327 \ccaSaveCurStruct{cca@id@document}%
```

`\cca@id@document@dummy` is the internal ID of a placeholder tag node inserted immediately after the root tag node is opened. This can be used to be replaced by the document's main title tag node using the `\ccaReplaceStruct` command, which should get this macro as its sole argument. Since `ltpdfa` removes all empty structure elements, if the dummy left unused, it will be removed at the very end.

```
328 \ccaStructStart{dummy}%
329 \ccaSaveCurStruct{cca@id@document@dummy}%
330 \ccaStructEnd{dummy}%
```

`\PDF@FinishDoc` is re-defined to disable `hyperref` filling the PDF's DocumentInfo with empty strings, as they are written by `ltpdfa`, instead.

```
331 \renewcommand\PDF@FinishDoc{%
332   \Hy@DisableOption{pdfauthor}%
```

```

333 \Hy@DisableOption{pdftitle}%
334 \Hy@DisableOption{pdfsubject}%
335 \Hy@DisableOption{pdfcreator}%
336 \Hy@DisableOption{pdfcreationdate}%
337 \Hy@DisableOption{pdfmoddate}%
338 \Hy@DisableOption{pdfproducer}%
339 \Hy@DisableOption{pdfkeywords}%
340 \Hy@DisableOption{pdftrapped}%
341 \Hy@DisableOption{pdfinfo}%
342 }%
343 }%
344 \AtEndDocument{%
345 \ifpackageloaded{coco-headings}{\cchResetNesting}{}%
346 \ifclassloaded{cocotex}{\if@backmatter\ccaVstructEnd{Backmatter}\fi}{}%
347 \ccaVstructEnd{Document}
348 \ifclassloaded{cocotex}{%
349 \ifarticle\else
350 \ccaAddRolemap{Frontmatter}{Part}%
351 \ccaAddRolemap{Mainmatter}{Part}%
352 \ccaAddRolemap{Backmatter}{Part}%
353 \fi
354 }{}%
355 \directlua{ltpdfa.endDocument()}%
356 }%
357 }

```

1.14 Math Tagging

```

358 \AtBeginDocument{%
359 \apptocmd{\({}\}{\ccaStructStart{Formula}\ccaAddPlacement{Inline}}{}{\cca@patch@error\({}
360 \pretocmd{\)}{\ccaStructEnd{Formula}}{}{\cca@patch@error\)}}
361 \apptocmd{\[{}\]{\ccaVstructStart{P}\ccaStructStart{Formula}}{}{\cca@patch@error\[{}
362 \pretocmd{\]}{\ccaStructEnd{Formula}\ccaVstructEnd{P}}{}{\cca@patch@error\]}}
363 \@ifpackageloaded{amsmath}{%
364 \pretocmd{\start@align}{\ccaVstructStart{P}\ccaStructStart{Formula}}{}{\cca@patch@error\
365 start@align}%
366 \apptocmd{\endalign}{\ccaStructEnd{Formula}\ccaVstructEnd{P}}{}{\cca@patch@error\endalign}%
367 }{}%
368 }

```

1.15 Default Role Mapping

Note that this section contains only the role mappings that didn't thematically fit into other CoCoT_EX modules.

```

368 % none so far.

```

Finally, we hook `ltpdfa`'s page processor into `AtBeginShipoutBox`:

```

369 \ccWhenAlly{\AddToHook{shipout/before}{\directlua{ltpdfa.pageprocessor(tex.box["ShipoutBox"])}}}
%

```

End of T_EX source code.

```

</a11y-sty>

```

2 Lua code

```
<*a11y-lua>
```

2.1 Local Variables and Tables

`ltpdfa` is an instance of the `ltpdfa` Lua table.

```
372 local ltpdfa = require('ltpdfa')
```

2.2 Meta Data Extraction

`meta` is a table that holds the metadata that are extracted from the `\jobname.xmp` file via its `extract` member.

```
373 meta = {
374   Author = '',
375   Title = '',
376   Creator = '',
377   Producer = '',
378   Keywords = ''
```

The method `meta.extract()` reads the meta data from the `\jobname.xmp` and stores certain values to be accessed by LaTeX. This is used to fill the DocumentInfo when a xmp file is available during the expansion of `\cct@write@pdf@meta` from the `coco-title` module (see Sect. 2).

```
379 extract = function(self)
380   local xmpfile = ltpdfa.metadata.xmphandler.fromFile(ltpdfa.config.metadata.xmpfile)
381   local f = io.open(xmpfile, "rb")
382   local content = f:read("*all")
383   f:close()
```

First, we extract the document title.

```
384 if (content:find('<dc:title>')) then
385   local title = content:gsub('.*<dc:title>[~<]*<rdf:Alt>[~<]*<rdf:li>[~>]*>(.*)</rdf:li>
386   >[~<]*</rdf:Alt>[~<]*</dc:title>.*', "%1")
387   self.Title = title
388 end
```

Then, we extract the authors from the `dc:creator` list.

```
388 local authors
389 local author = {}
390 if (content:find('<dc:creator>')) then
391   authors = content:gsub('.*<dc:creator>[~<]*<rdf:Seq>(.*)</rdf:Seq>[~<]*</dc:creator>.*', "%1")
392   for k in string.gmatch(authors, "<rdf:li>([~>]+)</rdf:li>") do
393     table.insert(author, k)
394   end
395   self.Author = table.concat(author, '\\and ')
396 end
```

Then, we extract the keywords from the `dc:subject` list. If that doesn't exist, we try to extract the keywords from the `pdf:Keywords` Element, instead.

```

397     local keywords
398     local keyword = {}
399     if (content:find('dc:subject')) then
400         keywords = content:gsub('.*<dc:subject>[~<]*<rdf:Bag>(.)</rdf:Bag>[~<]*</dc:subject>.*', "%1")
401         for k in string.gmatch(keywords, "<rdf:li>([~>]+)</rdf:li>") do
402             table.insert(keyword, k)
403         end
404         self.Keywords = table.concat(keyword, '\\and ')
405     elseif (content:find('pdf:Keywords')) then
406         local keyword = content:gsub('.*<pdf:keywords>(.)</pdf:Keywords>.*', "%1")
407         self.Keywords = keyword
408     end

```

Then, we extract the PDF producer from the `pdf:Producer` element, if it exists:

```

409     if (content:find('<pdf:Producer>')) then
410         local prod = content:gsub('.*<pdf:Producer>(.)</pdf:Producer>.*', "%1")
411         self.Producer = prod
412     end

```

Finally, we extract the PDF CreatorTool from the `xmp:CreatorTool` element, if it exists:

```

413     if (content:find('<xmp:CreatorTool>')) then
414         local creatortool = content:gsub('.*<xmp:CreatorTool>(.)</xmp:CreatorTool>.*', "%1")
415         self.Creator = creatortool
416     end
417 end
418 }

```

2.3 Public Methods

`cocotex` is the base table that contains all public methods and sub-tables available in the CoCoT_EX framework. Here, it is defined unless it is already defined elsewhere.

```

419 if type(cocotex) ~= 'table' then
420     cocotex = {}
421 end

```

`cocotex.ally` is a globally available namespace for coco-accessibility specific lua tables.

```

422 cocotex.ally = {
423     meta = meta
424 }

```

After loading `coco-accessibility.lua` via the `require()` method, a `cocotex.ally` table is returned.

```

425 return cocotex.ally

```

no more lua code.

```

</ally-lua>

```


Module 5

coco-meta.dtx

```
<*meta>
```

This file provides some macros that are used to process meta data, both for the whole document, as well as parts of a document.

File preamble

```
23 %%
24 %% module for CoCoTeX that provides handling of a document's meta data.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-meta}
30   [2025/07/14 v0.6.2 CoCoTeX meta module]
31 \RequirePackage{coco-common}
```

CommonMeta is an abstract Container for commonly used meta data, both for whole documents as well as parts of documents.

```
32 \ccDeclareContainer{CommonMeta}{%
33   \ccDeclareType{Components}{%
34     \ccDeclareRole[author]{Author}%
35     \ccm@declare@comp
36     \ccm@extended@common@macros
37     \ccm@declare@affils
38   }%
39   \ccDeclareType{Properties}{}%
40 }
```

1 Counted Container Handlers

1.1 Generic Blocks

`\ccm@generic@comp` is used to define a generic meta data block. It provides two Components for each instance, one for the block's Heading or Label, and one for its Content.

```
41 \def\ccm@generic@comp{%
```

`CC` is the formatted list of all GenericMeta components.

```
42   \ccDeclareComponent{GenericMetaBlock}{\expandafter\global}{}%
```

```
43   \ccDeclareComponentGroup{GenericMeta}{%
```

`Heading` is the label of a generic meta datum

```
44 \ccDeclareCountedComponent{Heading}%
```

`Heading` is the content of a generic meta datum

```
45 \ccDeclareCountedComponent{Content}%
```

```
46 }}
```

`\ccm@generic@eval` evaluates the Components and tells the Framework how the generic counted Sub-Containers should be rendered.

```
47 \def\ccm@generic@eval{%
48   \def\cc@cur@cont{titlepage}%
49   \ccComposeCollection{GenericMeta}{generic-meta-format}{GenericMetaBlock}
50 }
```

1.2 Contributor Roles

Contributors are Counted Containers that represent the meta data of people that share a role in contributing content to a document. Examples for such roles are an article/chapter/book's authors, or a collection/series' editors.

1.3 Declaring Contributor Role Blocks

`\ccDeclareRoleBlock` is used to create a new Collection Container (named `\ccPrefix#2#3`) for a Role with the name `{#2}`. A Role Block is a Component of the Parent Container, which contains a formatted list of certain Components of all members of the Role. Format and selection of the utilised Components are specified via the Property given in `{#4}`. Role Blocks can also be directly used inside the Parent Container as Overrides.

The optional argument `[#1]` tells the evaluator in the Parent Container's `end` macro how the collector is to be composed. Valid values are `compose` (default), which uses `\ccComposeCollection` to compose the Collection Component, or `apply`, which uses `\ccApplyCollection`, instead.

```
51 \def\ccDeclareRoleBlock{\@ifnextchar[\cc@declare@role@block{\cc@declare@role@block[compose]}}%
52 \def\cc@declare@role@block[#1]#2#3#4{%
53   \ifcsdef{ccm@role@#1}
54     {\ccDeclareComponent{#2#3}{\expandafter\global}{}%
55      \csgdef{ccm@role@\cc@cur@cont @#2@#3}{#4}%
56      \csappto{ccm@role@eval@\cc@cur@cont @#2}
57       {\csname ccm@role@#1\endcsname{#2}{#3}}}
58   {\ccPackageError{Meta}{Argument}
59    {Invalid optional argument in \string\ccDeclareRoleBlock!}
60    {Only 'apply' or 'compose' are allowed as values~^Jin the optional argument of \string\
      ccDeclareRoleBlock!}}}%

```

`\ccm@role@eval` creates the name lists for the role. `{#1}` is the name of the role.

```
61 \def\ccm@role@eval#1{\csname @ccm@role@eval@\cc@cur@cont @#1\endcsname}
```

`\ccm@eval@{#1}` is the name of the macro used to compose the Collection (either `\ccComposeCollection`, or `\ccApplyCollection`)

`{#2}` is the name of the role

`{#3}` is the name of the list.

The access Component is `\ccPrefix #2#3`, i.e., the prefix and both arguments together.


```
62 \def\ccm@eval@role#1#2#3{%
```

First, we check if the Collection Component has already been set in the input. If so, we set an internal flag to indicate that the Collection Component has been filled manually.

```
63 \ccIfComp{#2#3}{\cslet{cc@used@#2#3@override}\@empty}{%
```

Second, we check if the counter for the Role is defined and greater than 0. If neither is the case, this means that the Group does not occur in the input, at all, so we don't need to do anything.

```
64 \ifcsdef{cc#2Cnt}
65 {\expandafter\ifnum\csname cc#2Cnt\endcsname>\z@
```

otherwise, we call the Property that is stored in `\ccm@role@\cc@cur@cont @#2@#3` and store the result in the Component `#2#3`.

```
66 #1{#2}{\csname ccm@role@\cc@cur@cont @#2@#3\endcsname}{#2#3}%
67 \fi
68 }{}}}
```

`\ccm@role@apply` #1 is the name of the role and #2 is the name of the composition. This macro applies (i.e. *fully expands*) the `\ccm@role@\cc@cur@cont @#1@#2` Property and stores the result in the `#1#2` Component.

```
69 \def\ccm@role@apply#1#2{\ccm@eval@role\ccApplyCollection{#1}{#2}}
```

`\ccm@role@compose` #1 is the name of the role and #2 is the name of the composition. This stores the *unexpanded* contents of the `\ccm@role@\cc@cur@cont @#1@#2` Property in the `#1#2` Component.

```
70 \def\ccm@role@compose#1#2{\ccm@eval@role\ccComposeCollection{#1}{#2}}
```

Declaring Contributor Roles

`\ccDeclareRole` is used to declare the Components that belong to each member of a contributor role.

`[#1]` is the internal name of the Role's formatting Property. If omitted, it is the same as `{#2}`

`{#2}` is the name of the role

The output of all members of a role is controlled by a Component called “`<role>NameList`” that is formatted according to the `<role>-format` Property. For reasons of naming conventions, the role names for a Component and its respective Property do not necessarily need to be identical.

```
71 \def\ccDeclareRole{\cc@opt@second\cc@declare@role}%
72 \def\cc@declare@role[#1]#2{%
73 \ccDeclareComponentGroup[%
74 {\ccDeclareAttributeHandler*{corresp}{\ccSetProperty{is-corresp}{true}}}%
75 ]{#2}{%
```

Contributor Role Components

`FullNameOR` is the full name of the Role member. If omitted, it is calculated by the `gear@role-full-name-format` Property.

```
76 \ccDeclareCountedComponent{FullName}%
```

`CiteNameOR` is the Full Role member name that is used for citation advices. If omitted, it is calculated by the `gear@role-cite-name-format` Property.

```
77 \ccDeclareCountedComponent{CiteName}%
```

ShortCiteNameOR is a shortened version of the **CiteName** Component. If omitted, it is calculated by the **role-short-cite-name-format** Property.

```
78 \ccDeclareCountedComponent{ShortCiteName}%
```

PDFInfoNameOR is the version of the Role member name that is used in the PDF Info dictionary. If omitted, it is calculated by the **role-pdfinfo-name-format** Property.

```
79 \ccDeclareCountedComponent{PDFInfoName}%
```

InitialOR holds the initials of the Role member. If omitted, the initials are attempted to be calculated from the **FirstName** and **MidName** Components via the **initials-format** Property.

```
80 \ccDeclareCountedComponent{Initial}%
```

LastName is the surname of the Role member.

```
81 \ccDeclareCountedComponent{LastName}%
```

FirstName is the first name of the Role member.

```
82 \ccDeclareCountedComponent{FirstName}%
```

MidName is/are the middle name(s) of the Role member.

```
83 \ccDeclareCountedComponent{MidName}%
```

Honorific is a other honorific title for the Role member.

```
84 \ccDeclareCountedComponent{Honorific}%
```

Lineage is the name suffix, typically something like “jr.” or “the 3rd”.

```
85 \ccDeclareCountedComponent{Lineage}%
```

ORCID is the ORCID (Open Researcher and Contributor ID) of the Role member. Depending on the publisher style, this can be a full URL or just the identifier.

```
86 \ccDeclareCountedComponent{ORCID}%
```

AffilRef is the ID of an entry in the Affil Component Group.

```
87 \ccDeclareCountedComponent{AffilRef}% for references to the Affil Group
```

Affiliation is the Affiliation of the Role member.

Note that only one **AffilRef** or **Affiliation** should be used for any Role member, not both at the same time.

```
88 \ccDeclareCountedComponent{Affiliation}% for affiliations as direct Author meta data
```

Email is the email/contact address of the Role member.

```
89 \ccDeclareCountedComponent{Email}%
```

`CorrespondenceAsOR` is how the Role member is to be addressed when she is the corresponding Role member. If omitted, it is calculated by the `role-correspondence-as-format` Property.

```
90 \ccDeclareCountedComponent{CorrespondenceAs}%
```

Contributor Role Group Handlers

The Group Handlers fill the previously defined Override Components when they are not explicitly given inside the Component Group.

```
91 }%
92 \ccDeclareGroupHandler{#2}{%
93   \ccUnlessComp{FullName}{\ccComponent{FullName}{\ccUseProperty{#1-full-name-format}}}%
94   \ccUnlessComp{Initial}{\ccComponent{Initial}{\ccUseProperty{initials-format}}}%
95   \ccUnlessComp{CiteName}{\ccComponent{CiteName}{\ccUseProperty{#1-cite-name-format}}}%
96   \ccUnlessComp{ShortCiteName}{\ccComponent{ShortCiteName}{\ccUseProperty{#1-short-cite-name-
97     format}}}%
97   \ccUnlessComp{PDFInfoName}{\ccComponent{PDFInfoName}{\ccUseProperty{#1-pdfinfo-name-format
98     }}}%
98   \ccUnlessComp{CorrespondenceAs}{\ccComponent{CorrespondenceAs}{\ccUseProperty{#1-
99     correspondence-as-format}}}%
99   \ccWhenComp{AffilRef}{\ccWhenComp{Affiliation}{%
100     \ccPackageError{Meta}{Ambiguity}
101     {You cannot use both Containers AffilRef and Affiliation in the same '\ccPrefix#2' Sub-
102       Container}
102     {At least one '\ccPrefix#2' Sub-Container contains both AffilRef and Affiliation. This
103       is not allowed. Please decide for one affiliation strategy: Either two lists with
103       cross-references, or affiliations directly as an author's meta-data.}}}%
103 }%
```

Declaring the Contributor Role's Collection Components

Recall that the Collection Component's name are all prefixed by the Role's name, e.g., the actual Collection Component `➡NameList` of a Role named "Author" is accessed by calling the `➡AuthorNameList` Component.

`NameListCL` is the formatted list of all Role member's `➡FullName` Components according to the `role-block-print-format` Property.

```
104 \ccDeclareRoleBlock{#2}{NameList}{#1-list-print-format}%
```

`CitationListCL` is the formatted list of all Role member's `➡CiteName` according to the `role-block-cite-format` Property.

```
105 \ccDeclareRoleBlock{#2}{CitationList}{#1-list-cite-format}%
```

`ShortCitationListCL` is the formatted list of all Role member's `➡ShortCiteName` Component `role-block-short-cite-format` Property.

```
106 \ccDeclareRoleBlock{#2}{ShortCitationList}{#1-list-short-cite-format}%
```

`PDFInfoCL` is the formatted string that is sent to the PDF's Info dictionary. Its format is determined by the `role-block-pdfinfo-format` Property.

```
107 \ccDeclareRoleBlock[apply]{#2}{PDFInfo}{#1-list-pdfinfo-format}%
```

`\CorrespondenceCL` is the list of all Role member's `\CorrespondanceAs` Components according to the `\role-block-correspondence-format` Property.

```
108 \ccDeclareRoleBlock{#2}{\Correspondence}{#1-\list-correspondence-format}%
109 }
```

`\ccAddToRole` appends another Component declaration block `{#2}` to a pre-defined Role `{#1}`.

TODO
make into LaTeX kernel
hook

```
110 \def\ccAddToRole#1#2{\csgappto{\cc@group@#1@hook}{#2}}
```

2 Labeled Components

Labeled Components are two Components, one for the Content and one for the Label.

`\ccDeclareLabeledComp` declares two Components: one named `\ccPrefix #2` for the value, and another one named `\ccPrefix #2Label` for its corresponding label. `#3` is used for Property overrides. The optional Argument `#1` allows to set a default value for the Label.

```
111 \def\ccDeclareLabeledComp{\cc@opt@empty\cc@declare@labeled@comp}
112 \def\cc@declare@labeled@comp[#1]#2#3{%
113   \ccDeclareComponent{#2}{\expandafter\global}{}}%
114   \ccDeclareComponent{#2Label}{\expandafter\global}{}}%
115   \csxdef{labeled-meta-property-infix-\cc@cur@cont-#2}{#3}%
116   \if!#1!\else
117     \long\csgdef{\cc@cc@cur@cont @#2Label}{#1}%
118   \fi\ignorespaces}
```

`\ccUseLabeledComp` returns the Labeled Component with its label. The starred version omits automatic Tagging if the `coco-accessibility` module is active.

```
119 \def\ccUseLabeledComp{\@ifstar{\global\let\ccm@no@tag\relax\cc@use@labeled@comp}{\
    cc@use@labeled@comp}}
120 \def\cc@use@labeled@comp#1{%
121   \ccWhenComp{#1}{%
```

`\ccCurInfix` stores the currently active Property infix for the Labeled Component. Is is used to call the right format Property for the Labeled Component, which defaults to `\labeled-meta-[\ccCurInfix]-format`. If this Property doesn't exists, formatting falls back to `\labeled-meta-format`.

```
122 \letcs\ccCurInfix{labeled-meta-property-infix-\cc@cur@cont-#1}%
```

`\ccCurComp` stores the currently active name of the Labeled Component, which is used in the generic `\labeled-meta-format` Property.

```
123 \def\ccCurComp{#1}%
```

```
124 \ifx\ccm@no@tag\relax\else
125   \ccaStructStart{MetaDatum}%
126   \ccaAddToStruct{\cch@id@cur@meta}%
127 \fi
```

```

128 \ccIfProp{labeled-meta-\ccCurInfix-format}
129 {\ccUseProperty{labeled-meta-\ccCurInfix-format}}
130 {\ccUseProperty{labeled-meta-format}}%
131 \ifx\ccm@no@tag\relax\else\ccaStructEnd{MetaDatum}\fi
132 }\global\let\ccm@no@tag\@undefined}

```

3 Meta Data Rolemaps for Tagged PDFs

Role mapping for accessibility tagging:

```

133 \ccaAddRolemap{Authors}{P}
134 \ccaAddRolemap{Editors}{P}
135 \ccaAddRolemap{SeriesEditors}{P}
136 \ccaAddRolemap{Affiliations}{P}
137 \ccaAddRolemap{MetaDatum}{Div}
138 \ccaAddRolemap{MetaDatumLabel}{P}
139 \ccaAddRolemap{MetaDatumValue}{P}
140 \ccaAddRolemap{MetaDatumBlock}{Div}
141 \ccaAddRolemap{Abstract}{Div}
142 \ccaAddRolemap{AbstractLabel}{P}
143 \ccaAddRolemap{AbstractText}{Div}
144 \ccaAddRolemap{Keywords}{Div}
145 \ccaAddRolemap{KeywordsLabel}{P}
146 \ccaAddRolemap{KeywordsText}{P}

```

4 Common Meta Data

`\ccm@declare@comp` defines some commonly used meta Components

```

147 \def\ccm@declare@comp{%

```

`Copyright` holds the Copyright notice.

```

148 \ccDeclareComponent{Copyright}{\expandafter\global}{}}% Copyright text

```

`LicenceLogo` is a component for a license logo. This usually contains an `\includegraphics`.

```

149 \ccDeclareComponent{LicenceLogo}{}}%

```

`LicenceName` is the name of the license.

```

150 \ccDeclareComponent{LicenceName}{}}%

```

```

151 }%

```

`article-meta` is an abstract Container that holds meta data specific to a journal's Article.

```

152 %% for single articles
153 \ccDeclareContainer{article-meta}{%
154 \ccDeclareType{Components}{%

```

`StartPage` is the number of the starting page of an article

```
155 \ccDeclareGlobalComponent{StartPage}
```

`EndPage` is the number of the ending page of an article

```
156 \ccDeclareGlobalComponent{EndPage}
```

`CiteAs` holds a string as to how the article should be cited in other publications.

```
157 \ccDeclareLabeledComp[Cite as]{CiteAs}{cite-as}
```

`Submitted` holds the date when the article was submitted to the journal.

```
158 \ccDeclareLabeledComp[Submitted]{Submitted}{submitted}
```

`Received` holds the date when the article was received by the journal

```
159 \ccDeclareLabeledComp[Received]{Received}{received}
```

`Revised` holds the date when the article was revised by its author(s)

```
160 \ccDeclareLabeledComp[Revised]{Revised}{revised}
```

`Reviewed` holds the date when the article was reviewed by the editors or reviewers.

```
161 \ccDeclareLabeledComp[Reviewed]{Reviewed}{reviewed}
```

`Accepted` holds the date when the article was accepted for publication by the journal.

```
162 \ccDeclareLabeledComp[Accepted]{Accepted}{accepted}
```

`Published` holds the date when the article is due to be published.

```
163 \ccDeclareLabeledComp[Published]{Published}{published}
```

`COIStatement` holds the author's Conflict of Interest statement

```
164 \ccDeclareLabeledComp[Conflict of Interest]{COIStatement}{coi-statement}
```

```
165 }}
```

`\ccm@extended@common@macros` provides some extended markup. Some headings use these Components for compilations of contributions by different authors. They are also loaded by article title pages.

```
166 \def\ccm@extended@common@macros{%
```

`Abstract` holds the contribution's abstract or content summary.

```
167 \ccDeclareLabeledComp[Abstract]{Abstract}{abstract}%
```

`Keywords` holds a list of keywords related to the contribution.

```
168 \ccDeclareLabeledComp[Keywords]{Keywords}{keyword}%
```

DOI holds the Digital Object Identifier. Depending on the Publisher style, this may be the full URI, or just the identifier.

```
169 \ccDeclareLabeledComp{DOI}{doi}%
```

TitleEn holds the English title of the publication when the contribution's main language is *not* english.

```
170 \ccDeclareLabeledComp{TitleEn}{title-en}%
```

```
171 \ccm@generic@comp
172 }
```

4.1 Affiliations

`\ccm@declare@affils` is a wrapper that creates the user-level macros for the affiliations.

```
173 \def\ccm@declare@affils{%
```

AffilBlockCC is the Collection Component for the contribution's Affiliations list. **Note** that the [↗AffilBlock](#) itself is not generated in this module. The two modules `coco-headings` and `coco-title` that both depend on the `coco-meta` module have their own mechanisms to build their respective [↗AffilBlock](#) Collection Components.

```
174 \ccDeclareComponent{AffilBlock}{\expandafter\global}{}%
```

```
175 \ccDeclareComponentGroup{Affil}{%
```

AffiliationOR is the fully formatted Affiliation string. If omitted, the Component is built using the [⚙️affiliation-format](#) Property.

```
176 \ccDeclareCountedComponent{Affiliation}%
```

Address is the address where the Role member is working.

```
177 \ccDeclareCountedComponent{Address}%
```

Institute is the name of the university, department or institution where the Role member is working

```
178 \ccDeclareCountedComponent{Institute}%
```

Country is the country where the institution is located in.

```
179 \ccDeclareCountedComponent{Country}%
```

Department is the department where the Role member is working.

```
180 \ccDeclareCountedComponent{Department}%
```

AffilIDOR is the internal identifier that is referenced by the Role member's [↗AffilRef](#) Component. If omitted, the ID is the value of an automatic counter that is incremented by one at the beginning of each Affil Group Container counter in the same Parent Container.

```
181 \ccDeclareCountedComponent{AffilID}%
```

```

182 }%
183 \ccDeclareGroupHandler{Affil}{%
184   \ccUnlessComp{AffilID}{\ccComponentEA{AffilID}{\ccAffilCnt}}}%
185   \ccUnlessComp{Affiliation}{\ccComponent{Affiliation}{\ccUseProperty{affiliation-format}}}%
186 }%
187 }

```

5 Meta Data Properties

```

188 \ccAddToType{Properties}{CommonMeta}{%

```

5.1 Initials

`initials-format` <any> generates an Role member's initials from the ➡`FirstName` and ➡`MidName` Components.

```

189 \ccSetProperty{initials-format}{%
190   \expandafter\ifx\csname cc@\cc@cur@cont @\cc@cnt@grp-FirstName-\the\ccCurCount\endcsname\
      cc@long@empty\else
191   \expandafter\ifx\csname cc@\cc@cur@cont @\cc@cnt@grp-FirstName-\the\ccCurCount\endcsname\
      relax\else
192   \expandafter\expandafter\expandafter\@car\csname cc@\cc@cur@cont @\cc@cnt@grp-FirstName-\
      the\ccCurCount\endcsname\relax\@nil\ccUseProperty{initials-period}%
193   \expandafter\ifx\csname cc@\cc@cur@cont @\cc@cnt@grp-MidName-\the\ccCurCount\endcsname\
      cc@long@empty\else
194   \expandafter\ifx\csname cc@\cc@cur@cont @\cc@cnt@grp-MidName-\the\ccCurCount\endcsname\
      relax\else
195   \ccUseProperty{initials-sep}%
196   \expandafter\expandafter\expandafter\@car\csname cc@\cc@cur@cont @\cc@cnt@grp-MidName-\
      the\ccCurCount\endcsname\relax\@nil\ccUseProperty{initials-period}%
197   \fi\fi
198   \fi\fi
199 }

```

`initials-sep` <any> is the separator between two ➡`Initials`.

```

200 \ccSetProperty{initials-sep}{~}

```

`initials-period` <any> is the symbol that is inserted at the end of each ➡`Initial`.

```

201 \ccSetProperty{initials-period}{.}

```

5.2 Member Role Composition Properties

5.3 Overrides Within a Role Counted Component

The next Properties control how the Component Overrides within a single Role Counted Component are composed.

`role-full-name-format` <any> how the ➡`FullName` Component for each Role member is generated.

```

202 \ccSetProperty{role-full-name-format}{%
203   \if\ccUseComp{Honorific}\relax
204   \else

```



```

205     \ccUseComp{Honorific}\space
206     \fi
207     \ccUseComp{FirstName}\space
208     \if\ccUseComp{MidName}\relax
209     \else
210         \ccUseComp{MidName}\space
211     \fi
212     \ccUseComp{LastName}%
213     \if\ccUseComp{Lineage}\relax
214     \else
215         \space\ccUseComp{Lineage}%
216     \fi%
217 }%

```

`role-cite-name-format` <any> how the [CiteName](#) for each Role member is formatted.

```

218 \ccSetProperty{role-cite-name-format}{\ccIfComp{LastName}{\ccUseComp{LastName},~\ccUseComp{
    Initial}}{\ccUseComp{FullName}}}% How CiteName for each name is built

```

`role-short-cite-name-format` <any> how the [ShortCiteName](#) Component of a Role member is formatted

```

219 \ccSetProperty{role-short-cite-name-format}{\ccUseComp{LastName}}%

```

`role-pdfinfo-name-format` <any> how the [PDFInfoName](#) of a Role member is formatted

```

220 \ccPropertyLet{role-pdfinfo-name-format}{role-cite-name-format}%

```

`role-correspondence-as-format` <any> How the [CorrespondenceAs](#) string of a Role Member is formatted

```

221 \ccSetProperty{role-correspondence-as-format}{\ccUseComp{Email}}%

```

5.4 Format of Single Role Collection Component Items

the next properties control how single items in the Parent container's Collection Components are to be formatted.

`role-block-print-format` <any> How a single entry in the [NameList](#) is formatted.

```

222 \ccSetProperty{role-block-print-format}{\ccUseComp{FullName}\ifnum\ccCurCount<\ccTotalCount\
    ccUseProperty{counted-name-sep}\fi}%

```

`role-block-cite-format` <any> how a single entry in the [CitationList](#) is formatted

```

223 \ccSetProperty{role-block-cite-format}{\ccUseComp{CiteName}\ifnum\ccCurCount<\ccTotalCount\
    ccUseProperty{counted-name-sep}\fi}%

```

`role-block-short-cite-format` <any> how a single entry in the [ShortCitationList](#) is formatted

```

224 \ccSetProperty{role-block-short-cite-format}{\ccUseComp{ShortCiteName}\ifnum\ccCurCount<\
    ccTotalCount\ccUseProperty{counted-name-sep}\fi}%

```

`role-block-pdfinfo-format` <any> how a single entry in the [PDFInfo](#) Component is formatted

```

225 \ccSetProperty{role-block-pdfinfo-format}{\ccUseComp{PDFInfoName}\ifnum\ccCurCount<\
    ccTotalCount\and\fi}% How each item in the Component <Role>PDFInfo is formatted

```

role-block-correspondence-format <any> how a single entry in the ➡Correspondence Component is formatted

```

226 \ccSetProperty{role-block-correspondence-format}{%
227   \ccIfPropVal{is-corresp}{true}
228   {\ifx\is@first@corresp\relax
229     \ccUseProperty{corresp-sep}%
230   \else
231     \global\let\is@first@corresp\relax
232   \fi
233   \ccUseComp{CorrespondenceAs}%
234 }{}}%
```

counted-name-sep <any> how single entries in ➡NameList are separated

```

235 \ccSetProperty{counted-name-sep}{,\space}%
```

name-and <any> is a Property that can be used when composing Role specific Collection Components. Is is usually used between the penultimate and the last entry in the Collection Component.

```

236 \ccSetProperty{name-and}{\space and\space}%
```

name-et al <any> is a Property that can be used when composing Role specific Collection Components. Is is usually used after the first entry in the list, when the total number of entries is too large.

```

237 \ccSetProperty{name-et al}{\space et~al.}%
```

name-sep <any> is the default separator between entries in a Role specific Collection Component.

```

238 \ccSetProperty{name-sep}{,\space}%
```

keywords-sep <any> is the default separator between Entries in the Keywords list.

```

239 \ccSetProperty{keywords-sep}{,\space}%
```

corresp-mark <any> is the default marker for the “Correspondence” Role member, i.e., the Role member who is designated the primary contact person of a contribution.

```

240 \ccSetProperty{corresp-mark}{*}%
```

corresp-sep <any> is the default separator between entries in the ➡Correspondence Collection Component.

```

241 \ccSetProperty{corresp-sep}{,\space}%
```

5.5 Collection Component Properties Specific to Author Role

The Properties defined here are mostly aliases of the more generic Properties defined above.

Author Instance Override Properties

author-cite-name-format <any> how an Author’s ➡CiteName is formatted.

```

242 \ccPropertyLet{author-cite-name-format}{role-cite-name-format}%
```

author-short-cite-name-format <any> how an author’s ➡ShortCiteName is formatted.

```

243 \ccPropertyLet{author-short-cite-name-format}{role-short-cite-name-format}%
```

author-full-name-format <any> how an author's [Full Name](#) Component is composed

```
244 \ccPropertyLet{author-full-name-format} {role-full-name-format}%
```

author-pdfinfo-name-format <any> how an author's [PDFInfoName](#) Component is composed

```
245 \ccPropertyLet{author-pdfinfo-name-format} {role-pdfinfo-name-format}%
```

author-correspondence-as-format <any> how an author's [CorrespondenceAs](#) entry is to be formatted

```
246 \ccPropertyLet{author-correspondence-as-format} {role-correspondence-as-format}%
```

Author-Specific Collection Component Override Properties

author-list-print-format <any> is the format of each entry in the [AuthorNameList](#) Component.

```
247 \ccPropertyLet{author-list-print-format} {role-block-print-format}%
```

author-list-cite-format <any> is the format of each entry in the [AuthorCitationList](#) Component.

```
248 \ccPropertyLet{author-list-cite-format} {role-block-cite-format}%
```

author-list-short-cite-format <any> is the format of each entry in the [AuthorShortCitationList](#) Component.

```
249 \ccPropertyLet{author-list-short-cite-format} {role-block-short-cite-format}%
```

author-list-pdfinfo-format <any> is the format of each entry in the [AuthorPDFInfo](#) Component.

```
250 \ccPropertyLet{author-list-pdfinfo-format} {role-block-pdfinfo-format}%
```

author-list-correspondence-format <any> is the format of the [AuthorContribution](#) Collection Component.

```
251 \ccPropertyLet{author-list-correspondence-format} {role-block-correspondence-format}%
```

5.6 Format of Affiliation Lists

affiliation-format <any> is the format of the [Affiliation](#) Component for each [Affil](#) Instance.

```
252 \ccSetProperty{affiliation-format}{%
253   \ccWhenComp{Institute}{\ccUseComp{Institute}}%
254   \ccWhenComp{Department}{, \ccUseComp{Department}}%
255   \ccWhenComp{Address}{, \ccUseComp{Address}}%
256 }%
```

affil-sep <any> is the separator between the entries of the [AffilBlock](#) Collection Component.

```
257 \ccSetProperty{affil-sep}{\par}
```

affil-block-item-face <any> are the font parameters used to print each entry in the [AffilBlock](#) Collection Component.

```
258 \ccSetProperty{affil-block-item-face}{%}
```

`affil-block-item-format` is the format of each entry in the `↗AffilBlock` list

```

259 \ccSetProperty{affil-block-item-format}{%
260   \textsuperscript{\ccUseComp{AffilID}}%
261   \bgroup
262   \ccUseProperty{affil-block-item-face}%
263   \ccUseComp{Affiliation}
264   \egroup%
265   \ifnum\ccCurCount<\ccTotalCount\relax\ccUseProperty{affil-sep}\fi%
266 }

```

`affil-block-face` <any> is the font used to print the `↗AffilBlock` Collection Component.

```

267 \ccSetProperty{affil-block-face}{\small\normalfont}%

```

`affil-block-format` <any> prints the `↗AffilBlock` Collection Component.

```

268 \ccSetProperty{affil-block-format}{%
269   \ccWhenComp{AffilBlock}
270   {\bgroup
271     \ccUseProperty{affil-block-face}%
272     \ccUseComp{AffilBlock}%
273   \egroup
274   \par
275 }}

```

5.7 Properties for Labeled Componentns

`labeled-meta-format` <any> is the generic Property that determines how Labeled Components are composed. It checks for implicit formatting properties specific to each labeled Component and falls back to generic defaults if those are not defined by the user or publisher style.

```

276 \ccSetProperty{labeled-meta-format}{%
277   \ccIfProp{labeled-meta-before-\ccCurInfix}
278   {\ccUseProperty{labeled-meta-before-\ccCurInfix}}
279   {\ccUseProperty{labeled-meta-before}}%
280   \bgroup
281   \ifx\ccm@no@tag\relax\else\ccaStructStart{MetaDatumLabel}\fi
282   \ccIfProp{labeled-meta-\ccCurInfix-face}
283   {\ccUseProperty{labeled-meta-\ccCurInfix-face}}
284   {\ccUseProperty{labeled-meta-face}}%
285   \ccIfProp{labeled-meta-\ccCurInfix-label-format}
286   {\ccUseProperty{labeled-meta-\ccCurInfix-label-format}}
287   {\ccUseProperty{labeled-meta-label-format}}%
288   \ifx\ccm@no@tag\relax\else\ccaStructEnd{MetaDatumLabel}\fi
289   \ifx\ccm@no@tag\relax\else\ccaStructStart{MetaDatumValue}\fi
290   \ccUseComp{\ccCurComp}%
291   \ifx\ccm@no@tag\relax\else\ccaStructEnd{MetaDatumValue}\fi
292   \egroup
293   \ccIfProp{labeled-meta-after-\ccCurInfix}
294   {\ccUseProperty{labeled-meta-after-\ccCurInfix}}
295   {\ccUseProperty{labeled-meta-after}}%
296 }

```

`labeled-meta-label-format` <any> is the generic format of the label of a Labeled Component.

```

297 \ccSetProperty{labeled-meta-label-format}{%
298   \ccWhenComp{\ccCurComp Label}{%
299     \bgroup

```

```

300 \ccUseProperty{labeled-meta-before-\ccCurInfix-label}%
301 \ccIfProp{labeled-meta-\ccCurInfix-label-face}
302   {\ccUseProperty{labeled-meta-\ccCurInfix-label-face}}
303   {\ccUseProperty{labeled-meta-label-face}}%
304 \ccUseComp{\ccCurComp Label}%
305 \ccIfProp{labeled-meta-\ccCurInfix-label-sep}
306   {\ccUseProperty{labeled-meta-\ccCurInfix-label-sep}}
307   {\ccUseProperty{labeled-meta-label-sep}}%
308 \egroup
309 }

```

labeled-meta-label-face <any> is the font setting for the Label of a Labeled Component.

```

310 \ccSetProperty{labeled-meta-label-face}{\bfseries}

```

labeled-meta-label-sep <any> is the default and fallback separator between the Labeled Component's [Label](#) and its value.

```

311 \ccSetProperty{labeled-meta-label-sep}{:\enskip}

```

labeled-meta-face <any> is the face of a Labeled Component. It applies to both the Label and the Value, but can be locally overridden by the [labeled-meta-label-face](#) Property.

```

312 \ccSetProperty{labeled-meta-face}{}

```

labeled-meta-before <any> is the code expanded before the Labeled Component is printed.

Note that the Property is expanded *outside* the local group of the Labeled Component.

```

313 \ccSetProperty{labeled-meta-before}{}

```

labeled-meta-after <any> is the code expanded after the Labeled Component is printed.

Note that the Property is expanded *outside* the local group of the Labeled Component.

```

314 \ccSetProperty{labeled-meta-after}{\par}

```

```

315 }

```

```

</meta>

```


Module 6

coco-headings.dtx

```
<*headings>
```

This module provides handlers for headings like parts, chapters, sections, or inline headings common to all CoCoTeX projects.

```
23 %%
24 %% module for CoCoTeX that extends heading objects.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-headings}
30 [2025/07/14 v0.6.2 CoCoTeX headings module]
31 \RequirePackage{coco-meta}
```

Headings are handled differently with `cocotex.cls` compared to standard $\text{\LaTeX}$, since cocotex manuscripts tend to have a whole collection of additional information that are pressed into the headings, like subtitles or section authors down to subsection level, etc. Therefore, the `\@startsection` and `\@make[s]chapterhead` facilities from $\text{\LaTeX}$ are no longer sufficient. At the same time, the package does not redefine those macros and keeps them available for backwards compatibility.

First, we load the `bookmark` package:

```
32 \RequirePackage{bookmark}%
```

Since we use our own heading levels, we disable all automatically generated bookmarks.

```
33 \hypersetup{bookmarksdepth=-999}%
```

1 Facility for declaring heading levels and their layouts

`Heading` is an abstract parent Container for headings. It inherits from `CommonMeta`.

```
34 \ccDeclareContainer{Heading}{%
35   \ccInherit{Components,Properties}{CommonMeta}%
36   \ccDeclareType{Parent}{}%
37   \ccDeclareType{Components}{%}
```

We already have the `Author` Component inherited from the `CommonMeta` Container. We therefore just need to declare the overrides.

```
38   \cch@provide@authors%
```

The remaining Components are built as usual.

`Title` is the main title of the heading.

```
39 \cch@provide@comp{Title}%
```

`Subtitle` is an optional second-level title of the heading.

```
40 \cch@provide@comp{Subtitle}%
```

`Number` is the heading's counter.

```
41 \cch@provide@comp{Number}%
```

`RefLabel` is a unique ID of an heading. It is targeted by cross references and replaces L^AT_EX's `\label` command.

```
42 \ccDeclareComponent{RefLabel}{\{}}%
```

```
43 \cch@provide@quotes
44 }%
45 \ccDeclareType{Attributes}{%
```

`class <string>` is the style class of the heading.

```
46 \ccDeclareAttributeHandler{class}{%
47   \let\cch@current@class\ccAttrVal
48   \expandafter\ccUseStyleClass\expandafter{\ccAttrVal}{Heading}%
49 }%
```

`notag` is a flag that if set tags the entire heading and its Components as Artifacts. Content after the heading is *not* tagged as its own `<Sect/>`, but belongs to the `<Sect/>` of the last, non-flagged, heading.

```
50 \ccDeclareAttributeHandler*{notag}{%
51   \let\ccaEnable\relax
52   \let\ccaProtect\ccaDisable
53   \global\let\cch@notag\relax
54   \global\advance\cch@total@nesting@level\m@ne\relax
55   \ccaArtifact\ccaDisable
56 }%
```

```
57 }
58 \ccDeclareType{Properties}{}%
59 \ccDeclareEnv{\cch@heading}{\cch@end@heading}%
60 }
```

`\ccDeclareHeading` is the user-level macro to declare new headings. There also exists a *starred* version of this macro, which exempts the declared heading from auto-tagging.

- #1 (optional) inherit-from: load all properties from that heading level, first.
- #2 level: used for toc entries. -1 for part, 0 for chapter, 1 for section, etc.
- #3 name: part, chapter, section, etc, to be used in toc, head lines, bookmarks, etc.
- #4 Property definitions and switches

```
61 \long\def\ccDeclareHeading{\global\let\cch@star@hdg\@undefined\ifstar{\global\let\cch@star@hdg\
62   relax\cc@declare@heading}{\cc@declare@heading}}
63 \long\def\cc@declare@heading{\cc@opt@empty\cc@declare@heading}
64 \long\def\cc@declare@heading[#1]#2#3#4{%
```


First, we check if the heading has already been declared.

```
64 \ifcsdef{cc@container@#3}{%
```

If yes, then we check if the new declaration's parameters match with the pre-existing one. We start with the heading level.

```
65 \ccPackageInfo{Headings}{}{Appending to `#3'}%
66 \ifcsstring{cch@#3@level}{#2}{}%
67 \ccPackageError{Headings}
68 {Level Mismatch}
69 {Level of heading `#3' cannot be altered!}
70 {The already existing heading `#3' has toc level `\csname cch@#3@level\endcsname', but
71 your^^J%
72 re-declaration states `#2'.^^J%
73 ^^J%
74 Consider declaring a new heading altogether with `#3' as parent,^^J%
75 or add Properties to `#3' using \string\ccAddToType\string{Properties}\string{#3}\string}.}%
76 }
```

we also check the parent.

```
76 \if!#1!\else
77 \ifcsstring{cc@parent@#3}{#1}{}%
78 \ccPackageError{Headings}
79 {Parent Mismatch}
80 {Parent of heading `#3' cannot be altered!}
81 {The already existing heading `#3' inherits from `\csname cc@parent@#3\endcsname',^^J%
82 but your re-declaration sets Parent to `#1'.^^J%
83 ^^J%
84 Consider declaring a new heading altogether with `#1' as parent.}%
85 }%
86 \fi
```

and finally pass the new Properties to the existing heading.

```
87 \ccAddToType{Properties}{#3}{#4}%
```

Finally, we need to re-define the `\ccUseHeading` macro so that changes to the heading's Property list will be taken into account for all dependend constructions like list-ofs and toc-entries.

```
88 \cch@declare@heading{#2}{#3}%
89 }{% \ifcsdef cc@container@#3 else
```

If the heading does not already exist, we build a new one.

Each new heading constitutes its own Sub-Container of the heading Container. The name of this Sub-Container is the headings name.

```
90 \ccDeclareContainer{#3}{%
```

`\cch@#3@level` stores the numeric heading level for the heading

```
91 \csgdef{cch@#3@level}{#2}%
```

```
92 \ccPackageInfo{Headings}{}{Declaring heading `#3'}%
93 \edef\@argi{#1}%
94 \ccDeclareType{Parent}{\cch@create@parent{#1}{#3}}
```

We inherit everything from the heading levels parent, or from the default heading if no parent is present.

```

95 \ifx\@argi\@empty
96 \ccInherit{Components,Properties}{Heading}%
97 \else
98 \ccInherit{Components,Properties,Parent}{#1}%
99 \fi

```

The main body of the heading Declaration is a list of Property definitions which we append to the Sub-Container's "Property" Type.

```

100 \ccDeclareType{Properties}{%
101 #4%
102 }%

```

For each heading we declare some common macros like the ToC entry handlers, the heading's counters and its hooks.

```

103 \ccDeclareType{Init}{%
104 \cch@init@hooks{#3}%
105 \let\@cch@cur@cont\cc@cur@cont
106 \def\cc@cur@cont{Heading}%
107 \cc@init@l@{toc}{#2}{#3}%
108 \let\cc@cur@cont\@cch@cur@cont
109 \cch@init@cnt{#3}%
110 }%

```

Unlike other Sub-Containers, headings form no own L^AT_EX environment. Instead, headings are specifications of one common `\ccPrefix Heading` environment. Is is outsourced into the internal `\cch@declare@heading` macro, which is defined below.

The reason for that is that we don't want to define versions of the same property macros for each and every single heading level. Instead, we locally re-define the general low-level macros that represent the heading's properties for each instance of the generalised `Heading` container.

```

111 \cch@declare@heading{#2}{#3}%
112 }% \ccDeclareContainer{#3}
113 \ccWhenAlly{\ccaAddRolemap{#3}{Sect}}%
114 }% \ifcsdef cc@container@#3 fi
115 }% \cc@declare@heading

```

`\cch@create@parent` stores the heading level's name and its parent, if it exists.

```

116 \def\cch@create@parent#1#2{%
117 \def\ccCurSecName{#2}%
118 \if!#1!\else
119 \ccCheckParent{#1}{#2}%
120 \fi%
121 }

```

`\cch@declare@heading` consists of two parts: In the first part, the inheritance mechanism and the initializers for each new heading level are triggered.

#1 is the numeric heading level, #2 is the name of the heading.

```

122 \def\cch@declare@heading#1#2{%
123 \ccEvalType{Parent}%
124 \ccEvalType{Init}%

```

`\ccUseHeading` is defined as second step. It is called at the end of each `\ccPrefix Heading` environment to process the Components within the Container instance. Each heading level has its own “version” of this macro.

```
125 \csgdef{ccUseHeading#2}{%
```

Since heading levels don’t define their own environments, we make sure that `Heading` is the namespace we are working in.

```
126 \ccSetContainer{Heading}%
127 \setpar{\@par}%
```

Properties are stored in macros specific to the current heading Sub-Container, therefore we evaluate the level’s Properties, not those of the `Heading` Container. However, since we made use of the inheritance mechanism earlier, each Sub-Container’s Property list also contains the general `Heading` Property list.

```
128 \def\cchLevel{#1}%
129 \ccEvalType{#2}{Properties}%
```

Processing the author name list (from `coco-meta.sty`).

```
130 \ccm@role@eval{Author}%
131 \ccComposeCollection{Author}{author-contact-block-format}{AuthorContactBlock}%
132 \ccComposeCollection{Affil}{affil-block-item-format}{AffilBlock}%
```

Processing the `Quote` Group Container, if any.

```
133 \ccComposeCollection{Quote}{quote-block-format}{QuoteBlock}%
```

Hyperref related stuff.

```
134 \def\Hy@toclevel{#1}%
```

Call the mechanism to calculate the heading’s counter.

```
135 \cch@auto@number{#1}{#2}%
```

Here, the actual construction of the heading begins.

```
136 \ccUseProperty{heading-par}%
137 \cch@use@hook{before}{#2}%
138 \ccUseProperty{before-heading}%
```

Add vertical space before the heading

```
139 \cch@add@before@skip
```

The counters we calculated earlier and the space needed to render them are evaluated

```
140 \cc@format@number{}{}{#1}%
```

The value of `after-skip` is essential to determine whether the heading is to be displayed as block or inline element. In case, some heading definition omits setting a proper value, we build a fallback.

```
141 \ccIfProp{after-skip}{\expandafter\global\expandafter\@tempskipa\expandafter=
  after-skip}\relax}{\global\@tempskipa=1sp\relax}%
```

```
142 \cch@use@hook{print/before}{#2}%
143 \def\@svsec{%
```

The `heading block` is the composition of all of the heading's Components that are to be printed where the `heading` environment is in the source.

```
144 \ccUseProperty{before-heading-block}%
```

In an earlier incarnation of CoCoTeX, at this point `\ccCreateLabel` was called, which invokes a `\write`, which resets `\lastskip`. After `\ccCreateLabel` was moved to `\cchGenerateToC`, spacing in some publisher styles suddenly broke. So in order to preserve the exact previous behavior, we need to reset the (vertical) `\lastskip` manually while preserving horizontal spacing:

```
145 \ifpackageloaded{coco-xerif}{\@bsphack\kern\z@\@esphack}{}%
146 \leftskip\ccUseProperty{margin-left}%
147 \rightskip\ccUseProperty{margin-right}%
148 \bgroup
```

If Accessibility features are active, we add the start Tag for `<SectMeta>` that contains all meta data belonging to the heading (i.e. all heading Components sans the title, which is made into the head of the section). We immediately retrieve the Tag's ID since we will move it, later.

```
149 \ccWhenAlly{%
150 \ccaStructStart{SectMeta}%
151 \ccaSaveCurStruct{cch@id@cur@meta}%
152 }%
153 \ccUseProperty{heading-block}%
```

This is the default place to generate the entries for the Table of contents and the bookmarks.

If the `heading-block` Property is projected to spread across over more than a single page, the `\cchGenerateToC` macro should be called from inside the Property definition to ensure heading links point to the correct page. The recommended place is right after the main Title is printed.

```
154 \cchGenerateToC
155 \ccIfProp{extended}{\ccUseProperty{extended-heading}}{}%
```

Here, we end the `</SectMeta>` tag.

```
156 \ccWhenAlly{\ccaStructEnd{SectMeta}}{%
157 \egroup%
158 \cchGenerateRun%
159 \ccUseProperty{after-heading-block}%
160 \cch@use@hook{after}{#2}%
161 }%
```

Finally, we decide whether the printable material we stored in `\@svsec` is to be rendered as a block or inline. This is adopted from L^AT_EX's `\@startsection`. The distinction is made by the sign of `after-skip`: a positive value yields a block heading, a negative value yields an inline heading.

```
162 \ifdim\@tempskipa <\z@\relax
163 \cch@make@inline%
164 \else
165 \cch@make@block%
166 \fi
```

This macro is called at the end of the heading environment. In order to deal with possible vertical spaces after the heading, we wait until the group of the heading environment is closed before we actually print the fully composed heading. The definition of `\next` happens in either `\cch@make@inline` or `\cch@make@block`.

```
167 \aftergroup\next%
168 }%
169 }
```

`\cch@use@hook` recursively includes a hook #1 from the heading #2's parent before expanding its own version.

```

170 \def\cch@use@hook#1#2{%
171   \expandafter\ifx\csname cc@parent@#2\endcsname\relax\else
172     \letcs\cch@parent{cc@parent@#2}%
173     \cch@use@hook{#1}{\csname cc@parent@#2\endcsname}%
174   \fi
175   \UseHook{cc/headings/#2/#1}%
176   \ignorespaces}

```

`\cch@add@before@skip` is a routine that determines the skip that is inserted before a heading.

```

177 \def\cch@add@before@skip{%
178   \setlength\@tempskipa{ccUseProperty{before-skip}}%
179   \ifdim\@tempskipa<\z@\relax
180     \def\do@skip{\minusvspace{-\@tempskipa}}%
181   \else
182     \def\do@skip{\addvspace{\@tempskipa}}%
183   \fi%
184   \if@nobreak
185     \everypar{}%
186     \do@skip
187   \else
188     \addpenalty\@secpenalty
189     \do@skip
190   \fi}

```

1.1 Initializers for New Heading Levels

`\cch@init@hooks` initializes the Hooks for heading level #1.

```

191 \def\cch@init@hooks#1{%

```

`cc/headings/[level]/toc/before` is expanded before the ToC entry is printed

```

192   \NewHook{cc/headings/#1/toc/before}%

```

`cc/headings/[level]/toc/after` is expanded after the ToC entry is printed

```

193   \NewHook{cc/headings/#1/toc/after}%

```

`cc/headings/[level]/before` is expanded before the before-heading property called

```

194   \NewHook{cc/headings/#1/before}%

```

`cc/headings/[level]/after` is expanded after the `after-heading-block` property was called.

```

195   \NewHook{cc/headings/#1/after}%

```

`cc/headings/[level]/print/before` is expanded just before `\@svsec` is locally defined.

```

196   \NewHook{cc/headings/#1/print/before}%

```

`cc/headings/[level]/run/after` is expanded after the local `RunTitle` has been generated

```

197   \NewHook{cc/headings/#1/run/after}%

```

`env/Heading/[level]/begin` is the hook that is called at the begin of each defined heading level with the name `[name]`. It is called at the beginning of every `Heading` environment whose mandatory argument matches `[name]` immediately before the Instance's Attributes are evaluated.

```
198 \NewHook{env/\ccPrefix Heading/#1/begin}%
```

If the current heading is derived from a parent, we want the parent's hooks to also apply to the child:

```
199 \ifcsname cc@parent@#1\endcsname
200   \AddToHook{env/\ccPrefix Heading/#1/begin}{\UseHook{env/\ccPrefix Heading/\csname cc@parent@
201     #1\endcsname/begin}}%
202 \fi
```

`\cch@init@cnt` initialises a counter with the name #1 for automatic numbering if it doesn't exist, yet.

```
203 \def\cch@init@cnt#1{\ifcsname c@#1\endcsname\else\@definecounter{#1}\fi}
```

1.2 Initializers for Instances of Heading Levels

`\cch@auto@number` advances the heading counter if the `numbering` Property is set to `auto` and the current heading is not overridden by the `Number` Component. #1 is the numeric level of the heading, #2 is the name of the heading's counter.

```
204 \def\cch@auto@number#1#2{%
205   \ccIfPropVal{numbering}{auto}
206   {\expandafter\ifx\csname c@#2\endcsname\relax\cch@init@cnt{#2}\fi
207   \ccIfAttrIsSet{Heading}{nonumber}{%
208     {\ccIfComp{Number}
209       {}
210       {\ifnum #1>\c@secnumdepth\relax\else
211         \stepcounter{#2}%
212         \edef\@tempa{\csname the#2\endcsname}%
213         \ccComponentEA{Number}{\@tempa}%
214         \fi}}
215   }{}}
```

2 Externalisation of Heading Components

Components of headings may be used far away from the heading itself. Since, by design, Components are defined strictly local within their containers, those external usages demand special treatment.

2.1 Common Stuff

`\cch@set@author@name@list` sets the `#1AuthorNameList` Component.

```
216 \def\cch@set@author@name@list#1{%
```

first, we look if the `Override` was given in the `Heading` Container. If so, we do nothing.

```
217 \ccUnlessComp{#1AuthorNameList}{%
```

If not, we look whether or not the general `AuthorNameList` override was given in the `Heading` Container.

```
218 \ifx\cc@used@AuthorNameList@override\@empty
```

If yes, then we copy its value to `#1AuthorNameList`.

```
219 \ccComponent{#1AuthorNameList}{\cc@Heading@AuthorNameList}%
220 \else
```

Or else, we re-build the `#1AuthorNameList` from the raw `Author` Subcontainers by using the `author-list-print-format` Property.

```
221 \ifnum\ccAuthorCnt>\z@
222 \ccedefFromCountedComp\cch@tempa{Author}{author-list-print-format}%
223 \ifx\cch@tempa\relax\else
224 \ccComponent{#1AuthorNameList}{\cch@tempa}%
225 \fi
226 \fi
227 \fi
228 }}%
```

2.2 Table of Contents Entry

`\cchGenerateToC` is a macro to be used inside of the `heading-block` Property, preferably right after printing the `Title` Component. The macro generates the ToC and Bookmark entries. It is self-deactivating, i.e., it should be used only once in each `heading` Container, as only its first occurrence generates the ToC and BM entries.

```
229 \def\cchGenerateToC{%
230 \ifx\cch@open@toc\relax
231 \ccCreateLabel{\ccCurSecName}% generate a new label for the bookmarks and toc
232 \ccIfPropVal{no-toc}{true}{\cch@make@toc}% ToC entries
233 \if\cc@pdf@std X\else
234 \ccIfPropVal{no-BM}{true}{\cch@make@bookmarks}% Bookmarks
235 \fi
236 \ccUseProperty{toc-hook}%
237 \global\let\cch@open@toc\@undefined
238 \fi
239 }
```

`\cch@make@toc` initializes the creation of a `Heading` instance's entry in the table of contents.

Each entry is in itself treated as a Container. As such, it consists of Components that are written into the `.toc` file.

```
240 \def\cch@make@toc{%
241 \cc@check@empty{Heading}{Title}{Toc}%
242 \cc@check@empty{Heading}{Number}{Toc}%
243 \cc@check@empty{Heading}{Subtitle}{Toc}%
244 \cch@set@author@name@list{Toc}%
245 \ccIfAttrIsSet{Heading}{notoc}{%
246 {\protected@edef\cch@toc@entry{%
247 \ccIfComp{TocTitle}{\string\ccComponent{TocTitle}{\string\ignorespaces\space\expandonce{\cc@Heading@TocTitle}}}{%
248 \ccIfComp{TocNumber}{\string\ccComponent{TocNumber}{\string\ignorespaces\space\expandonce{\cc@Heading@TocNumber}}}{%
249 \ccIfComp{TocAuthorNameList}{\string\ccComponent{TocAuthorNameList}{\string\ignorespaces\space\expandonce{\cc@Heading@TocAuthorNameList}}}{%
250 \ccIfComp{TocSubtitle}{\string\ccComponent{TocSubtitle}{\string\ignorespaces\space\expandonce{\cc@Heading@TocSubtitle}}}{%
251 }%
252 \ccIfProp{toc-level}
```

```

253     {\edef\cch@toc@sec@name{\ccUseProperty{toc-level}}}
254     {\let\cch@toc@sec@name\ccCurSecName}%
255     \protected@write\@auxout
256     {\ccGobble\ccaProtect}%
257     {\string\@writefile{toc}{\protect\ccContentsline{\cch@toc@sec@name}{\cch@toc@entry}{\
258       thepage}{\@currentHref}\protected@file@percent}}\relax
259     \ccCreateContentListEntries{Heading}{\cch@toc@sec@name}{\cch@toc@entry}{\thepage}{\
260       @currentHref}%
261     \ccCreateContentListEntries{\cch@toc@sec@name}{\cch@toc@sec@name}{\cch@toc@entry}{\thepage
262       }{\@currentHref}%
263     \protected@write\@auxout{\ccaEnable}{}%
264   }}

```

`\cc@toc@extract@data` is called within the `\l@<level>` macro to extract the Components for each entry in the .toc file. #1 is the numerical heading level, #2 is the name of the heading level, #3 is the content of the toc entry (which holds the Components), #4 is the page number.

```

262 \def\cc@toc@extract@data#1#2#3#4{%
263   \ccSetContainer{Heading}%
264   \ccEvalType[#2]{Properties}%
265   \ccDeclareComponent{TocPage}{}{}%
266   \ccComponent{TocPage}{\ccUseProperty{toc-page-face}#4}%
267   \ccDeclareComponent{TocTitle}{}{}%
268   \ccDeclareComponent{TocSubtitle}{}{}%
269   \ccDeclareComponent{TocNumber}{}{}%
270   \ccDeclareComponent{TocAuthorNameList}{}{}%
271   \cc@expand@l@contents{#3}{Heading}{Toc}{Title}%%
272   \cc@format@number{toc-}{Toc}{#1}%
273 }

```

`\cc@toc@print@entry` is also called within the `\l@<level>` macro and eventually prints the entry by expanding a `Heading`'s toc-specific Properties.

```

274 \def\cc@toc@print@entry#1{%
275   \bgroup
276   \cch@use@hook{toc/before}{#1}%
277   \ccUseProperty{toc-before-entry}%
278   \ccUseProperty{toc-format}%
279   \cch@use@hook{toc/after}{#1}%
280   \ccUseProperty{toc-after-entry}%
281   \egroup}

```

2.3 Facility to create the running title macros

`\cchGenerateRun` is a user-level macro to generate the running title overrides. It is self-deactivating, meaning it can be used only once per `heading` environment.

```

282 \def\cchGenerateRun{%
283   \ifx\cch@open@run\relax
284     \cch@make@run% Running headers
285     \cch@use@hook{run/after}{\ccCurSecName}%
286     \global\let\cch@open@run\undefined
287   \fi
288 }

```


`\cch@make@run` prepares the Components used to compose the running titles. It checks if the user provides page header specific overrides in the `Heading` instance. If not, it uses the non-specific Components instead, as long as they are not empty.

After all the header-specific Components are set, the heading level specific property `running-heading` is evaluated and passed to the corresponding `\<level>mark` macros iff they exist.

```

289 \def\cch@make@run{%
290   \cc@check@empty{Heading}{Title}{Run}%
291   \cc@check@empty{Heading}{Number}{Run}%
292   \cc@check@empty{Heading}{Subtitle}{Run}%
293   \cch@set@author@name@list{Run}%
294   \ccUseProperty{running-extra}%
295   \ccIfProp{running-level}
296     {\letcs\cch@mark@name{\ccUseProperty{running-level}mark}}
297     {\letcs\cch@mark@name{\ccCurSecName mark}}%
298   \letcs\cch@parent{cc@parent@\ccCurSecName}%
299   \ifx\cch@mark@name\undefined
300     \ifx\cch@parent\relax\else
301       \letcs\cch@mark@name{\cch@parent mark}%
302     \fi
303   \fi
304   \ifx\cch@mark@name\undefined\else
305     \begingroup
306     \ccGobble
307     \protected@edef\@tempa{\csname cc@Heading@running-heading\endcsname}%
308     \expandafter\cch@mark@name\expandafter{\@tempa}%
309     \endgroup
310   \fi
311 }

```

2.4 Facility to create PDF bookmarks

`\cch@make@bookmarks` generates an entry that is directly written as Bookmark into the PDF file. This is done using the `bookmark` package.

```

312 \def\cch@make@bookmarks{%
313   \cc@check@empty{Toc}{Heading}{Title}{BM}%
314   \cc@check@empty{Toc}{Heading}{Number}{BM}%
315   \cc@check@empty{Toc}{Heading}{AuthorNameList}{BM}%
316   \cc@check@empty{Toc}{Heading}{Subtitle}{BM}%
317   \ccIfAttrIsSet{Heading}{noBM}{}
318   {\ccIfProp{bookmark-level}{\edef\Hy@toclevel{\ccUseProperty{bookmark-level}}}{}}%
319   \begingroup
320   \ccGobble
321   \protected@edef\@tempa{\csname cc@Heading@bookmark\endcsname}%
322   \bookmark[level=\Hy@toclevel,dest=\@currentHref]{\expandonce{\@tempa}}%
323   \endgroup
324 }

```

3 Rendering the Headings

3.1 Inline Headings

`\cch@make@inline` Inline headings are stored in a temporary box and expanded after the next (non-heading) paragraph is opened.

```

325 \newbox\cch@inline@sec@box
326 \def\cch@make@inline{%
327   \ccIfProp{after-indent}{\global\@afterindenttrue}{\global\@afterindentfalse}%
328   \global\setbox\cch@inline@sec@box\hbox{\ifvoid\cch@inline@sec@box\else\unhbox\
      cch@inline@sec@box\ccUseProperty{interline-para-sep}\fi\@svsec}%
329   \ccaEnable
330   \@nobreakfalse
331   \global\@noskipsectrue
332   \gdef\next{%
333     \global\everypar{%
334       \if@noskipsec
335         \global\@noskipsecfalse
336         {\setbox\z@lastbox}%
337         \clubpenalty\@M
338         \begingroup
339         \unhbox\cch@inline@sec@box
340         \endgroup
341         \unskip
342         \hskip -\@tempskipa
343       \else
344         \clubpenalty \@clubpenalty
345         \global\setbox\cch@inline@sec@box\box\voidb@x
346         \everypar{}%
347       \fi}%
348   \ignorespaces}}

```

3.2 Block Headings

`\cch@make@block` is used to print block headings.

```

349 \def\cch@make@block{%
350   \@svsec
351   \ccUseProperty{after-heading-par}%
352   \ccIfProp{after-indent}{\global\@afterindenttrue}{\global\@afterindentfalse}%
353   \gdef\next{%
354     \ifdim\parskip>\z@\relax\advance\@tempskipa-\parskip\relax\fi
355     \vskip \@tempskipa
356     \@afterheading
357     \ignorespaces}}

```

4 The Heading environment

4.1 Environment Macros

`\cch@heading` is the macro called at the begin of the `Heading` environment. Optional #1 stores the headings local parameters, #2 is the level of the heading.

```

358 \def\cch@heading{\cc@opt@empty\@cch@heading}%
359 \def\@cch@heading[#1]#2{%

```

Some L^AT_EX kernel macros are saved, the namespace is set and counted groups from previous headings are reset.

```
360 \cch@reserve
```

`\ccCurSecName` stores the name of the current heading level.

```
361 \xdef\ccCurSecName{#2}%
362 \ccEvalAttributes[Heading]{#1}%
```

After opening the environment, if accessibility Features are enabled, we check the current absolute nesting level and close all open Sectioning tags if the nominal level of the current heading is lower than the *nominal* level of the last opened heading. *Nominal* in this case refers the level given to the heading when it was defined in contrast to the actual, *absolute*, nesting level in the tex document.

```
363 \ifx\cch@notag\relax\else
364 \ccWhenAlly{%
```

`\ccPrevSecLevel` is the previously opened, *nominal*, heading level.

```
365 \global\let\ccPrevSecLevel=\ccCurSecLevel\relax
```

`\ccCurSecLevel` stores the nominal level of the current heading.

```
366 \xdef\ccCurSecLevel{\csname cch@#2@level\endcsname}%
```

Now, we call the auto-close mechanism defined below in Sect. 6,

```
367 \cchAutoClose%
368 \ccDebugMsg[a11y]{Level after close: \the\cch@total@nesting@level}%
369 \fi
```

Then, we call the heading level specific `\begin/Heading/[level]/begin` hook.

```
370 \UseHook{env/\ccPrefix Heading/#2/begin}%
```

```
371 \ccEvalType[#2]{Components}%
372 \ignorespaces
373 }
```

`\cch@end@heading` is stuff that happens at the end of the `Heading` environment.

```
374 \def\cch@end@heading{%
375 \expandafter\ifx\csname ccUseHeading\ccCurSecName\endcsname\relax
376 \PackageError{coco-headings.sty}{Heading level \ccCurSecName\space unknown!}{A Heading with
level \ccCurSecName\space is unknown. Use the \string\ccDeclareHeading\space macro to
declare heading levels.}%
377 \else
378 \csname ccUseHeading\ccCurSecName\endcsname%
379 \fi
380 \cch@reset
381 }
```

4.2 Content Handlers

`\cch@reserve` re-directs some of L^AT_EX's kernel macros and makes sure that some other macros have their default values:

```

382 \def\cch@reserve{%
383   \ccSetContainer{Heading}%
384   \let\cch@ltx@dbl@backslash\%
385   \letcs\{\ccPrefix Break}%
386   \let\cc@ltx@label\label%
387   \def\ccAuthorCnt{\z@}%
388   \def\ccAffilCnt{\z@}%
389   \cc@reset@components{\cc@cur@cont}%
390   \let\cch@current@class\relax
391   \global\let\cch@id@cur@meta@undefined
392   \global\let\cch@id@cur@head@undefined
393   \global\let\cch@no@tag@undefined%
394   \global\let\cch@ccaEnable\ccaEnable
395   \global\let\cch@ccaProtect\ccaProtect
396   \global\let\cch@open@toc\relax
397   \global\let\cch@open@run\relax
398   \ignorespaces}

```

`\cch@reset` restores L^AT_EX's default definitions (however, this should be unnecessary since `Heading` is an environment and therefore constitutes a closed group).

```

399 \def\cch@reset{%
400   \let\cc@cur@cont\relax
401   \let\cch@ltx@dbl@backslash
402   \let\label\cc@ltx@label
403   \let\ccCurSecName\relax
404   \global\let\ccaEnable\cch@ccaEnable
405   \global\let\ccaProtect\cch@ccaProtect
406   \ifx\cch@notag\relax\endccaArtifact\fi
407 }

```

`\cch@provide@quotes` covers multiple quotation blocks associated with a heading.

```

408 \def\cch@provide@quotes{%

```

`QuoteBlockCL` is the Collection Component for one or more ➡ `Quote` Component Groups.

```

409   \ccDeclareComponent{QuoteBlock}{-}{-}%

```

`QuoteGC` is a Component Group for quotes that belong to a heading.

```

410   \ccDeclareComponentGroup{Quote}{-}%

```

`QuoteTextCC` is the quotation text

```

411   \ccDeclareCountedComponent{QuoteText}%

```

`QuoteSourceCC` is the source of the quotation.

```

412   \ccDeclareCountedComponent{QuoteSource}%
413 }%
414 }

```

`\cch@provide@authors` sets up the additional Components for the ➡ `Author` Role specific to headings.

```

415 \def\cch@provide@authors{%
416   \ccAddToRole{Author}{-}%

```

`AuthorContactCC` holds the contact information of an author.

```
417 \ccDeclareCountedComponent{AuthorContact}%
418 }%
```

`AuthorContactBlockCL` is the Collection Component for the Counted Component `AuthorContact`.

```
419 \ccDeclareRoleBlock{Author}{ContactBlock}{author-contact-block-format}%
```

```
420 \ccDeclareGroupHandler{Author}{%
421 \ccIfComp{AuthorContact}{-}{\ccComponent{AuthorContact}{\ccUseProperty{author-contact-format
422 }}}{}%
423 }%
```

`AuthorNameListCL` is the Collection Component for the Author names.

```
423 \cc@provide@overrides{AuthorNameList}%
424 }
```

`\cch@provide@comp` is a wrapper that creates the user-level macros for the Component itself and its overrides. #1 is the Component name.

```
425 \def\cch@provide@comp#1{%
426 \ccDeclareComponent{#1}{-}{-}%
427 \cc@provide@overrides{#1}%
428 }
```

`\cc@provide@overrides` declares the Component macros for a Heading Component's overrides. #1 is the Component name. The overrides allow a four-way distinction between *i* the data printed in-situ (`#1`), *ii* data sent to toc (`Toc#1`), (iii) data sent to the page styles (`Run#1`), and (iv) the data sent to the PDF bookmarks (`BM#1`).

```
429 \def\cc@provide@overrides#1{%
430 \ccDeclareComponent{Toc#1}{-}{-}% toc overrides
431 \ccDeclareComponent{Run#1}{-}{-}% running overrides
432 \ccDeclareComponent{BM#1}{-}{-}% bookmark overrides
433 }
```

5 Defaults

```
434 \ccAddToProperties{Heading}{%
```

`interline-para` [`true`|<empty>] is a switch that if non-empty prevents two adjacent inline headings from being set in the same paragraph.

```
435 \ccSetProperty{interline-para}{-}%
```

`interline-para-sep` <any> is the material that is printed between two adjacent inline headings.

```
436 \ccSetProperty{interline-para-sep}{\space}
```

heading-par <any> is the material added to the very beginning of a heading.

```
437 \ccSetProperty{heading-par}{%
438   \ccWhenProp{interline-para}{\if@noskipsec \leavevmode \fi}%
439   \par
440   \global\@afterindenttrue
441 }%
```

after-heading-par <any> is expanded at the very end of non-inline headings.

```
442 \ccSetProperty{after-heading-par}{\par \nobreak}%
```

before-heading <any> is expanded immediately before any vertical skips of a heading are inserted, but after the  **begin-hook**.

```
443 \ccSetProperty{before-heading}{}%
```

title-face <any> is the style of the heading's main title.

```
444 \ccSetProperty{title-face}{\bfseries}%
```

subtitle-face <any> is the style of the heading's subtitle.

```
445 \ccSetProperty{subtitle-face}{\normalfont}%
```

author-face <any> is the face of the heading's printed Author Component.

```
446 \ccSetProperty{author-face}{\normalfont}%
```

quote-face <any> is the style of a quotation.

```
447 \ccSetProperty{quote-face}{\raggedleft}%
```

quote-source-face <any> is the style of a quotation's source line.

```
448 \ccSetProperty{quote-source-face}{}%
```

quote-block-format <any> is the format of a single quotation. By default, it uses the  **QuoteText** and  **QuoteSource** Components.

```
449 \ccSetProperty{quote-block-format}{%
450   \bgroup
451   \ccUseProperty{quote-face}%
452   \ccUseComp{QuoteText}\par
453   \ccIfComp{QuoteSource}{\ccUseProperty{quote-source-face}--\space\ccUseComp{QuoteSource}}\
454   \egroup}
```

heading-block <any> is the format of the main heading. It uses the  **Subtitle**,  **AuthorNameList**,  **QuoteBlock** and  **AffilBlock** Components.

```
455 \ccSetProperty{heading-block}
456   {\ccUseProperty{main-title-format}%
457   \ccaStructStart{Div}%
458   \ccWhenComp{Subtitle}{\ccUseProperty{subtitle-face}\ccaStructStart{P}\ccUseComp{Subtitle}\
459     ccaStructEnd{P}}\par\nobreak}%
459   \ccWhenComp{AuthorNameList}{\ccUseProperty{author-face}\ccaStructStart{P}\ccUseComp{
460     AuthorNameList}\ccaStructEnd{P}}\par\nobreak}%
460   \ccWhenComp{QuoteBlock}{\ccaStructStart{Div}\ccUseComp{QuoteBlock}\ccaStructEnd{Div}}%
461   \ccWhenComp{AffilBlock}{\ccUseProperty{affil-block-face}\ccaStructStart{Div}\ccUseComp{
462     AffilBlock}\ccaStructEnd{Div}}\par}%
```

```

462 \ccaStructEnd{Div}%
463 }%

```

main-title-format <any> is the format of the heading's main title. It should also enclose the heading's ➤ **Number** and ➤ **Title** Components with Tags that are mapped to  <H/> or  <Hn/> with $1 < n < 6$. The number is tagged as  <Lbl/> if present.

```

464 \ccSetProperty{main-title-format}{%
465   \ccUseProperty{title-face}%
466   \cchHeadTagStart
467   \ccIfComp{Number}%
468     {\ccaStructStart{Lbl}\ccUseProperty{hang-number}\ccaStructEnd{Lbl}}%
469     {\leftskipOpt}%
470   \ccUseComp{Title}%
471   \cchHeadTagEnd
472   \par\nobreak
473 }

```

extended-heading <any> is the format of extended headings which incorporates the ➤ **Abstract** and ➤ **Keywords** Labeled Components. Requires the  **extended** Property to be non-empty.

➤ **Abstract** and ➤ **Keywords** are tagged with  <Abstract/> and  <Keyword/>, their labels with  <AbstractLabel/> and  <KeywordsLabel/>, and their values with  <AbstractText/> and  <KeywordsText/>, respectively.

```

474 \ccSetProperty{extended-heading}{%
475   \ccWhenComp{Abstract}{%
476     \par\vskip\baselineskip
477     \ccaStructStart{Abstract}%
478     \bgroup
479       \bfseries
480       \ccaStructStart{AbstractLabel}%
481       \ccIfComp{AbstractLabel}
482         {\ccUseComp{AbstractLabel}}
483         {Abstract}%
484       \ccaStructEnd{AbstractLabel}%
485     \egroup
486     \par\nobreak
487     \bgroup
488       \itshape\small
489       \ccaStructStart{AbstractText}%
490       \ccUseComp{Abstract}%
491       \ccaStructEnd{AbstractText}%
492     \egroup
493     \par
494     \ccaStructEnd{Abstract}%
495   }%
496   \ccWhenComp{Keywords}{%
497     \par\vskip\baselineskip
498     \ccaStructStart{Keywords}%
499     \bgroup
500       \bfseries
501       \ccaStructStart{KeywordsLabel}%
502       \ccIfComp{KeywordsLabel}
503         {\ccUseComp{KeywordsLabel}}
504         {Keywords}%
505       \ccaStructEnd{KeywordsLabel}%
506     \egroup%
507     \par\nobreak
508     \bgroup
509       \itshape\small

```

```

510     \ccaStructStart{KeywordsText}%
511     \ccUseComp{Keywords}%
512     \ccaStructEnd{KeywordsText}%
513     \par%
514     \egroup
515     \ccaStructEnd{Keywords}%
516 }%

```

before-skip <skip> the vertical space before heading. Positive values are set with L^AT_EX's `\addvspace`, while negative values are set with coco-common's `.`

TODO
 values < 0pt use
`\minusvspace`, else
`\addvspace`. L^AT_EX's
 default behaviour of
`\@afterindent` is
 relocated to the
 after-indent property.

```

517 \ccSetProperty{before-skip}{\z@skip}%

```

after-heading-block <any> is expanded at the very end of the printed heading.

```

518 \ccSetProperty{after-heading-block}{}%

```

before-heading-block <any> is expanded at the very beginning of `@svsec`.

```

519 \ccSetProperty{before-heading-block}{\parindent\z@ \parskip\z@}%

```

toc-hook <any> is called after ToC and Bookmark entries are written and allows for material to be added to the toc file.

```

520 \ccSetProperty{toc-hook}{}% Called, after ToC and BM entries have been written to the .aux file

```

after-indent <any> if non-empty, the first paragraph after the heading will be indented.

```

521 \ccSetProperty{after-indent}{}%

```

margin-left [auto|<dimen>|<empty>] is the left margin of the heading. Its value can either be a fixed dimension, the string auto, or empty. If the Property is set to auto or an empty string, the margin is calculated from the  **indent** (see below). Otherwise the fix value is used.

```

522 \ccSetProperty{margin-left}{}%

```

margin-right <skip> is the right margin of the heading block.

```

523 \ccSetProperty{margin-right}{\@flushglue}%

```

after-skip <skip> is the vertical space after the heading block. If the value is greater than or equal to 0pt, the heading is formatted in block, while it is formatted as inline heading if the value is negative.

```

524 \ccSetProperty{after-skip}{1sp}%

```

indent [auto|auto-global|<dimen>] is the offset of the first line of the heading relative to  **margin-left**.

If the value is auto, the indent of the heading is the width of the widest  **Number** Component of *all headings with the same level*.

If the value is auto-global, the indent is the width of the widest Number component across *all heading levels*. Both auto and auto-global require at least two L^AT_EX runs. See Sect. 3.3 in Module Module 3 for more details.


```
525 \ccSetProperty{indent}{auto}%
```

`number-width` <dimen> is the (actual) width of the Number component.

```
526 \ccSetProperty{number-width}{}%
```

`number-sep` <any> Is the separator between the Number and the Title components

```
527 \ccSetProperty{number-sep}{\space}%
```

`number-align` [left|center|right] is the horizontal alignment of the Number component inside its surrounding `\hbox`.

```
528 \ccSetProperty{number-align}{left}%
```

`number-format` <any> is the format of a heading's counter. It prints the [RunTitle](#) component and the `number-sep` Property, and stylizes them both with the `title-face` and `number-face` Properties.

```
529 \ccSetProperty{number-format}{%
530   \bgroup
531   \ccUseProperty{title-face}%
532   \ccUseProperty{number-face}%
533   \ccUseComp{Number}%
534   \ccUseProperty{number-sep}%
535   \egroup}
```

`numbering` [auto|<any>] if non-auto, headings are not numbered automatically if no Number component is given. This property can be overridden in a local instance with the `nonumber` Attribute.

```
536 \ccSetProperty{numbering}{auto}%
```

`running-level` <name> is an override that allows the heading's running title to appear as another level's running title. Usually, the `RunTitle` Component is passed to `\<level>mark` for the page header, but if this Property is non-empty, the heading will be passed to `\<running-level>mark`, instead.

```
537 \ccSetProperty{running-level}{}% override level for running title, name
```

`running-heading` <any> is the format of the material passed to the `\<level>mark` or `\<running-level>mark` command. It uses the [RunTitle](#) and [RunAuthorNameList](#) Components.

```
538 \ccSetProperty{running-heading}{%
539   \ccIfComp{RunAuthorNameList}{\ccUseComp{RunAuthorNameList}:\space}{}%
540   \ccUseComp{RunTitle}%
541 }%
542 %% ToC
```

`no-toc` [true|false] whether or not the heading does *not* create an entry in the table of contents (true means no toc entry, false means toc entry).

```
543 \ccSetProperty{no-toc}{false}%
```

`no-BM` [true|false] whether or not the heading does *not* create a bookmark (true means no bookmark, false means bookmark).

```
544 \ccSetProperty{no-BM}{false}%
```

`toc-margin-top` <skip> vertical space before the ToC entry.

```
545 \ccSetProperty{toc-margin-top}{\z@}%
```

`toc-margin-bottom` <skip> vertical space after the ToC entry.

```
546 \ccSetProperty{toc-margin-bottom}{\z@}%
```

`toc-margin-left` [auto|<dimen>] left margin of the toc entry. See `margin-left` for the meaning of auto.

```
547 \ccSetProperty{toc-margin-left}{auto}%
```

`toc-margin-right` <dimen> right margin of the ToC entry.

```
548 \ccSetProperty{toc-margin-right}{\@pnumwidth}%
```

`toc-title-face` <any> style of the title in the ToC entry.

```
549 \ccSetProperty{toc-title-face}{}%
```

`toc-indent` [auto|auto-global|<dimen>] offset of the ToC entry's first line relative to margin-left. See `indent`.

```
550 \ccSetProperty{toc-indent}{auto}%
```

`toc-number-width` <dimen> the actual width of the TocNumber Component.

```
551 \ccSetProperty{toc-number-width}{}%
```

`toc-number-align` [left|center|right] the alignment of the TocNumber within the surrounding `\hbox`.

```
552 \ccSetProperty{toc-number-align}{left}%
```

`toc-number-face` <any> style of the TocNumber component.

```
553 \ccPropertyLet{toc-number-face}{toc-title-face}%
```

`toc-number-sep` <any> separator between the `TocNumber` and `TocTitle` Components

```
554 \ccSetProperty{toc-number-sep}{\enskip}%
```

`toc-number-format` <any> is the format of the ➡ `TocNumber` Component, using the ⚙ `toc-number-face` and ⚙ `toc-number-sep` Properties.

```
555 \ccSetProperty{toc-number-format}{%
556   \bgroup
557   \ccUseProperty{toc-number-face}%
558   \ccUseComp{TocNumber}%
559   \ccUseProperty{toc-number-sep}%
560   \egroup}
```

`toc-page-sep` <any> separator between the `TocTitle` and the page counter. The dotted line is tagged as ➡ `<leaders/>`, which is mapped to be an artifact of type ➡ `<leaders/>`.

```
561 \ccSetProperty{toc-page-sep}{\ccaAddKeep\ccaStructStart[Document]{leaders}\dotfill\
    ccaStructEnd{leaders}}%
```

`toc-page-face` <any> style of the page counter

```
562 \ccSetProperty{toc-page-face}{}%
```

toc-page-format <any> format of the page counter using the **toc-page-sep** and **toc-page-face** Properties. The number itself is tagged as a ****.

```
563 \ccSetProperty{toc-page-format}{%
564   \ccUseProperty{toc-page-sep}%
565   \bgroup
566   \ccUseProperty{toc-page-face}%
567   \ccaStructStart{Span}\ccUseComp{TocPage}\ccaStructEnd{Span}%
568   \egroup}%%
```

toc-level <name> name of another heading level as which the ToC entry should be rendered.

```
569 \ccSetProperty{toc-level}{}%
```

toc-before-entry <any> is expanded before any ToC entry is rendered. Should setup margins, alignment, line-breaking rules, etc.

```
570 \ccSetProperty{toc-before-entry}{%
571   \addvspace{\ccUseProperty{toc-margin-top}}%
572   \parindent \z@
573   \let\\\@centercr
574   \hyphenpenalty=\@M
575   \rightskip \ccUseProperty{toc-margin-right} \@plus 1fil\relax
576   \parfillskip -\rightskip
577   \leftskip \ccUseProperty{toc-margin-left}%
578   }%
```

toc-after-entry <any> is expanded at the very end of a ToC entry. By default, it sets the skip after the entry to **toc-margin-bottom**.

```
579 \ccSetProperty{toc-after-entry}{\par\addvspace{\ccUseProperty{toc-margin-bottom}}}%
```

toc-format <any> format of the ToC entry itself. It uses the **toc-title-face**, **toc-hang-number** and **toc-page-format** Properties to print the **TocNumber**, **TocAuthorNameList**, **TocTitle**, and **TocPage** Components. Tagging should incorporate the **<TOCI/>**, as well as for numbered ToC entries the **<P/>** and **<Reference/>** tags that span the entire entry, as well as **<Lb1/>** for the **TocNumber**, and **** for the remaining Components of the entry. Unnumbered Entries are simply **<TOCI/>** with **<Spans/>** for the various Components.

```
580 \ccSetProperty{toc-format}{%
581   \ccUseProperty{toc-title-face}%
582   \ccaVstructStart{TOCI}%
583   \ccWhenComp{TocNumber}{\ccaStructStart{P}\ccaStructStart{Reference}}%
584   \ccTocLink{%
585     \ccIfComp{TocNumber}
586       {\ccaStructStart{Lb1}\ccUseProperty{toc-hang-number}\ccaStructEnd{Lb1}}
587       {\leftskipOpt\leavevmode}%
588     \ccWhenComp{TocAuthorNameList}{\ccaStructStart{Span}\ccUseComp{TocAuthorNameList}:\space\
589       ccaStructEnd{Span}}%
589     \ccaStructStart{Span}\ccUseComp{TocTitle}\ccaStructEnd{Span}%
590     \ccUseProperty{toc-page-format}%
591   }%
592   \ccWhenComp{TocNumber}{\ccaStructEnd{Reference}\ccaStructEnd{P}}%
593   \ccaVstructEnd{TOCI}%
594   }%
```

bookmark-level <num> number(!) of the heading level as which the Bookmark entry should be rendered.

```
595 \ccSetProperty{bookmark-level}{}%
```

`bookmark <any>` is the format of the bookmark, which by default is built only from the `BMNumber` and `BMTITLE` Components.

```
596 \ccSetProperty{bookmark}{%
597   \ccIfComp{BMNumber}{\ccUseComp{BMNumber}\space}{}%
598   \ccUseComp{BMTITLE}%
599 }%
```

`orcid-link <any>` how an `ORCID` link is rendered.

```
600 \ccSetProperty{orcid-link}{%
601   \ccIfComp{ORCID}{\ccCompLink{ORCID}{\includegraphics[height=1em]{logos/ORCID.pdf}}}{}%
602 }%
```

`author-contact-format <any>` how a single Author Component's contact information should be rendered. By default, it uses the Author's `FullName`, the value of the `AffilRef` component as superscript, and the `orcid-link` Property.

```
603 %% a single Author's contact information block
604 \ccSetProperty{author-contact-format}{%
605   \ccUseComp{FullName}\ccWhenComp{RefAffil}{\textsuperscript{\ccUseComp{AffilRef}}}%
606   \ccUseProperty{orcid-link}%
607 }%
```

`author-list-format <any>` how a single entry in the `AuthorNameList` Collection Component should be rendered.

```
608 \ccPropertyLet{author-list-format}{author-list-print-format}%
```

`author-contact-block-format <any>` is the Collection Property for the `AuthorContactBlock` Collection Component and sets how each single entry in the Collection should be formatted. By default, it uses the `AuthorContact` Counted Component and appends the `counted-name-sep` to all instance of that Component but the last.

```
609 \ccSetProperty{author-contact-block-format}{%
610   \ccUseComp{AuthorContact}\ifnum\ccCurCount<\ccTotalCount\ccUseProperty{counted-name-sep}\fi
611 }
```

6 Accessibility Features

`\cch@total@nesting@level` stores the absolute nesting level opened with each heading.

```
612 \newcount\cch@total@nesting@level \cch@total@nesting@level=\z@ \relax
```

`\cch@max@nesting@level` stores the highest absolute nesting level throughout the document.

```
613 \newcount\cch@max@nesting@level \cch@max@nesting@level=\z@ \relax
```

Since PDF 1.7 allows only `<H1/>`–`<H6/>`, more nesting levels need to be mapped to one of the Standard tags. The Tagged PDF Best Practice Guide recommends mapping higher levels to `<P/>`, which we do at the end of the document. All this is not necessary if PDF/UA-2 is used, since first it allows any `Hn` for as long as `Hns` are nested sequentially, and second, because then we can use the `<Title/>` tag instead of `<Hn/>`.

```
614 \ccWhenAlly{\ifcc@pdf@two\else\AtEndDocument{\cch@add@h@rolemap}\fi}%
```

`\cch@add@h@rolemap` adds the Rolemap entries for H_n , where $n > 6$.

```

615 \def\cch@add@h@rolemap{%
616   \@tempcnta=7\relax
617   \loop
618   \unless\ifnum\@tempcnta>\cch@max@nesting@level\relax
619     \ccaAddRolemap{H\the\@tempcnta}{P}%
620     \advance\@tempcnta\@ne\relax
621   \repeat
622 }

```

`\cchAutoClose` checks if any open `<Sect/>` tags (or Tagging mapped to `<Sect/>`) need to be closed and closes them if applicable.

For that, we check if the value of `\ccPrevSecLevel` (i.e., the *nominal* heading level of the last opened heading) is larger or equal to the *nominal* level of the currently opened heading. If so, we add closing `</Sect>` tags until either the *absolute* nesting level is 1 or an decrementing counter that starts at `\ccPrevSecLevel` gets less than `\ccCurSecLevel`. At each step, we also decrease the `\cch@total@nesting@level` counter by one.

```

623 \DeclareAccessibilityCommand\cchAutoClose{%
624   \let\cca@next\relax
625   \ccDebugMsg[a11y]{Calling AutoClose for \ccCurSecName\space (\ccCurSecLevel).}
626   \ifx\ccPrevSecLevel\@undefined\else
627     \ifx\ccCurSecLevel\@undefined\else
628       \ifnum\cch@total@nesting@level=\@ne\relax\else
629         \ccDebugMsg[a11y]{prev: \ccPrevSecLevel, cur: \ccCurSecLevel, abs: \the\
          cch@total@nesting@level.}
630         \ifnum\ccPrevSecLevel=\ccCurSecLevel\relax
631           \ccDebugMsg[a11y]{Closing \the\cch@total@nesting@level.}
632           \global\advance\cch@total@nesting@level\m@ne\relax
633           \ccaVstructEnd{\csname cch@sec@\the\cch@total@nesting@level @name\endcsname}%
634         \else
635           \ifnum\ccPrevSecLevel<\ccCurSecLevel\relax
636             \csxdef{cch@prev@parent@level@\ccCurSecLevel}{\ccPrevSecLevel}%
637           \else
638             \ifnum\ccPrevSecLevel>\ccCurSecLevel\relax
639               \ccDebugMsg[a11y]{Closing (inner) \the\cch@total@nesting@level.}
640               \global\advance\cch@total@nesting@level\m@ne\relax
641               \ccaVstructEnd{\csname cch@sec@\the\cch@total@nesting@level @name\endcsname}%
642               \ifnum\cch@total@nesting@level=\@ne\relax\else
643                 \xdef\ccPrevSecLevel{\csname cch@prev@parent@level@\ccPrevSecLevel\endcsname}%
644                 \let\cca@next\cchAutoClose
645               \fi
646             \fi
647           \fi
648         \fi
649       \fi
650     \fi
651   \fi
652   \cca@next
653 }

```

`\cchResetNesting` resets the *absolute* heading nesting level to 0 by closing all currently open `</Sect>` tags. This macro is intended to be used at the very end of the document or at major document partitions (e.g., `\mainmatter`, `\appendix`, etc.)

```

654 \DeclareAccessibilityCommand\cchResetNesting{%
655   \ccWhenAlly{%
656     \ifnum\cch@total@nesting@level>\z@\relax
657     \loop

```

```

658     \ccaVstructEnd{\csname cch@sec@\the\cch@total@nesting@level @name\endcsname}%
659     \global\advance\cch@total@nesting@level\m@ne\relax
660     \ifnum\cch@total@nesting@level>\z@\relax
661     \repeat
662   \fi
663 }}

```

The advancement of the `\cch@total@nesting@level` counter is done in the `\env/Heading/before` hook.

```

664 \AddToHook{env/\ccPrefix Heading/before}{%
665   \ccDebugMsg[a1y]{Processing heading}%
666   \global\advance\cch@total@nesting@level\@ne\relax
667 }

```

In the `\env/Heading/after` hook, first, the `</SectMeta>` Tag is closed. Only then, we open the tag for actual `<Sect>`. Here, we also define the name of the Tag to be re-used later in the `\cchAutoClose` mechanism. Finally, we move the `<Hn/>` Tagging node (which tags the section's title) and the `<SectMeta/>` node (which contains the tagged meta data realized by the `\Heading` Container's other Components) into the `<Sect/>` node.

```

668 \AddToHook{env/\ccPrefix Heading/after}{%
669   \ifx\cch@notag\relax
670     \global\let\cch@notag\@undefined
671   \else
672     \ccaVstructStart{\ccCurSecName}%
673     \csxdef{cch@sec@\the\cch@total@nesting@level @name}{\ccCurSecName}%

```

For inline headings, the actual title is not yet printed by the time the heading Container's environment is closed. In this case, we don't move the node. For one, because it is likely not yet set, and second because it most likely already is in the "right" spot, i.e. inside the `<Sect/>` tag.

```

674   \ifx\cch@id@cur@head\@undefined\else
675     \ccaMoveStruct{\cch@id@cur@head}%
676   \fi

```

If `\cch@id@cur@meta` is undefined when the Heading container's environment is closed, then this indicates an inline heading. In this case, we don't need to move the `<SectMeta/>` node, as it most likely already is in the right place.

```

677   \ifx\cch@id@cur@meta\@undefined\else
678     \ccaMoveStruct{\cch@id@cur@meta}%
679   \fi%
680 \fi
681 }

```

`\cchHeadTagStart` is used inside Property declarations to insert the start tag of the current section

```

682 \def\cchHeadTagStart{%
683   \if@cc@pdf@two
684     \ccaVstructStart{Title}%
685   \else
686     \ccaVstructStart{H\the\numexpr\cch@total@nesting@level\relax}%
687     \ifnum\cch@total@nesting@level>\cch@max@nesting@level
688       \global\cch@max@nesting@level=\cch@total@nesting@level\relax
689     \fi
690   \fi
691   \ccaSaveCurStruct{cch@id@cur@head}%
692 }

```

`\cchHeadTagEnd` is used inside Property declarations to insert the end tag of the current section, unless the heading level was declared with the starred version of `\ccDeclareHeading`.

```
693 \def\cchHeadTagEnd{%
694   \if@cc@pdf@two
695     \ccaVstructEnd{Title}%
696   \else
697     \ccaVstructEnd{H\the\numexpr\cch@total@nesting@level\relax}%
698   \fi}
```

The `<SectMeta/>` tag is mapped to a simple `<Div/>`.

```
699 \ccaAddRolemap{SectMeta}{Div}
```

7 Miscellaneous

7.1 Alternative paragraph separation

`\ccNewPar` is a user-level macro to have a vertical skip between two local paragraphs and no indent in the second one. The amount of vertical space between the paragraphs can be adjusted with the optional argument. If #1 is omitted, `\ccnewparskip` is inserted, which defaults to `1\baselineskip` if the dimension isn't set to something other than Opt in the preamble. This macro is intended to be used at the end of the first of the paragraphs.

```
700 \newdimen\ccnewparskip \AtBeginDocument{\ifdim\ccnewparskip=\z@\relax \ccnewparskip=1\
       baselineskip\relax\fi}
701 \def\ccNewPar{\@ifnextchar[{\cc@newpar}{\cc@newpar[\the\ccnewparskip]}}%
702 \def\cc@newpar[#1]{%
703   \ifhmode\par\fi
704   \vskip#1\relax
705   \@afterheading
706 }
707 \cslet{\ccPrefix NewPar}\ccNewPar
```

WARNING!
The following section is
deprecated and will be
changed or deleted in
future releases.

`\TitleBreak`

```
708 \letcs\TitleBreak{\ccPrefix Break}
```

`</headings>`

Module 7

coco-notes.dtx

```
<*endnotes>
```

This file contains the code for foot- and endnote handling. It provides a switch between endnotes and footnotes as well as options to handle the resetting of footnote/endnote counters.

```
23 %%
24 %% module for CoCoTeX that handles footnote/endnote switching.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-notes}
30 [2025/07/14 v0.6.2 le-tex coco notes module]
```

1 Internal Switches and Package Options

1.1 Package Switches

`\ifccn@use@en` is an internal switch for endnotes (`\ccn@use@entrue`) or footnotes (`\ccn@use@enfalse`, default).

```
31 \newif\ifccn@use@en \ccn@use@enfalse
```

1.2 Package Options

The `endnotes` option causes all footnotes to be rendered as endnotes.

```
32 \ExplSyntaxOn
33 \keys_define:nn { cocotex/notes }
34 {
35   endnotes .code:n = { \global\@ccn@use@entrue },
```

The option `ennotoc` prevents headings in the Notes section from creating entries in the Table of Contents.

```
36   ennotoc .code:n = { \global\let\ccn@en@no@toc\relax },
```

The option `resetnotesperchapter` resets foot- and endnote counters at the start of each `chapter` level heading. If omitted (default) foot- or endnotes are numbered throughout the whole document

```
37   resetnotesperchapter .code:n = { \global\let\ccn@reset@notes@per@chapter\relax },
```

The option `endnoteswithchapters` implies `endnotes` and causes chapter headings to be repeated in the `printnotes` chapter as sections.

```

38 endnoteswithchapters .code:n =
39 {
40   \global\@ccn@use@entrue
41   \global\let\ccn@en@with@chapters\relax
42 },

```

The option `endnotelinks` is now defunct, because back-linking is necessary for tagging.

```

43 endnotelinks .code:n = {}
44 }
45 \ProcessKeyOptions[cocotex/notes]
46 \ExplSyntaxOff

```

1.3 Hard Requirements

The `footnote` package is mandatory since it provides the `\savenotes` and `\spewnotes` macros.

```

47 \RequirePackage{footnote}

```

2 Endnote Handling

`\if@enotesopen` is a switch from the `endnotes` package. but since the package is loaded only with the `endnotes` options set, we need to define the conditional, anyhow.

```

48 \newif\if@enotesopen

```

`\ccn@parindent` is the par indent used in the endnotes section. It defaults to the value of `parindent` at the very end of the L^AT_EX Preamble.

```

49 \AtBeginDocument{\edef\ccn@parindent{\the\parindent}}

```

`\enindent` is the left margin and hanging indent of the endnotes section.

```

50 \newdimen\enindent \enindent=2em\relax

```

If endnotes are activated via a Package option, we include the `endnotes` package.

```

51 \if@ccn@use@en
52   \RequirePackage{endnotes}

```

`\ccn@use@TeX@heading` is a switch that defines itself when the CoCoT_EX Headings module is loaded.

```

53 \ifpackageloaded{coco-headings}{\let\ccn@use@TeX@heading\relax}{}

```

`\@endnotemark` is re-defined when endnotes should back-reference. In this case, we insert a L^AT_EX `\label` for later referencing.

TODO
This macro should be
patched, not re-defined!

```

54 \global\newcount\endnoteLinkCnt \global\endnoteLinkCnt\z@
55 \def\@endnotemark{%
56   \leavevmode
57   \ifhmode\edef\x@sf{\the\spacefactor}\nobreak\fi
58   \phantomsection%
59   \label{endnote-\the\endnoteLinkCnt}%
60   \ccaVstructStart{FootnoteMark}\ccaVstructStart{Lbl}%
61   \hyperref[endnotetext-\the\endnoteLinkCnt]{\makeenmark}%
62   \ccaVstructEnd{Lbl}\ccaVstructEnd{FootnoteMark}%
63   \ifhmode\spacefactor\x@sf\fi%
64   \relax%
65 }
66 %\fi

```

`\footnote` is re-defined to be an alias of the `\endnote` macro.

```

67 \def\footnote{\cc@opt@empty\ccn@endnote}
68 \long\def\ccn@endnote[#1]#2{%
69   \ccaStructStart{Footnote}%
70   \ccaSaveCurStruct{abs@enote@\the\endnoteLinkCnt}%
71   \def\@argi{#1}\ifx\@argi\@empty
72     \endnote{#2}%
73   \else
74     \endnote[#1]{#2}%
75   \fi
76   \ccaStructEnd{Footnote}}

```

`\enotesize` holds the font size of the endnotes section.

```

77 \def\enotesize{\normalsize}%

```

`\enoteformat` is the format of an endnote. We create the label right at the start of the endnote text to prevent erroneous pointing to the next page.

```

78 \def\enoteformat{%
79   \phantomsection%
80   \label{endnotetext-\currentEndnote}%
81   \noindent
82   \leavevmode
83   \hskip-\enindent\hb@xt@\enindent{%
84     \ccaVstructStart{Lbl}%
85     \hyperref[endnote-\currentEndnote]{\@themark}\hss%
86     \ccaVstructEnd{Lbl}%
87   }%
88   \expandafter\parindent\ccn@parindent\relax\expandafter%
89 }%

```

`\enoteheading` is a macro that is expanded at the beginning of the endnotes section. Originally, it was intended to hold the endnote section's heading, we mis-use it to set the leftskip. Apparently, the intention is to re-define the macro style-wise if needed...

```

90 \gdef\enoteheading{%
91   \leftskip\enindent
92 }%

```

`\printnotes` is the macro that eventually prints the endnote section in its stead.

```

93 \def\printnotes{%
94   \ifx\ccn@en@with@chapters\relax
95     \ccn@end@notes
96   \fi
97   \if@enotesopen
98     \ifx\ccn@reset@notes@per@chapter\relax
99       \global\c@endnote\z@%
100     \fi
101     \bgroup
102     \parskip\z@
103     \theendnotes
104     \egroup
105   \fi}
106 \else

```

`\c@endnote` is defined to ensure upward-compatibility.

```

107 \newcount\c@endnote \c@endnote\z@
108 \let\printnotes\relax
109 \fi

```

3 Processing Package Options

3.1 Endnotes With Chapters

`\ccn@end@notes` stores the number of endnotes in a chapter in a generic macro `\ccn@notes@in@the\realchap`.

```

110 \def\ccn@end@notes{\csxdef{ccn@notes@in@the\realchap}{\the\endnoteLinkCnt}}

```

`\realchap` is a counter that increases by one with each (coco-headings) chapter.

```

111 \newcount\realchap \realchap\z@

```

If endnotes are printed chapter-wise, we need to hook into the `\chapter` heading level using `\cc/headings/chapter/print/before`. There, we check if the last chapter did actually contain endnotes. If yes, we pass the chapter's `\Title` and `\RunTitle` components into the endnote temporary `.ent` file as a `\section` heading.

```

112 \AtBeginDocument{%
113   \AddToHook{cc/headings/chapter/print/before}{%
114     \ifx\ccn@en@with@chapters\relax
115       \ccn@end@notes
116       \global\advance\realchap\@ne
117       \ifx\ccn@reset@notes@per@chapter\relax\global\c@endnote\z@\fi
118       \def\ccn@par@number{\ccIfComp{TocNumber}{\ccUseComp{TocNumber}}{\ccIfComp{Number}{\ccUseComp{Number}}{\relax}}}%
119       \def\ccn@par@title{\ccIfComp{TocTitle}{\ccUseComp{TocTitle}}{\ccUseComp{Title}}}%
120       \def\ccn@par@runttitle{\ccIfComp{RunTitle}{\ccUseComp{RunTitle}}{\ccUseComp{Title}}}%
121       \addtoendnotes{%
122         \noexpand\ifnum\noexpand\csname ccn@notes@in@the\realchap\endcsname>\noexpand\csname
123           ccn@notes@in@the\numexpr\the\realchap-\@ne\relax\endcsname\relax
124         \noexpand\edef\noexpand\prev@leftkip{\noexpand\the\noexpand\leftskip}%
125         \noexpand\leftskip\noexpand\z@
126         \noexpand\begin{\ccPrefix Heading}\noexpand[notag\ifx\ccn@en@no@toc\relax ,notoc\fi\
127           noexpand]{section}%
128         \noexpand\ccComponent{Number}{\ccn@par@number}%

```

```

127     \noexpand\ccComponent{Title}{\ccn@par@title}%
128     \noexpand\ccComponent{RunTitle}{\ccn@par@runtile}%
129     \noexpand\end{\ccPrefix Heading}%
130     \noexpand\leftskip\noexpand\prev@leftkip\noexpand\relax%
131     \noexpand\fi}%
132 \else

```

3.2 Chapter-wise Resetting

If we don't use endnotes with chapters, we check if the `resetnotesperchapter` option is set and, if it is set, we instead set both counters for endnotes and footnotes to zero.

```

133     \ifx\ccn@reset@notes@per@chapter\relax
134         \global\c@footnote\z@
135         \global\c@endnote\z@
136     \fi
137 \fi
138 }%
139 }

```

3.3 Back-Referencing Endnotes

Linking endnotes requires overwriting the `\@endnotetext` macro to save a global counter to the *.ent file.

```

140 \global\newif\if@haveenotes
141 \long\def\@endnotetext#1{%
142     \global\@haveenotestru
143     \if@enotesopen \else \@openenotes \fi
144     \immediate\write\@enotes{%
145         \string\def\string\currentEndnote{\the\endnoteLinkCnt}%
146         \noexpand\ccaVstructStart{FootnoteText}%
147         \noexpand\expandafter\noexpand\ccaAddToStruct\noexpand\expandafter{\noexpand\csname
148             abs@enote@\the\endnoteLinkCnt\noexpand\endcsname}%
149         \doanenote{\@theenmark}%
150     }%
151     \begingroup
152     \def\next{#1}%
153     \newlinechar='40
154     \immediate\write\@enotes{\meaning\next}%
155 \endgroup
156 \immediate\write\@enotes{\noexpand\ccaAddID{auto}\noexpand\ccaVstructEnd{FootnoteText}\
157     @endanenote}%
158 \global\advance\endnoteLinkCnt\@ne%
159 }

```

3.4 Allow Non-Numerical Endnote Counters

`\@xendnote` is an override of `endnote`'s macro of the same name to account for manual entnote counters that include non-numerical symbols.

```

158 \ifdefined\@xendnote
159     \patchcmd\@xendnote
160         {\c@endnote=#1\relax
161         \unrestored@protected@xdef\@theenmark{\theendnote}}
162         {\sbox\z@{\@tempcnta#1\relax}%

```

```

163 \ifdim\wd\z@>\z@\relax
164   %\global\advance\c@endnote\@ne\relax
165   \unrestored@protected@xdef\@theenmark{#1}%
166 \else
167   \c@endnote=#1\relax
168   \unrestored@protected@xdef\@theenmark{\theendnote}%
169 \fi}
170 {}{\cc@patch@error{notes}\@xendnote}%
171 \fi

```

4 Adjusting Regular Footnotes

4.1 Allowing Multiple Paragraphs in Footnotes

First, we make a small adjustment to the `\fn@fntext` macro from the `footnote` package by making it `\long` and therefore allowing `\par` inside its argument.

```

172 \long\def\fn@fntext#1{%
173   \ifx\ifmeasuring@\@undefined%
174     \expandafter\@secondoftwo\else\expandafter\@iden%
175   \fi%
176   {\ifmeasuring@\expandafter\@gobble\else\expandafter\@iden\fi}%
177   {%
178     \global\setbox\fn@notes\vbox{%
179       \unvbox\fn@notes%
180       \fn@startnote%
181       \@makefntext{%
182         \rule\z@\footnotesep%
183         \ignorespaces%
184         #1%
185         \@finalstrut\strutbox%
186       }%
187       \fn@endnote%
188     }%
189   }%
190 }

```

4.2 Allowing Non-Numeric Footnote Counters

Re-definition of `footnote` package's footnote mark retriever to allow non-numeric values in the optional argument of `\footnote`.

```

191 \def\fn@getmark@i#1[#2]{%
192   \sbox\z@{\@tempcnta#2\relax}%
193   \ifdim\wd\z@>0\p@\relax
194     \def\thempfn{#2}%
195     \fn@getmark@iii%
196   \else
197     \csname c@\@mpfn\endcsname#2%
198     \fn@getmark@ii%
199   \fi
200 }
201 \def\fn@getmark@iii#1{%
202   \unrestored@protected@xdef\@thefnmark{\thempfn}%
203   \endgroup%

```

```

204 #1%
205 }

```

And the same for plain L^AT_EX:

```

206 \long\def\xfootnote[#1]#2{%
207   \ccaStructStart{Footnote}%
208   \begingroup
209   \sbox\z@{\@tempcnta0#1\relax}%
210   \ifdim\wd\z@>0\p@{\relax
211     \unrestored@protected@xdef\@thefnmark{#1}%
212   \else
213     \csname c@{\@mpfn}\endcsname #1\relax
214     \unrestored@protected@xdef\@thefnmark{\thempfn}%
215   \fi
216 \endgroup
217 \@footnotemark\@footnotetext{#2}%
218 \ccaStructEnd{Footnote}%
219 }

```

5 Tagging Footnotes

Adding artifact tagging to the footnoterule:

```

220 \pretocmd\footnoterule{\ccaVstructStart [Document] {footnoterule}}{\cca@patch@error\footnoterule
221   }
221 \apptocmd\footnoterule{\ccaVstructEnd{footnoterule}}{\cca@patch@error\footnoterule}

```

patching `\@footnotemark` to introduce the  `<FootnoteMark/>` tag which will be mapped to the  `<Reference/>` tag, later.

```

222 \pretocmd\@footnotemark{%
223   \protected@xdef\@lt@fn@parent{\ccaGetCurStruct{idx}}%
224   \ccaStructStart{FootnoteMark}%
225 }{\cca@patch@error\@footnotemark}
226 \apptocmd\@footnotemark{%
227   \ccaStructEnd{FootnoteMark}%
228 }{\cca@patch@error\@footnotemark}

```

patching `\@makefnmark` for the  `<Lbl/>` tag both in the text body and in the footnote insert.

```

229 \pretocmd\@makefnmark{%
230   \ccaStructStart{Lbl}%\addAltText{\@thefnmark}
231 }{\cca@patch@error\@makefnmark}
232 \apptocmd\@makefnmark{%
233   \ccaStructEnd{Lbl}%\addAltText{\@thefnmark}
234 }{\cca@patch@error\@makefnmark}

```

patching `\@makefntext` to introduce the  `<FootnoteText/>` tag, which will be mapped to  `<Note/>`, below.

```

235 \pretocmd\@makefntext{%
236   \ccaStructStart{FootnoteText}%
237   \ifx\@lt@fn@parent\@empty\relax\else\ccaAddToStruct{\@lt@fn@parent}\fi%
238 }{\cca@patch@error\@makefntext}
239 \apptocmd\@makefntext{%
240   \ccaAddID{auto}\ccaStructEnd{FootnoteText}%
241 }{\cca@patch@error\@makefntext}

```

Finally, we add the  `<FootnoteMark/>` and  `<FootnoteText/>` PDF tags to the rolemap.

```
242 \ccaAddRolemap{Footnote}{Span}  
243 \ccaAddRolemap{FootnoteMark}{Reference}  
244 \ccaAddRolemap{FootnoteText}{\ifcc@pdf@two Aside\else Note\fi}
```

```
</endnotes>
```


Module 8

coco-script.dtx

```
<*script>
```

This package is used to handle non-latin based script systems like Japanese, Chinese, Armenian and the like.

```
23 %% module for CoCoTeX that handles script switching.
24 %%
25 %% Maintainer: p.schulz@le-tex.de
26 %%
27 %% requires LuaLaTeX!
28 %%
29 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
30 \ProvidesPackage{coco-script}
31 [2025/07/14 v0.6.2 CoCoTeX script module]
```

The argument of the `usescript` option is a list of script systems that are used in the document. It is used to determine the additional fonts that are to be loaded via the babel package.

```
32 \RequirePackage{coco-kernel}
33 \let\usescript\relax
34 \ExplSyntaxOn
35 \keys_define:nn { cocotex/script }
36 {
37   usescript .code:n = { \gdef\usescript{#1} }
38 }
39 \ProcessKeyOptions[cocotex/script]
40 \ExplSyntaxOff
41 \RequirePackage[quiet]{fontspec}
42 \def\parse@script#1,#2,\relax{%
43   \ccs@callback{#1}%
44   \edef\@argii{#2}%
45   \let\next\relax
46   \ifx\@argii\@empty\else
47     \def\next{\parse@script#2,\relax}%
48   \fi\next}
49 \ifx\usescript\relax\else
50   \def\ccs@callback#1{\expandafter\global\expandafter\let\csname use@script@#1\endcsname\@empty}
51   \expandafter\parse@script\usescript,,\relax
52 \fi
53 \ccPackageInfo{Script}{Info}{Fonts loaded: \meaning\usescript}
```

If babel's bidirectional feature is loaded without need, there might be errors, so we do some checks first:

```
54 \def\cc@bidi{}
55 \@tempwafalse
56 \ifx\usescript\relax\else
57   \expandafter\ifx\csname use@script@arabic\endcsname\@empty\@tempwattrue
58   \else\expandafter\ifx\csname use@script@hebrew\endcsname\@empty\@tempwattrue
59   \else\expandafter\ifx\csname use@script@amharic\endcsname\@empty\@tempwattrue
60   \else\expandafter\ifx\csname use@script@ethiop\endcsname\@empty\@tempwattrue
```

```

61 \else\expandafter\ifx\curname use@script@syriac\endcurname\@empty\@tempwattrue
62 \fi\fi\fi\fi\fi\fi
63 \if@tempwa\def\cc@bidi{,bidi=basic}\fi
64 \RequirePackage[silent\cc@bidi]{babel}

```

1 Fallback fonts

`\ccsTestFont` is used to test the currently active font family. For that, we compare the base name of `\f@family` with the (fully expanded) value given in the first argument.

`{#1}` is the comparison value that is tested against L^AT_EX's `\f@family`.

`{#2}` is the `true` branch, executed if the base name of `{#1}` matches against the base name `\f@family`.

`{#3}` is the `else` branch.

```

65 \def\ccsTestFont#1#2#3{\edef\@argi{#1}\expandafter\expandafter\expandafter\ccs@testfont\
    \expandafter\f@family\expandafter.\expandafter\@nil\@argi.\@nil{#2}{#3}\@nil}
66 \def\ccs@testfont #1.#2\@nil#3.#4\@nil#5#6\@nil{\expandafter\ifnum\pdf@strcmp{#1}{#3}=\z@\relax
    #5\else#6\fi}

```

Comparison values of the three basic font families of L^AT_EX (`roman`, `sans-serif` and `monospace`) are defined after all packages are loaded. Macros for comparing Custom font families need to be defined manually.

```

67 \AddToHook{begindocument}{%

```

`\ccs@rmdefault` holds the default roman font

```

68 \begingroup\rmfamily\xdef\ccs@rmdefault{\f@family}\endgroup%

```

`\ccs@sfdefault` holds the default sans-serif font

```

69 \begingroup\sffamily\xdef\ccs@sfdefault{\f@family}\endgroup%

```

`\ccs@ttdefault` holds the default monospace font

```

70 \begingroup\ttfamily\xdef\ccs@ttdefault{\f@family}\endgroup%
71 }

```

1.1 Default Fallback Font: Noto

The default fall backfont is the NotoSans Font Family

```

72 \newfontfamily\fallbackfont{NotoSerif-Regular.ttf}%
73 [BoldFont = NotoSerif-Bold.ttf,%
74 ItalicFont = NotoSerif-Italic.ttf,%
75 BoldItalicFont = NotoSerif-BoldItalic.ttf,%
76 Path = ./fonts/Noto/Serif/,%
77 WordSpace = 1.25]
78 \newfontfamily\sffallbackfont{NotoSans-Regular.ttf}%
79 [BoldFont = NotoSans-Bold.ttf,%
80 ItalicFont = NotoSans-Italic.ttf,%
81 BoldItalicFont = NotoSans-BoldItalic.ttf,%
82 Path = ./fonts/Noto/Sans/,%
83 WordSpace = 1.25]

```

```

84 \newfontfamily\ttfallbackfont{NotoSansMono-Regular.ttf}%
85 [BoldFont = NotoSansMono-Bold.ttf,%
86 ItalicFont = NotoSansMono-Light.ttf,%
87 BoldItalicFont = NotoSansMono-SemiBold.ttf,%
88 Path = ./fonts/Noto/Mono/,%
89 WordSpace = 1.25]

```

`\textfallback` is a text command that switches to the serif fallback font for its argument `{#1}`.

```

90 \DeclareTextFontCommand\textfallback{\fallbackfont}

```

`\textsffallback` is a text command that switches to the sans-serif fallback font for its argument `{#1}`.

```

91 \DeclareTextFontCommand\textsffallback{\sffallbackfont}

```

`\textttfallback` is a text command that switches to the monospace fallback font for its argument `{#1}`.

```

92 \DeclareTextFontCommand\textttfallback{\ttfallbackfont}

```

`\ccFallback` is a “smart” font macro that detects if it has been used inside `rm`, `sf`, or `tt` contexts and switches to the corresponding fallback font automatically to print its argument `{#1}`.

```

93 \DeclareTextFontCommand\ccFallback{%
94   \ccsTestFont{\ccs@rmdefault}
95   {\fallbackfont}
96   {\ccsTestFont{\ccs@sfdefault}
97    {\sffallbackfont}
98    {\ttfallbackfont}}}%

```

1.2 Emojis

A font and a text command for using plain, black emojis.

```

99 \newfontfamily\emojifont{NotoEmoji-Regular.ttf}%
100 [BoldFont = NotoEmoji-Bold.ttf,%
101 Path = ./fonts/Noto/Emoji/]
102 \DeclareTextFontCommand\textemoji{\emojifont}

```

1.3 Support for medieval scripts and special characters

Warning: Junicode provides supports only for the `rm` font family!

```

103 \babelfont{mdv}[%
104 Path=fonts/Junicode/,%
105 ItalicFont = Junicode-Italic.ttf,%
106 BoldFont = Junicode-Bold.ttf,%
107 BoldItalicFont = Junicode-BoldItalic.ttf,%
108 ]{Junicode.ttf}
109 \def\mdvfont#1{{\mdvfamily#1}}

```

1.4 International Phonetic Alphabet

Since the IPA character inventory seems to be included in the Noto fonts, we simply define the `\ccTextipa` as a context-sensitive alias.

`\ccTextipa` is a text command for the International Phonetic Alphabet. By default, the global Noto fallback font is used and sensitive to `rm`, `sf`, and `tt` contexts.

Warning! Noto has been chosen because it contains all IPA symbols, but there are some short-comings: in italic contexts, [a] and [] become indistinguishable for roman and hardly distinguishable in sans-serif contexts. Also, the bow used to bind vowels is misplaced in NotoSerif. We therefore provide an easy way to locally re-define alternative IPA fonts:

`\ccIpaFont` for roman contexts,

```
110 \let\ccIpaFont\fallbackfont
```

`\ccIpaSfFont` for sans-serif contexts, and

```
111 \let\ccIpaSfFont\sffallbackfont
```

`\ccIpaTtFont` for monospace contexts.

```
112 \let\ccIpaTtFont\ttfallbackfont
```

Those macros are used to select the appropriate IPA font inside the `\ccTextipa` text command:

```
113 \DeclareTextFontCommand\ccTextipa{%
114   \ccsTestFont{\ccs@rmdefault}
115   {\ccIpaFont}
116   {\ccsTestFont{\ccs@sfdefault}
117    {\ccIpaSfFont}
118    {\ccIpaTtFont}}}%
```

2 Generic Fonts Declaration Mechanism

- #1 Options passed to `\babelprovide`
- #2 language
- #3 argument(s) passed to `\babelfont{rm}`
- #4 argument(s) passed to `\babelfont{sf}`

```
119 \def\ccDeclareBabelFont{\cc@opt@empty\ccs@declare@babel@font}%
120 \def\ccs@declare@babel@font[#1]#2#3#4{%
121   \expandafter\ifx\csname use@script@#2\endcsname\@empty
122     \babelprovide[#1]{#2}%
123     \message{^^J [coco-script Loaded Script: #2]^^J}%
124     %%
125     \expandafter\gdef\csname ccs@babel@rm@font@#2\endcsname{#3}%
126     \expandafter\gdef\csname ccs@babel@sf@font@#2\endcsname{#4}%
127     \if!#2!\else
128       \def\ccs@tempa{\babelfont[#2]{rm}}%
129       \expandafter\expandafter\expandafter\ccs@tempa\csname ccs@babel@rm@font@#2\endcsname
130     \fi
131     \if!#3!\else
132       \def\ccs@tempa{\babelfont[#2]{sf}}%
133       \expandafter\expandafter\expandafter\ccs@tempa\csname ccs@babel@sf@font@#2\endcsname
```

```

134 \fi
135 \fi
136 }

```

Top level macro to declare a font alias.

#1 font family alias
#2 font family fallback

```

137 \def\ccBabelAlias#1#2{%
138 \ifx\usescript\relax\else
139 \def\ccs@callback##1{%
140 \expandafter\ifx\csname ccs@no@fallback@##1\endcsname\relax
141 \expandafter\ifx\csname ccs@babel@#2@font@##1\endcsname\relax
142 \PackageError
143 {coco-script.sty}
144 {\expandafter\string\csname #2family\endcsname\space for Language `##1' was not
145 declared!}
146 {You attempted to declare an alias towards a font family that has not been declared
147 for the language `##1', yet.}%
148 \else
149 \def\ccs@tempa{\babelfont[##1]{#1}}%
150 \expandafter\expandafter\expandafter\ccs@tempa\csname ccs@babel@#2@font@##1\endcsname
151 \fi
152 \else
153 \PackageInfo{coco-script.sty}{^^J\space\space\space\space No fallback for `##1';^^J\space
154 \space\space\space Skipping font family `#1'->`#2'}%
155 \fi}%
156 \expandafter\parse@script\usescript,,\relax
157 \fi}

```

3 Predefined script systems

3.1 Support for Armenian script

```

155 \ccDeclareBabelFont{armenian}{[%
156 Path=./fonts/Noto/Armenian/,
157 BoldFont = NotoSerifArmenian-Bold.ttf,%
158 WordSpace = 1.25]{NotoSerifArmenian-Regular.ttf}}
159 {[%
160 Path=./fonts/Noto/Armenian/,
161 BoldFont = NotoSansArmenian-Bold.ttf,%
162 WordSpace = 1.25]{NotoSansArmenian-Regular.ttf}%
163 }

```

Legacy and backwards compatibility:

```

164 \def\armenian{\foreignlanguage{armenian}}

```

3.2 Support for Chinese script

```

165 \ccDeclareBabelFont{chinese}{[%
166 Path=./fonts/Noto/Chinese/,
167 BoldFont = NotoSerifSC-Bold.otf,%

```

```

168     WordSpace = 1.25]{NotoSerifSC-Regular.otf}}
169 {[%
170     Path=./fonts/Noto/Chinese/,
171     BoldFont = NotoSansSC-Bold.otf,%
172     WordSpace = 1.25]{NotoSansSC-Regular.otf}%
173 }

```

3.3 Support for Japanese script

```

174 \ccDeclareBabelFont{japanese}{[%
175     Path=./fonts/Noto/Japanese/,
176     BoldFont = NotoSerifJP-Bold.otf,%
177     WordSpace = 1.25]{NotoSerifJP-Regular.otf}
178 }{[%
179     Path=./fonts/Noto/Japanese/,
180     BoldFont = NotoSansJP-Bold.otf,%
181     WordSpace = 1.25]{NotoSansJP-Regular.otf}
182 }

```

3.4 Support for Korean script

```

183 \ccDeclareBabelFont{korean}{[%
184     BoldFont = NotoSerifKR-Bold.otf,%
185     ItalicFont = NotoSerifKR-Regular.otf,%
186     BoldItalicFont = NotoSerifKR-Medium.otf,%
187     Path=./fonts/Noto/Korean/,%
188     Script=CJK%
189 ]{NotoSerifKR-Regular.otf}}
190 {[%
191     BoldFont = NotoSansKR-Bold.otf,%
192     ItalicFont = NotoSansKR-Regular.otf,%
193     BoldItalicFont = NotoSansKR-Medium.otf,%
194     Path=./fonts/Noto/Korean/,%
195     Script=CJK%
196 ]{NotoSansKR-Regular.otf}%
197 }

```

3.5 Support for Hebrew script

```

198 \ccDeclareBabelFont{hebrew}{[%
199     Renderer=Harfbuzz,%
200     Scale=MatchUppercase,%
201     Path=./fonts/Noto/Hebrew/,%
202     Ligatures=TeX,%
203     Script=Hebrew,%
204     BoldFont = NotoSerifHebrew-Bold.ttf]{NotoSerifHebrew-Regular.ttf}%
205 }{[%
206     Renderer=Harfbuzz,%
207     Scale=MatchUppercase,%
208     Path=./fonts/Noto/Hebrew/,%
209     Ligatures=TeX,%
210     Script=Hebrew,%
211     BoldFont = NotoSansHebrew-Bold.ttf]{NotoSansHebrew-Regular.ttf}%
212 }

```

3.6 Support for Arabic script

```

213 \ccDeclareBabelFont{arabic}{[%
214     BoldFont = NotoNaskhArabic-Bold.ttf,%
215     Path = ./fonts/Noto/Arabic/,%
216     Script=Arabic%
217     ]{NotoNaskhArabic-Regular.ttf}}
218 {[%
219     BoldFont = NotoSansArabic-Bold.ttf,%
220     Path = ./fonts/Noto/Arabic/,%
221     Script=Arabic%
222     ]{NotoSansArabic-Regular.ttf}%
223 }

```

3.7 Support for Greek script

```

224 \ccDeclareBabelFont{greek}{[%
225     BoldFont = NotoSerif-Bold.ttf,%
226     ItalicFont = NotoSerif-Italic.ttf,%
227     BoldItalicFont = NotoSerif-BoldItalic.ttf,%
228     Path = ./fonts/Noto/Serif/,%
229     Script=Greek,%
230     WordSpace = 1.25
231     ]{NotoSerif-Regular.ttf}}
232 {[%
233     BoldFont = NotoSans-Bold.ttf,%
234     ItalicFont = NotoSans-Italic.ttf,%
235     BoldItalicFont = NotoSans-BoldItalic.ttf,%
236     Path = ./fonts/Noto/Sans/,%
237     Script=Greek,%
238     WordSpace = 1.25%
239     ]{NotoSans-Regular.ttf}%
240 }

```

3.8 Support for Ethiopian/Amharic script

```

240 \ccDeclareBabelFont{ethiop}{[%
241     BoldFont = NotoSerifEthiopic-Bold.ttf,%
242     ItalicFont = NotoSerifEthiopic-Regular.ttf,%
243     BoldItalicFont = NotoSerifEthiopic-Bold.ttf,%
244     Path = ./fonts/Noto/Ethiop/,%
245     WordSpace = 1.25
246     ]{NotoSerifEthiopic-Regular.ttf}}
247 {[%
248     BoldFont = NotoSansEthiopic-Bold.ttf,%
249     ItalicFont = NotoSansEthiopic-Regular.ttf,%
250     BoldItalicFont = NotoSansEthiopic-Bold.ttf,%
251     Path = ./fonts/Noto/Ethiop/,%
252     WordSpace = 1.25%
253     ]{NotoSansEthiopic-Regular.ttf}%
254 }
254 \ccDeclareBabelFont{amharic}{[%
255     BoldFont = NotoSerifEthiopic-Bold.ttf,%
256     ItalicFont = NotoSerifEthiopic-Regular.ttf,%
257     BoldItalicFont = NotoSerifEthiopic-Bold.ttf,%
258     Path = ./fonts/Noto/Ethiop/,%
259     WordSpace = 1.25
260     ]{NotoSerifEthiopic-Regular.ttf}}

```

```

261 { [BoldFont = NotoSansEthiopic-Bold.ttf,%
262     ItalicFont = NotoSansEthiopic-Regular.ttf,%
263     BoldItalicFont = NotoSansEthiopic-Bold.ttf,%
264     Path = ./fonts/Noto/Ethiop/,%
265     WordSpace = 1.25%
266     ] {NotoSansEthiopic-Regular.ttf}%
267 }

```

3.9 Support for Georgian script

```

268 \ccDeclareBabelFont{georgian}{%
269     [%
270         Path = ./fonts/Noto/Georgian/,%
271         BoldFont = NotoSerifGeorgian-Bold.ttf,%
272         ItalicFont = NotoSerifGeorgian-Regular.ttf,%
273         BoldItalicFont = NotoSerifGeorgian-Bold.ttf,%
274         Scale=0.85%
275     ] {NotoSerifGeorgian-Regular.ttf}%
276 }{%
277     [%
278         Path = ./fonts/Noto/Georgian/,%
279         BoldFont = NotoSerifGeorgian-Bold.ttf,%
280         ItalicFont = NotoSerifGeorgian-Regular.ttf,%
281         BoldItalicFont = NotoSerifGeorgian-Bold.ttf,%
282         Scale=0.85%
283     ] {NotoSerifGeorgian-Regular.ttf}%
284 }

```

3.10 Support for Syrian script

Since Babel does not support the Syrian script natively, we create a `babel-syriac.ini` file and include it, if it is needed. If we don't, the kerning and ligatures of Syriac text will be off.

Please note that due to the restrictions of the `listings`-Package, some Unicode characters cannot be displayed correctly in the documentation of the following code. Therefore, Syriac letters appear as “x” in the following source code listing.

```

285 \expandafter\ifx\csname use@script@syriac\endcsname\@empty%
286 \RequirePackage{filecontents}
287 \begin{filecontents*}{babel-syriac.ini}
288 [identification]
289 charset = utf8
290 version = 0.1
291 date = 2019-08-25
292 name.local = ??????????
293 name.english = Classical Syriac
294 name.babel = classicalsyriac
295 tag.bcp47 = syc
296 tag.opentype = SYR
297 script.name = Syriac
298 script.tag.bcp47 = Syrc
299 script.tag.opentype = syrc
300 level = 1
301 encodings =
302 derivate = no
303 [captions]
304 [date.gregorian]

```



```

305 [date.islamic]
306 [time.gregorian]
307 [typography]
308 [characters]
309 [numbers]
310 [counters]
311 \end{filecontents*}
312 \fi

```

Now, we can create the fallback font and import the newly created ini file:

```

313 \ccDeclareBabelFont[import=syriac]{syriac}{[%
314     BoldFont = NotoSansSyriac-Black.ttf,%
315     ItalicFont = NotoSansSyriac-Regular.ttf,%
316     BoldItalicFont = NotoSansSyriac-Black.ttf,%
317     Path = ./fonts/Noto/Syriac/,%
318     WordSpace = 1.25
319     ]{NotoSansSyriac-Regular.ttf}}
320 {[BoldFont = NotoSansSyriac-Black.ttf,%
321     ItalicFont = NotoSansSyriac-Regular.ttf,%
322     BoldItalicFont = NotoSansSyriac-Black.ttf,%
323     Path = ./fonts/Noto/Syriac/,%
324     WordSpace = 1.25%
325     ]{NotoSansSyriac-Regular.ttf}%
326 }

```

```
</script>
```


Module 9

coco-title.dtx

```
<*title>
```

This file provides macros and facilities for title pages.

```

23 %%
24 %% module for CoCoTeX for maketitle.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-title}
30   [2025/07/14 v0.6.2 CoCoTeX title module]
31 \RequirePackage{coco-meta}
```

1 Top-Level Interface

`titlepage` is the main Container for the whole document's meta data.

```

32 \ccDeclareContainer{titlepage}{%
33   \ccInherit {Components,Properties}{CommonMeta}%
34   \ifarticle\ccInherit{Components}{article-meta}\fi
35   \ccDeclareType{Components}{%
36     \cct@simple@comps
37     \cct@fundings@comp
38     \cct@role@handlers{author}{Author}%
39     \cct@declare@role{editor}{Editor}%
40     \cct@declare@role{series-editor}{SeriesEditor}%
41   }%
42   \ccDeclareType{Properties}{}%
43   \ccDeclareEnv[Meta]{\cctmeta}{\endcctmeta}%
44 }
```

`\cct@declare@role` declares the roles for editors and series editors and initializes the biography meta block for both.

```

45 \def\cct@declare@role#1#2{%
46   \ccDeclareRole[#1]{#2}%
47   \cct@role@handlers{#1}{#2}%
48 }
```

`\cct@role@handlers` adds title page specific Components and Handlers to the Author, Editor and Series-Editor Roles.

```

49 \def\cct@role@handlers#1#2{%
```

```

50 \ccAddToRole{#2}{%
51   \ccDeclareCountedComponent{Bio}%
52   \ccDeclareCountedComponent{Biography}}%
53 \ccDeclareGroupHandler{#2}{%
54   \ccIfComp{Biography}{\ccIfComp{Bio}{\ccComponent{Biography}{\ccUseProperty{#1-biography-
55     format}}}}}%
56 \ccDeclareRoleBlock[apply]{#2}{BioBlock}{#1-bio-block-format}%
57 }

```

`\ccDeclareTitlepage` is the default titlepage declarator with the next token being added the titlepage's Property list.

```

58 \def\ccDeclareTitlepage{\ccAddToType{Properties}{titlepage}}

```

`\cctmeta` is the code executed at the beginning of the `\ccPrefix Meta` Container

```

59 \def\cctmeta{\cc@opt@empty\cct@meta}
60 \def\cct@meta[#1]{%
61   \UseHook{env/meta/begin}%
62   \ccEvalAttributes[titlepage]{#1}%
63   \ccEvalType{Components}%
64   \let\and\relax
65 }

```

`\ccAddTitleRole` is a user-level macro to add both a new Role with the name `{#2}` and a controlling Property `{#1}` to the `titlepage` container.

```

66 \def\ccAddTitleRole#1#2{%
67   \ccAddToType{Components}{titlepage}{\cct@declare@role{#1}{#2}}%
68   \ccAddTitleEval{\cct@eds@eval{#2}}%
69 }

```

`\ccAddTitleEval` is a User-level macro to add additional Material titlepage evaluators (the next token).

```

70 \def\ccAddTitleEval{\csgappto{\cct@add@eval}}

```

`\cct@add@eval` is a hook for additional titlepage evaluators

```

71 \def\cct@add@eval{}

```

`\endcctmeta` is the code executed at the end of the `Meta` Container

```

72 \def\endcctmeta{%
73   \ccSetContainer{titlepage}%
74   \ccEvalType{Properties}%
75   \cct@maketitle
76   \ccm@role@eval{Author}%
77   \ccApplyCollection{Affil}{affil-block-item-format}{AffilBlock}%
78   \cct@eds@eval{Editor}%
79   \cct@eds@eval{SeriesEditor}%
80   \ccm@generic@eval
81   \cct@fundings@eval
82   \cct@add@eval
83   \cc@if@preamble\cct@set@pdfmeta\relax

```

Now, we expand the `\cct/document/meta`.

```

84 \UseHook{cct/document/meta}%
85 \let\cct@cur@cont\@empty
86 }

```

2 Processing of PDF Meta Data

The next few macros handle the content that is written directly into the pdf as meta data.

`\cct@set@pdfmeta` is the wrapper for the whole meta data handling.

```

87 \def\cct@set@pdfmeta{%

```

`\cct@write@pdf@meta@string` handles meta data that are stored as plain *strings* in the XMP file.

`{#1}` is the hyperref name for the meta datum
`{#2}` is the ltpdfa name for the meta datum
`{#3}` is the fallback value.

The fallback value is only chosen if either no XMP file exists, or if the XMP file does not contain the required data field.

In any case, the string is sanitized through `\ccSanitizeStr`.

```

88 \def\cct@write@pdf@meta@string##1##2##3{%
89   \let\cct@cur@data\@empty
90   \ccIfAlly
91     {\ccSanitizeStr\cct@cur@data{\directlua{tex.print(cocotex.ally.meta.##2)}}}%
92     \ifx\cct@cur@data\@empty
93       \ccSanitizeStr\cct@cur@data{##3}%
94     \fi
95     \ifx\cct@cur@data\@empty
96       \ccDebugMsg[pdfmeta]{##2 is empty (3: ##3); doing nothing}%
97     \else
98       \ccDebugMsg[pdfmeta]{Writing \string\setDocinfo[utf-8]{##2}{\expandafter\strip@prefix\
          meaning\cct@cur@data}}%
99       \edef\x{\noexpand\ccaSetDocinfo[utf-8]{##2}}\expandafter\x\expandafter{\cct@cur@data}%
100     \fi
101   {\cct@write@hr@infodict{##1}{##2}{##3}}}%

```

`\cct@write@pdf@meta@list` handles meta data that are represented as *lists* in the XMP file.

`{#1}` is the hyperref name for the meta datum
`{#2}` is the ltpdfa name for the meta datum
`{#3}` is the fallback value.
`{#4}` is the handler for the list. This is a macro that takes one argument. Each item of the list will be passed to that macro as argument.
`{#5}` is the separator between the items

```

102 \def\cct@write@pdf@meta@list##1##2##3##4##5{%
103   \let\cct@cur@data\@empty
104   \ccIfAlly
105     {\protected@edef\cct@cur@data{\directlua{tex.print(cocotex.ally.meta.##2)}}}%
106     \ifx\cct@cur@data\@empty
107       \protected@edef\cct@cur@data{##3}%
108     \ifx\cct@cur@data\@empty\else
109       \edef\x{\noexpand\cct@split@pdf@meta{\noexpand##4}{##5}}\x%

```

```

110     \fi
111     \else
112     \cct@split@pdf@meta{##4}{\and}%
113     \fi}
114     {\cct@write@hr@infodict{##1}{##2}{##3}}}%

```

`\cct@split@pdf@meta` is a helper function to split the list-form meta datum stored `\cct@cur@data` in into its items.

`{#1}` is a CS token that takes one argument. Each item of the list is recursively passed to that CS token

`{#2}` is the separator between the items

```

115 \def\cct@split@pdf@meta##1##2{%
116 \def\cct@split@pdf@meta####1##2####2\@nil####3{%
117 \if\relax\detokenize{####1}\relax\else
118 ####3{####1}%
119 \if!####2!\else
120 \edef\@argi{##2}%
121 \edef\@argii{####2}%
122 \ifx\@argi\@argii\else
123 \cct@split@pdf@meta####2\@nil{####3}%
124 \fi
125 \fi
126 \fi
127 }%
128 \expandafter\cct@split@pdf@meta\cct@cur@data##2\@nil{##1}%
129 }%

```

`\cct@write@hr@infodict` writes the PDF info dictionary entry via `hyperref` with the key `{#1}` and the value `{#2}`.

```

130 \def\cct@write@hr@infodict##1##2##3{%
131 \if\cc@pdf@std X%
132 \pdfstringdef\cct@cur@data{##3}%
133 \pdfextension info{/##2 (\cct@cur@data)}%
134 \else
135 \protected@edef\x{\noexpand\hypersetup{##1={\expandonce{##3}}}}\x
136 \fi
137 }%

```

After we decided how we want to process the PDF meta data, we now start to collect the necessary data points:

```

138 \cct@title@insert@xmp
139 \ifx\cc@color@enc\relax\else
140 \cct@title@process@oi
141 \fi
142 \cct@title@process@bkc
143 \cct@title@process@bkt
144 \cct@title@process@bkk
145 \cct@title@process@bka
146 \cct@title@process@bkl
147 }

```

`\cct@title@process@oi` processes the OutputIntent, which is needed by each PDF/A, UA and X standards, but handled differently: handling for PDF/UA is taken care of by `ltpdfa`, while PDF/X is handled here, directly.

```

148 \def\cct@title@process@oi{%
149 \ccIfAlly
150 {\edef\x{\noexpand\ccaAddToConfig{intent}{\ccUseProperty{output-intent}}}\x}
151 {\if\cc@pdf@std X%

```

```

152 \immediate\pdfobj stream attr{/N \ccIfComp{IccComponents}{\ccUseComp{IccComponents}}{\
    cct@default@icc@comp}} file{\ccIfComp{IccProfileFile}{\ccUseComp{IccProfileFile}}{suppl
    /\cc@color@enc.icc}}
153 \pdfcatalog{%
154   /OutputIntents [ <<
155     /Type /OutputIntent
156     /S /GTS_PDFX
157     /DestOutputProfile \the\pdflastobj\space 0 R
158     /OutputConditionIdentifier(\ccIfComp{IccIdentifier}{\ccUseComp{IccIdentifier}}{\
        cct@default@icc@iden})
159     /RegistryName(http://www.color.org)
160   >> ]
161 }
162 \fi
163 }}

```

2.1 Processing of the Document's Main Language

`\cct@title@process@bkl` writes the document's main language into the Info dictionary if no accessibility features are active. If it is active, the document language is handled by `ltpdfa`. The language tag is extracted via `babel`'s ini configuration file or the main document language.

```

164 \def\cct@title@process@bkl{%
165   \ccUnlessAlly{%
166     \edef\cct@lang@id{\localeinfo*{name.english}}%tag.bcp47
167     \cct@write@hr@infodict{pdflang}{\cct@lang@id}%
168   }%
169 }

```

2.2 Processing of the Document's Title

`\cct@title@process@bkt` processes the document's main title

```

170 \def\cct@title@process@bkt{%
171   \cslet{\ccPrefix Break}\space
172   \protected@xdef\@title{\ccUseProperty{doc-book-title}}%
173   \cct@write@pdf@meta@string{pdftitle}{Title}{\@title}%
174   \ccpgdefFromProperty{RunBookTitle}{run-book-title}%
175 }

```

2.3 Processing of the Document's Author

`\cct@title@process@bka` processes the document's main author or, if that doesn't exist, the main editor, or throws a warning if neither exist.

```

176 \def\cct@title@process@bka{%
177   \@tempwattrue
178   \begingroup
179   \ccGobble
180   \renewcommand\foreignlanguage[2]{\##2}%
181   \ccIfComp{AuthorPDFInfo}
182     {\ccpgdefFromComp{RunBookName}{AuthorPDFInfo}}
183     {\ccIfComp{EditorPDFInfo}
184       {\ccpgdefFromComp{RunBookName}{EditorPDFInfo}}
185     {\ccIfComp{AuthorNameList}

```

```

186     {\ccpgdefFromComp{RunBookName}{AuthorNameList}}
187     {\ccIfComp{EditorNameList}
188     {\ccpgdefFromComp{RunBookName}{EditorNameList}}
189     {\ifnum\ccAuthorCnt>\z@
190     \@setpar{\@par}%
191     \ccpxdefFromCountedComp{RunBookName}{Author}{author-list-pdfinfo-format}%
192     \else
193     \ifnum\ccEditorCnt>\z@
194     \ccpxdefFromCountedComp{RunBookName}{Editor}{editor-list-pdfinfo-format}%
195     \else
196     \ccPackageWarning{transcript-title}{Meta Data}{No author or editor given!}%
197     \@tempswafalse
198     \fi
199     \fi
200     }}}}%
201 \if@tempwa
202   \ccSanitizeStr\@author{\csname\ccPrefix RunBookName\endcsname}%
203   \cct@write@pdf@meta@list{pdfauthor}{Author}{\csname\ccPrefix RunBookName\endcsname}{\
204     ccaAddAuthor}{\and}%
205   \fi
206 \endgroup

```

2.4 Processing of the PDF's Creator, Producer, and Keywords Meta Data

`\cct@title@process@bkc` processes the metadata for the pdf creator and producer.

```

207 \def\cct@title@process@bkc{%
208   \cct@write@pdf@meta@string{pdfcreator}{Creator}{\ccWhenComp{PDFCreator}{\ccUseComp{PDFCreator
209     }}}}%
210   \cct@write@pdf@meta@string{pdfproducer}{Producer}{\ccWhenComp{PDFProducer}{\ccUseComp{
211     PDFProducer}}}%
212 }

```

`\cct@title@process@bkk` processed the metadata for the keywords.

```

211 \def\cct@title@process@bkk{%
212   \cct@write@pdf@meta@list{pdfkeywords}{Keywords}{\ccWhenComp{Keywords}{\ccUseComp{Keywords}}}{\
213     ccaAddKeyword}{\ccUseProperty{keywords-sep}}}%
214 }

```

2.5 Including the XMP Meta Data

`\cct@title@insert@xmp` inserts the contents of the XMP meta data file into the pdf, if it exists. There are two versions, depending on whether coco-accessibility is active or not.

```

214 \def\cct@title@insert@xmp{%
215   \edef\cca@xmp@file@name{\ccUseComponentFrom{titlepage}{XmpFile}.xmp}%
216   \ccIfAlly
217     {\cct@title@insert@xmp@ltpdfa}
218     {\cct@title@insert@xmp@direct}%
219 }

```

`\cct@title@insert@xmp@direct` is the default version which writes the xmp meta data directly into the PDF.


```

220 \def\cct@title@insert@xmp@direct{%
221   \IfFileExists{\cca@xmp@file@name}{%
222     \begingroup
223       \immediate\pdfobj stream attr {/Type /Metadata /Subtype /XML}
224       file{\cca@xmp@file@name}
225       \pdfcatalog{/Metadata \the\pdflastobj\space 0 R }
226       \if\cc@pdf@std X%
227         \ccWhenComp{PDFPart}{%
228           \pdfextension info{/GTS_PDFXConformance(PDF/\ccUseComp{PDFStandard}-\ccUseComp{PDFPart}
229             }\ccIfCompEmpty{PDFLevel}}{\ccUseComp{PDFLevel}}\ccIfCompEmpty{PDFIssue}}{\ccUseComp{PDFIssue}}) }%
230           \pdfextension info{/GTS_PDFXVersion(PDF/\ccUseComp{PDFStandard}-\ccUseComp{PDFPart}\
231             \ccIfCompEmpty{PDFLevel}}{\ccUseComp{PDFLevel}}\ccIfCompEmpty{PDFIssue}}{\ccUseComp{PDFIssue}}) }%
232         }
233       \fi
234     \endgroup%
235   }%
236 }

```

`\cct@title@insert@xmp@ltpdfa` is the version that uses `ltpdfa`'s mechanism to write XMP meta data into the PDF.

First we check if the specified xmp file exists. If it exists, the `DocumentInfo` is extracted from the XMP file. Otherwise, we set the `DocumentInfo` from the contents of the `titlepage` Container and let `ltpdfa` generate the `xmp` file.

```

235 \def\cct@title@insert@xmp@ltpdfa{%
236   \IfFileExists{\cca@xmp@file@name}
237     {\ccaAddToConfig{metadata}{xmpfile=\cca@xmp@file@name}%
238       \directlua{ally.meta:extract()}}
239     {\ccPackageWarning{A11y}{File}{%
240       \cca@xmp@file@name\space not found.^^J
241       Note that the ltpdfa package will create one^^J
242       from the Components given in the Meta Container.}}}

```

3 Intermediate Level Interfaces

`cct/maketitle/before` is expanded right before the titlepage is printed.

```

243 \NewHook{cct/maketitle/before}

```

`cct/maketitle/after` is expanded at the end of the titlepage.

```

244 \NewHook{cct/maketitle/after}

```

`cct/document/meta` is expanded at the very end of the Meta Container.

```

245 \NewHook{cct/document/meta}

```

`env/meta/begin` is used to add code to be executed at the very beginning of the Meta Container's main environment, before the Attributes are evaluated.

```

246 \NewHook{env/meta/begin}

```

`\cct@article@titlepage` is the prototype for article title pages.

```
247 \def\cct@article@titlepage{%
248   \ccUseProperty{article-title}%
249 }
```

`\cct@journal@titlepage` is the prototype for journal title pages.

```
250 \def\cct@journal@titlepage{%
251   \ccUseProperty{before-titlepage}%
252   \ccWhenComp{Cover}{\ccUseProperty{coverpage}}%Cover ist kein Bild, wird von uns gebaut
253   \ccUseProperty{before-titlepage-roman}%
254   \ccUseProperty{titlepage-roman}%
255   \ccUseProperty{after-titlepage}%
256 }
```

`\cct@book@titlepage` is the prototype for book (monographs and collections) title pages.

```
257 \def\cct@book@titlepage{%
258   \ccUseProperty{before-titlepage}%
259   \ccWhenComp{Cover}{\ccUseProperty{coverpage}}%
260   \ccUseProperty{before-titlepage-roman}%
261   \ccUseProperty{titlepage-roman}%
262   \ccUseProperty{after-titlepage}%
263 }
```

`\cct@maketitle` assigns one of the above definitions to the `\ccPrefix Maketitle` macro.

```
264 \def\cct@maketitle{%
265   \expandafter\gdef\csname\ccPrefix Maketitle\endcsname{%
266     \let\cc@cnt@grp\@empty
```

Here, we expand the `\cct/maketitle/before`.

```
267   \UseHook{cct/maketitle/before}%
268   \bgroup
269   \ccSetContainer{titlepage}%
270   \ccEvalType{Properties}%
271   \ifarticle
272     \cct@article@titlepage
273   \else
274     \ifjournal
275       \cct@journal@titlepage
276     \else
277       \cct@book@titlepage
278     \fi
279   \fi
280   \egroup
281   \UseHook{cct/maketitle/after}%
282 }%
283 }
```

3.1 Funds, Grants, and Supporters

This is a Subcontainer within `\ccPrefix Meta` which allows to set up multiple funding, grant, or supporter callouts.

`\cct@fundings@comp` wrapper to set up the Subcontainer

```

284 \def\cct@fundings@comp{%
285   \ccDeclareGlobalComponent[2]{FundingColumns}%
286   \ccDeclareComponent{FundingBlock}{\expandafter\global}{}%
287   \ccDeclareComponentGroup{Funding}{%
288     \ccDeclareCountedComponent{FundName}%
289     \ccDeclareCountedComponent{FundLogo}%
290     \ccDeclareCountedComponent{FundAltText}%
291     \ccDeclareCountedComponent{FundID}%
292   }{}%
293 }
```

`\cct@fundings@eval` Evaluator for the funding

```

294 \def\cct@fundings@eval{%
295   \def\cc@cur@cont{titlepage}%
296   \ccComposeCollection{Funding}{fund-format}{FundingBlock}%
297   \UseHook{cc/titlepage/funding}%
298 }%
299 \NewHook{cc/titlepage/funding}
```

`\cct@eds@eval` evaluator for the editors

```

300 \def\cct@eds@eval#1{%
301   \ccm@role@eval{#1}%
302   \cct@create@editor@string{#1}}
```

`\cct@create@editor@string` evaluates the editor string and adds a suffix.

```

303 \def\cct@create@editor@string#1{%
304   \expandafter\ifx\csname cc@cc@cur@cont @#1NameList\endcsname\relax\else
305     \csgappto{cc@cc@cur@cont @#1NameList}{\letcs\ccTotalCount{cc#1Cnt}\ccUseProperty{editor-
306       suffix}}}%
307 }
```

3.2 Simple Component Declarations

`\cct@simple@comps` wrapper for the Titlepage's simple Components.

```

308 \def\cct@simple@comps{%
```

General Information

`Cover` holds the path(!) to the cover image (without `\includegraphics!`)

```

309 \ccDeclareGlobalComponent{Cover}%
```

`Dedication` is a dedication.

```

310 \ccDeclareGlobalComponent{Dedication}%
```

`Acknowledgements` self explanatory.

```

311 \ccDeclareGlobalComponent{Acknowledgements}%
```

Statement additional publication statement

```
312 \ccDeclareGlobalComponent{Statement}%
```

Editorial generic statement by the editors of a periodical or collection.

```
313 \ccDeclareGlobalComponent{Editorial}%
```

Titles and Names

Title is the document's (printed) main title.

```
314 \ccDeclareGlobalComponent{Title}%
```

ShortTitleOR is a shortened version of the document title. If set, this is the title that is written into the PDF meta data (unless DocTitle is also set) and stored as running title (unless RunTitle is also set). If the Component is not used, the Title component is used, instead.

```
315 \ccDeclareGlobalComponent{ShortTitle}%
```

DocTitleOR is an override for the title that is written into the PDF meta data (unless an XMP file is used). This Component's value should only contain alphanumeric characters, ideally from the ASCII code block, but must not contain any L^AT_EX markup.

```
316 \ccDeclareGlobalComponent{DocTitle}%
```

RunTitleOR is an override for the title that is used as running title for page headers. It should contain only robust L^AT_EX markup.

```
317 \ccDeclareGlobalComponent{RunTitle}%
```

AltTitle is an alternative title for bastard title pages, etc. It is not used by CoCoT_EX, but some publisher styles may need it.

```
318 \ccDeclareGlobalComponent{AltTitle}%
```

Subtitle is the document's subtitle.

```
319 \ccDeclareGlobalComponent{Subtitle}%
```

TitleNote additional printed information tied to the document's title.

```
320 \ccDeclareGlobalComponent{TitleNote}%
```

RunNamesOR is an override for the document author's names, which is intended to be used in running page headers. If not set, the (calculated) ➤ **AuthorNameList** or ➤ **EditorNameList** is used, instead.

```
321 \ccDeclareGlobalComponent{RunNames}%
```

AltNames is an alternative data field for additional names.

```
322 \ccDeclareGlobalComponent{AltNames}%
```

Series

Series is the series title.

```
323 \ccDeclareGlobalComponent{Series}%
```

SubSeries is the subtitle for the series.

```
324 \ccDeclareGlobalComponent{SubSeries}%
```

SeriesNote are additional notes concerning the series.

```
325 \ccDeclareGlobalComponent{SeriesNote}%
```

Volume is the volume of the document within a series.

```
326 \ccDeclareGlobalComponent{Volume}%
```

Number is the number of the document within the series.

```
327 \ccDeclareGlobalComponent{Number}%
```

EditorNameListCC is the Collection Component for the Editor's names.

```
328 \ccDeclareGlobalComponent{EditorNameList}%
```

SeriesEditorNameListCC is the Collection Component for the Series-Editor's names.

```
329 \ccDeclareGlobalComponent{SeriesEditorNameList}%
```

Publisher Information

Publisher is the publisher name

```
330 \ccDeclareGlobalComponent{Publisher}%
```

PubDivision is the publisher division.

```
331 \ccDeclareGlobalComponent{PubDivision}%
```

PubDivInfo holds additional information about the publisher division.

```
332 \ccDeclareGlobalComponent{PubDivInfo}%
```

PubPlace holds the place of publication or the publisher's address.

```
333 \ccDeclareGlobalComponent{PubPlace}%
```

PubLogo holds the publisher logo. Depending on the publisher style, this may be just a path, or a complete `\includegraphics` expression.

```
334 \ccDeclareGlobalComponent{PubLogo}%
```

PubNote additional generic notes about the publisher.

```
335 \ccDeclareGlobalComponent{PubNote}%
```

PubWeb holds the url or email contact address of a publisher.

```
336 \ccDeclareGlobalComponent{PubWeb}%
```

Publication Meta

XmpFile is the basename of the XMP meta data file without the .xmp file ending (which is added automatically). The default value is `\jobname`.

```
337 \ccDeclareGlobalComponent[\jobname]{XmpFile}%
```

PDFCreator is the tool with which the original document was created (for *xerif*, this is usually M\$ Word).

```
338 \ccDeclareGlobalComponent{PDFCreator}%
```

PDFProducer is the tool with which the PDF was created. Defaults to “le-tex xerif with CoCoTeX v(CoCoTeX version)”

```
339 \ccDeclareGlobalComponent[le-tex xerif with CoCoTeX v.v0.6.2]{PDFProducer}%
```

Year the year of the publication

```
340 \ccDeclareGlobalComponent{Year}%
```

Date the date of writing/finishing. Defaults to `\today`.

```
341 \ccDeclareGlobalComponent[\today]{Date}%
```

Edition the edition of the publication.

```
342 \ccDeclareGlobalComponent{Edition}%
```

EditionNote additional notes about the particular edition.

```
343 \ccDeclareGlobalComponent{EditionNote}%
```

ISBNPreText text added before the ISBN block

```
344 \ccDeclareGlobalComponent{ISBNPreText}%
```

ISBN the publication’s international standard book number.

```
345 \ccDeclareGlobalComponent{ISBN}%
```

ISSN the publication’s international standard serial number for periodicals.

```
346 \ccDeclareGlobalComponent{ISSN}%
```

EISSN additional ISSN for electronic publications

347 \ccDeclareGlobalComponent{EISSN}%

EpubPreText additional text between ISBN and eISBN

348 \ccDeclareGlobalComponent{EpubPreText}%

EISBN ISBN for electronic publications.

349 \ccDeclareGlobalComponent{EISBN}%

EpubISBN ISBN for EPUBs.

350 \ccDeclareGlobalComponent{EpubISBN}%

ElibPDF additional serial number for electronic libraries.

351 \ccDeclareGlobalComponent{ElibPDF}%

BibLISSN additional ISSN for special libraries.

352 \ccDeclareGlobalComponent{BibLISSN}%

BibleISSN additional electronic ISSN for special libraries.

353 \ccDeclareGlobalComponent{BibleISSN}%

Funding

FundingPreText additional text before the funding list.

354 \ccDeclareGlobalComponent{FundingPreText}%

FundingPostText additional text after the funding list.

355 \ccDeclareGlobalComponent{FundingPostText}%

Imprint Meta Data

Biblio bibliographic information block.

356 \ccDeclareGlobalComponent{Biblio}%

BiblioTitle the title of the bibliographic information block

357 \ccDeclareGlobalComponent{BiblioTitle}%

Print the name and address of the printing company.

358 \ccDeclareGlobalComponent{Print}%

PrintNote additional information about the printing process.

```
359 \ccDeclareGlobalComponent{PrintNote}%
```

Lectorate name and address of the lectorate

```
360 \ccDeclareGlobalComponent{Lectorate}%
```

Translator name of the document's translator

```
361 \ccDeclareGlobalComponent{Translator}%
```

CoverConcept concept creator of the front page cover

```
362 \ccDeclareGlobalComponent{CoverConcept}%
```

CoverDesign designer of the front page cover.

```
363 \ccDeclareGlobalComponent{CoverDesign}%
```

Component creator of the front page cover image

```
364 \ccDeclareGlobalComponent{CoverImage}%
```

Typesetter name and address of the typesetter

```
365 \ccDeclareGlobalComponent{Typesetter}%
```

QA name of the person(s) responsible for quality assurance.

```
366 \ccDeclareGlobalComponent{QA}%
```

UsedFont information about the fonts used throughout the document

```
367 \ccDeclareGlobalComponent{UsedFont}%
```

Conversion name of the person(s) responsible for data conversion

```
368 \ccDeclareGlobalComponent{Conversion}%
```

EnvDisclaimer environmental disclaimer, used paper, etc.

```
369 \ccDeclareGlobalComponent{EnvDisclaimer}%
```

Advertise advertisements.

```
370 \ccDeclareGlobalComponent{Advertise}%
```

Licencing

LicenceText License Description

```
371 \ccDeclareGlobalComponent{LicenceText}%
```


LicenseLogo the path(!) to the licence logo. `\includegraphics` is added automatically.

372 `\ccDeclareGlobalComponent{LicenceLogo}%`

LicenceLink URL to the license.

373 `\ccDeclareGlobalComponent{LicenceLink}%`

LicenceName the plain name of the license.

374 `\ccDeclareGlobalComponent{LicenceName}%`

CopyrightDisclaimer self explanatory...

375 `\ccDeclareGlobalComponent{CopyrightDisclaimer}%`

Journal-specific Meta Data

JournalName Full name of the journal.

376 `\ccDeclareGlobalComponent{JournalName}%`

JournalAbbrev short name of the journal.

377 `\ccDeclareGlobalComponent{JournalAbbrev}%`

Issue of the journal.

378 `\ccDeclareGlobalComponent{Issue}%`

PubCycle Publication cycle

379 `\ccDeclareGlobalComponent{PubCycle}%`

Prices of the journal issues or subscription models

380 `\ccDeclareGlobalComponent{Prices}%`

MemberList in case of publishing organizations, this Component may hold a list of members.

381 `\ccDeclareGlobalComponent{MemberList}%`

Startpage is the start page of the Journal

382 `\ccDeclareGlobalComponent{Startpage}%`

Generic additional information

AddNoteI additional information for the first title page.

383 `\ccDeclareGlobalComponent{AddNoteI}%`

AddNoteII additional information for the second title page.

```
384 \ccDeclareGlobalComponent{AddNoteII}%
```

AddNoteIII additional information for the third title page.

```
385 \ccDeclareGlobalComponent{AddNoteIII}%
```

AddNoteIV additional information for the fourth title page.

```
386 \ccDeclareGlobalComponent{AddNoteIV}%
```

Colour Profile and Output Intent

The following Components control the output intent and included icc profile for the document.

➡ **IccProfileFile** holds the path (relative to the main tex file) and name of the .icc file.

```
387 \ccDeclareGlobalComponent{IccProfileFile}
```

➡ **IccComponents** holds the number of components in the color profile

```
388 \ccDeclareGlobalComponent{IccComponents}
```

➡ **IccIdentifier** holds the identifier of the color profile

```
389 \ccDeclareGlobalComponent{IccIdentifier}%
```

PDF Standard Conformance

The following Components control the PDF standard and conformity.

The defaults for the following Components are determined by the **pdf-standard** class option. For instance, in **PDF/X-1a:2003**, **X** is the Standard, **1** is the Part, **a** is the Level, and **2003** is the year when the standard was issued.

Passing it through the Meta container allows style authors to override local settings:

```
\ccAddToType{Components}{titlepage}{%
  \if\cc@pdf@std X\relax
    \ccComponent{PDFStandard}{X}
    \ccComponent{PDFPart}{3}
    \ccComponent{PDFLevel}{1}
    \ccComponent{PDFIssue}{2003}
  \else
    \ccComponent{PDFStandard}{A}
    \ccComponent{PDFPart}{2}
    \ccComponent{PDFLevel}{A}
    \ccComponent{PDFIssue}{1}
  \fi
}
```

which would always generate **PDF/X-3:2003** for any PDF/X value requested via the class options, and always **PDF/A-2a** for any other value.

➡ **PDFStandard** is the PDF Standard itself. Currently supported: X, A and UA.

```
390 \ccDeclareGlobalComponent[\cc@pdf@std]{PDFStandard}
```

➡ **PDFPart** holds the selected part of the PDF standard that the document should fulfill.

```
391 \ccDeclareGlobalComponent[\cc@pdf@part]{PDFPart}
```

➔**PDFLevel** holds the standard-specific conformance level.

```
392 \ccDeclareGlobalComponent[\cc@pdf@level]{PDFLevel}
```

➔**PDFIssue** contains the issue year to which the PDF Standard should conform.

```
393 \ccDeclareGlobalComponent[\cc@pdf@issue]{PDFIssue}
394 }
```

4 Default Settings

```
395 \ccAddToProperties{titlepage}{%
```

article-title <any> is the title of a single article. It defaults to what standard L^AT_EX does with **\maketitle** in the **article** class without the **titlepage** class option and uses the ➔**Title**, ➔**AuthorNameList**, and ➔**Date** Components.

```
396 \ccSetProperty{article-title}{%
397   \newpage
398   \null
399   \vskip 2em
400   \begin{center}%
401     \ccaStructStart{Titlepage}%
402     \let \footnote \thanks
403     {\LARGE\ccaStructStart{Title}\ccaReplaceStruct{\cca@id@document@dummys}\ccUseComp{Title}\
404       \ccaStructEnd{Title}\par}%
405     \vskip 1.5em%
406     {\large
407       \lineskip .5em%
408       \ccaStructStart{Authors}\ccUseComp{AuthorNameList}\ccaStructEnd{Authors}\par
409     }%
410     \vskip 1em%
411     {\large \ccaStructStart{P}\ccUseComp{Date}\ccaStructEnd{P}}}% % Set date in \large size.
412   \end{center}%
413   \ccaStructEnd{Titlepage}%
414   \par
415   \vskip 1.5em
416 }
```

```
416 % Title page hooks
417 % Before \ccPrefix Maketitle and outside the group
418 \ccSetProperty{before-titlepage}{%
419   \pagestyle{empty}%
420   \parindent\z@
421   \parskip\z@
422 }%
423 \ccSetProperty{after-titlepage}{\pagestyle{headings}}%
424 % Pages of title
425 %% Cover page
426 \ccSetProperty{coverpage}{%
427   \bgroup
428   \def\thepage{\@alph{c@page}}%
429   \smash{\rlap{%
430     \raise\dimexpr\headheight+\headsep+\topmargin+\topskip-\paperheight\relax
431     \vtop{%
432       \hskip-\oddsidemargin
```

```

433     \includegraphics[width=\paperwidth,height=\paperheight]{\ccUseComp{Cover}}}%
434   }}}}
435   \ccUseProperty{after-coverpage}%
436   \egroup
437 }%
438 \ccSetProperty{after-coverpage}{\cleardoublepage}%
439 \ccSetProperty{titlepage-roman}{%
440   \ccUsePropertyEnv{titlepage-i}%
441   \clearpage
442   \ccUsePropertyEnv{titlepage-ii}%
443   \clearpage
444   \ccUsePropertyEnv{titlepage-iii}%
445   \clearpage
446   \ccUsePropertyEnv{titlepage-iv}%
447   \clearpage
448 }%
449 %% Generic meta blocks
450 \ccSetProperty{generic-meta-heading-face}{\large}% format of the heading of a generic meta block
451 \ccSetProperty{generic-meta-format}{% Format of a single generic meta-block
452   \ccIfComp{Heading}{\ccUseProperty{generic-meta-heading-face}\ccUseComp{Heading}\par}\vskip\
453     baselineskip}}}%
454   \ccUseComp{Content}%
455   \par%
456 }%
457 %% Funding
458 \ccSetProperty{funding-format}{}%

```

Fallback for the width in case someone sets up a fixed value for a fund's width.

```

458 \ccSetProperty{funding-columns}{\ccUseComp{FundingColumns}}
459 \ccSetProperty{fund-width}{\dimexpr((\textwidth - (\ccUseProperty{funding-sep}*(\ccUseProperty
460   {funding-columns}-1)))/\ccUseProperty{funding-columns})\relax}
461 \ccSetProperty{fund-vertical-sep}{\baselineskip}%
462 \ccSetProperty{fund-sep}{%
463   \expandafter\@tempcnta\CalcModulo{\ccCurCount}{\ccUseProperty{funding-columns}}%
464   \strut
465   \ifnum\@tempcnta=\z@
466     \ifnum\ccCurCount<\ccTotalCount\relax
467       \vskip\ccUseProperty{fund-vertical-sep}%
468     \fi
469   \else
470     \hfill
471   \fi}
472 \ccSetProperty{fund-format}{% Format of a single fund/grant/sponsor
473   \strut\vtop{%
474     \hsize\ccUseProperty{fund-width}%
475     \ccWhenComp{FundName}{\ccaStructStart{P}\ccUseComp{FundName}\ccaStructEnd{P}}\{[1ex]}%
476     \ccaStructStart{Figure}%
477     \ccaAddPlacement{Block}%
478     \ccdefFromComp\cca@Gin@alt{FundAltText}%
479     \ccSanitizeStr\cca@Gin@alt{\cca@Gin@alt}%
480     \ccaAddAltText{\cca@Gin@alt}%
481     \includegraphics[max width=\hsize]{\ccUseComp{FundLogo}}%
482     \ccaStructEnd{Figure}%
483   }%
484   \ccUseProperty{fund-sep}%
485 }%
486 \ccSetProperty{funding-sep}{4mm}%
487 \ccSetProperty{funding-block}{%
488   \bgroup
489   \ccUseProperty{funding-format}%

```

```

489 \ccGetComp{FundingPreText}%
490 \ccGetComp*{FundingBlock}%
491 \ccGetComp{FundingPostText}%
492 \par
493 \egroup
494 }
495 %% before the roman part of the title pages but after cover page
496 \ccSetProperty{before-titlepage-roman}{%
497 \setcounter{page}{1}%
498 \def\thepage{\roman{page}}%
499 }%
500 \ccSetProperty{titlepage-i}{%
501 \ccaStructStart{Titlepage}%
502 \ifmonograph
503 \ccaStructStart{Authors}\ccUseComp{AuthorNameList}\ccaStructEnd{Authors}%
504 \else
505 \ccaStructStart{Editors}\ccUseProperty{EditorNameList}\ccaStructEnd{Editors}%
506 \fi%
507 \vskip\baselineskip
508 \bgroup
509 \ccUseProperty{title-face}\ccaStructStart{Title}\ccaReplaceStruct{\cca@id@document@dummys}\
    ccUseComp{Title}\ccaStructEnd{Title}%
510 \egroup
511 \ccaStructEnd{Titlepage}%
512 }%
513 \ccSetProperty{titlepage-ii}{%
514 \ccaStructStart{Titlepage}%
515 \ccGetComp{Editorial}%
516 \ccGetComp{SeriesNote}%
517 \ccGetComp{GenericMetaBlock}%
518 \vfill
519 \ccUseProperty{bio-output}%
520 \ccaStructEnd{Titlepage}%
521 }%
522 \ccSetProperty{titlepage-iii}{%
523 \ccaStructStart{Titlepage}%
524 \ifmonograph
525 \ccaStructStart{Authors}\ccUseComp{AuthorNameList}\ccaStructEnd{Authors}%
526 \else
527 \ccaStructStart{Editors}\ccUseProperty{EditorNameList}\ccaStructEnd{Editors}%
528 \fi%
529 \par
530 \ccUseProperty{title-format}
531 \ccGetComp{Edition}%
532 \ccGetComp{EditionNote}%
533 \ccaStructEnd{Titlepage}%
534 \vfill
535 \clearpage
536 }%
537 \ccSetProperty{titlepage-iv}{%
538 \ccaStructStart{Titlepage}%
539 \ccGetComp{Dedication}% Dedication
540 \ccGetComp{Acknowledgements}% Dedication
541 \ccUseProperty{funding-block}%
542 \vfill
543 \bgroup
544 \ccUseProperty{imprint-face}%
545 \ccaStructStart{MetaDatumBlock}%
546 \ccWhenComp{Biblio}{%
547 \ccaStructStart{MetaDatum}%
548 {\bfseries

```

```

549     \ccaStructStart{MetaDatumLabel}%
550     \ccGetComp*{BiblioTitle}%
551     \ccaStructEnd{MetaDatumLabel}%
552   }%
553   \ccaStructStart{MetaDatumValue}%
554   \ccGetComp*{Biblio}%
555   \ccaStructEnd{MetaDatumValue}%
556   \ccaStructEnd{MetaDatum}%
557 }%
558 \ccUseProperty{imprint-sep}%
559 \ccUseProperty{imprint}%
560 \ccaStructEnd{MetaDatumBlock}%
561 \egroup
562 \ccaStructEnd{Titlepage}
563 \clearpage
564 }%
565 %% predefined face and format Properties
566 \ccSetProperty{title-face}{\Huge\sffamily\bfseries}%

```

The document's main title is tagged with the  `<Title/>` tag, which in PDF/UA-1 should be mapped to  `<H1/>`.

```

567 \ccSetProperty{title-format}{%
568   \bgroup
569   \ccaVstructStart{Title}% PDF 2.0
570   \ccUseProperty{title-face}%
571   \ccUseComp{Title}\par
572   \ccaVstructEnd{Title}% PDF 2.0
573 \egroup
574 \ccWhenComp{Subtitle}{\ccUseProperty{subtitle-format}}%
575 \ccWhenComp{TitleNote}{\ccUseProperty{title-note-format}}%
576 \ccGetComp{Statement}%
577 \vskip\baselineskip
578 }%
579 \ccSetProperty{title-note-face}{\large\sffamily}%
580 \ccSetProperty{title-note-format}{%
581   \bgroup
582   \ccUseProperty{title-note-face}%
583   \ccUseComp{TitleNote}%
584 \egroup
585 \par
586 }%
587 \ccSetProperty{subtitle-face}{\Large\sffamily\bfseries}%
588 \ccSetProperty{subtitle-format}{%
589   \bgroup
590   \ccUseProperty{subtitle-face}%
591   \ccUseComp{Subtitle}%
592 \egroup
593 \par
594 }%
595 %% Imprint
596 \ccSetProperty{imprint-face}{\footnotesize}%
597 \ccSetProperty{imprint-sep}{\ifhmode\par\fi\addvspace{\baselineskip}}%
598 \ccSetProperty{imprint}{%
599   \ccUseProperty{publisher}%
600   \ccGetComp{Qualification}%
601   \ccGetComp{Conversion}%
602   \ccGetComp{CoverDesign}%
603   \ccGetComp{CoverImage}%
604   \ccGetComp{Lectorate}%
605   \ccGetComp{QA}%
606   \ccGetComp{Translator}%

```

```

607 \ccGetComp{Appraiser}%
608 \ccGetComp{Discussion}%
609 \ccGetComp{Typesetter}%
610 \ccGetComp{Print}%
611 \ccGetComp{UsedFont}%
612 \ccGetComp{DOI}%
613 \ccGetComp{Keywords}%
614 \ccUseProperty{imprint-sep}%
615 \ccGetComp{ISBNPreText}%
616 \ccGetComp{ISBN}%
617 \ccGetComp{EpubPreText}%
618 \ccGetComp{EISBN}%
619 \ccGetComp{EpubISBN}%
620 \ccUseProperty{imprint-sep}%
621 \ccGetComp{EnvDisclaimer}%
622 }%
623 \ccSetProperty{journal-meta}{%
624   \ccUseLabeledComp{Submitted}%
625   \ccUseLabeledComp{Received}%
626   \ccUseLabeledComp{Revised}%
627   \ccUseLabeledComp{Accepted}%
628   \ccUseLabeledComp{Published}%
629   \ccUseLabeledComp{Copyright}%
630   \ccUseLabeledComp{COIStatement}%
631   \ccUseLabeledComp{Keywords}%
632 }%
633 \ccSetProperty{licence}{%
634   \ccIfComp{LicenceLogo}{\includegraphics{\ccUseComp{LicenceLogo}}\par}{}%
635   \ccGetComp{LicenceText}%
636 }%
637 \ccSetProperty{copyright}{%
638   \ccaStructStart{MetaDatum}%
639   \ccaStructStart{P}%
640   \ccIfComp{Copyright}
641     {\ccUseComp{Copyright}\par}
642     {\textcopyright\space\ccUseComp{Year}\space\ccUseComp{Publisher},\space\ccUseComp{PubPlace}
643       }\par}%
643   \ccaStructEnd{P}%
644   \ccaStructEnd{MetaDatum}%
645 }%
646 \ccSetProperty{publisher}{%
647   \ccGetComp{PubDivInfo}%
648   \ccUseProperty{copyright}%
649   \ccGetComp{PubNote}%
650   \ccGetComp{PubWeb}%
651 }%
652 % Name Formats
653 \ccSetProperty{counted-meta-sep}{\ifnum\ccCurCount<\ccTotalCount\relax\vskip\baselineskip\fi}%
654   separator between multiple instances of the same meta datum
655 \ccSetProperty{counted-name-sep}{% Separator between multiple names; titlepage-specific override of
656   the same Property in coco-meta!
657   \ifnum\ccTotalCount>1\relax
658     \ifnum\ccCurCount<\ccTotalCount\relax
659       \ifnum\ccCurCount<\numexpr\ccTotalCount-1\relax
660         \ccUseProperty{name-sep}%
661       \else
662         \ccUseProperty{name-and}%
663       \fi
664     \fi
665   \fi
666 }%

```

```

665 % Aliases for different Roles, see coco-meta.sty for the actual Property values:
666 %% editors:
667 \ccPropertyLet{editor-cite-name-format} {role-cite-name-format}%
668 \ccPropertyLet{editor-short-cite-name-format} {role-short-cite-name-format}%
669 \ccPropertyLet{editor-full-name-format} {role-full-name-format}%
670 \ccPropertyLet{editor-pdfinfo-name-format} {role-pdfinfo-name-format}%
671 \ccPropertyLet{editor-correspondence-as-format} {role-correspondence-string-format}%
672 %
673 \ccPropertyLet{editor-list-print-format} {role-block-print-format}%
674 \ccPropertyLet{editor-list-cite-format} {role-block-cite-format}%
675 \ccPropertyLet{editor-list-short-cite-format} {role-block-short-cite-format}%
676 \ccPropertyLet{editor-list-pdfinfo-format} {role-block-pdfinfo-format}%
677 \ccPropertyLet{editor-list-correspondence-format} {role-block-correspondence-format}%
678 %% series-editors:
679 \ccPropertyLet{series-editor-cite-name-format} {role-cite-name-format}%
680 \ccPropertyLet{series-editor-short-cite-name-format} {role-short-cite-name-format}%
681 \ccPropertyLet{series-editor-full-name-format} {role-full-name-format}%
682 \ccPropertyLet{series-editor-pdfinfo-name-format} {role-pdfinfo-name-format}%
683 \ccPropertyLet{series-editor-correspondence-as-format} {role-correspondence-as-format}%
684 %
685 \ccPropertyLet{series-editor-list-print-format} {role-block-print-format}%
686 \ccPropertyLet{series-editor-list-cite-format} {role-block-cite-format}%
687 \ccPropertyLet{series-editor-list-short-cite-format} {role-block-short-cite-format}%
688 \ccPropertyLet{series-editor-list-pdfinfo-format} {role-block-pdfinfo-format}%
689 \ccPropertyLet{series-editor-list-correspondence-format} {role-block-correspondence-format}%
690 %% name Separators
691 \ccSetProperty{editor-suffix-sgl}{(Ed.)}%
692 \ccSetProperty{editor-suffix-pl}{(Eds.)}%
693 \ccSetProperty{editor-suffix}{%
694   \space
695   \ifnum\ccTotalCount=\@ne\relax
696     \ccUseProperty{editor-suffix-sgl}%
697   \else
698     \ccUseProperty{editor-suffix-pl}%
699   \fi
700 }%
701 % Biography
702 % those Properties control how (Role specific) Biography Blocks are formatted, i.e. the list of all
703 %   Biographies of a specific Role:
704 \ccSetProperty{role-bio-block-face}{}% face for the entire, role-specific, Biography Block
705 \ccSetProperty{role-bio-block-format}{\ccUseProperty{role-bio-block-face}\ccUseComp{Biography}}
706 \ccPropertyLet{author-bio-block-format} {role-bio-block-format}% Override for single author meta
707 \ccPropertyLet{editor-bio-block-format} {role-bio-block-format}% Override for single editor meta
708 \ccPropertyLet{series-editor-bio-block-format} {role-bio-block-format}% Override for single
709 %   series editor meta info
710 % those Properties control how a (Role specific) Biography is formatted:
711 \ccSetProperty{role-biography-format}{\bfseries\ccUseComp{FullName}:}\space\ccUseComp{Bio}\par}%
712 % Format of a single entry in the Role specific Biography
713 \ccPropertyLet{author-biography-format} {role-biography-format}% Override for single author meta
714 \ccPropertyLet{editor-biography-format} {role-biography-format}% Override for single editor meta
715 \ccPropertyLet{series-editor-biography-format} {role-biography-format}% Override for single
716 %   series editor meta info
717 \ccSetProperty{bio-output-format}{%
718   \ccGetComp{AuthorBioBlock}%
719   \ccGetComp{EditorBioBlock}%
720   \ccGetComp{SeriesEditorBioBlock}%

```



```

717 }%
718 % Running headers
719 \ccSetProperty{run-book-title}{%
720   \ccIfComp{RunTitle}
721     {\ccUseComp{RunTitle}}
722     {\ccIfComp{ShortTitle}
723       {\ccUseComp{ShortTitle}}
724       {\ccIfComp{Title}{\ccUseComp{Title}}{No title given!}}}%
725 }%
726 \ccSetProperty{run-book-name}{%
727   \ccIfComp{RunNames}
728     {\ccUseComp{RunNames}}
729     {\ifmonograph
730       \ccIfComp{AuthorNameList}
731         {\ccUseComp{AuthorNameList}}
732         {no author defined!}%
733       \else
734         \ccIfComp{EditorNameList}
735           {\ccUseComp{EditorNameList}}
736           {no editor defined!}%
737       \fi}%
738 }%
739 \ccSetProperty{doc-book-title}{%
740   \ccIfComp{DocTitle}
741     {\ccUseComp{DocTitle}}
742     {\ccIfComp{ShortTitle}
743       {\ccUseComp{ShortTitle}}
744       {\ccUseComp{Title}}}%
745 }%

```

output-intent <see below> sends the output intent information to the ltpdfa package. It must contain of three data fields:

profile with the name of the to-be-embedded .icc file,
componetns with an integer telling the pdfwriter how many values are coded by each color (e.g., 4 for cmyk, 3 for rgb)
identifier with the identifying name of the profile (e.g., **Coated FOGRA39** for the included cmyk profile, etc.)

```

746 \ccSetProperty{output-intent}{%
747   profile=\ccIfComp{IccProfileFile}{\ccUseComp{IccProfileFile}}{\suppl/\cc@color@enc.icc};%
748   components=\ccIfComp{IccComponents}{\ccUseComp{IccComponents}}{\cct@default@icc@comp};%
749   identifier=\ccIfComp{IccIdentifier}{\ccUseComp{IccIdentifier}}{\cct@default@icc@iden}%
750 }

```

```

751 }

```

5 Accessibility Features

5.1 Encoding of the PDF-A Conformance

If we generate PDF/UA, metadata is left to the ltpdfa package, so we only set the conformance level.

```

752 \ccWhenAlly{%
753   \AddToHook{cct/document/meta}{%
754     \edef\x{\noexpand\ccaSetDocinfo{conformance}}{%
755       \if\cc@pdf@std U

```

```

756     pdfuaid=\if@cc@pdf@two2\else1\fi
757   \else
758     ;pdfaid=\cc@pdf@part
759     ;level=\cc@pdf@level
760   \fi
761 }}%
762 \x}%

```

5.2 Titlepage Specific Role Maps

According to the “Tagged PDF Best Practice Guide” by the PDF Association, the main title of the document should be mapped to  `<P/>` until the more appropriate  `<Title/>` tag becomes widely accepted with the PDF 2.0 Standard.

```

763   \if@cc@pdf@two\else\ccaAddRolemap{Title}{P}\fi
764   \ccaAddRolemap{Titlepage}{Div}
765 }%ccIfAlly

```

```
</title>
```

Module 10

coco-floats.dtx

Output driver for `coco-floats.sty`.

```
<floats>
```

This module provides handlers for floating objects like tables and figures common to all CoCoTeX projects

Note that we take the term “Float” quite liberally: “Floats” basically mean “*things that may have a caption and which are somewhat outside the main text body*”, whether they actually float (i. e., moved into the `\@toplist` or `\@botlist` by L^AT_EX), or not.

```

23 %%
24 %% module for CoCoTeX that extends floating objects.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-floats}
30   [2025/07/14 v0.6.2 CoCoTeX floats module]
31 \ExplSyntaxOn
32 \keys_define:nn { cocotex/floats }
33 {
34   nofigs .code:n = {\global\let\ccf@no@figs\relax}
35 }
36 \ProcessKeyOptions[cocotex/floats]
37 \ExplSyntaxOff

```

1 Package Setup

1.1 Hard requirements

For the list-of mechanism, we need the CoCoTeX common module, which also loads the CoCoTeX kernel module.

```
38 \RequirePackage{coco-common}
```

For landscape images, we load the `rotating` package.

```
39 \RequirePackage{rotating}
```

Since file names form word often contain spaces and periods, we also include the `grffile` package.

```
40 \RequirePackage{grffile}
```

In order to save footnotes in captions, we require the `footnote` package.

```
41 \RequirePackage{footnote}
```

The `adjustbox` package is needed to restrict the maximum dimensions of image files.

```
42 \RequirePackage[Export]{adjustbox}
```

Finally, we need the `stfloats` package to allow bottom placed images on pages that start L^AT_EX's twocolumn mode.

```
43 \usepackage{stfloats}
44 \setcounter{dblbotnumber}{5}
```

1.2 Adjustments at the Beginning of the Document

```
45 \AtBeginDocument{%
```

The first adjustment implements the `nofigs` option by deactivating the `\includegraphics` macro.

```
46 \ifx\ccf@no@figs\relax
47   \renewcommand\includegraphics[2][{}]{%
48     \fi
```

`\ccf@ltx@includegraphics` stores the final definition of the `\includegraphics` macro for later use.

```
49 \global\let\ccf@ltx@includegraphics\includegraphics
```

Adjustments to the `htmltabs` package, if it is used:

```
50 \ifpackageloaded{htmltabs}
51   {\global\let\cc@uses@htmltabs\relax
52     \def\ht@adjust@linewidth{%
53       \advance\ht@h@offset\leftskip
54       \advance\ht@h@offset\totalleftmargin
55       \advance\linewidth-\rightskip
56     }%
57   }{}%
```

In order to catch the actual dimensions of the float box, we need to hook into L^AT_EX's `\@endfloatbox` macro. This macro is low-level enough so it covers regular, double-column, and rotated floats. Those values will later be written into the `.aux` file for each float. The values, together with the float's overall width, are stored in a macro called `cc-float-\the\ccf@int@cnt-dimens`.

```
58 \gappto\@endfloatbox{%
59   \global\ccf@total@height=\ht\@currbox\relax%
60   \global\ccf@total@depth=\dp\@currbox\relax%
61 }%
62 }%
```

1.3 Document Class-Option Overrides

Since CoCoTeX is mainly developed for automatic typesetting and float positioning, we set rather high tolerances for macros from L^AT_EX's standard `.clo` files:

```
63 \def\topfraction{0.9}
64 \def\textfraction{0.1}
65 \def\bottomfraction{0.8}
66 \def\totalnumber{8}
67 \def\topnumber{8}
```

```

68 \def\bottomnumber{8}
69 \def\floatpagefraction{0.8}
70 \@fptop\z@
71 \@fpbot\@flushglue

```

1.4 Internal Registers

`\ccf@floatbox` is for measuring the dimensions of the whole float

```

72 \newbox \ccf@floatbox

```

`\ccf@sub@box` is for measuring a single sub-float.

```

73 \newbox \ccf@sub@box

```

`\ccf@int@cnt` is an internal global counter that numbers all top-level floats sequentially.

```

74 \newcount\ccf@int@cnt \ccf@int@cnt\z@

```

`\ccSubFloatCnt` counts the sub-floats within a parent float Container instance.

```

75 \newcount\ccSubFloatCnt \ccSubFloatCnt=\z@\relax

```

`\ccf@int@sub@flt@cnt` is a temporary counter that holds the total number of subfloats inside a parent float Container instance.

```

76 \newcount\ccf@int@sub@flt@cnt \ccf@int@sub@flt@cnt\z@

```

Various dimension registers that store dimensions and spaces of floats and sub-floats:

`\ccf@sub@maxheight` stores and self-updates the height of the largest sub-float inside a float

```

77 \newdimen\ccf@sub@maxheight \ccf@sub@maxheight=\z@\relax

```

`\ccf@sub@sep` is the space between sub-floats

```

78 \newdimen\ccf@sub@sep \ccf@sub@sep=\fboxsep\relax

```

`\ccf@total@width` stores the cumulated overall width of the entire float

```

79 \newdimen\ccf@total@width \ccf@total@width=\textwidth\relax

```

`\ccf@total@height` is the overall height of a float

```

80 \newdimen\ccf@total@height \ccf@total@height=\textwidth\relax

```

`\ccf@total@depth` is the overall depth of a float

```

81 \newdimen\ccf@total@depth \ccf@total@depth=\textwidth\relax

```

`\ccf@calc@width` is an internal dimension used to calculate the ratio between multiple sub-floats that should be scaled to the same height

```

82 \newdimen\ccf@calc@width \ccf@calc@width=\ccf@total@width\relax

```

`\ccf@sep@top` holds the actual vertical skip inserted at the top of a float. If the float is floating, this equals to `intext-skip`, or `float-skip`, otherwise.

```
83 \newskip\ccf@sep@top \ccf@sep@top=\z@\relax
```

`\ccf@sep@bottom` is the same for the bottom vertical skip.

```
84 \newskip\ccf@sep@bottom \ccf@sep@bottom=\z@\relax
```

Internal dimensions for the horizontal margins:

`\ccf@margin@r` holds the right side margin

```
85 \newdimen\ccf@margin@r \ccf@margin@r=\z@\relax
```

`\ccf@margin@l` holds the left side margin

```
86 \newdimen\ccf@margin@l \ccf@margin@l=\z@\relax
```

`\ccf@margin@i` holds the inner margin

```
87 \newdimen\ccf@margin@i \ccf@margin@i=\z@\relax
```

`\ccf@margin@o` holds the outer margin

```
88 \newdimen\ccf@margin@o \ccf@margin@o=\z@\relax
```

`\ifccf@break@capt` is a locally adjustable switch that indicates whether captions are allowed to break across pages (true) or not (false).

```
89 \newif\ifccf@break@capt \@ccf@break@captfalse
```

`\ifccf@sameheight` determines if subfloats should be scaled such that they are all the same height.

```
90 \newif\ifccf@sameheight \@ccf@sameheighttrue
```

2 Internal macros

2.1 Generic resetter

Some macros are re-evaluated for each new top-level float.

`\ccf@reset@defaults` resets the those macros. It is called at the very beginning of each new float.

```
91 \def\ccf@reset@defaults{%
92   \global\ccSubFloatCnt=\z@
93   \global\ccf@total@width=\z@
94   \global\let\ccf@has@capt@top\@undefined
95   \global\let\ccf@has@capt@bottom\@undefined
96   \global\let\ccf@has@subcapt@top\@undefined
97   \global\let\ccf@has@subcapt@bottom\@undefined
98   \global\let\ccf@sub@contentsline@store\@empty
99   \global\ccf@sub@maxheight=\z@\relax
```

```

100 \tempcnta=\z@ \relax
101 \cc@reset@components{\cc@cur@cont}%
102 \let\ccf@prefix\@empty
103 \let\ht@cur@element\ccfCapType
104 \global\let\ccf@current@class\relax
105 \global\let\ccf@landscape\relax
106 }

```

2.2 Wrapper for L^AT_EX's Native float Environments

`\ccf@set@env` determines the low-level L^AT_EX float environment depending on orientation and document options. If no `float-pos` is given (implicitly or determined), the object is not treated as a float at all.

```

107 \def\ccf@set@env{%
108   \ifx\ccf@floatpos\@empty
109     \let\ccf@begin@env\bgroup
110     \let\ccf@end@env\egroup
111   \else
112     \ifx\ccf@landscape\@empty
113       \edef\ccf@env@name{sideways\ccfCapType}%
114       \edef\ccf@begin@env{\noexpand\begin{\ccf@env@name\ifx\ccf@do@dbl\relax*\fi}}%
115       \edef\ccf@end@env{\noexpand\end{\ccf@env@name\ifx\ccf@do@dbl\relax*\fi}}
116     \else
117       \edef\ccf@env@name{\ifx\ccf@do@dbl\relax db\fi float}%
118       \edef\ccf@begin@env{\expandafter\noexpand\csname @x\ccf@env@name\endcsname {\ccfCapType}[\ccf@floatpos]}%
119       \edef\ccf@end@env{\expandafter\noexpand\csname end@\ccf@env@name\endcsname}%
120     \fi
121   \fi}

```

`\ccf@get@seps` determines the top and bottom skips dependent on float position and orientation

```

122 \def\ccf@get@seps{%
123   \ifx\ccf@floatpos\@empty
124     \expandafter\ccf@sep@top\dimexpr\ccUseProperty{intext-skip-top}\relax%
125   \else
126     \expandafter\ccf@sep@top\dimexpr\ccUseProperty{float-skip-top}\relax%
127   \fi
128   \ifx\ccf@landscape\relax
129     \ifx\ccf@floatpos\@empty
130       \expandafter\ccf@sep@bottom\dimexpr\ccUseProperty{intext-skip-bottom}\relax%
131     \else
132       \expandafter\ccf@sep@bottom\dimexpr\ccUseProperty{float-skip-bottom}\relax%
133     \fi
134   \fi}

```

`\ccf@set*@@sep` Hooks to apply top and bottom skips, respectively.

```

135 \def\ccf@set@top@sep{\addvspace{\ccf@sep@top}}
136 \def\ccf@set@bot@sep{\addvspace{\ccf@sep@bottom}}

```

3 The Generic float Container

Components in Containers that are derived from the abstract `float` are in fact all Counted Components, where top-level instances use 0 as their internal counter and sub-floats are counted incrementally. Thus, we can *simplify* the **internal** names to `<Componentname>-<Counter>`, which is done via a custom wrapper for the `\cc@def@counted@comp` Component declarator.

`\ccfMakeComp` is a shortcut for float Component declarations.

`{#1}` is the generic name of the Component.

```
137 \def\ccfMakeComp#1{%
138   \cc@def@counted@comp{#1-\the\ccSubFloatCnt}{#1}{\}%
139 }
```

`\ccfMakeCompL` is a shortcut to declare Float Components together with their *list-of* overrides.

`{#1}` is the generic name of the Component.

```
140 \def\ccfMakeCompL#1{%
141   \ccfMakeComp{#1}%
142   \ccfMakeComp{Listof#1}}
```

`float` is the main parent Container for all floats.

```
143 \ccDeclareContainer{float}{%
```

3.1 Common Float Components

```
144 \ccDeclareType{Components}{%
```

First, we set the naming scheme of the internal Component macros which is then valid for all Component declarations by locally re-defining `\cc@counted@comp@scheme`.

```
145 \def\cc@counted@comp@scheme#1{#1-\the\ccSubFloatCnt}%
```

`Content` is the main content holder of a float.

```
146 \ccfMakeComp{Content}%
```

`Caption` is the main caption of a float.

`ListofCaptionOR` is the corresponding list-of-entry

```
147 \ccfMakeCompL{Caption}%
```

`Legend` is a legend to a float.

`ListofLegendOR` is the corresponding list-of-entry

```
148 \ccfMakeCompL{Legend}%
```

`Source` is the source of a float.

`ListofSourceOR` is the corresponding list-of-entry

```
149 \ccfMakeCompL{Source}%
```

`Number` is the counter of the float (including the label)

`ListofNumberOR` is the corresponding list-of-entry

```
150 \ccfMakeCompL{Number}%
```

`RefLabel` is the float's ID used for cross-references (replaces L^AT_EX's `\label` command)

```
151 \ccfMakeComp{RefLabel}%
```

`ListofEntryCL` is the Collection Component for the entire Listof entry.

```
152 \ccfMakeComp{ListofEntry}%
153 }%
```

3.2 Common Float Properties

```
154 \ccDeclareType{Properties}{%
```

Placement and Spacing

`intext-skip-top` <skip> vertical space between the text body and following non-floating floats

```
155 \ccSetProperty{intext-skip-top}{\intextsep}%
```

`intext-skip-bottom` <skip> vertical space between non-floating floats and the following text body

```
156 \ccSetProperty{intext-skip-bottom}{\intextsep}%
```

`float-skip-top` <skip> vertical space between text body and following floating floats

```
157 \ccSetProperty{float-skip-top}{\z@}%
```

`float-skip-bottom` <skip> vertical space between floating floats and following text body

```
158 \ccSetProperty{float-skip-bottom}{\z@}%
```

`sub-float-sep` <skip> horizontal space between sub-floats

```
159 \ccSetProperty{sub-float-sep}{\ccf@sub@sep}%
```

`margin-inner` <skip> inner margins of floats in twopage mode, i. e., left margin on odd pages and right margin on even pages, respectively.

```
160 \ccSetProperty{margin-inner}{\z@}%
```

`margin-outer` <skip> outer margin of floats in twopage mode, i. e., right margin on odd pages and left margin on even pages, respectively.

```
161 \ccSetProperty{margin-outer}{\z@}%
```

margin-left <skip>, auto horizontal space between the left page area boundary and the float. If auto, the left margin of the caption is controlled by the left margin of the Content object.

```
162 \ccSetProperty{margin-left}{\z@}%
```

margin-right <skip>, auto horizontal space between the right page area boundary and the float. If auto, the right margin of the caption is controlled by the right margin of the Content object.

```
163 \ccSetProperty{margin-right}{\z@}%
```

before-float <any> is the code that is executed before a float's content is evaluated.

```
164 \ccSetProperty{before-float}{\parindent\z@}%
```

fix-dimen [true|false] if true, the content is *always* scaled to **\hsize**, i.e., the **\textwidth** minus the left and right margins, even if it means to scale *up* an image file.

```
165 \ccSetProperty{fix-dimen}{false}%
```

inline-content-h-sep <any> how content instances inside the same ➡Content Component are separated horizontally.

```
166 \ccSetProperty{inline-content-h-sep}{\hfill}%
```

inline-content-h-sep-auto <dim> if multiple content instances inside the same ➡Content Component is measured from auto calculation of left and right margins, this is the horizontal space that is inserted inbetween them.

```
167 \ccSetProperty{inline-content-h-sep-auto}{0.5em}%
```

inline-content-v-sep <any> how content instances inside the same ➡Content Component are separated vertically.

```
168 \ccSetProperty{inline-content-v-sep}{\par\hskip\ccf@margin@1}%
```

Properties for Float-Type Handlers

float-content <any> is the material that is put into the **\ccf@sub@box** for measuring.

```
169 \ccSetProperty{float-content}{\ccUseComp{Content}}%
```

subfloat-content <any> is the material that is put into the **\ccf@sub@box** for measuring.

```
170 \ccSetProperty{subfloat-content}{\ccUseComp{Content}}%
```

float-render <any> the output routine for top-level float type specific contents

```
171 \ccSetProperty{float-render}{\ccUseComp{Content}}%
```

subfloat-render <any> the output routine for second-level float type specific contents.

```
172 \ccSetProperty{subfloat-render}{\ccUseComp{Content}}%
```

measure-content <any> defines how the width of single floats are measured (relevant if one of the horizontal margins is set to auto). By default, this does nothing, but the Property may be overridden by sub-Containers.

```
173 \ccSetProperty{measure-content}{}
```

Properties for Captions

caption-face <any> style applied to both top and bottom placed captions

```
174 \ccSetProperty{caption-face}{}%
```

caption-face-top <any> style applied to top placed captions only

```
175 \ccSetProperty{caption-face-top}{}%
```

caption-face-bottom <any> style applied to bottom placed captions only

```
176 \ccSetProperty{caption-face-bottom}{}%
```

source-face <any> style applied to the printed **Source** Component.

```
177 \ccSetProperty{source-face}{}%
```

legend-face <any> style applied to the printed **Legend** Component.

```
178 \ccSetProperty{legend-face}{}%
```

caption-sep-top <skip> vertical space between top caption and content, i. e., the skip *after* the top placed caption.

```
179 \ccSetProperty{caption-sep-top}{\z@}%
```

caption-sep-bottom <skip> vertical space between bottom caption and content, i.e., the skip *before* the bottom placed caption.

```
180 \ccSetProperty{caption-sep-bottom}{\z@}%
```

caption-top <any> the content of the top placed caption

```
181 \ccSetProperty{caption-top}{%
182   \ccIfComp{Number}{\ccUseProperty{number-face}\ccUseComp{Number}\ccUseProperty{number-sep}
183   }{}%
184   \ccUseComp{Caption}%
185 }%
```

caption-bottom <any> the content of the bottom placed caption

```
185 \ccSetProperty{caption-bottom}{%
186   \ccIfComp{Legend}{\ccUseProperty{legend-face}\ccUseComp{Legend}}{}%
187   \ccIfComp{Source}{%
188     \ccIfComp{Legend}{\par\nopagebreak}{}%
189     {\ccUseProperty{source-face}%
190     \ccUseComp{Source}}{}%
191   }%
192 }
```

subcaption-face <any> the style of captions of second level floats

```
191 \ccPropertyLet{subcaption-face}{caption-face}%
```

subcaption-face-top <any> the style of top placed captions of second level floats

```
192 \ccSetProperty{subcaption-face-top}{\ccUseProperty{caption-face-top}}%
```

subcaption-face-bottom <any> the style of bottom placed captions of second level floats

```
193 \ccSetProperty{subcaption-face-bottom}{\ccUseProperty{caption-face-bottom}}%
```

subcaption-add-sep-top <skip> additional vertical space between top caption and top sub-caption

194 \ccSetProperty{subcaption-add-sep-top}{\z@}%

subcaption-add-sep-bottom <skip> additional vertical space between bottom sub-caption and bottom caption

195 \ccSetProperty{subcaption-add-sep-bottom}{\z@}%

subcaption-sep-top <skip> vertical space between top placed sub-captions and content, i. e., the space *after* top placed sub-captions.

196 \ccSetProperty{subcaption-sep-top}{\ccUseProperty{caption-sep-top}}%

subcaption-sep-bottom <skip> vertical space between content and top placed sub-captions, i. e., the space before bottom placed sub-captions.

197 \ccSetProperty{subcaption-sep-bottom}{\ccUseProperty{caption-sep-bottom}}%

subcaption-top <any> the content of top placed sub-captions

198 \ccSetProperty{subcaption-top}{\ccUseProperty{caption-top}}%

subcaption-bottom <any> the content of bottom placed sub-captions

199 \ccSetProperty{subcaption-bottom}{\ccUseProperty{caption-bottom}}%

subcaption-valign-top [top|bottom|middle] vertical alignment of neighboring top-placed sub-captions

200 \ccSetProperty{subcaption-valign-top}{top}%

subcaption-valign-bottom [top|bottom|middle] vertical alignment of neighboring bottom-placed sub-captions

201 \ccSetProperty{subcaption-valign-bottom}{top}%

Properties for Counters

auto-number-prefix <any> Prefix for auto-generated Number components

202 \ccSetProperty{auto-number-prefix}{\csname\ccfCapType name\endcsname}%

auto-number-prefix-sep <any> Separator between the auto-generated number prefix and the auto-generated Number component.

203 \ccSetProperty{auto-number-prefix-sep}{~}%

numbering [auto|<any>] if auto, float counters in instances without the Number component are generated automatically. Any other value suppresses auto-numbering.

204 \ccSetProperty{numbering}{auto}%

numbering [auto|<any>] if auto, subfloat counters in instances without the Number component are generated automatically. Any other value suppresses auto-numbering.

Note: this Property has only effect when subfloats are second-level. In first-level sub-floats, the **numbering** Property is used.

205 \ccSetProperty{sub-numbering}{~}%

number-sep <any> separator between the printed float number and the caption

206 `\ccSetProperty{number-sep}{\enskip}%`

number-face <any> style of number, additional to caption-format

207 `\ccSetProperty{number-face}{\bfseries}%`

sub-number-sep <any> separator between number and caption in sub-floats

208 `\ccSetProperty{sub-number-sep}{\,,}%`

sub-number-style [arabic|Alph|alph|roman|Roman] numbering style for automatically generated subfloat counters

209 `\ccSetProperty{sub-number-style}{alph}%`

sub-number-face <any> style of the number of a subfloat

210 `\ccSetProperty{sub-number-face}{}%`

sub-number-before <any> stuff that is put immediately before the automatically generated subfloat counter

211 `\ccSetProperty{sub-number-before}{({}%`

sub-number-after <any> stuff that is put immediately after the automatically generated subfloat counter

212 `\ccSetProperty{sub-number-after}{})}%`

sub-number-format <any> the format of the number

213 `\ccSetProperty{sub-number-format}{%`
 214 `\ccUseProperty{float-number}%`
 215 `\ccUseProperty{sub-number-sep}%`
 216 `\ccUseProperty{sub-number}}%`

label-pos [top|bottom] position of the cross reference anchor, referring to top or bottom placed captions.

217 `\ccSetProperty{label-pos}{top}%`

sublabel-pos [top|bottom] position of the cross reference anchor for sub-floats, referring to top or bottom placed sub-captions.

218 `\ccSetProperty{sublabel-pos}{top}%`

Properties for List-Of Entries

list-of-page-sep <any> separator between the listof-entry and the page

219 `\ccSetProperty{list-of-page-sep}{\ccaVstructStart [Document] {leaders}\dotfill\ccaVstructEnd{`
`leaders}}%`

list-of-number-face <any> style of the listof-entry

220 `\ccPropertyLet{list-of-number-face}{list-of-caption-face}%`

list-of-number-sep <any> separator between the number and the listof entry.

221 `\ccSetProperty{list-of-number-sep}{\enskip}%`

`list-of-number-align` [`left`|`center`|`right`] horizontal alignment of the listof number within its local hbox.

```
222 \ccSetProperty{list-of-number-align}{left}%
```

`list-of-number-format` <any> format of the number in listof entries.

```
223 \ccSetProperty{list-of-number-format}{%
224   \bgroup
225   \ccUseProperty{list-of-number-face}%
226   \ccUseComp{ListofNumber}%
227   \ccUseProperty{list-of-number-sep}%
228   \egroup}%
```

`list-of-parfillskip` <skip> parfillskip of an entry in the listof

```
229 \ccSetProperty{list-of-parfillskip}{-\rightskip}%
```

`list-of-margin-right` <skip> right margin of the listof entry

```
230 \ccSetProperty{list-of-margin-right}{\@pnumwidth \@plus 1fil}%
```

`list-of-margin-left` [`auto`|<skip>] right margin of the listof entry

```
231 \ccSetProperty{list-of-margin-left}{auto}%
```

`list-of-indent` [`auto`|<dimen>] horizontal offset of the first line of an listof-entry, relative to margin-left.

```
232 \ccSetProperty{list-of-indent}{auto}%
```

`list-of-block` <any> format of the entire listof entry.

```
233 \ccSetProperty{list-of-block}{%
234   \ccUseProperty{list-of-caption-face}%
235   \ccaStructStart{TOCI}%
236   \ccaStructStart{P}\ccaStructStart{Reference}%
237   \ccIfComp{ListofNumber}
238     {\ccaStructStart{Lb1}\ccUseComp{list-of-hang-number}\ccaStructEnd{Lb1}}
239     {\leftskipOpt}%
240   \ccaStructStart{Span}%
241   \ccUseComp{ListofCaption}%
242   \ccUseProperty{list-of-page-sep}\ccUseComp{ListofPage}%
243   \ccaStructEnd{Span}%
244   \ccaStructEnd{Reference}\ccaStructEnd{P}%
245   \ccaStructEnd{TOCI}%
246 }%
```

`list-of-before-entry` <any> material inserted at the beginning of each listof entry

```
247 \ccSetProperty{list-of-before-entry}{%
248   \ccGobble
249   \leftskip\ccUseProperty{list-of-margin-left}\relax%
250   \rightskip \ccUseProperty{list-of-margin-right}\relax%
251   \parfillskip \ccUseProperty{list-of-parfillskip}\relax
252   \parindent\z@
253   \@afterindenttrue
254   \interlinepenalty\@M
255   \leavevmode
256   \null\nobreak
257 }% list-of-float appearance
```

`list-of-after-entry` <any> material inserted at the end of a listof entry.

```
258 \ccSetProperty{list-of-after-entry}{\par}%
259 }% /Properties
260 \ccDeclareType{Attributes}{%
```

`class` <string> is the class of the Float.

```
261 \ccDeclareAttributeHandler{class}{\xdef\ccf@current@class{\ccAttrVal}}%
```

`break-caption` <bool> whether or not the caption is allowed to break across pages

```
262 \ccDeclareAttributeHandler*{break-caption}[\ccf@break@captfalse]{\ccf@break@capttrue}%
```

`float-pos` [(h|t|p|b)*|h!] the float position of the float. h! means that the float is not actually floating and is equivalent to omitting the Attribute.

```
263 \ccDeclareAttributeHandler{float-pos}[\let\ccf@floatpos@empty]{\ccf@attr@pos{\ccAttrVal}}%
```

`orientation` landscape is whether the float is rotated by 90° (landscape) or not (if omitted, default)

```
264 \ccDeclareAttributeHandler{orientation}{\ccf@attr@orient{\ccAttrVal}}
```

`debug` <flag> if set, additional debugging is written into the shell and log file.

```
265 \ccDeclareAttributeHandler*{debug}[\let\ccf@debug\relax]{\let\ccf@debug\ccf@attr@debug}
```

`no-same-height` <flag> if set, the same-height calculations are de-activated for this float.

```
266 \ccDeclareAttributeHandler*{no-same-height}{\ccf@sameheightfalse}%
```

```
267 }
268 }% /Container
```

3.3 The Generic float Environment

This section defines the macros for the float's Container-specific LaTeX environment.

`\ccf@float` is a mid-level Macro that provides the common floating L^AT_EX environment. #1 is the float environment's kv-attribute list.

#1 float position (optional)

```
269 \def\ccf@float{\cc@opt@empty\ccf@float}
270 \def\ccf@float[#1]{%
271 \par
272 \begingroup
273 \cc@is@finalfalse
274 \global\advance\ccf@int@cnt\@ne
275 \ccEvalType{FloatEnvInfo}%
276 \ccf@reset@defaults
277 \ccToggleCountedConditionals
278 \ccEvalType{Properties}%
279 \ccIfPropVal{subfloat-same-height}{true}{\global\ccf@sameheighttrue}{\global\ccf@sameheightfalse}
```

```

280 \ccEvalAttributes{#1}%
281 \ccf@eval@class
282 \ccf@set@hsize
283 \ccf@get@seps
284 \ccEvalType{Components}%
285 \ccUseProperty{before-float}%
286 \ccf@set@env
287 \ifx\ccf@floatpos\empty\else\savenotes\fi
288 \cc@is@finaltrue
289 \ignorespaces}

```

`\endccf@float` is the end of the common float environment.

```

290 \def\endccf@float{%
291 \ccf@begin@env
292 \cc@is@finalfalse
293 \ccf@set@top@sep
294 \ccf@int@sub@flt@cnt=\ccSubFloatCnt\relax
295 \ccSubFloatCnt=\z@\relax
296 \cc@iterate{\ccSubFloatCnt}{\z@}{\the\ccf@int@sub@flt@cnt}
297 {\ccf@create@counter
298 \ccf@compose@listof}%
299 \ccSubFloatCnt=\ccf@int@sub@flt@cnt\relax
300 \ifnum\ccSubFloatCnt=\z@\relax
301 \ccUseProperty{measure-content}%
302 \fi
303 \ccf@test@caption{0}{\top}%
304 \ccf@test@caption{0}{\bottom}%
305 \bgroup
306 \cc@is@finaltrue

```

Floats as a whole are tagged as `<Aside/>` when PDF/UA standard is 2.0, or as `<Div/>`, otherwise.

```

307 \ccaStructStart{\ifcc@pdf@two\relax Aside\else Div\fi}%
308 \edef\ccf@parstruct@id{\ccaGetCurStruct{idx}}%
309 \hsize\ccf@total@width
310 \ccf@process
311 \ccaStructEnd{\ifcc@pdf@two Aside\else Div\fi}%
312 \par
313 \egroup
314 \ccSavePage
315 \ccf@set@bot@sep
316 \ccf@end@env
317 \ccf@debug%
318 \ifx\ccf@floatpos\empty\else\spewnotes\fi
319 \endgroup
320 \ccf@store@dimens
321 \global\let\ccf@current@class\relax
322 }

```

`\ccf@store@dimens` writes the float's final dimensions into the aux file.

```

323 \def\ccf@store@dimens{%
324 \immediate\write\auxout
325 {\string\expandafter\string\gdef\string\csname\space cc-float-\the\ccf@int@cnt-dimens\string
326 \endcsname{%
327 {\the\ccf@total@width}%
328 {\the\ccf@total@height}%
329 {\the\ccf@total@depth}%
330 }}%

```



```
330 }
```

`\ccfAdjustMargins` adjusts the left and right margins of the final float according to which is set to `auto`, where `{#1}` is the width of the content object.

If both are set to `auto`, the float is horizontally centered and the caption is the same width as the float. If only the right (or left margin) is set to `auto`, the image and its caption gets aligned to the left (or right), respectively (note the inversion!).

```
331 \def\ccfAdjustMargins#1{%
332   \global\@tempdima=\dimexpr\textwidth-#1\relax
333   \ifdim\@tempdima>z@\relax\ifdim\wd\ccf@sub@box>z@\relax
334     \ccIfStrEqual{\ccUseProperty{margin-left}}{auto}
335     {\global\ccf@total@width=\textwidth\relax
336      \ccIfStrEqual{\ccUseProperty{margin-right}}{auto}
337      {\global\ccf@margin@l=.5\@tempdima\relax
338       \global\ccf@margin@r=.5\@tempdima\relax}
339      {\global\ccf@margin@l=\@tempdima\relax}%
340       \global\advance\ccf@total@width-\ccf@margin@r\relax}
341     {\ccIfStrEqual{\ccUseProperty{margin-right}}{auto}
342      {\global\ccf@total@width=\textwidth\relax
343       \global\ccf@margin@r=\@tempdima\relax
344       \global\advance\ccf@total@width-\ccf@margin@r\relax}
345     {}}%
346   \fi\fi}
```

3.4 The SubFloat Environment

The SubFloat Sub-Container

Second-level floats (or SubFloats) are sub-containers of the float container.

`\ccSubFloat` is the user-level environment for sub-floats

```
347 \def\ccSubFloat{%
348   \ifx\ccf@is@subfloat\relax
349     \PackageError{coco-floats.sty}{Nested ccSubFloats detected!}{You cannot (yet) nest a `
350       ccSubFloat' environment into another `ccSubFloat' environment!}%
351   \else
352     \global\let\ccf@is@subfloat\relax
353     \global\advance\ccSubFloatCnt\@ne
354   \fi
355   \global\cslet{ccf@made@label@for@the\ccSubFloatCnt}\relax
356   \ignorespaces}
```

`\endccSubFloat` is the end of the sub-float environment

```
356 \def\endccSubFloat{%
357   \ifhmode\unskip\fi
358   \setbox\ccf@sub@box\hbox{%
359     \ccGobble
360     \@cc@is@finalfalse
361     \let\includegraphics\ccf@measuregraphics
362     \ccUseProperty{subfloat-content}}%
363   }%
364   \expandafter\xdef\csname ccf@\cc@cur@cont @width-\the\ccSubFloatCnt\endcsname{\the\wd\
365     ccf@sub@box}%}
```

```

365 \expandafter\xdef\csname ccf@cc@cur@cont @height-\the\ccSubFloatCnt\endcsname{\the\ht\
    ccf@sub@box}%
366 \expandafter\xdef\csname ccf@cc@cur@cont @depth-\the\ccSubFloatCnt\endcsname{\the\dp\
    ccf@sub@box}%
367 \@tempdima=\dimexpr\the\ht\ccf@sub@box+\the\dp\ccf@sub@box\relax
368 \@tempdimb=\dimexpr\the\wd\ccf@sub@box\relax
369 \ifdim\@tempdima>\ccf@sub@maxheight\relax
370 \global\ccf@sub@maxheight=\@tempdima\relax
371 \fi
372 \global\setbox\ccf@sub@box\box\voidb@x
373 \global\let\ccf@is@subfloat\@undefined
374 \aftergroup\ignorespaces
375 }

```

Printing the Subfloats

`\ccfRenderSubFloats` iterates through the single sub-floats and renders them in a nice row. #1 is the subfloat counter, #2 is the Component name that contains the actual contents of the sub-float, for `\ccPrefix Figure` it is `Fig`, for `\ccPrefix Table` it is `Content`.

```

376 \long\def\ccfRenderSubFloats#1#2{%
377 \leavevmode
378 \savenotes
379 \ifnum#1>\@ne\relax\hfill\fi
380 \vtop\bgroup
381 \expandafter\hsize\csname cc@cc@cur@cont @res@width-#1\endcsname\relax
382 \let\includegraphics\ccf@includesubgraphics
383 \leavevmode
384 \ccf@render@sub{#1}{#2}%
385 \egroup
386 \spewnotes
387 }

```

`\ccf@render@sub` renders a single sub-float. For the arguments, see `\ccfRenderSubFloats`, above.

```

388 \long\def\ccf@render@sub#1#2{%
389 \ccSubFloatCnt=#1\relax
390 \ccf@make@subcaption{top}%
391 \bgroup\strut\ccUseComp{#2}\strut\par\egroup%
392 \ccf@make@subcaption{bottom}}

```

3.5 Attribute Handlers

The following macros handle the Attributes of Float Container instances.

`\ccf@eval@class` expands the style class specific Properties.

```

393 \def\ccf@eval@class{%
394 \ccUseStyleClass{default}{\ccfCapType}%
395 \ifx\ccf@current@class\relax\else
396 \ccUseStyleClass{\ccf@current@class}{\ccfCapType}%
397 \fi}

```

`\ccf@attr@pos` is the handler for determining the float position. Some float Properties and Attributes restrict and override the explicit float positions, e.g., fully rotated floats must be positioned in `p` mode (i.e., as float page).

`{#1}` is the value of the float-pos Attribute. It may be any combination of h, t, p, b; or h!, which means that the float is non-floating (which is equivalent to an omitted float-pos Attribute)

```

398 \def\ccf@attr@pos#1{%
399   \edef\ccf@floatpos{#1}%
400   \def\@tempa{h!}\ifx\ccf@floatpos\@tempa\let\ccf@floatpos\@empty\fi
401   \def\@tempa{h}\ifx\ccf@floatpos\@tempa\def\ccf@floatpos{htpb!}\fi
402   \ifx\ccf@do@dbl\relax
403     \ifx\ccf@floatpos\@empty\def\ccf@floatpos{htpb!}\fi% 11514
404     \linewidth\dimexpr2\columnwidth+\columnsep\relax
405     \hsize\linewidth\relax
406   \fi
407 }

```

`\ccf@attr@orient` is the handler for the orientation Attribute.

`{#1}` is the value of the orientation Attribute. Currently, the only value that does things is landscape.

```

408 \def\ccf@attr@orient#1{%
409   \ccIfStrEqual{#1}{landscape}
410   {\linewidth\textheight
411     \hsize\linewidth
412     \global\let\ccf@landscape\@empty
413     \def\ccf@floatpos{p}}{}}

```

`\ccf@attr@debug` prints some debug information to `stdout` for a single float that has the Attribute `debug` set.

```

414 \def\ccf@attr@debug{%
415   \message{^^J[CoCo Float Debug]^^J
416     Textheight:\space\the\textheight^^J
417     Type:\space\space\space\space\space\space\space\space\cc@cur@cont^^J
418   \ifx\ccf@capType\cc@str@figure
419     Path: \space\space\space\space\space\space\space\ccf@fig@path^^J
420   \fi
421     Class:\space\space\space\space\space\space\space\ccf@current@class^^J
422     Floatpos:\space\space\space\space\ccf@floatpos^^J
423     Environ:\space\space\space\space\expandafter\noexpand\ccf@begin@env...\expandafter\noexpand
424       \ccf@end@env^^J
425     Subfloat:\space\space\space \the\ccSubFloatCnt^^J
426   \ifnum\ccSubFloatCnt=\z@
427     Width:\space\space\space\space\space\space\space\the\ccf@total@width^^J
428     Height:\space\space\space\space\space\space\space\the\ccf@total@height^^J
429     Depth:\space\space\space\space\space\space\space\the\ccf@total@depth^^J
430   \else
431     Width \the\ccSubFloatCnt:\space\space\space\space\space\space\space\expandafter\meaning\csname
432       ccf@\cc@cur@cont @width-\the\ccSubFloatCnt\endcsname^^J
433     Height \the\ccSubFloatCnt:\space\space\space\space\space\space\space\expandafter\meaning\csname ccf@
434       \cc@cur@cont @height-\the\ccSubFloatCnt\endcsname^^J
435     Depth \the\ccSubFloatCnt:\space\space\space\space\space\space\space\expandafter\meaning\csname
436       ccf@\cc@cur@cont @depth-\the\ccSubFloatCnt\endcsname^^J
437   \fi}}

```

3.6 Handling of List-of Entries

`\ccf@generate@listof@handlers` generates handlers for listof-entries.

- #1 is the file ending
- #2 is the caption type

#3 is the Container name

```
434 \def\ccf@generate@listof@handlers#1#2#3{%
```

`cc@listof@extract@data` The first macro that is dynamically defined, is the Component collector.

##1 is a numeric level that represents the order of the listof-entries

##2 is the caption type

##3 is the content of the `l@<level>` macro

##4 is the page number associated with that entry.

```
435 \expandafter\gdef\csname cc@#2@extract@data\endcsname##1##2##3##4{%
436   \ccSetContainer{#3}%
437   \def\ccfCapType{#2}%
438   \ccEvalType[#3]{Properties}%
439   \ccDeclareComponent{ListofCaption}{-}{-}%
440   \ccDeclareComponent{ListofLegend}{-}{-}%
441   \ccDeclareComponent{ListofSource}{-}{-}%
442   \ccDeclareComponent{ListofNumber}{-}{-}%
443   \ccDeclareComponent{ListofPage}{-}{-}%
444   \ccComponent{ListofPage}{\ccUseProperty{list-of-page-face}##4}%
445   \cc@expand@l@contents{##3}{#3}{Listof}{Caption}%
446   \cc@format@number{list-of}{Listof}{##1}%
447 }%
```

`\cc@listof@print@entry` The second dynamically defined macro is the entry renderer. It applies the Listof properties and selects the components to be printed. ##1 is the caption type of the float.

```
448 \expandafter\gdef\csname cc@#2@print@entry\endcsname##1{%
449   \bgroup
450   \UseHook{cc/listof/##1/before}% was list-of-before-hook-##1
451   \ccUseProperty{list-of-before-entry}%
452   \@ccis@finaltrue
453   \ccUseProperty{list-of-block}%
454   \UseHook{cc/listof/##1/after}% was list-of-after-hook-##1
455   \ccUseProperty{list-of-after-entry}%
456   \egroup}%
```

`cc/listof/[type]/before` is expanded before the List-of entry for a float of type `[type]` is printed

```
457 \NewHook{cc/listof/#2/before}%
```

`cc/listof/[type]/after` is expanded after the Listof entry for a float of type `[type]` is printed, but before the Property `list-of-after-entry` is called.

```
458 \NewHook{cc/listof/#2/after}%
459 }
```

`\ccf@addcontentsline` fork of L^AT_EX's `\addtocontents` macro.

```
460 \def\ccf@addcontentsline{%
461   \ccWhenComp{ListofEntry}{%
462     \protected@write\@auxout
463     {\ccGobble\ccaProtect}%
464     {\string\@writefile{\ccf@cap@list@type}
465     {\protect\ccContentsline
466     {\ifnum\ccSubFloatCnt>\z@\ccIfAttr{\ccfCapType}{subfloat}{sub}{-}\fi\ccfCapType}
```

```

467     {\ccUseComp{ListofEntry}}
468     {\thepage}
469     {\@currentHref}\protected@file@percent}}\relax%
470     \protected@write\@auxout{\ccaEnable}{\relax}%
471   }%
472 }

```

`\ccf@check@empty` is a wrapper for CoCoTeX kernel's `\cc@check@empty`

```

473 \def\ccf@check@empty#1{\cc@check@empty{\cc@cur@cont}{#1-\the\ccSubFloatCnt}{Listof}}

```

`\ccf@compose@listof` is the Component Group Handler for `Listof` Components.

```

474 \def\ccf@compose@listof{%
475   \ccf@check@empty{Number}%
476   \ccf@check@empty{Caption}%
477   \ccf@check@empty{Legend}%
478   \ccf@check@empty{Source}%
479   \let\ccf@listof@entry\relax
480   \ccWhenComp{ListofCaption}{\csgappto{\ccf@listof@entry}{\string\ccComponent{ListofCaption}{\ccUseComp{ListofCaption}}}}%
481   \ccWhenComp{ListofNumber}{\csgappto{\ccf@listof@entry}{\string\ccComponent{ListofNumber}{\ccUseComp{ListofNumber}}}}%
482   \ccWhenComp{ListofLegend}{\csgappto{\ccf@listof@entry}{\string\ccComponent{ListofLegend}{\ccUseComp{ListofLegend}}}}%
483   \ccWhenComp{ListofSource}{\csgappto{\ccf@listof@entry}{\string\ccComponent{ListofSource}{\ccUseComp{ListofSource}}}}%
484   \ifx\ccf@listof@entry\relax\else
485     \bgroup
486     \ccGobble
487     \ccaProtect
488     \protected@edef\@ccf@listof@entry{\ccf@listof@entry}%
489     \ccComponentEA{ListofEntry}{\@ccf@listof@entry}%
490     \egroup
491   \fi
492 }%

```

`\ccf@write@listof` The last macro to be defined here is the list-of writer. This macro is responsible to write the entry into TeX's auxiliary file system.

```

493 \def\ccf@write@listof{%
494   \ccUnlessAttr{\ccfCapType}{nolist}
495   {\ifnum\ccSubFloatCnt=\z@\relax
496     \ccIfAttr{\ccfCapType}{subfloat}
497     {\ccSubFloatCnt=\z@\relax
498       \cciterate{\ccSubFloatCnt}{\z@}{\the\ccf@int@sub@flt@cnt}
499       {\ccf@addcontentsline}}%
500     {\ccf@addcontentsline}%
501   \else
502     \ccIfAttr{\ccfCapType}{subfloat}{\ccf@addcontentsline}%
503   \fi}}

```

3.7 Label and Referencing mechanisms

Generation of Number Components

`\ccf@create@counter` checks for the various parameters that control whether or not a Number component is auto-generated for each sub-float.

```

504 \def\ccf@create@counter{%
505   \ccIfAttrIsSet{\ccfCapType}{nonumber}{%
506     {\ccUnlessComp{Number}
507       {\ccIfPropVal{numbering}{auto}
508         {\ccIfAttr{\ccfCapType}{subfloat}
509           {\ifnum\ccSubFloatCnt=\z@\relax
510             \ccf@set@top@counter%
511           \else
512             \ccIfPropVal{sub-numbering}{auto}
513             {\ccf@set@subcounter}{}%
514           \fi}
515     {\ccf@set@top@counter}}{}}}%

```

`\ccf@set@top@counter` generates first level float counter.

```

516 \def\ccf@set@top@counter{%
517   \ccWhenComp{Caption}{%
518     \global\expandafter\advance\csname c@\ccfCapType\endcsname\@ne\relax
519     \ccdefFromProperty\ccf@name@prefix{auto-number-prefix}%
520     \ccdefFromProperty\ccf@name@sep{auto-number-prefix-sep}%
521     \protected@edef\@tempa{\ccf@name@prefix\ccf@name@sep\expandafter\the\csname c@\ccfCapType\endcsname}%
522     \ccComponentEA{Number}{\@tempa}%
523   }%
524 }

```

`\ccf@set@subcounter` generates second level counters for numbered sub-floats. #1 is the sub-float counter.

```

525 \def\ccf@set@subcounter{%

```

`float-number` <any> the counter of a first-level float

```

526   \ccSetPropertyVal{float-number}{\csname cc@\cc@cur@cont @Number-0\endcsname}%

```

`sub-number` <any> the counter of a second-level float

```

527   \ccSetPropertyVal{sub-number}{%
528     \begingroup
529     \expandonce{\ccUseProperty{sub-number-face}}%
530     \relax\ccUseProperty{sub-number-before}%
531     \csname @\ccUseProperty{sub-number-style}\endcsname{\the\ccSubFloatCnt}%
532     \ccUseProperty{sub-number-after}%
533   \endgroup}%
534   \ccComponent{Number}{\ccUseProperty{sub-number-format}}%
535 }

```

Generation of $\LaTeX$ Labels

`\ccfCreateLabel` creates labels

```

536 \def\ccfCreateLabel{%
537   \ccIfComp{Number}
538   {\def\cc@fallback@anchor{%
539     \ccGobble
540     \ccdefFromComp\@currentlabel{Number}%
541     \ccdefFromComp\@currentlabelname{ListofCaption}}%
542   \def\cc@labelname@comp{Caption}}
543   {\def\cc@fallback@anchor{\phantomsection}}%
544   \expandafter\ccCreateLabel\expandafter{\ccfCapType}

```

3.8 Processing the Float

Sizes, Spacing and Margins

`\ccf@set@hsize` calculates the available maximum width for the float contents and captions according to the values of the `margin-right` and the `margin-left` properties.

```

545 \def\ccf@set@hsize{%
546   \expandafter\ccf@sub@sep\ccUseProperty{sub-float-sep}\relax%
547   \global\ccf@total@width=\hsize\relax
548   \ccf@set@margin{l}{left}%
549   \ccf@set@margin{r}{right}%
550   \ccf@set@margin{i}{inner}%
551   \ccf@set@margin{o}{outer}%
552   \ccf@set@margins
553   \global\advance\ccf@total@width-\ccf@margin@r\relax
554 }

```

`\ccf@set@margin` is used to set the internal margin macros

`{#1}` is the location of the margin, either l, r, i, or o

```

555 \def\ccf@set@margin#1#2{%
556   \ccIfStrEqual{\ccUseProperty{margin-#2}}{auto}
557   {\csname ccf@margin@#1\endcsname=\z@\relax}
558   {\csname ccf@margin@#1\expandafter\endcsname\ccUseProperty{margin-#2}\relax}%
559 }

```

`\ccf@set@margins` realises inner and outer margins via the left and right margins.

```

560 \def\ccf@set@margins{%
561   \ccTestPage
562   \if@cc@odd
563     \advance\ccf@margin@l\ccf@margin@i
564     \advance\ccf@margin@r\ccf@margin@o
565   \else
566     \advance\ccf@margin@l\ccf@margin@o
567     \advance\ccf@margin@r\ccf@margin@i
568   \fi
569 }

```

Processing the Contents of the Float Environment

`\ccf@process` calculates the dimensions of the content of a float environment (including captions and spacing) and eventually prints the contents using the `float-render` and `subfloat-render` Properties.

```

570 \def\ccf@process{%
571   \ifx\ccf@has@capt@top\@empty\leavevmode\fi
572   \ccf@make@outer@caption{top}%
573   \ifnum\the\ccSubFloatCnt=\z@\relax
574     \bgroup\advance\hsize-\ccf@margin@l
575     \cc@is@finaltrue
576     \ccUseProperty{float-render}%
577     \egroup
578   \else
579     \ccf@test@subcapt
580     \ccf@calc@sameheight
581     \def\ccf@prefix{sub}%
582     \ifx\ccf@has@subcapt@top\@empty\ccf@calc@row@ht{top}\fi
583     \ifx\ccf@has@subcapt@bottom\@empty\ccf@calc@row@ht{bottom}\fi

```

```

584 \bgroup
585 \advance\hsize-\ccf@margin@l
586 \@cc@is@finaltrue
587 \ccUseProperty{subfloat-render}%
588 \egroup
589 \let\ccf@prefix\@empty
590 \fi
591 \ccf@make@outer@caption{bottom}%
592 }

```

`\ccf@calc@row@ht` calculates the heights of all captions in the same row.

`{#1}` determines if the `top` or `bottom` row is calculated.

```

593 \def\ccf@calc@row@ht#1{%
594 \@tempcnta\z@
595 \@tempdima\z@
596 \cc@iterate{\@tempcnta}{\@ne}{\ccSubFloatCnt}{%
597 \setbox\z@\vbox{%
598 \ccSubFloatCnt\@tempcnta\relax
599 \expandafter\hsize\expandafter\dimexpr\csname cc@\cc@cur@cont @res@width-\the\@tempcnta\
600 endcsname\relax
601 \ccGobble
602 \ccUseProperty{\ccf@prefix caption-face}%
603 \ccUseProperty{\ccf@prefix caption-face-#1}%
604 \leavevmode
605 \strut\ccUseProperty{caption-#1}\strut%
606 }%
607 \expandafter\ifdim\dimexpr\ht\z@+\dp\z@\relax>\@tempdima \@tempdima\dimexpr\ht\z@+\dp\z@\
608 relax\fi
609 }%
610 \expandafter\edef\csname ccf@capt@row@height@#1\endcsname{\the\@tempdima}%
611 }

```

`\ccf@calc@sameheight` calculates the target width of each sub-image in the  Figure Container if each of the sub-images is required to match a uniform height.

```

610 \def\ccf@calc@sameheight{%
611 \if@ccf@sameheight

```

If all sub figures should be scaled to the same height, we reserve two registers: dimension `\@tempdima`, which holds the cumulated widths of all adjusted sub images, and `\@tempcnta`, which serves as a temporary index for each processed sub float.

```

612 \@tempdima=\z@\relax
613 \@tempcnta=\z@\relax

```

The `\ccf@calc@width` dimension register holds the target width of the total sub float block. It is initialized to `\ccf@total@width` and reduced by the amount of the left margin.

```

614 \ccf@calc@width=\ccf@total@width\relax
615 \advance\ccf@calc@width-\ccf@margin@l\relax

```

now, we iterate through the sub floats, storing the current index in `\@tempcnta`.

```

616 \cc@iterate{\@tempcnta}{\@ne}{\ccSubFloatCnt}{%

```

First, we calculate the ratio between the height of the largest image in the Figure container, and the height of the current image, resulting in `\@tempa` being a rational number ≥ 1 .


```

617 \edef\@tempa{\CalcRatio{\ccf@sub@maxheight}{\csname ccf@cc@cur@cont @height-\the\@tempcnta
\endcsname}}}%

```

Now, we subtract the mandatory space between to sub images (as indicated by the Property) to the total target width `\ccf@calc@width`.

```

618 \ifnum\the\@tempcnta>\@ne\relax
619 \advance\ccf@calc@width-\ccf@sub@sep\relax%
620 \fi

```

Temporary length register `\@tempdimc` holds the natural width of the current image.

```

621 \expandafter\@tempdimc\csname ccf@cc@cur@cont @width-\the\@tempcnta\endcsname\relax

```

Temporary length register `\@tempdimb` holds the width of the down-scaled image. The scaling factor is `\@tempa` calculated earlier.

```

622 \@tempdimb=\@tempa\@tempdimc\relax

```

This values is stored in `cc@\cc@cur@cont @adj@width-\the\@tempcnta` for each image.

```

623 \csedef{cc@\cc@cur@cont @adj@width-\the\@tempcnta}{\the\@tempdimb}%

```

This width is added to the `\@tempdima` register.

```

624 \advance\@tempdima\@tempdimb
625 }%

```

Once we calculated the new width of each image, we reset the temporary sub-float counter `\@tempcnta`, and the temporary length registers `\@tempdimb` and `\@tempdimc`.

```

626 \@tempcnta=\z@\relax
627 \@tempdimb=\z@\relax
628 \@tempdimc=\z@\relax

```

Up to this point, we calculated the ratio of each image in relation to the laregst image and what width each image would have if we scaled it *up* to this maximum height. As a by-product, we also calculated the target width of all sub images by substracting the separator space inserted between each image.

Now, we need to calculate the acual scaling factor. For that, we loop through all subfloats once more, re-using the temproary counter `\@tempcnta` as index of the current image.

```

629 \cc@iterate{\@tempcnta}{\@ne}{\ccSubFloatCnt}{%

```

`\@tempdima` stores the sum of the widths of all images if they were upscaled to match the highest image. We now calculate the ratio between the width of the current, upscaled, image and and sum of the widths of all upscaled images. The result is stored in `tempa` as a rational number between 0 and 1.

```

630 \edef\@tempa{\CalcRatio{\csname cc@\cc@cur@cont @adj@width-\the\@tempcnta\endcsname}{\
@tempdima}}}%

```

We now use this factor to calculate the target width of the sub float by multiplying this scaling factor with the actually available width of the Container stored in `\ccf@calc@width`.

```

631 \csedef{cc@\cc@cur@cont @res@width-\the\@tempcnta}{\dimexpr\@tempa\ccf@calc@width\relax}%

```

We now store the natural height of the current image in `\@tempdimc`...

```

632 \@tempdimc\dimexpr\csname ccf@\cc@cur@cont @height-\the\@tempcnta\endcsname\relax

```

...and also multiply it by the scaling factor such that `\@tempdimc` now holds the actual height of the current image after down-scaling.

```
633 \@tempdimc\dimexpr\@tempa\@tempdimc\relax
```

`\@tempdimb` stores the total height of the down-scaled images. If all calculations are correct, `\@tempdimb` should be equal for each iteration.

```
634 \ifdim\@tempa\@tempdimb<\@tempdimc\relax
635 \@tempdimb=\@tempdimc\relax
636 \fi
637 }%
638 \else
```

If images should not be scaled to the same height, we simply set the target width of each sub image to its natural value.

```
639 \cciterate{\@tempcnta}{\@ne}{\ccSubFloatCnt}{%
640 \csletcs{cc@\cc@cur@cont @res@width-\the\@tempcnta}{ccf@\cc@cur@cont @width-\the\@tempcnta}
641 }%
642 \fi
```

In any case, the total height of the entire image Container is stored in `cc@\cc@cur@cont @res@height`.

```
643 \csedef{cc@\cc@cur@cont @res@height}{\the\@tempdimb}%
644 }
```

3.9 Caption mechanism

`\ccf@test@caption` tests if the current sub-float has any top or bottom caption that needs to be printed.

- #1 is the value of the sub-float counter
- #2 indicates if the caption belongs to the whole float (empty) or a sub-float (sub)
- #3 top or bottom

We compare the caption of the current `\SubCounter` level with a caption of a non-existing, negative, float level in case there is non-expandable material hard-coded into the `caption-#3` Property. If we were to compare the width of the `\hbox` with `\z@`, this scenario would give us false positives.

Warning: Long captions can cause the `\hbox`'s width to exceed `\maxdimen`. To avoid L^AT_EX errors in this case, we compare `sp` instead of `pt`. This, however, means that if the difference is less than 1pt, the test fails and no caption is printed!

```
645 \def\ccf@test@caption#1#2#3{%
646 \ccaDisable\@cc@is@finalfalse
647 \setbox\cc@tempboxa\hbox{\ccGobble\ccSubFloatCnt=0#1\relax\ccUseProperty{#2caption-#3}\relax}%
648 \setbox\cc@tempboxb\hbox{\ccGobble\ccSubFloatCnt\m@ne\relax\ccUseProperty{#2caption-#3}\relax}
649 \edef\my@wda{\expandafter\strip@pt\wd\cc@tempboxa sp}%
650 \edef\my@wdb{\expandafter\strip@pt\wd\cc@tempboxb sp}%
651 \ifdim\my@wda>\my@wdb\relax
652 \expandafter\global\expandafter\let\csname ccf@has@#2capt@#3\endcsname\@empty
653 \fi
654 \ccaEnable\@cc@is@finaltrue
655 }
```

`\ccf@test@subcapt` tests if the current float has any top or bottom captions that need to be printed

```

656 \def\ccf@test@subcapt{%
657   \cc@iterate{\@tempcnta}{\@ne}{\ccSubFloatCnt}{%
658     \ccf@test@caption{\the\@tempcnta}{sub}{top}%
659     \ccf@test@caption{\the\@tempcnta}{sub}{bottom}%
660   }%
661 }

```

`\ccf@capt@top@offset` determines the spacing inserted **above both captions**.

```

662 \def\ccf@capt@top@offset#1{%
663   \ccIfStrEqual{#1}{top}{%
664     \par\ifccf@break@capt\else\nopagebreak\fi%
665     \expandafter\@tempskipa\ccUseProperty{\ccf@prefix caption-sep-bottom}\relax%
666     \advance\@tempskipa\dimexpr-\topskip+\dp\strutbox\relax
667     \ifccf@break@capt\advance\@tempskipa\dimexpr-\baselineskip-\ht\strutbox+\topskip\relax\fi
668     \ifx\ccf@has@subcapt@bottom\empty
669       \ifnum\the\ccSubFloatCnt=\z@
670         %% subcapt-bot exists and capt-bot is rendered
671         \advance\@tempskipa\dimexpr\dp\strutbox\relax
672         \expandafter\advance\expandafter\@tempskipa\ccUseProperty{subcaption-add-sep-bottom}\relax%
673       \fi
674     \fi
675     \vskip\@tempskipa
676     \leavevmode
677   }}

```

`\ccf@capt@bottom@offset` determines the spacing inserted **below the captions**.

```

678 \def\ccf@capt@bottom@offset#1{%
679   \ccIfStrEqual{#1}{top}{
680     {\@tempskipa=\z@}\relax
681     \expandafter\advance\expandafter\@tempskipa\ccUseProperty{\ccf@prefix caption-sep-top}%
682     \ifnum\the\ccSubFloatCnt=\z@\relax
683       \ifx\ccf@has@subcapt@top\empty
684         %% subcapt-top exists and capt-top is rendered
685         \advance\@tempskipa\dimexpr\ht\strutbox-\topskip-\p@\relax
686         \expandafter\advance\expandafter\@tempskipa\ccUseProperty{subcaption-add-sep-top}\relax%
687       \else
688         \advance\@tempskipa\dimexpr-\dp\strutbox\relax
689       \fi
690     \fi
691     \vskip\@tempskipa
692     \par\ifccf@break@capt\else\nopagebreak\fi}
693   {\ifnum\the\ccSubFloatCnt>\z@\relax
694     \vskip\dp\strutbox
695     \fi}}

```

`\ccf@make@caption` prints the caption.

- #1 is the placement (top, bottom)
- #2 is the vertical alignment (top, middle, bottom)

```

696 \long\def\ccf@make@caption#1#2{%
697   \ccf@capt@top@offset{#1}%
698   \ifnum\the\ccSubFloatCnt=\z@\relax
699     \def\ccf@caption@box{%

```

```

700 \ifx\ccf@landscape@empty
701 \setbox\@tempboxa\vbox\bgroup\hsize\textheight
702 \else
703 \hskip\ccf@margin@1%
704 \setbox\@tempboxa\vbox\bgroup\advance\hsize-\ccf@margin@1%
705 \fi}%
706 \else
707 \expandafter\cc@tempskipa\csname ccf@capt@row@height@#1\endcsname\relax
708 \expandafter\advance\expandafter\cc@tempskipa\dimexpr-\baselineskip+\topskip\relax
709 \def\ccf@caption@box{\setbox\@tempboxa\vbox to \cc@tempskipa\bgroup}%
710 \fi
711 \ccf@caption@box%
712 \ccIfStrEqual{#2}{top}{-}{\if@ccf@break@capt\else\vss\fi}%
713 \ccUseProperty{\ccf@prefix caption-face}%
714 \ccUseProperty{\ccf@prefix caption-face-#1}%

```

The caption as a whole is tagged with `<Caption/>`.

```

715 \ccaStructStart{Caption}%
716 \cc@topstrut\ccUseProperty{\ccf@prefix caption-#1}\strut%
717 \ccaStructEnd{Caption}%
718 \ifx\ccf@measure\relax\else
719 \ccIfPropVal{label-pos}{#1}{%
720 \ccfCreateLabel%
721 \ccf@write@listof%
722 }{}%
723 \fi
724 \ccIfStrEqual{#2}{bottom}{-}{\if@ccf@break@capt\else\vss\fi}%
725 \egroup%
726 \if@ccf@break@capt\unvbox\@tempboxa\else\box\@tempboxa\fi%
727 \ccf@capt@bottom@offset{#1}%
728 }

```

`\ccf@make@outer@caption` is a shell for the outer captions. #1 is the placement (top or bottom)

```

729 \def\ccf@make@outer@caption#1{%

```

now, we print the actual captions, if they contain contents.

```

730 \expandafter\ifx\csname ccf@has@capt@#1\endcsname\empty
731 \setbox\z@\vbox{%
732 \@cc@is@finalfalse
733 \let\ccf@measure\relax
734 \ccGobble
735 \ccSubFloatCnt\z@
736 \ccf@make@caption{#1}{top}%
737 }%
738 \immediate\write\@auxout{\string\expandafter\string\gdef\string\csname\space ccFloat\the\
739 \ccf@int@cnt Cap#1\string\endcsname{\the\dimexpr \ht\z@+\dp\z@\relax}}%
740 \bgroup
741 \savenotes
742 \if@ccf@break@capt\else\nopagebreak\fi
743 \ccSubFloatCnt\z@
744 \ccf@make@caption{#1}{top}%
745 \spewnotes
746 \egroup
747 \ccIfStrEqual{#1}{top}{-}{\if@ccf@break@capt\else\nopagebreak\fi}{-}%

```

If no caption at {#1} is given, we need to issue a `\par`, which otherwise would come from `\ccf@capt@top@offset`:

```

748 \par
749 \fi}

```

`\ccf@make@subcaption` creates the caption for subfloats. #1 is the position (`top` or `bottom`).

```

750 \def\ccf@make@subcaption#1{%
751 \expandafter\ifx\cscname ccf@has@\ccf@prefix capt@#1\endcsname\@empty
752 \ccf@make@caption{#1}{\ccUseProperty{\ccf@prefix caption-valign-#1}}%
753 \fi}

```

4 Generic User-Level Float Containers

`\ccDeclareFloat` is a user-level macro used to declare a new `ccFloat` environment.

- [#1] Name of the float Container from which the declared Container should inherit Properties (*optional*)
- {#2} top-level name of the float environment (e.g., `\ccPrefix Table`, `\ccPrefix Figure`)
- {#3} caption type (e.g., `table`, `figure`)
- {#4} list (e.g., `lot`, `lof`)
- {#5} additional Component body, use this to add to Types or introduce custom Handlers to the Float Container.

```

754 \def\ccDeclareFloat{\cc@opt@empty\ccf@declare@float}
755 \long\def\ccf@declare@float[#1]#2#3#4#5{%

```

First, we check if the Container already exists. If so, we issue an error message. May we force the style programmers learn to make use of CoCoT_EX's extensive toolbox.

```

756 \ifcsdef{cc@container@#2}{%
757 \ccPackageError{Float}{}
758 {Attempt to re-define pre-existing float Container `#2'}
759 {You cannot re-define an existing float Container. Use
760 \string\ccAddToType{<Type>}{#2}{<code>} to alter the #2 container!}}{%

```

Otherwise, we declare the new Container and invoke all the Initializers.

```

761 \def\ccf@parent{#1}%
762 \ccDeclareContainer{#2}{%
763 \ccPackageInfo{Floats}{}{Declaring float `#2'}%
764 \ifx\ccf@parent\@empty
765 \ccInherit{Properties,Components,Attributes}{float}
766 \else
767 \ccInherit{Properties,Components,Attributes}{\ccf@parent}
768 \fi
769 \ccDeclareType{FloatEnvInfo}{%
770 \ccSetContainer{#2}%
771 \def\ccf@capType{#3}%
772 \def\ccf@cap@list@type{#4}%
773 }% /FloatEnvInfo

```

The macro actually defines two L^AT_EX environments; a normal one for one-column floats, and a starred one for page-wide floats in two-column mode.

```

774 \ccDeclareEnv{#2}{\ccf@float}{\endccf@float}%
775 \ccDeclareEnv{#2*}{\if@twocolumn\let\ccf@do@dbl\relax\else\fi\ccf@float}{\if@twocolumn\let\
ccf@do@dbl\relax\fi\endccf@float}%
776 \ccDeclareType{Components}{}%
777 \ccDeclareType{Properties}{}%

```

Generating the Handlers for the list-of entries and define the corresponding `\@` macros

```

778 \ccf@generate@listof@handlers{#4}{#3}{#2}%
779 \bgroup
780 \def\cc@cur@cont{#2}%
781 \cc@init@l@list-of{#3}{0}{#3}% Generate listof-Entries for first level floats
782 \cc@init@l@list-of{#4}{1}{sub#3}% Generate listof-Entries for sub-floats
783 \egroup
784 #5
785 }% /container
786 }

```

5 Image Containers

5.1 Abstract Graphics Container

`Graphic` is an abstract Container that represents an image file.

```

787 \ccDeclareContainer{Graphic}{%
788 \ccDeclareType{Components}{%
789 \def\cc@counted@comp@scheme#1{#1-\the\ccSubFloatCnt}%

```

`Fig` holds the `\includegraphics` with the path to and the options for the actual image file.

```

790 \ccfMakeComp{Fig}%

```

`AltText` is the alternative text for accessibility.

```

791 \ccfMakeComp{AltText}%
792 }%
793 \ccDeclareType{Properties}{}%
794 }

```

5.2 Floating Figure Container

`Figure` is the user-level Container for display-style images or image clusters including their respective captions. Figures may either be placed as free-standing in-situ blocks or as floats.

```

795 \ccDeclareFloat{Figure}{figure}{lof}{%
796 \ccInherit{Properties,Components}{Graphic}%
797 \ccDeclareType{Properties}{%

```

`subfloat-same-height` [true|false] Whether all images in subfloats should be scaled to the same height (true) or not (false).

```

798 \ccSetProperty{subfloat-same-height}{true}%

```

`float-content` <any>

```

799 \ccSetProperty{float-content}{%
800 \ifx\ccf@no@figs\relax
801 \rule{0pt}{1pt}\rule{1pt}{0pt}%

```

```

802 \else
803 \ccUseComp{Fig}%
804 \fi}%

```

subfloat-content <any>

```

805 \ccPropertyLet{subfloat-content}{float-content}

```

float-render <any> figure specific output routine.

```

806 \ccSetProperty{float-render}{\ccfFigureRender}%

```

subfloat-render <any> figure specific output routine for sub-floats.

```

807 \ccSetProperty{subfloat-render}{\ccfSubFigureRender}%

```

measure-content <any> figure specific content measuring mechanism.

```

808 \ccSetProperty{measure-content}{\ccfMeasureFigContent}%

```

placement-tag <any> when PDF tagging is active, this placement attribut is used.

```

809 \ccSetProperty{placement-tag}{\ccaAddPlacement{Block}}

```

```

810 }}

```

5.3 Measuring Single Fig Components for Variable Width Captions

\ccImageBreak is used to separate multiple **\includegraphics** inside the same **Fig** component. By definition, those images are stacked *vertically*.

```

811 \def\ccImageBreak{\ccUseProperty{inline-content-v-sep}}

```

\ccImageSep is used to separate multiple **\includegraphics** inside the same **Fig** component. By definition, those images are stacked *horizontally*.

```

812 \def\ccImageSep{\ccUseProperty{inline-content-h-sep}}

```

\ccfMeasureFigContent measures the width of a single image. It further checks if and/or is set to auto and re-sets the internal margin and total width dimension registers accordingly.

```

813 \def\ccfMeasureFigContent{%
814 \ccaDisable\@cc@is@finalfalse
815 \bgroup

```

Note: Currently, if there are more than one **\includegraphics** separated by **\ccImageBreak** inside the same **Fig** Component, we only measure the *last* occurrence. This may change in a future update.

```

816 \def\ccImageBreak{\egroup\global\setbox\ccf@sub@box\hbox\bgroup\ccGobble\let\includegraphics
\ccf@measuregraphics}%
817 \def\ccImageSep{\hskip\ccUseProperty{inline-content-h-sep-auto}}%
818 \global\setbox\ccf@sub@box\hbox{\ccGobble
819 \let\includegraphics\ccf@measuregraphics
820 \ccUseProperty{float-content}%
821 }%
822 \ccfAdjustMargins{\wd\ccf@sub@box}%

```

```

823 \egroup
824 \global\setbox\ccf@sub@box\box\voidb@x
825 \ccaEnable\@cc@is@finaltrue
826 }

```

5.4 Figure Output Routines

`\ccfFigureRender` tells the float Container how the main content Component of Figure-type Floats is to be rendered. It is called via the Property.

```

827 \def\ccfFigureRender{%
828   \bgroup
829   \ifx\ccf@landscape\@empty
830     \hsize\dimexpr\textwidth-\ccf@margin@r-\ccf@margin@l\relax%
831   \fi
832   \let\includegraphics\ccf@includesubgraphics
833   \hskip\ccf@margin@l
834   \strut\ccUseComp{Fig}\strut
835 \egroup}

```

`\ccfSubFigureRender` tells the abstract float Container how the main content Component of Figure-type sub-floats are to be rendered. It is called via the Property.

```

836 \def\ccfSubFigureRender{%
837   \hskip\ccf@margin@l
838   \cc@iterate{\@tempcnta}{\@one}{\ccSubFloatCnt}{%
839     \ccfRenderSubFloats{\the\@tempcnta}{Fig}%
840   }}

```

`\ccf@includesubgraphics` is an override of L^AT_EX's `\includegraphics` patched to adjust for maximum width and height, and to capture the `alt` key in order to pass it down to `ltpdfa`.

In order to capture multiple images in the same Figure Container (i.e. real or fake Sub-Floats), tagging as `<Figure>` of images takes place here, as does assignment of alternative text(s). Those can be submitted by the `AltText` Component or by the `alt` key in the optional argument of `\includegraphics`. If both are given, the `alt` key takes precedence.

```

841 \def\ccf@includesubgraphics{\cc@opt@empty\@ccf@includesubgraphics}%
842 \def\@ccf@includesubgraphics[#1]#2{%
843   \let\cca@Gin@alt\relax
844   \def\@igopts{max width=\hsize,max height=\vsize}%
845   \ccIfPropVal{fix-dimen}{true}{\apptocmd\@igopts{,width=\hsize}{}}{}%
846   \if!#1!\else
847     \apptocmd\@igopts{,#1}{}{}%
848   \fi
849   \gdef\ccf@fig@path{#2}%
850   \if@cc@is@final
851     \ccaStructStart{Figure}%
852     \ccUseProperty{placement-tag}%
853   \fi%
854   \expandafter\ccf@ltx@includegraphics\expandafter[\@igopts]{#2}%
855   \if@cc@is@final
856     \ifx\cca@Gin@alt\undefined\let\cca@Gin@alt\relax\fi
857     \ifx\relax\cca@Gin@alt\relax
858       \ccWhenComp{AltText}{\ccSanitizeStr\cca@Gin@alt{\ccUseComp{AltText}}}%
859     \else
860       \ccSanitizeStr\cca@Gin@alt{\cca@Gin@alt}%
861     \fi

```


If the alt-text contains unbalanced parentheses, the PDF-Syntax will break, so we pass the text through `hyperref`'s `\pdfstringdef` macro one more time to transform all non-alphanumeric symbols to octals before we pass it down to `ltpdfa`.

```

862 \ifx\@cca@Gin@alt\relax\else
863 \Hy@pdfstringdef\@cca@Gin@alt{\@cca@Gin@alt}%
864 \ccaAddAltText{\@cca@Gin@alt}%
865 \fi
866 \ccaStructEnd{Figure}%
867 \fi
868 }

```

`\ccf@measuregraphics` is an override of $\text{\LaTeX}$'s `\includegraphics` that is used to measure the natural dimensions of the included image. It also checks if the `\includegraphics` has either an height or width explicitly given. if so, we de-activate the same-height calculations for the entire float.

```

869 \def\ccf@measuregraphics{\cc@opt@empty\@ccf@measuregraphics}
870 \def\@ccf@measuregraphics[#1]#2{%
871 \begingroup
872 \setkeys{Gin}{#1}%
873 \ifx\Gin@ewidth\Gin@exclamation
874 \ifx\Gin@eheight\Gin@exclamation\else
875 \global\@ccf@sameheightfalse
876 \fi
877 \else
878 \global\@ccf@sameheightfalse
879 \fi
880 \endgroup
881 \ccf@ltx@includegraphics[#1]{#2}%
882 }

```

5.5 Inline Figures

Inline Figure Container

`InlineFigure` is the user-level Container for inline graphics (e. g., images in tables or symbols inside the main text body). Note that this Container is *not* derived from the abstract `float` Container. Also, there is no $\text{\LaTeX}$ environment for that Container but a simple macro.

```

883 \ccDeclareContainer{InlineFigure}{%
884 \ccInherit{Properties,Components}{Graphic}%
885 \ccDeclareType{Attributes}{}%
886 \ccDeclareType{Properties}{%

```

`smash` [true|false] whether the image is allowed to stretch the line it is in (false) or not (true) if the height exceeds `\baselineskip`.

```

887 \ccSetProperty{smash}{false}

```

`vertical-align` [top|middle|bottom] the vertical alignment of the inline image relative to the baseline of the surrounding text. If the value is `bottom`, the bottom border of the image is aligned with the baseline, `top` aligns the top border of the image at baseline + $\text{\ht\strutbox}$, `middle` centers the image at baseline + $0.5 \times \text{\ht\strutbox}$.

```

888 \ccSetProperty{vertical-align}{bottom}

```

`float-render` <any> specific output routine for inline figures

```

889 \ccSetProperty{float-render}{\ccUseComp{Fig}}

```

`placement-tag` <any> inline figures should be placed inline.

```
890 \ccSetProperty{placement-tag}{\ccaAddPlacement{Inline}}
```

```
891 }%
```

```
892 }
```

Inline Figure User Macro

`\ccInlineFigure` is the Handler for an inline figure's main content Component.

`[#1]` is the attribute list for the figure

`{#2}` is the Container Body

```
893 \def\ccInlineFigure{\cc@opt@empty\cc@inline@figure}
894 \def\cc@inline@figure[#1]#2{%
895   \beginngroup
896     \ccSetContainer{InlineFigure}%
897     \def\ccfCapType{figure}%
898     \ccToggleCountedConditionals
899     \ccEvalType{Properties}%
900     \ccEvalAttributes{#1}%
901     \ccf@eval@class
902     \ccEvalType{Components}%
903     \ignorespaces
904     #2%
905     \ccSubFloatCnt=\z@\relax
906     \bgroup
907       \ccUseProperty{float-render}%
908       \egroup
909       \ccf@debug%
910       \ccf@store@dimens
911     \endgroup
912 }
913 \csdef{\ccPrefix InlineFigure}{\ccInlineFigure}%
```

6 Table Containers

6.1 The Abstract Tabular Container

CoCoTeX's float module supports the three basic Standard L^AT_EX tabular environments (`tabular`, `tabularx` and `tabulary`) as well as `htmltab` from the `htmltabs` package. For the measuring to work correctly, we need to render the tables as a whole and store the result inside `\ccf@floatbox` for measuring and further processing.

`Tabular` is an abstract Container that represents raw table data. Its main purpose is to provide a unified interface to patch some of L^AT_EX's standard `tabular` environments, as well as the `htmltab` environment, if the `htmltabs` package is loaded.

```
914 \ccDeclareContainer{Tabular}{%
915   \ccDeclareType{Properties}{}%
916   \ccDeclareType{Components}{}%
917   \ccf@reserve@tabular
918 }%
919 }
```

`\ccf@reserve@tabular` is a shell macro that temporarily stores the default macro definitions for various tabular environments and patches them such that the contents are stored inside the `\ccf@floatbox`. The macro is called at the very beginning of the Table Container's environment and the patches only hold inside that environment. Thus, all tabular environments can be used in their vanilla state outside CoCoT_EX's Table environments.

```

920 \def\ccf@reserve@tabular{%
921   \ccf@reserve@tab{%
922     \ccf@reserve@tab{x}%
923     \ccf@reserve@tab{y}%
924     \ccf@reserve@htmltab%
925   }

```

`\ccf@reserve@tab` stores the default definitions for a specific vanilla-L^AT_EX tabular environment and re-defines the macros in a way that the tabulars are stored in the `\ccf@floatbox` instead of printed onto the page.

```

926 \def\ccf@reserve@tab#1{%
927   \csletcs{orig@tabular#1}{\tabular#1}%
928   \csletcs{orig@endtabular#1}{\endtabular#1}%
929   \csdef{\tabular#1}{%
930     \global\setbox\ccf@floatbox
931     \vbox\bgroup
932       \if!#1!\else
933         \let\tabular\orig@tabular
934         \let\endtabular\orig@endtabular
935       \fi
936       \csname orig@tabular#1\endcsname}%
937   \csdef{\endtabular#1}{\csname orig@endtabular#1\endcsname\egroup}%
938 }

```

6.2 The User-Level Table Container

Table is a user-level Container for display-style tables including their captions. They may either be placed as free-standing in-situ blocks or as floats.

```

939 \ccDeclareFloat{Table}{table}{\lot}{%
940   \ccInherit{Properties,Components}{Tabular}%
941   \ccDeclareType{Properties}{%
942     \ccSetProperty{subcaption-valign-top}{bottom}%
943     \ccSetProperty{subfloat-content}{%
944       \PackageError{coco-floats.sty}
945       {ccSubFloat does not support sub-tables (yet)!}
946       {You cannot yet use a tables within the `ccSubFloat'!}%
947     }%
948     \ccSetProperty{float-render}{\ccfTableRender}%
949     \ccSetProperty{subfloat-render}{\ccfSubTableRender}%
950     \ccSetProperty{measure-content}{\ccfMeasureTabContent}
951   }%
952 }

```

`\ccf@reserve@htmltab` special handler for tables using the `htmltabs` package:

```

953 \AtBeginDocument{%
954   \ifpackageloaded{htmltabs}{%
955     \def\ccf@reserve@htmltab{%
956       \let\ccf@add@style\@empty
957       \ifx\ccf@floatpos\@empty
958         \expandafter\ifx\csname ccFloat\the\ccf@int@cnt Captop\endcsname\relax\else
959           \htInitSkip\csname ccFloat\the\ccf@int@cnt Captop\endcsname

```

```

960     \advance\htInitSkip\ccf@sep@top%
961   \fi
962   \expandafter\ifx\csname ccFloat\the\ccf@int@cnt Capbottom\endcsname\relax\else
963     \htAddToBottom\csname ccFloat\the\ccf@int@cnt Capbottom\endcsname
964     \advance\htAddToBottom\ccf@sep@bottom%
965   \fi
966   \else
967     \def\ccf@add@style{;break-table:false;}%
968   \fi
969   \edef\cc@tempa{margin-left:\ccf@margin@l\ccf@add@style}%
970   \expandafter\htAddStyle\expandafter{\cc@tempa}%
971   \global\setbox\htTableBox\box\voidb@x
972   \let\htOutputTable\relax
973   }\let\ccf@reserve@htmltab\relax}%
974 }

```

6.3 The Table Output Handler

`\ccfGetTableContent` returns the `\ccf@floatbox` if it is not un-itialized or void.

```

975 \def\ccfGetTableContent{%
976   \ifx\htTableBox\@undefined\else
977     \ifvoid\htTableBox\else
978       \let\ccf@floatbox\htTableBox%
979     \fi\fi}

```

`\ccfTableRender` is the content of the Property specific for tables.

```

980 \def\ccfTableRender{%
981   \ccfGetTableContent
982   \ccComponent{Content}{\unvbox\ccf@floatbox}%
983   \ccUseComp{Content}%
984   \ccaStructStart{Table}%
985   \ifx\ht@structID@THead\@undefined
986     \ifx\ht@structID@TBody\@undefined\else\ccaMoveStruct{\ht@structID@TBody}\fi%
987   \else
988     \ccaMoveStruct{\ht@structID@THead}%
989     \ifx\ht@structID@TBody\@undefined\else\ccaMoveStruct{\ht@structID@TBody}\fi%
990   \fi
991   \ifx\ht@structID@TFoot\@undefined\else\ccaMoveStruct{\ht@structID@TFoot}\fi%
992   \par\if@ccf@break@capt\else\nopagebreak\fi
993   \vskip\dp\strutbox
994   \ccaStructEnd{Table}%
995 }

```

`\ccfSubTableRender` Is the content of the table-specific Property

Note that table sub-floats aren't allowed yet, so this definition is un-used at the moment. TeX will crash with an error message before this Property is ever expanded.

```

996 \def\ccfSubTableRender{%
997   \cc@iterate{\@tempcnta}{\@ne}{\ccSubFloatCnt}{%
998     \ccfGetTableContent
999     \cc@is@finalfalse
1000     \ccComponent{Content}{\unvbox\ccf@floatbox}%
1001     \cc@is@finaltrue
1002     \ccfRenderSubFloats{\the\@tempcnta}{Content}%
1003   }}

```

`\ccfMeasureTabContent` measures the width of a table in order to adjust the left and right margins if either is set to auto.

```

1004 \def\ccfMeasureTabContent{%
1005   \ccfGetTableContent
1006   \ifvoid\ccf@floatbox\else
1007     \ccfAdjustMargins{\wd\ccf@floatbox}%
1008   \fi
1009 }
```

7 Other Float-Related Macros

`\ccFloatBarrier` can be used to force all pending floats to be printed at the next shipout.

```

1010 \def\ccFloatBarrier{\AtBeginShipoutNext{\clearpage}}
```

Output Driver for the `coco-floats.sty`.

```
</floats>
```


Module 11

coco-frame.dtx

This file provides facilities to visualise crop marks and the print area.

```

22 <*>frame>

23 %%
24 %% module for CoCoTeX for crop marks and print area frames.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-frame}
30 [2025/07/14 v0.6.2 coco-frame]\relax

```

1 Top-Level Interface

```

31 \let\cc@frame@mode n
32 \ExplSyntaxOn
33 \bool_new:N \g_cocotex_frame_none_bool \bool_set_true:N \g_cocotex_frame_none_bool
34 \bool_new:N \g_cocotex_frame_guide_bool \bool_set_false:N \g_cocotex_frame_guide_bool
35 \bool_new:N \g_cocotex_frame_crops_bool \bool_set_false:N \g_cocotex_frame_crops_bool
36 \bool_new:N \g_cocotex_frame_boxes_bool \bool_set_false:N \g_cocotex_frame_boxes_bool
37 \keys_define:nn { cocotex/frame }
38 {
39   guide .code:n = {
40     \bool_gset_true:N \g_cocotex_frame_guide_bool
41     \bool_gset_false:N \g_cocotex_frame_none_bool
42   },
43   cropmarks .code:n = {
44     \bool_gset_true:N \g_cocotex_frame_crops_bool
45     \bool_gset_true:N \g_cocotex_frame_boxes_bool
46     \bool_gset_false:N \g_cocotex_frame_none_bool
47   },
48   pdfboxes .code:n = {
49     \bool_gset_true:N \g_cocotex_frame_boxes_bool
50     \bool_gset_false:N \g_cocotex_frame_none_bool
51   }
52 }

```

The value of the `offset` option is the space added equally to all sides of the paper. Cropmarks are printed within this margin. Default is `4em`.

```

53 \dim_new:N \g_coco_frame_offset_dim
54 \dim_gset:Nn \g_coco_frame_offset_dim {4em}
55 \keys_define:nn { cocotex/frame }
56 {

```

```

57 offset .dim_gset:N = \g_coco_frame_offset_dim
58 }

```

The bleed width is given via the `bleed` package option. Default is `4mm`.

```

59 \dim_new:N \g_coco_frame_bleed_dim
60 \dim_gset:Nn \g_coco_frame_bleed_dim {4mm}
61 \keys_define:nn { cocotex/frame }
62 {
63   bleed .dim_gset:N = \g_coco_frame_bleed_dim
64 }

```

LEGACY

```

65 \keys_define:nn { cocotex/frame }
66 {
67   frame .choice:,
68   frame / none .code:n = { },%\bool_gset_false:N \g_cocotex_frame_print_bool \global\let\cc@frame@mode n
69   frame / crop .code:n = { \keys_set:nn { cocotex/frame } {pdfboxes,cropmarks} },%\bool_gset_true:
        N \g_cocotex_frame_print_bool \global\let\cc@frame@mode c
70   frame / frame .code:n = { \keys_set:nn { cocotex/frame } {frame} },%\bool_gset_false:N \g_cocotex
        _frame_print_bool \global\let\cc@frame@mode w
71 }
72 \ProcessKeyOptions[cocotex/frame]

```

2 Cropmark printer

Apparently, the crop package relies on old pdf dimension macros. If they aren't defined, we load the `luatex85` package and set the values of the type area by hand:

```

73 \@ifundefined{pdfpagewidth}{%
74   \RequirePackage{luatex85}
75   \pdfpagewidth\paperwidth
76   \pdfpageheight\paperheight
77 }{}

```

Now, we determine the dimensions of the PDF boxes.

```

78 \bool_if:NT \g_cocotex_frame_boxes_bool
79 {
80   \ifx\cc@frame@@offset\@undefined
81     \newdimen\cc@frame@@offset
82     \cc@frame@@offset\g_coco_frame_offset_dim\relax
83   \fi
84   \ifx\bleed\@undefined
85     \newdimen\bleed
86     \bleed\g_coco_frame_bleed_dim\relax
87   \fi
88   \bool_if:NTF \g_cocotex_frame_crops_bool
89   {%
90     \voffset\dimexpr\cc@frame@@offset-1in\relax
91     \hoffset\dimexpr\cc@frame@@offset-1in\relax
92   }
93   {
94     \bleed\z@\relax
95     \cc@frame@@offset\z@\relax
96   }

```



```

97 \edef\l@offset{\strip@pt\dimexpr\cc@frame@@offset*7200/7227\relax}
98 \edef\r@offset{\strip@pt\dimexpr(\cc@frame@@offset+\paperwidth)*7200/7227\relax}
99 \edef\u@offset{\strip@pt\dimexpr(\cc@frame@@offset)*7200/7227\relax}
100 \edef\o@offset{\strip@pt\dimexpr(\cc@frame@@offset+\paperheight)*7200/7227\relax}
101 \edef\b@l@offset{\strip@pt\dimexpr(\cc@frame@@offset-\bleed)*7200/7227\relax}
102 \edef\b@r@offset{\strip@pt\dimexpr(\cc@frame@@offset+\paperwidth+\bleed)*7200/7227\relax}
103 \edef\b@u@offset{\strip@pt\dimexpr(\cc@frame@@offset-\bleed)*7200/7227\relax}
104 \edef\b@o@offset{\strip@pt\dimexpr(\cc@frame@@offset+\paperheight+\bleed)*7200/7227\relax}
105 \edef\@tempa{%
106   /TrimBox [\l@offset\space\u@offset\space\r@offset\space\o@offset]
107   /BleedBox [\b@l@offset\space\b@u@offset\space\b@r@offset\space\b@o@offset]
108   %/CropBox [\b@l@offset\space\b@u@offset\space\b@r@offset\space\b@o@offset]
109   %/MediaBox [\b@l@offset\space\b@u@offset\space\b@r@offset\space\b@o@offset]
110 }

```

and set them as PDF attributes:

```

111 \expandafter\pdfpageattr\expandafter{\@tempa}

```

Setting PDF boundaries

```

112 \edef\stockwidth{\the\dimexpr\paperwidth+\cc@frame@@offset+\cc@frame@@offset\relax}
113 \edef\stockheight{\the\dimexpr\paperheight+\cc@frame@@offset+\cc@frame@@offset\relax}
114 }

```

Cropmarks and page area frames both are painted via the `crop` package, which we load and setup if any of the Module's modes is loaded via the package options.

```

115 \bool_if:NF \g_cocotex_frame_none_bool {
116   \RequirePackage{crop}
117   \renewcommand*\CROP@marks{%
118     \CROP@setmarkcolor
119     \CROP@user@b
120     \vskip1in\hskip1in\relax
121     \CROP@ulc\null\hfill\CROP@@@info\CROP@upedge\hfill\null\CROP@urc\hskip-1in\null
122     \vfill
123     \CROP@l@edge\hfill\CROP@redge
124     \vfill
125     \hskip1in\relax
126     \CROP@llc\null\hfill\CROP@l@edge\hfill\null\CROP@lrc\hskip-1in\null
127     \vskip-1in}%
128 }

```

If the user requested them, we tell the crop package to draw the crop marks:

```

129 \bool_if:NT \g_cocotex_frame_crops_bool {\crop[cam]}

```

The code for the page area guides

```

130 \bool_if:NT \g_cocotex_frame_guide_bool {%
131   \@tempdima\dimexpr\textheight\relax
132   \divide\@tempdima by\baselineskip
133   \multiply\@tempdima by65536\relax
134   \edef\cnt@baselines{\strip@pt\@tempdima}%
135   \def\cc@frame@lines{%
136     \@tempcnta\z@
137     \loop\advance\@tempcnta\@ne
138     \hsize1em\relax
139     \ifodd\count\z@
140       \vrule\@width1em\@height0.2\p@\@depth0.02\p@

```

```

141 \llap{\smash{\the\@tempcnta\,}}%
142 \fi%
143 \rlap{%
144 \ifodd\count\z@\else\fi
145 \vrule\@width\columnwidth\@height0.00005\p@\@depth0\p@
146 \if@twocolumn
147 \kern\columnsep\vrule\@width\columnwidth\@height0.00005\p@\@depth0\p@
148 \else
149 \ifx\col@number\@undefined\else
150 \ifnum\col@number<\tw@\relax
151 \else
152 \@tempcntb\col@number
153 \loop
154 \advance\@tempcntb\m@ne
155 \kern\columnsep\vrule\@width\columnwidth\@height0.00005\p@\@depth0\p@
156 \ifnum\@tempcntb>\@ne
157 \repeat
158 \fi
159 \fi
160 \fi
161 \ifodd\count\z@\else
162 \vrule\@width1em\@height0.00005\p@\@depth0\p@%
163 \llap{\smash{\the\@tempcnta\,}}%
164 \fi
165 }%
166 \break
167 \ifnum\@tempcnta<\cnt@baselines
168 \repeat
169 }%
170 \def\cc@frame@margin{%
171 \ifdim\marginparwidth>\z@
172 \vrule height\textheight%
173 \hskip-\marginparwidth\relax
174 \vbox to\textheight{\hsize\marginparwidth\relax
175 \rlap{\vbox to\z@{\hrule width\marginparwidth}}}%
176 \null\vss
177 \rlap{\vbox to\z@{\hrule width\marginparwidth}}}%
178 }%
179 \vrule height\textheight%
180 \fi
181 }
182 \renewcommand*\CROP@frame{%
183 \vskip0in%
184 \color[cmYk]{0.4,0,0,0}%
185 \ifodd\count\z@\let\@themargin\oddsidemargin\else\let\@themargin\evensidemargin\fi
186 \advance\@themarginlin
187 \moveright\@themargin
188 \vbox to\z@{\baselineskip\z@skip\lineskip\z@skip\lineskiplimit\z@
189 \vskip\topmargin\vbox to\z@{\vss\hrule width\textwidth}%
190 \vskip\headheight\vbox to\z@{\vss\hrule width\textwidth}%
191 \vskip\headsep\vbox to\z@{\vss\hrule width\textwidth}%
192 \hbox to\textwidth{%
193 \ifodd\count\z@
194 \rlap{\hskip\dimexpr\textwidth+\marginparsep+\marginparwidth\relax\cc@frame@margin}%
195 \else
196 \rlap{\hskip-\marginparsep\relax\cc@frame@margin}%
197 \fi
198 \llap{\vbox to\textheight{\tiny\let\@tempa\font\selectfont
199 \vskip\topskip\cc@frame@lines\null\vss}}}%
200 \llap{\vrule height\textheight}%
201 \if@twocolumn

```

```

202     \hskip\columnwidth\rlap{\vrule height\textheight}%
203     \hskip\columnsep\rlap{\vrule height\textheight}%
204   \else
205     \ifx\col@number\@undefined\else
206       \ifnum\col@number<\tw@\relax
207         \else
208           \@tempcntb\col@number
209           \loop
210             \advance\@tempcntb\m@ne
211             \hskip\columnwidth\rlap{\vrule height\textheight}%
212             \hskip\columnsep\rlap{\vrule height\textheight}%
213             \ifnum\@tempcntb>\@ne
214               \repeat
215           \fi
216         \fi
217       \fi
218     \hfil\vrule height\textheight
219   }%
220   \vbox to\z@{\vss\hrule width\textwidth}%
221   \vskip\footskip\vbox to\z@{\vss\hrule width\textwidth}%
222   \vss}%
223 \vbox to\z@{\baselineskip\z@skip\lineskip\z@skip\lineskiplimit\z@%
224 \vskip-0in\rlap{\hskip1in%
225   \vbox to\z@{\vbox to\z@{\vss\hrule width\paperwidth}%
226     \hbox to \paperwidth{\llap{\vrule height\paperheight}\hfil%
227       \vrule height\paperheight}%
228     \vbox to\z@{\vss\hrule width\paperwidth}%
229     \vss}}\vss}}
230 \crop[frame,noinfo]%
231 }

```

```

232 </frame>

```


Module 12

coco-lists.dtx

```
< *lists >
```

This module provides handlers for list-like environments like item lists, enumerations, glossaries and descriptions.

Note: The `coco-lists` module diverges somewhat from the other CoCoTeX modules insofar as that its main Container does not follow the CoCoTeX’s usual “collect all–process later” approach, but all Properties are processed at the beginning of each Container’s instances and the contents are processed as they are parsed by the `\LaTeX` interpreter, just like “reguar” $\LaTeX$ lists. Configuration of lists, however, follows the CoCoTeX playbook.

1 Preamble

```
23 %%
24 %% Common document class for \textit{xerif} projects.
25 %%
26 %% Maintainer: p.schulz@le-tex.de
27 %%
28 \NeedsTeXFormat{LaTeX2e}[2023/11/01]
29 \ProvidesPackage{coco-lists}
30   [2025/07/14 v0.6.2 CoCoTeX lists module]
31 \RequirePackage{coco-common}
```

1.1 Package Options

If the `replace` option is set, LaTeX’s default lists are replaced by `coco-lists` module. This effects $\LaTeX$ ’s `enumerate`, `itemize`, and `description` environments.

```
32 \newif\if@ccl@replace \@ccl@replacefalse
33 \ExplSyntaxOn
34 \keys_define:nn { cocotex/lists }
35 {
36   replace .code:n = {\global\@ccl@replacetrue}
37 }
```

The option `inherit` defines how nested lists inherit their properties. Currently, there are two ways: `common`: All nested lists of the same type inherit only from the same, generic type definition; `conseq`: nested lists of the same type inherit from the next-higher level list of the same type, and from the generic type definition.

For example, if `inherit=common`, 3rd level `itemize` and 2nd level `itemize` both inherit only the property values of the same generic `itemize` list type. If `inherit=conseq`, 3rd level inherits the property lists from 2nd level `itemize`.

Since inheritance is a transitive relation, 3rd level `itemize` will ultimately also inherit the Properties from generic `itemize`, but in contrast to `common`, `conseq` allows 2nd level `itemize` to override some Properties of generic `itemize`, which will be propagate down to 3rd level `itemize`, while with `inherit=common`, the override on 2nd level `itemize` would have no effect on 3rd level `itemize`.

`\ccl@ih@common` is used for comparisons. It represents the `inherit=common` package option.

```
38 \def\ccl@ih@common{common}
```

`\ccl@ih@conseq` is used for comparisons. It represents the `inherit=conseq` package option.

```
39 \def\ccl@ih@conseq{conseq}%
```

`\ccl@inherit` stores the value of the `inherit` package option.

```
40 \let\ccl@inherit\ccl@ih@common
41 \keys_define:nn { cocotex/lists }
42 {
43   inherit .choice:,
44   inherit / conseq .code:n = { \global\let\ccl@inherit\ccl@ih@conseq },
45   inherit / common .code:n = { \global\let\ccl@inherit\ccl@ih@common },
46   inherit .initial:n = common
47 }
```

`\ccl@str@local` is a string for comparison. It represents the `nesting=local` option.

```
48 \def\ccl@str@local{local}%
```

`\ccl@str@global` is a string for comparison. It represents the `nesting=global` option.

```
49 \def\ccl@str@global{global}%
```

`\ccl@nesting` The nesting option sets whether the nesting level of a list should be counted list-specific (value `local`), or globally (value `global`, default).

```
50 \let\ccl@nesting\ccl@str@global
51 \keys_define:nn { cocotex/lists }
52 {
53   nesting .choice:,
54   nesting / local .code:n = { \global\let\ccl@nesting\ccl@str@local },
55   nesting / global .code:n = { \global\let\ccl@nesting\ccl@str@global },
56   nesting .initial:n = global
57 }
```

```
58 \ProcessKeyOptions[cocotex/lists]
59 \ExplSyntaxOff
```

2 The List Container

`List` is the most abstract Container for lists.

```
60 \ccDeclareContainer{List}{%
```

2.1 List Properties

```
61 \ccDeclareType{Properties}{%
```

List Boundaries

before-list <any> is expanded at the very beginning of a (nested) list.

```
62 \ccSetProperty{before-list}{% at the very beginning of each (nested) list
63 \if@noskipsec \leavevmode \fi
64 \ifvmode\else
65 \unskip \par
66 \fi
```

🔑 **<L>** is the opening List tag

```
67 \ccaStructStart{L}%
68 }%
```

after-list <any> is expanded at the very end of a (nested) list. By default, it calls the ⚙️ **after-item** Property.

🔑 **</L>** is the closing List tag

```
69 \ccSetProperty{after-list}{%
70 \ccUseProperty{after-item}%
71 \ccaStructEnd{L}% end tag for the (nested) list
72 }%
```

List Margins

margin-top <skip> is the vertical skip at the beginning of each List instance.

```
73 %% list margins
74 \ccSetProperty{margin-top}{\z@}%
```

margin-bottom <skip> is the vertical skip at the end of each List instance.

```
75 \ccSetProperty{margin-bottom}{\z@}% vertical space before the list.
```

margin-left [auto|<skip>] is the horizontal space to the left of each list instance, from left boundary of the page area. **auto** means that the left margin is set to the width of widest label + ⚙️ **prev-margin-left**. The value is passed through **\dimexpr**, so basic arithmetic is allowed.

```
76 \ccSetProperty{margin-left}{\csname leftmargin\@roman\cclCurDepth\endcsname-\ccUseProperty{
    label-sep}+\ccUseProperty{prev-margin-left}}%
```

max-label-width <dimen> is the maximum space reserved for a list item's label.

```
77 \ccSetProperty{max-label-width}{.33\textwidth}%
```

margin-right <skip> is the right margin of the list instance.

```
78 \ccSetProperty{margin-right}{\z@}% horizontal space to the right of each list item
```

Between List Items

item-sep <skip> is the vertical space between two adjacent list items. Note that the real value value is advanced by the value of the ⚙️ **par-skip** Property.

```
79 \ccSetProperty{item-sep}{\z@}%
```

after-indent [true|false] determines whether the text paragraph after the (top-level) list is indented (**true**) or not (**false**).

```
80 \ccSetProperty{after-indent}{false}%
```

`at-begin-item-body` <any> is expanded right at the beginning of a new item body and sets the `<LBody>` tag.

```
81 \ccSetProperty{at-begin-item-body}{\ccaVstructStart{LBody}}%
```

`at-end-item-body` <any> is expanded at the very end of an item body, but before the final `\par`. By default, it only sets the closing `</LBody>` tag.

```
82 \ccSetProperty{at-end-item-body}{\ccaVstructEnd{LBody}}%
```

`after-item` <any> is expanded after each list item. It calls the `at-end-item-body` Property and closes the item's final paragraph as well as the `</LI>` tag.

```
83 \ccSetProperty{after-item}{%
84   \ccUseProperty{at-end-item-body}%
85   \ccaVstructEnd{LI}% Close list item tags
86   \par}%
```

`before-item` <any> is called at the very beginning of each list item. If the current item is the first item, the `\ifcclFirst` conditional is set to false. All non-first items of the same List instance call the `after-item` Property and add a vertical skip of `item-sep` amount.

After that, the paragraph formatting parameters for the list-item `par-indent`, `par-skip`, and `par-fill-skip`, as well as the starting `<LI>` tag are set.

```
87 \ccSetProperty{before-item}{%
88   \ifcclFirst
89     \global\cclFirstfalse
90   \else
91     \ccUseProperty{after-item}%
92     \vskip\ccUseProperty{item-sep}%
93   \fi
94   \parindent\ccUseProperty{par-indent}\relax%
95   \parskip\ccUseProperty{par-skip}\relax%
96   \parfillskip\ccUseProperty{par-fill-skip}\relax%
97   \noindent
98   \leavevmode
99   \ccaVstructStart{LI}% Start tag for a list item
100 }%
```

`item-offset` <any> calculates `\cclItemIndent` from the `indent` and `label-sep` Properties and sets the horizontal offset of the first line of the list item. After that, the value of the macro is unsigned.

```
101 \ccSetProperty{item-offset}{%
102   \cclItemIndent\ccUseProperty{indent}%
103   \advance\cclItemIndent\dimexpr-\ccUseProperty{label-sep}\relax
104   \hskip\cclItemIndent\relax%
105   \ifdim\ccUseProperty{indent}>\z@
106     \cclItemIndent\ccUseProperty{indent}%
107   \else
108     \cclItemIndent-\ccUseProperty{indent}%
109   \fi
110 }%
```

`par-indent` <skip> is the indent of the first line of a *new* paragraph inside a list item

```
111 \ccSetProperty{par-indent}{\parindent}%
```

`par-fill-skip` <skip> is the skip at the end of the last line of each paragraph inside a list item

```
112 \ccSetProperty{par-fill-skip}{\@flushglue}%
```


par-skip <dimen> vertical space between two adjacent paragraphs inside the same List item

```
113 \ccSetProperty{par-skip}{\z@}%
```

Label Formatting

label <any> prints the ➡**Label** component.

```
114 \ccSetProperty{label}{\ccUseComp{Label}}%
```

indent [auto|auto-global|<dimen>] is the indent of each List item's first line (relative to ⚙**margin-left**).

If the value is **auto**, the real indent and left margin of a item's first line is calculated using coco-common's indentation mechanism (see Sect. 3.3 in Module Module 3). The first-line indent will thereby be calculated from the widest width of all labels of the same list type and nesting level.

Note: the value **auto-global** is allowed, but it causes *all* lists – regardless of the nesting level – to have the same left margin and indent!

```
115 \ccSetProperty{indent}{-\dimexpr\csname leftmargin\@roman\cclCurDepth\endcsname-\ccUseProperty{label-sep}\relax}%
```

label-sep <dimen> is the horizontal space between the label and the item body.

```
116 \ccSetProperty{label-sep}{.5em}%
```

label-face <any> is the style of the label.

```
117 \ccSetProperty{label-face}{}%
```

label-align [left|center|right] is the alignment of the label within its local **\hbox**.

```
118 \ccSetProperty{label-align}{left}%
```

label-format <any> is the format of the label. It should call the ⚙**label-face** and ⚙**label** properties and enclose the latter with ➡**<Lb1>** and ➡**</Lb1>**.

```
119 \ccSetProperty{label-format}{%
120   \ccUseProperty{label-face}%
121   \ccaVstructStart{Lb1}%
122   \ccUseProperty{label}%
123   \ccaVstructEnd{Lb1}%
124 }%
```

label-box <any> is the property that builds a local **\hbox** into which the ➡**Label** Component is printed. It should respect the ⚙**label-align** Property and call ⚙**label-format**.

```
125 \ccSetProperty{label-box}{%
126   \hbox to \cclItemIndent{%
127     \ccIfPropVal{label-align}{left}{\hss}%
128     \ccUseProperty{label-format}%
129     \ccIfPropVal{label-align}{right}{\hss}}%
130 }%
```

item-format <any> contains material printed at the beginning of a new item. It should call the ⚙**before-item**, ⚙**item-offset**, ⚙**label-box** and ⚙**label-sep** Properties.

```
131 \ccSetProperty{item-format}{%
132   \ccUseProperty{before-item}%
133   \ccUseProperty{item-offset}%
```

```

134     \ccUseProperty{label-box}%
135     \hskip\ccUseProperty{label-sep}%
136   }%
137 }%

```

2.2 List Components

```

138 \ccDeclareType{Components}{%

```

`Label` represents a List item's local label.

```

139   \ccDeclareComponent{Label}{-}{-}%

140 }%
141 \ccDeclareEnv{cc@list}{endcc@list}%
142 }

```

3 Declaring List Types

List Types are the next layer of abstraction for lists. This layer distinguishes numbered from unnnumbered and description lists.

`\DeclareListType` declares a new list type. #1 is the name of the list type, #2 is the declaration body. Each new list type should declare at least an Attribute handler and a Label handler. #3 is a list of type specific properties that are appended to the generic list's property list.

```

143 \long\def\ccDeclareListType#1#2#3{%

```

`\DeclareAttributeHandler` declares a new handler for a list's attributes. ##1 is the definition body.

```

144   \def\DeclareAttributeHandler##1{\csdef{ccl@eval@attrs@#1}{##1}}%

```

`\DeclareLabelHandler` declares a new handler for each item's label. ##1 is the definition body. It should fill the Label Component with content in case the optional argument of item is omitted.

```

145   \def\DeclareLabelHandler##1{\csdef{ccl@make@label@#1}{##1}}%

```

```

146   \ccDeclareContainer{#1List}{%
147     \ccInherit{Components,Properties}{List}%
148     \ccDeclareType{Properties}{%

```

`list-type` <any> holds the name of the list type.

```

149   \ccSetProperty{list-type}{#1}%

150   #3%
151 }%
152 \ccDeclareEnv[#1-list]{\cc@list}{\endcc@list}%
153 }%
154 #2%
155 }

```

4 Declare Lists

The next layer of abstraction is the user-level List container. Each List container must be assigned to a list type from which it will inherit its type-specific properties.

`\ccDeclareList` defines a new list. #1 is the name of the list environment (sans `\ccPrefix`), #2 is the list type, #3 is the list-specific Property list.

```
156 \def\ccDeclareList#1#2#3{%
157   \csxdef{cc@cur@depth@#1}{\z@}%
158   \ccDeclareContainer{#1}{%
159     \ccInherit{Properties,Components}{#2List}%
160     \ccDeclareType{Properties}{#3}%
161     \ccDeclareEnv{#1}{\cc@list}{\endcc@list}%
162   }%
163   \ccDeclareNested{#1}{\z@}{}%
164 }
```

`\ccDeclareNested` can be used to declare Property overrides for nested lists. #1 is the list name, #2 is the nesting depth (#2th nesting level means that the Properties are used for the $n + 1$ -th list of the same name), #3 is the Property list.

```
165 \def\ccDeclareNested#1#2#3{%
166   \@tempcnta=#2\relax
167   \ifx\@tempcnta<\z@\relax
168     \ccPackageError{lists}{Nesting}{Invalid nesting level!}{You cannot declare nesting levels
169       less than 0!}%
169   \fi
170   \advance\@tempcnta\@ne\relax
171   \ccDeclareContainer{#1-\the\@tempcnta}{%
172     \ifcsdef{cc@container@#1}
173       {\ccInherit{Properties,Components}{#1}}
174       {\ccPackageError{lists}{Inheritance}
175         {List `#1' undefined!}
176         {You need to define the list `#1' before you can declare nested list overrides!}}%
177     \ccDeclareType{Properties}{#3}%
178   }%
179 }
```

We want to count each list type separately to ensure the correct item label is printed, but we also need to keep within the global nesting level limit. Therefore, we set two internal counters, one for the overall nesting level, and another one for each list type. Note that the latter is a macro, not a counter register.

`\ccl@depth` is the counter for the overall nesting level.

```
180 \newcount\ccl@depth
```

`\ccl@item@cnt` is the internal counter for the items within a (nested) list level.

```
181 \newcount\ccl@item@cnt
```

`\ifcclFirst` is true as long as the first item of a list is processed.

```
182 \newif\ifcclFirst \cclFirsttrue
```

`\ccl@advance@depth` is a helper macro to advance both the global list nesting level, as well as the list Container specific nesting level. #1 is the amount by which both counters should be advanced.

```
183 \def\ccl@advance@depth#1{\csname ccl@advance@depth@\ccl@nesting\endcsname{#1}}
```

`\ccl@advance@depth@global` is called when the nesting level should be counted for all lists equally without respecting the list type.

```
184 \def\ccl@advance@depth@global#1{%
185   \edef\cclPrevDepth{\the\ccl@depth}%
186   \global\advance\ccl@depth#1\relax
187   \edef\cclCurDepth{\the\ccl@depth}%
188 }
```

`\ccl@advance@depth@local` is called when the nesting level should be counted for each list type individually.

```
189 \def\ccl@advance@depth@local#1{%
190   \letcs\cclPrevDepth{cc@cur@depth@\cc@cur@cont}%
191   \expandafter\@tempcnta\csname cc@cur@depth@\cc@cur@cont\endcsname\relax
192   \advance\@tempcnta#1\relax
193   \csxdef{cc@cur@depth@\cc@cur@cont}{\the\@tempcnta}%
194   \edef\cclCurDepth{\csname cc@cur@depth@\cc@cur@cont\endcsname}%
195   \global\advance\ccl@depth#1\relax
196 }
```

`\cclItemIndent` stores the actual calculated indent of an List item's first line.

```
197 \newskip\cclItemIndent
```

`\cclTopID` is a counter that stores a unique number for each top-level List Instance. It is used to calculate the margins of both top-level items and items of nested lists.

```
198 \newcount\cclTopID \cclTopID\z@\relax
```

`\cclID` stores a unique “identifier” number for each list, irrespective their nesting levels.

```
199 \newcount\cclID \cclID\z@\relax
```

An internal global counter register `\ccl@total@list@cnt` is used to count the overall number of opening lists. Currently, the global ID of each list is unused.

```
200 \newcount\ccl@total@list@cnt \ccl@total@list@cnt\z@\relax
```

`\ccl@incr@count` stores the current list ID counter in a nesting-depth specific macro `ccl@prev@cnt@\the\ccl@depth`, advances the global internal list counter by one, and sets the publicly available counter `\cclID` to the resulting value. Also, if the nesting level is 1, the `\cclTopID` counter is incremented.

```
201 \def\ccl@incr@count{%
202   \csxdef{ccl@prev@cnt@\the\ccl@depth}{\the\cclID}%
203   \global\advance\ccl@total@list@cnt\@ne\relax
204   \global\cclID\ccl@total@list@cnt\relax
205   \ifnum\cclCurDepth=\@ne\relax
206     \global\advance\cclTopID\@ne\relax
207   \fi
208 }
```

`\ccl@decr@count` resets the list counter for the next lower nesting level, whenever a nested list is closed.

```
209 \def\ccl@decr@count{%
210   \global\cclID\csname ccl@prev@cnt@\the\ccl@depth\endcsname\relax
211 }
```

4.1 The List Environment

List environments have the same name as their respective containers (preixed by the `\ccPrefix`). However, they all call the low-level macros `\cc@list` and `\endcc@list`.

`\cc@list` is begin macro for the generalized coco-list environment. #1 is the attribute list of the environment.

```
212 \def\cc@list{\cc@opt@empty\cc@list}
213 \def\@cc@list[#1]{%
214   \ifx\cc@is@counted\relax
215     \ccUntoggleCountedConditionals
216   \fi
217   \ccl@advance@depth\@ne%
218   \ccl@incr@count%
219   \edef\ccl@cur@cont{\cc@cur@cont-\cclCurDepth}%
220   \global\cclFirsttrue
```

If the nesting goes deeper than the style programmer anticipated:

```
221 \ifcsdef\cc@container@\ccl@cur@cont\{
222   {\ifx\ccl@inherit\ccl@ih@common
223     \let\ccl@cur@cont\cc@cur@cont%
224     \else
225       \global\csletcs
226         {cc@type@Properties@\cc@cur@cont-\cclCurDepth}
227         {cc@type@Properties@\cc@cur@cont-\cclPrevDepth}%
228     \fi}%
```

Horizontal margin Properties from the previous nesting level are stored so that the nested lists can use them.

```
229 \edef\ccl@leftskip{\the\dimexpr\leftskip\relax}%
230 \edef\ccl@rightskip{\the\dimexpr\rightskip\relax}%
```

`prev-margin-left` <skip> stores the left margin of the next higher list level (i. e., the left margin of the list item that the current list is nested into)

```
231 \ccSetPropertyX{prev-margin-left}{\ccl@leftskip}%
```

`prev-margin-right` <skip> stores the superior list item's right margin.

```
232 \ccSetPropertyX{prev-margin-right}{\ccl@rightskip}%
233 \ccEvalType[\ccl@cur@cont]{Properties}%
234 \ccEvalType[\ccl@cur@cont]{Components}%
```

`\ccl@list@type` locally stores the current value of the `list-type` Property.

```
235 \edef\ccl@list@type{\ccUseProperty{list-type}}%
```

Processing of the optional argument.

```
236 \cclUseAttributeHandler{#1}%
```

The exact values of the margins are calculated.

```
237 \cclCalculateMarginLeft%
238 \cclCalculateVMargin{top}%
239 \cclCalculateVMargin{bottom}%
```

`\Item` is used to separate the single items of a list.

```
240 \csdef{\ccPrefix Item}{\cc@opt@empty\ccl@item}%
241 \def\ccl@item[##1]{%
242   \protected@edef\ccl@item@label{##1}%
243   \ifx\ccl@item@label\@empty
244     \cclUseLabelHandler%
245   \else
246     \ccComponent{Label}{##1}%
247   \fi
248   \sbox\z@{\@cc@is@finalfalse\ccUseProperty{label-format}}%
249   \@tempdima=\dimexpr\ccUseProperty{max-label-width}\relax
250   \ifdim\wd\z@<\@tempdima\relax
251     \@tempdima=\the\wd\z@\relax%
252   \fi
253   \bgroup
254     \def\cc@cur@cont{list}%
255     \cc@store@latest{\the\cclTopID-number-\cclCurDepth-maxwd}{\the\@tempdima}%
256     \cc@store@latest{\the\cclTopID-number-maxwd}{\the\@tempdima}%
257   \egroup
258   \ccSetPropertyX{label-width}{\the\@tempdima}%
259   \ccUseProperty{item-format}%
260   \ccUseProperty{at-begin-item-body}\ignorespaces%
261 }%
```

`\item` If default L^AT_EX macros are replaced per package option, `\item` is made into a copy of the local definition of `\ccPrefix Item`.

Warning: this might be dangerous when the User tries to embed something inside a CoCoT_EX list that uses L^AT_EX's standard `\list` or `\trivlist` environments!

```
262 \if@cc@replace\letcs\item{\ccPrefix Item}\fi%
```

Up to this point, we only managed Properties. From this point forward, we actually print the list. We start by using the `⚙ before-list` Property.

```
263 \ccUseProperty{before-list}%
```

then, we add the top vertical skip by `⚙ int-margin-top` amount.

```
264 \ccUseProperty{int-margin-top}%
```

and set the left and right margins using the `⚙ margin-left`, `⚙ label-sep` and `⚙ margin-right` Properties.

```
265 \leftskip\dimexpr\ccUseProperty{margin-left}+\ccUseProperty{label-sep}\relax%
266 \rightskip\dimexpr\ccUseProperty{margin-right}\relax%
267 }
```

`\endcc@list` is called at the end of each List Container's respective environment. It basically calls the `⚙ after-list` Property one last time, decrements the depth counter(s) and adds the `⚙ int-margin-bottom` vertical skip.

```
268 \def\endcc@list{%
269   \ccUseProperty{after-list}%
```

```

270 \ccl@decr@count%
271 \ccl@advance@depth\m@ne%
272 \ccUseProperty{int-margin-bottom}%

```

If the List is not nested, we eventually evaluate the `\after-indent` Property.

```

273 \ifnum\cclCurDepth=\z@\relax
274 \ccIfPropVal{after-indent}{false}{%
275 \global\@afterindentfalse
276 \aftergroup\cc@afterbox}{}%
277 \fi
278 }

```

`\cclCalculateVMargin` generates a macro that sets the internal margin Properties of the (nested) list. #1 is the orientation (`top` or `bottom`).

```

279 \def\cclCalculateVMargin#1{%
280 \ifdim\ccUseProperty{margin-#1}=\z@\relax
281 \ccSetProperty{int-margin-#1}{\relax}%
282 \else
283 \ccSetProperty{int-margin-#1}{\addvspace{\ccUseProperty{margin-#1}}}%
284 \fi
285 }

```

`\cclCalculateLeftMargin` generates the value that `\leftskip` is set to.

```

286 \def\cclCalculateMarginLeft{%
287 \ifcsdef{cc-list-\the\cclTopID-number-maxwd}
288 {\ccSetPropertyVal{number-width-max}{\csname cc-list-\the\cclTopID-number-maxwd\endcsname}}
289 {\ccSetPropertyVal{number-width-max}{1sp}}%
290 \ifcsdef{cc-list-\the\cclTopID-number-\cclCurDepth-maxwd}
291 {\ccSetPropertyVal{number-width-level-max}{\csname cc-list-\the\cclTopID-number-\cclCurDepth
-maxwd\endcsname}}
292 {\ccSetPropertyVal{number-width-level-max}{1sp}}%
293 \cc@get@indent[\ccl@calc@margin@left]{\the\cclTopID}%
294 }

```

`\ccl@calc@margin@left` is an override for coco-common's `\cc@calc@margin@left` specific for lists. According to `\cc@calc@margin@left`'s argument structure, #1 is the internal Property prefix, and #2 is the current value of the list depth counter. However, since we already stored the left margin of the previous depth level in the internal `\prev-margin-left` Property, we can gobble both arguments.

```

295 \def\ccl@calc@margin@left#1#2{%
296 \@tempdima=\ccUseProperty{prev-margin-left}\relax%
297 \ccSetPropertyX{margin-left}{\the\dimexpr\@tempdima-\ccUseProperty{indent}\relax}%
298 }

```

4.2 Unpacking the List Type-Specific Handlers

The caller macros for the two list type-specific Handlers for Attributes and Labels are defined here. They do some basic exception catching and then call the Handlers themselves if no error is detected.

`\cclUseLabelHandler` calls the list type specific Label handler to generate a label accordingly in cases where `\item` omits the optional argument.

```

299 \def\cclUseLabelHandler{%

```

```

300 \expandafter\ifx\csname ccl@make@label@\ccl@list@type\endcsname\relax
301 \ccPackageError{lists}{type}
302 {List type '\ccl@list@type' does not provide a Label Handler.}
303 {Make sure that the body of \ccl@list@type's declaration contains a \string\
    DeclareLabelHandler.}
304 \else
305 \csname ccl@make@label@\ccl@list@type\endcsname
306 \fi
307 }

```

`\cclUseAttributeHandler` checks if the list type specific attribute handler exists and applies it to the attribute list #1.

```

308 \def\cclUseAttributeHandler#1{%
309 \ccParseAttributes{\cc@cur@cont-\ccl@cur@depth}{#1}%
310 \expandafter\ifx\csname ccl@eval@attrs@\ccl@list@type\endcsname\relax
311 \ccPackageError{Lists}{Type}
312 {List type '\ccl@list@type' does not provide an Attribute Handler.}
313 {Make sure that the body of \ccl@list@type's declaration contains a \string\
    DeclareAttributeHandler.}
314 \else
315 \csname ccl@eval@attrs@\ccUseProperty{list-type}\endcsname
316 \fi
317 }

```

5 Default List Types

Vanilla CoCoT_EX supports three list types: numbered lists (corresponds to L^AT_EX's *enumerate* environment), unnumbered lists (*itemize*), and description lists (*description*).

5.1 Unnumbered Lists

`unnumbered` is technically an abstract child Container of the  `List` parent.

```

318 \ccDeclareListType{unnumbered}{%

```

`\ccl@make@label@unnumbered` generates the  `Label` Component of an unnumbered list type.

```

319 \DeclareLabelHandler{%
320 \ccComponent{Label}{\ccUseProperty{default-label}}

```

`\ccl@eval@attrs@itemize` is the handler for attributes of itemize-like list types. Currently, it does nothing.

```

321 \DeclareAttributeHandler{}

```

Itemize-Type List Specific Properties

`default-label` <any> is a property that holds a fallback label which is used when the optional argument of `\Item` is omitted.

```

322 {\ccSetProperty{default-label}{-}}

```


Itemize-Style Default Lists

`Itemize` is the user-level unnumbered  List Container.

```

323 \ccDeclareList{Itemize}{unnumbered}{\ccSetProperty{default-label}{\textbullet}}
324 \ccDeclareNested{Itemize}{1}{%
325   \ccSetProperty{label-face}{\normalfont\bfseries}%
326   \ccSetProperty{default-label}{\textendash}}
327 \ccDeclareNested{Itemize}{2}{\ccSetProperty{default-label}{\textasteriskcentered}}
328 \ccDeclareNested{Itemize}{3}{\ccSetProperty{default-label}{\textperiodcentered}}

```

5.2 Numbered Lists

`\ccl@item@adv` is an internal counter that holds the amount by which the counter of numebred lists should advance for each item.

```

329 \newcount\ccl@item@adv

```

`numbered` is an abstract child Container of the List parent that represents numbered lists.

```

330 \ccDeclareListType{numbered}{%

```

`\ccl@eval@attrs@numbered` is the handler for attributes specific to the enumerate-like list types.

```

331 \DeclareAttributeHandler{%

```

The attribute `step` indicates by what amount the internal counter should be advanced for each item. Defaults to +1 if none is given.

```

332   \ccIfAttr{\cc@cur@cont-\cclCurDepth}{step}
333     {\ccl@item@adv=\expandafter\numexpr\csname cc@\cc@cur@cont-\cclCurDepth @attr@step\
334       endcsname\relax}%
335     {\ccl@item@adv=\@one}%

```

The attribute `start` indicates the initial internal counter of the items in the list. The number itself is the counter of the first item, so we need to subtract the value of `step` from the given value such that `\item` can advance it by that same value. If the attribute is not given, the internal coutner is initialized to 0.

```

335   \ccIfAttr{\cc@cur@cont-\cclCurDepth}{start}
336     {\ccl@item@cnt=\expandafter\numexpr\csname cc@\cc@cur@cont-\cclCurDepth @attr@start\
337       endcsname\relax
338       \advance\ccl@item@cnt-\ccl@item@adv}%
339     {\ccl@item@cnt=\z@\relax}%
340   }

```

`\ccl@make@label@numbered` is the  Label handler of a numbered list type.

```

340 \DeclareLabelHandler{%
341   \advance\ccl@item@cnt \ccl@item@adv\relax
342   \expandafter\ifx\csname ccl@label@type@\ccUseProperty{enum-type}\endcsname\relax
343   \ccPackageWarning{lists}{type}{Enum type \ccUseProperty{enum-type} is unknown, revert to
344     numeric counters!}
345   \let\ccl@label\ccl@label@type@arabic%
346   \else
347   \letcs\ccl@label{\ccl@label@type@\ccUseProperty{enum-type}}%
348   \fi
349   \ccComponent{Label}{\ccl@label{\ccl@item@cnt}\ccUseProperty{default-label-sep}}
350 }%

```

```
350 }{%
```

Numbered List-Specific Properties

New Properties

`enum-type` [arabic|roman|Roman|Alph|alph] controls how the item counter is rendered when it is not given explicitly with the optional argument of `\item`. The default values are borrowed from LaTeX's default enumerate types and defined below.

```
351 \ccSetProperty{enum-type}{arabic}%
```

Properties with Deviating Default Values

By default, numeric ➡`Label` are followed by a period to accommodate L^AT_EX customs.

```
352 \ccSetProperty{default-label-sep}{.}
353 \ccSetProperty{label}{\ccUseComp{Label}}}
```

Available Counting Styles

`\ccl@label@type@arabic` transforms the value of the following (implicit) counter to arabic numerals.

```
354 \def\ccl@label@type@arabic{\@arabic}
```

`\ccl@label@type@roman` transforms the value of the following (implicit) counrer to lower case roman numerals.

```
355 \def\ccl@label@type@roman{\@roman}
```

`\ccl@label@type@Roman` transforms the value of the following (implicit) counrer to upper case roman numerals.

```
356 \def\ccl@label@type@Roman{\@Roman}
```

`\ccl@label@type@alph` transforms the value of the following (implicit) counrer to lower case alphabetic letters.

```
357 \def\ccl@label@type@alph{\@alph}
```

`\ccl@label@type@Alph` transforms the value of the following (implicit) counrer to upper case alphabetic letters.

```
358 \def\ccl@label@type@Alph{\@Alph}
```

Enumerate-Style Default Lists

`Enumerate` is the user-level Container for numbered 📦`List` Containers.

```
359 \ccDeclareList{Enumerate}{numbered}{%
360 \ccDeclareNested{Enumerate}{1}{% (
361 \ccSetProperty{default-label-sep}{})}%
362 \ccSetProperty{enum-type}{alph}%
363 }
364 \ccDeclareNested{Enumerate}{2}{\ccSetProperty{enum-type}{roman}}
365 \ccDeclareNested{Enumerate}{3}{\ccSetProperty{enum-type}{Alph}}
```

5.3 Description Lists

`text` is an abstract child Container of the List parent used for `description`-like list types.

```
366 \ccDeclareListType{text}{%
```

`\ccl@eval@attrs@text` is the handler for the attributes of description-like list types.

```
367 \DeclareAttributeHandler{%
368   \ccIfAttr{\cc@cur@cont-\cclCurDepth}{width}
369   {\ccSetPropertyVal{min-margin-left}{\expandafter\dimexpr\csname cc@\cc@cur@cont-\
370     cclCurDepth @attr@width\endcsname\relax}}%
371   {\ccSetProperty{min-margin-left}{2em}}%
372 \ccIfPropVal{label-growth}{down}
373   {\long\def\ccl@vbox##1{\smash{\vtop{##1}}}}
374   {\long\def\ccl@vbox##1{\vbox{##1}}}%
375 }
```

`\ccl@make@label@text` creates the label of a description-like list type.

```
375 \DeclareLabelHandler{%
376   \ccComponent{Label}}{%
377   }}
```

Description-Type Specific Properties

New Properties

`label-growth` [`up`|`down`] controls the direction labels “grow” into when they need more space than `*max-label-width`. On T_EX-primitive level, it controls whether the label is put into a `\vbox` or `\vtop` with `\hsize=\cclItemIndent`.

Important note: If the `label-growth` is set to ‘down’ and the description of an item uses less lines than its label, the label *will* flow into the next item. There is no (easy) way to catch that (automatically) without destroying the possibility to nesting lists.

```
378 {\ccSetProperty{label-growth}{up}}%
```

Properties with Deviating Default Values

The Properties `*margin-left` and `*indent` of text-type lists are by default set to `auto`.

```
379 \ccSetProperty{indent}{auto}%
380 \ccSetProperty{margin-left}{auto}%
```

To accommodate for the new `*label-grow` option, the `*label-box` has a conditional that switches between regular `\hbox` labels and the two `\vbox` variants described above.

```
381 \ccSetProperty{label-box}{%
382   \ifdim\ccUseProperty{label-width}<\ccUseProperty{max-label-width}\relax
383   \hbox to \cclItemIndent{%
384     \ccIfPropVal{label-align}{left}{\hss}%
385     \ccUseProperty{label-format}%
386     \ccIfPropVal{label-align}{right}{\hss}}%
387   \else
388     \ccl@vbox{\relax%
389       \hsize\dimexpr\cclItemIndent%
390       \leftskip\z@
391       \rightskip\z@}
```

```

392     \parindent\z@
393     \leavevmode
394     \ccUseProperty{label-format}%
395     \@@par
396   }%
397   \fi
398 }

```

Description-Type Default Lists

Description is the user-level Container for text type  List Containers.

As with the standard $\text{\LaTeX}$ `description` environment, there are no default definitions for nested Description-type lists.

```

399 \ccDeclareList{Description}{text}{%
400   \ccSetProperty{label-face}{\bfseries}
401 }

```

5.4 Replacing $\text{\LaTeX}$'s Default Lists

At the User's discretion (using the `replace` package option, see Sect. 1.1, above), $\text{\LaTeX}$'s default list environments `itemize`, `enumerate`, and `description` are re-defined to use CoCo $\text{\TeX}$'s list mechanism, instead.

```

402 \if@ccl@replace
403   \letcs\itemize{\ccPrefix Itemize}
404   \letcs\enditemize{end\ccPrefix Itemize}
405   \letcs\enumerate{\ccPrefix Enumerate}
406   \letcs\endenumerate{end\ccPrefix Enumerate}
407   \letcs\description{\ccPrefix Description}
408   \letcs\enddescription{end\ccPrefix Description}
409 \fi

```

```
</lists>
```

Index

Numbers written in *italic* refer to the page where the corresponding entry is described; numbers underlined refer to the definition; numbers in roman refer to the pages where the entry is used.

Macro and Environment Index

In this index, the cc(®)- and module specific ccX(®)-Prefixes were omitted when sorting the entries.

Symbols	
\@cc@url	57
\@ccf@calc@width	183
\@endnotemark	120
\@gobbleopt	45
\cc@parse@csv	36
\@xendnote	123
\cc_map_pdfstd:nn	8
A	
\cca@debug	60
\cca@declare@cmd	64
\cca@declare@cmd@firstopt	63
\cca@declare@cmd@secopt	63
\cca@do@doparas	60
\cca@do@dospaces	60
\cca@do@nodetree	59
\cca@do@showspaces	60
\cca@Gin@alt	67
\cca@id@document	72
\cca@id@document@dummy	72
\cca@lang@id	62
\cca@patch@error	60
\cca@protected@defs	63
\cca@relaxed@defs	62
\cca@saved@defs	62
\cca@set@docinfo	66
\cca@temp@signature	63
\ccaAddAltText	67
\ccaAddColSpan	71
\ccaAddFigure	70
\ccaAddID	66
\ccaAddKeep	71
\ccaAddLang	66
\ccaAddLastLink	67
\ccaAddNumbering	66
\ccaAddPlacement	66
\ccaAddRolemap	66
\ccaAddRowSpan	71
\ccaAddScope	71
\ccaAddToStruct	65
\ccaArtifact	70
\cc@abspage	46
\ccaConvertPdfString	72
\ccAddTitleEval	138
\ccAddTitleRole	138
\ccAddToComponentGroup	26
\ccAddToProperties	32
\ccAddToRole	82
\ccAddToType	20
\ccAddToTypeLocal	20
\ccaDisable	63
\ccaEnable	63
\ccaFigureEnd	71
\ccaFigureStart	71
\cc@afterbox	44
\ccaAfterClassHook	8
\ccaGetCurrentStruct	66
\ccaGetCurStruct	65
\ccaGetStructParent	68
\ccaMoveStruct	66
\ccaPagestyleArtifacts	69
\cc@apply@collection	28
\ccApplyCollection	27, 78
\ccAppPropLocal	34
\ccAppToProp	32
\ccaProtect	63
\ccaPstructEnd	65
\ccaPstructStart	65
\ccaReplaceStruct	66, 72
\cc@assign@res	27
\ccaStructEnd	65
\ccaStructStart	64
\ccAttrVal	37
\ccaVstructEnd	65
\ccaVstructStart	65
B	
\backmatter	10
\ccBreak	47
\ccBurl	56
\ccBurlTokenSep	56, 56

C

<code>\c@endnote</code>	122
<code>\cc@calc@margin@left</code>	52
<code>\CalcModulo</code>	45
<code>\CalcRatio</code>	45
<code>\cc@check@empty</code>	25
<code>\ccCheckParent</code>	20
<code>\cc@cnt@grp</code>	26
<code>\CoCoTeX</code>	40
<code>\cc@comp@edef</code>	29
<code>\ccCompLink</code>	55
<code>\ccComponent</code>	22
<code>\ccComponentEA</code>	22
<code>\cc@compose@collection</code>	28
<code>\ccComposeCollection</code>	27, 78
<code>\ccCompWarning</code>	22
<code>\ccContentsline</code>	48
<code>\cc@counted@comp@scheme</code>	30, 166
<code>\cc@create@label</code>	55
<code>\ccCreateContentListEntries</code>	49
<code>\ccCreateLabel</code>	54
<code>\CsToStr</code>	63
<code>\ccCurComp</code>	82
<code>\ccCurCount</code>	27
<code>\ccCurInfix</code>	82
<code>\ccCurSecLevel</code>	105, 115
<code>\ccCurSecName</code>	105

D

<code>\cc@debug@message</code>	17
<code>\ccDebugMsg</code>	17
<code>\DeclareAccessibilityCommand</code>	63
<code>\DeclareAttributeHandler</code>	208
<code>\ccDeclareAttributeHandler</code>	37
<code>\ccDeclareAttributeHandler*</code>	37
<code>\ccDeclareClass</code>	39
<code>\ccDeclareComponent</code>	21
<code>\ccDeclareComponentGroup</code>	25
<code>\ccDeclareContainer</code>	19
<code>\ccDeclareContentList</code>	49
<code>\ccDeclareCountedComponent</code>	30
<code>\ccDeclareEnv</code>	19
<code>\ccDeclareFloat</code>	187
<code>\ccDeclareGlobalComponent</code>	22
<code>\ccDeclareGroupHandler</code>	26
<code>\ccDeclareHeading</code>	94, 117
<code>\ccDeclareLabeledComp</code>	82
<code>\DeclareLabelHandler</code>	208
<code>\ccDeclareList</code>	209
<code>\DeclareListType</code>	208
<code>\ccDeclareNested</code>	209
<code>\ccDeclareRole</code>	79
<code>\ccDeclareRoleBlock</code>	78
<code>\ccDeclareTitlepage</code>	138
<code>\ccDeclareType</code>	19
<code>\cc@def@counted@comp</code>	30, 166
<code>\ccdefFromComp</code>	23
<code>\ccdefFromProperty</code>	33
<code>\ccdefTlFromComp</code>	23
<code>\discretionary</code>	57
<code>\cc@do@inherit</code>	21

E

<code>\ccedefFromCountedComp</code>	29
<code>\endcc@list</code>	212
<code>\endccaArtifact</code>	70
<code>\endccf@float</code>	174
<code>\endccSubFloat</code>	175
<code>\endcctmeta</code>	138
<code>\enindent</code>	120
<code>\enoteformat</code>	121
<code>\enoteheading</code>	121
<code>\enotesize</code>	121
<code>\ccEvalAttributes</code>	37
<code>\ccEvalType</code>	20
<code>\cc@expand@l@contents</code>	49
<code>\cc@extract@generic</code>	48

F

<code>\ccf@addcontentsline</code>	178
<code>\ccf@attr@debug</code>	177
<code>\ccf@attr@orient</code>	177
<code>\ccf@attr@pos</code>	176
<code>\ccf@calc@row@ht</code>	182
<code>\ccf@calc@sameheight</code>	182
<code>\ccf@calc@width</code>	163, 182, 183
<code>\ccf@capt@bottom@offset</code>	185
<code>\ccf@capt@top@offset</code>	185
<code>\ccf@check@empty</code>	179
<code>\ccf@compose@listof</code>	179
<code>\ccf@create@counter</code>	179
<code>\ccf@eval@class</code>	176
<code>\ccf@float</code>	173
<code>\ccf@floatbox</code>	163
<code>\ccf@generate@listof@handlers</code>	177
<code>\ccf@get@seps</code>	165
<code>\ccf@includesubgraphics</code>	190
<code>\ccf@int@cnt</code>	163
<code>\ccf@int@sub@flt@cnt</code>	163
<code>\ccf@ltx@includegraphics</code>	162
<code>\ccf@make@caption</code>	185
<code>\ccf@make@outer@caption</code>	186
<code>\ccf@make@subcaption</code>	187
<code>\ccf@margin@i</code>	164
<code>\ccf@margin@l</code>	164
<code>\ccf@margin@o</code>	164
<code>\ccf@margin@r</code>	164
<code>\ccf@measuregraphics</code>	191
<code>\ccf@process</code>	181
<code>\ccf@render@sub</code>	176
<code>\ccf@reserve@htmltab</code>	193
<code>\ccf@reserve@tab</code>	193
<code>\ccf@reserve@tabular</code>	193
<code>\ccf@reset@defaults</code>	164
<code>\ccf@sep@bottom</code>	164
<code>\ccf@sep@top</code>	164
<code>\ccf@set*@sep</code>	165
<code>\ccf@set@env</code>	165
<code>\ccf@set@hsize</code>	181
<code>\ccf@set@margin</code>	181
<code>\ccf@set@margins</code>	181
<code>\ccf@set@subcounter</code>	180
<code>\ccf@set@top@counter</code>	180

Container Index

A	
article-meta	<u>83</u>
C	
chapter	<u>122</u>
CommonMeta	<u>77, 93</u>
D	
Description	<u>218</u>
E	
Enumerate	<u>216</u>
F	
Figure	<u>182, 188</u>
float	<u>166</u>
G	
Graphic	<u>188</u>
H	
Heading	<u>93, 116</u>
I	
InlineFigure	<u>191</u>
Itemize	<u>215</u>
L	
List	<u>204, 214–216, 218</u>
M	
Meta	<u>42</u>
N	
numbered	<u>215</u>
S	
section	<u>122</u>
T	
Table	<u>193</u>
Tabular	<u>192</u>
text	<u>217</u>
titlepage	<u>137</u>
U	
unnumbered	<u>214</u>

Component Index

In this index, the name in parentheses denote the (abstract) Container within which the Component entry is defined.

A	
Abstract (Generic)	84, 109
Accepted (Generic)	84
Acknowledgements (titlepage)	145
AddNoteI (titlepage)	151
AddNoteII (titlepage)	152
AddNoteIII (titlepage)	152
AddNoteIV (titlepage)	152
Address (Generic)	85
Advertise (titlepage)	150
Affil (Generic)	89
AffilBlock (Generic)	85, 85, 89, 90, 108
Affiliation (Generic)	80, 80, 85, 89
AffilID (Generic)	85
AffilRef (Generic)	80, 80, 85, 114
AltNames (titlepage)	146
AltText (float)	188, 190
AltTitle (titlepage)	146
Author (Generic)	106
Author (Heading)	93
AuthorCitationList (Generic)	81, 89
AuthorContact (Generic)	107, 107, 114
AuthorContactBlock (Generic)	107, 114
AuthorContribution (Generic)	89
AuthorCorrespondence (Generic)	82
AuthorNameList (Generic)	81, 81, 89, 107, 108, 114
AuthorNameList (titlepage)	146, 153
AuthorPDFInfo (Generic)	81, 89
AuthorShortCitationList (Generic)	81, 89
B	
BibleISSN (titlepage)	149
Biblio (titlepage)	149
BiblioTitle (titlepage)	149
BiblISSN (titlepage)	149
BMNumber (Generic)	114
BMNumber (Heading)	94
BMSubtitle (Heading)	94
BMTitle (Generic)	114
BMTitle (Heading)	94
C	
Caption (float)	166
CC (Generic)	77
CitationList (Generic)	81, 87
CiteAs (Generic)	84
CiteName (Generic)	79, 80, 81, 87, 88
COIStatement (Generic)	84
Component (titlepage)	150
Content (float)	166, 168
Conversion (titlepage)	150
Copyright (Generic)	83
CopyrightDisclaimer (titlepage)	151
CorrespondanceAs (Generic)	82
Correspondence (Generic)	82, 88
CorrespondenceAs (Generic)	81, 87, 89
Country (Generic)	85
Cover (titlepage)	145
CoverConcept (titlepage)	150
CoverDesign (titlepage)	150
D	
Date (titlepage)	148, 153
Dedication (titlepage)	145
Department (Generic)	85
DocTitle (titlepage)	146
DOI (Generic)	85
E	
Edition (titlepage)	148
EditionNote (titlepage)	148
Editorial (titlepage)	146
EditorNameList (titlepage)	146, 147
EISBN (titlepage)	149
EISSN (titlepage)	149
ElibPDF (titlepage)	149
Email (Generic)	80
EndPage (Generic)	84
EnvDisclaimer (titlepage)	150
EpubISBN (titlepage)	149
EpubPreText (titlepage)	149
F	
Fig (float)	188, 189
FirstName (Generic)	80, 80, 86
FullName (Generic)	79, 81, 86, 89, 114
FundingPostText (titlepage)	149
FundingPreText (titlepage)	149
H	
Heading (Generic)	78
Honorific (Generic)	80
I	
Initial (Generic)	80, 86
Institute (Generic)	85
ISBN (titlepage)	148
ISBNPreText (titlepage)	148
ISSN (titlepage)	148
Issue (titlepage)	151
J	
JournalAbbrev (titlepage)	151
JournalName (titlepage)	151
K	
Keywords (Generic)	84, 109
L	
Label (Generic)	90, 91
Label (List)	207, 208, 214–216

Property Index

In this index, the name in parentheses denote the (abstract) Container within which the Property entry is defined.

A	
affil-block-face (Generic)	90
affil-block-format (Generic)	90
affil-block-item-face (Generic)	89
affil-block-item-format (Generic)	90
affil-sep (Generic)	89
affiliation-format (Generic)	85, 89
after-heading-block (Generic)	110
after-heading-par (Generic)	108
after-indent (Generic)	110
after-indent (List)	205, 213
after-item (List)	205, 206, 206
after-list (List)	205, 212
after-skip (Generic)	110
article-title (titlepage)	153
at-begin-item-body (List)	206
at-end-item-body (List)	206, 206
author-cite-name-format (Generic)	88
author-contact-block-format (Generic)	114
author-contact-format (Generic)	114
author-correspondence-as-format (Generic)	89
author-face (Generic)	108
author-full-name-format (Generic)	89
author-list-cite-format (Generic)	89
author-list-correspondence-format (Generic) ..	89
author-list-format (Generic)	114
author-list-pdfinfo-format (Generic)	89
author-list-print-format (Generic)	89
author-list-short-cite-format (Generic)	89
author-pdfinfo-name-format (Generic)	89
author-short-cite-name-format (Generic)	88
auto-number-prefix (float)	170
auto-number-prefix-sep (float)	170
B	
before-float (float)	168, 173
before-heading (Generic)	108
before-heading-block (Generic)	110
before-item (List)	206, 207
before-list (List)	205, 212
before-skip (Generic)	110
bookmark (Generic)	114
bookmark-level (Generic)	113
C	
caption-bottom (float)	169, 170, 184, 185
caption-face (float)	169, 182, 185
caption-face-bottom (float)	169, 169, 182, 185
caption-face-top (float)	169, 169, 182, 185
caption-sep-bottom (float)	170, 185
caption-sep-top (float)	169, 170, 185
caption-top (float)	169, 170, 184, 185
caption-valign-bottom (float)	187
corresp-mark (Generic)	88
corresp-sep (Generic)	88
counted-name-sep (Generic)	88, 114
D	
default-label (List)	214
E	
enum-type (List)	216
extended (Generic)	109
extended-heading (Generic)	109
F	
fix-dimen (float)	168
float-content (float)	168, 188
float-number (float)	171, 180
float-render (float)	168, 181, 189, 190, 191, 194
float-skip-bottom (float)	165, 167
float-skip-top (float)	165, 167
H	
heading-block (Generic)	108
heading-block (Heading)	98
heading-par (Generic)	108
I	
indent (Generic)	110, 110
indent (List)	206, 207, 213, 217
initials-format (Generic)	80, 86
initials-period (Generic)	86
initials-sep (Generic)	86
inline-content-h-sep (float)	168
inline-content-h-sep-auto (float)	168
inline-content-v-sep (float)	168
int-margin-bottom (List)	212, 213
int-margin-top (List)	212, 213
interline-para (Generic)	107
interline-para-sep (Generic)	107
intext-skip-bottom (float)	165, 167
intext-skip-top (float)	165, 167
item-format (List)	207
item-offset (List)	206, 207
item-sep (List)	205, 206
K	
keywords-sep (Generic)	88
L	
label (List)	207, 207, 216
label-align (List)	207, 207
label-box (List)	207, 207, 217
label-face (List)	207, 207, 218
label-format (List)	207, 207
label-grow (List)	217
label-growth (List)	217
label-pos (float)	171, 185
label-sep (List)	206, 207, 207, 212
labeled-meta-[ccCurInfix]-format (Generic) ...	82
labeled-meta-[infix]-after (Generic)	91
labeled-meta-[infix]-before (Generic)	91
labeled-meta-[infix]-face (Generic)	91
labeled-meta-[infix]-format (Generic)	90

labeled-meta-[infix]-label-face (Generic) 91
 labeled-meta-[infix]-label-format (Generic) . . 90
 labeled-meta-[infix]-label-sep (Generic) 91
 labeled-meta-after (Generic) 91
 labeled-meta-before (Generic) 91
 labeled-meta-before-[infix]-label (Generic) . . 90
 labeled-meta-face (Generic) 91
 labeled-meta-format (Generic) 82, 90
 labeled-meta-label-face (Generic) 91, 91
 labeled-meta-label-format (Generic) 90
 labeled-meta-label-sep (Generic) 91
 legend-face (float) 169, 169
 list-of-after-entry (float) 173, 178
 list-of-before-entry (float) 172, 178
 list-of-block (float) 172, 178
 list-of-caption-sep (float) 172
 list-of-indent (float) 172
 list-of-margin-left (float) 172, 172
 list-of-margin-right (float) 172, 172
 list-of-number-align (float) 172
 list-of-number-face (float) 171, 172
 list-of-number-format (float) 172
 list-of-number-sep (float) 171, 172
 list-of-page-face (float) 178
 list-of-page-sep (float) 171, 172
 list-of-parfillskip (float) 172, 172
 list-type (List) 208, 211

M

main-title-format (Generic) 109
 margin-bottom (List) 205, 213
 margin-inner (float) 167, 181
 margin-left (Generic) 110, 110
 margin-left (List) 205, 207, 212, 213, 217
 margin-left (float) 168, 175, 181, 189
 margin-outer (float) 167, 181
 margin-right (Generic) 110
 margin-right (List) 205, 212
 margin-right (float) 168, 175, 181, 189
 margin-top (List) 205, 213
 max-label-width (List) 205, 217
 measure-content (float) 168, 189

N

name-and (Generic) 88
 name-et-al (Generic) 88
 name-sep (Generic) 88
 no-BM (Generic) 111
 no-toc (Generic) 111
 number-align (Generic) 111
 number-face (Generic) 111
 number-face (float) 169, 171
 number-format (Generic) 111
 number-sep (Generic) 111, 111
 number-sep (float) 169, 171
 number-width (Generic) 111
 number-width-level-max (List) 213
 number-width-max (List) 213
 numbering (Generic) 111
 numbering (float) 170

O

orcid-link (Generic) 114, 114
 output-intent (titlepage) 159

P

par-fill-skip (List) 206, 206
 par-indent (List) 206, 206
 par-skip (List) 205, 206, 207
 placement-tag (float) 189, 192
 prev-margin-left (List) 205, 211, 213
 prev-margin-right (List) 211

Q

quote-block-format (Generic) 108
 quote-face (Generic) 108
 quote-source-face (Generic) 108

R

role-block-cite-format (Generic) 81, 87
 role-block-correspondence-format (Generic) . . 82, 88
 role-block-pdfinfo-format (Generic) 81, 87
 role-block-print-format (Generic) 81, 87
 role-block-short-cite-format (Generic) 81, 87
 role-cite-name-format (Generic) 79, 87
 role-correspondence-as-format (Generic) . . . 81, 87
 role-full-name-format (Generic) 79, 86
 role-pdfinfo-name-format (Generic) 80, 87
 role-short-cite-name-format (Generic) 80, 87
 running-heading (Generic) 111
 running-level (Generic) 111

S

smash (float) 191
 source-face (float) 169, 169
 sub-float-sep (float) 167, 181, 183
 sub-number (float) 171, 180
 sub-number-after (float) 180
 sub-number-before (float) 171, 180
 sub-number-face (float) 171, 180
 sub-number-format (float) 171, 180
 sub-number-sep (float) 171, 171
 sub-number-style (float) 171, 180
 subcaption-add-sep-bottom (float) 170, 185
 subcaption-add-sep-top (float) 170, 185
 subcaption-bottom (float) 170, 184, 185
 subcaption-face (float) 169, 182, 185
 subcaption-face-bottom (float) 169, 182, 185
 subcaption-face-top (float) 169, 182, 185
 subcaption-sep-bottom (float) 185
 subcaption-sep-top (float) 170, 185
 subcaption-top (float) 170, 184, 185
 subcaption-valign-bottom (float) 170, 187
 subcaption-valign-top (float) 170, 187
 subfloat-content (float) 168, 175, 189
 subfloat-render (float) 168, 181, 189, 190, 194
 subfloat-same-height (float) 188
 sublabel-pos (float) 171, 185
 subtitle-face (Generic) 108

T

title-face (Generic) 108, 111

<code>toc-after-entry</code> (Generic)	113	<code>toc-number-align</code> (Generic)	112
<code>toc-before-entry</code> (Generic)	113	<code>toc-number-face</code> (Generic)	112 , 112
<code>toc-format</code> (Generic)	113	<code>toc-number-format</code> (Generic)	112
<code>toc-hang-number</code> (Generic)	113	<code>toc-number-sep</code> (Generic)	112 , 112
<code>toc-hook</code> (Generic)	110	<code>toc-number-width</code> (Generic)	112
<code>toc-indent</code> (Generic)	112	<code>toc-page-face</code> (Generic)	112 , 113
<code>toc-level</code> (Generic)	113	<code>toc-page-format</code> (Generic)	113 , 113
<code>toc-margin-bottom</code> (Generic)	112 , 113	<code>toc-page-sep</code> (Generic)	112 , 113
<code>toc-margin-left</code> (Generic)	112 , 113	<code>toc-title-face</code> (Generic)	112 , 113
<code>toc-margin-right</code> (Generic)	112 , 113		
<code>toc-margin-top</code> (Generic)	111 , 113		
		V	
		<code>vertical-align</code> (float)	191

Hook Index

B	
begin-hook	108
C	
cc/headings/[level]/after	99
cc/headings/[level]/before	99
cc/headings/[level]/print/before	99
cc/headings/[level]/run/after	99
cc/headings/[level]/toc/after	99
cc/headings/[level]/toc/before	99
cc/headings/chapter/print/before	122
cc/listof/[type]/after	178
cc/listof/[type]/before	178
cca/document/begin	72
cct/document/meta	138, 143
cct/maketitle/after	143
cct/maketitle/before	143, 144
E	
env/Heading/[level]/begin	100, 105
env/Heading/after	116
env/Heading/before	116
env/meta/begin	143

Tag Index

Symbols		
/Span	67	KeywordsLabel 109
		KeywordsText 109
A		L
Abstract	109	L 205
AbstractLabel	109	Lbl 109, 113, 125, 207
AbstractText	109	LBody 206
Aside	174	leaders 112
		LI 206
C		N
Caption	186	Note 125
D		P
Div	117, 174	P 24, 113, 114, 160
F		R
Figure	190	Reference 113, 125
FootnoteMark	125, 126	
FootnoteText	125, 126	
H		S
H	109	Sect 94, 115, 116
H1	114, 156	SectMeta 98, 116, 117
H6	114	Span 113
Hn	109, 114, 116	Spans 113
K		T
Keyword	109	Title 114, 156, 160
		TOCI 113

Attribute Index

B		F	
break-caption (float)	<u>173</u>	float-pos (float)	<u>173</u>
C		N	
class (Heading)	<u>94</u>	no-same-height (float)	<u>173</u>
class (float)	<u>173</u>	notag (Heading)	<u>94</u>
D		O	
debug (float)	<u>173</u>	orientation (float)	<u>173</u>